

はじめてのLean

梅崎直也

ZEN大学

2026年9月28日

はじめての Lean

この教材の目的は、Lean のコードを**書ける**ようになることではなく、証明が検査される**仕組みを納得**することである。

目標

- **目標1** — Lean を読めるようになる。Lean のコードを読むとは、書かれた**項の型を推測**することである。そしてこの推測は**機械的な手順**で実行できる——だからコンピュータにも実行できる。実際に自分の手で型を推測できるようになること、少なくとも「これなら機械にもできそうだ」という感覚を持つことを目指す。
- **目標2** — **検証の仕組みを納得**する。項の型を推測するこの仕組みが、**定理の証明の検証にそのまま使える**ことを納得する。「なぜそれで証明の正しさを検証したと思えるのか」への答えがここにある。

この2つの目標に向けて、第1章・第3章で次の3段階を学ぶ。

- **型がどのように作られるか**を知る。
- その**型の項をどのように作り、使うか**を知る。
- 複雑な項についても、**部分項から順に型を推測**できるようになる。

第2章・第4章では、「**命題は型であり、証明はその型の項である**」という対応を学ぶ。これによって、目標1の「項の型を推測する」仕組みが、目標2の「証明を検証する」仕組みへ接続する。

仕組みのスローガン

納得したいことは、突き詰めれば次の2つである。

1. 文字列が**項**であるかどうか、つまり文法規則に従っているかは、**機械的に判定**できる。
2. 項が与えられたら、**型が付くかどうかを機械的に判定**でき、付く場合はその型を計算できる。

設計の方針

- **タクティクについて詳しくは説明しない**。第6章では、タクティクを証明項を組み立てるための仕組みとして簡単に紹介し、タクティクによる証明と、それに対応する項スタイルの証明を比較する。
- **mathlib は使わず、Lean 4 の標準環境（自動的に読み込まれる Prelude を含む）だけ**を使う。集合も位相もすべて自作する。
- どちらも理由は同じ——**仕組みを納得するために、ブラックボックスを減らしたい**からである。実際にコードを書く場面では、タクティクも mathlib も遠慮なく使えばよい。

- Lean を網羅的に紹介することは目的ではなく、必要な最低限の機能しか説明しない。

1. **第1・2章 — 依存関数型から、ならば・全称の証明へ。** 第1章で項と型、定義、関数・依存関数、暗黙引数を学ぶ。続く第2章では、仮定や対象を受け取り、適用して進む証明を読む。「単射どうしの合成は単射」の証明まで進み、型検査が証明の検査になることを確かめる。
2. **第3・4章 — 帰納型から、かつ・または・存在の証明へ。** 第3章で帰納型、場合分け・再帰、等号と自然数の不等式を学び、項を作る・分解する道具を増やす。続く第4章では、その道具を使って「かつ」「または」「矛盾」「存在」の証明を読む。偶数の和や全射の合成を例に、関数・構成子・場合分けを組み合わせ、証明検査の全体像を整理する。structure と部分型は第5章で扱う。

各 ⇨ 練習には折りたたみの解答が付いている（解答もすべて Lean の検査済み）。手元で動かす Lean のファイルは [GitHub のリポジトリ](#)にある（使い方はリポジトリの README）。

1. [型と項 I — 関数と依存関数型](#)
2. [型と命題 I — ならばと全称量化](#)
3. [型と項 II — 帰納型](#)
4. [型と命題 II — かつ・または・否定・存在量化](#)
5. [数学を記述する道具](#)
6. [位相空間](#) — 定義から「コンパクト空間からハウスドルフ空間への連続全単射は同相写像である」の証明まで
7. [発展演習: 位相空間の圏と自由忘却随伴](#) — 演習（sorry を自分で埋める）
8. [発展演習: 実数](#) — 演習（sorry を自分で埋める）
9. [発展演習: Ascoli の定理（ブルバキ流）](#) — 演習（sorry を自分で埋める）
10. [発展演習: 実数版 Arzelà–Ascoli の定理](#) — 演習（sorry を自分で埋める）
11. [発展演習: グラフの被覆](#) — 演習（sorry を自分で埋める）
12. [発展演習: 位相空間の被覆](#) — 演習（sorry を自分で埋める）
13. [発展演習: 基本亜群と円周](#) — 演習（sorry を自分で埋める）

型と項 I — 関数と依存関数型

Lean が証明を検査する仕組みを知るために、まずコードの読み方から始めよう。この章では、数や関数を例に、書かれた項がどんな型を持つかを一つずつ確かめる。その読み方を、次の章で証明に当てはめていく。

この教材の読み方

中心は、書かれたコードを解読することである。項を見るたびに型を予想し、`#check` の出力で確かめよう。各節の練習には、予想を言葉にする問題と、小さな項を書いて確かめる問題がある。本文にはその場で確認したい問題を置き、短い確認問題は本文の説明に続けて置き、組み合わせの練習や発展問題は章末に集めた。型を追う手順をつかむことが目的である。型検査がそのまま証明の検査になることを、次の章で経験する。

1. 項と型

Lean に書くものの基本の単位を**項** (term) と呼ぶ。項とは何かの正確な定義はここでは与えないが、さしあたり `#check` の右側を書いてエラーが出ないものは**項だ**と考えてよい。

型 (type) は、ひとまず**集合のようなもの**、その型の項は**要素のようなもの**と思ってもよい。ただし、これは大まかなイメージであり、型や項には、それらを作り、扱うための規則がある。以後、その規則を少しずつ見ていく。

`#check e` は、項 `e` の**型**を調べて表示するコマンドである。ここで、調べる対象の `e` は項だが、`#check e` 全体は項ではなく、Lean に対する指示である。後で使う `def` も、新しい定義を導入するコマンドであり、項とは区別する。エディタ右側のパネル (Infoview) に表示される結果を、本文では独立した枠で示す。実際に Lean を書くときも、Infoview の表示や項にマウスを重ねたときの型表示を使い、どのような型として読まれているか、どこで型が合わなくなったかを確かめながら進める。

```
#check 3
```

```
3 : Nat
```

これは「項 `3` は型 `Nat` (自然数の型) を持つ」と読む。数学の $3 \in \mathbb{N}$ のようなものと思ってよいが、厳密には異なり、`:` は項の型を示す記号であって、集合への所属を表す $\in$ とは区別する。

```
#check true
```

```
Bool.true : Bool
```

「項 `true` は型 `Bool` (真偽値の型) を持つ」。表示が `Bool.true` となっているのは、これが正式な名前だからである。ここで `.` は名前の区切りで、接頭辞 `Bool` と名前 `true` をつないでいる。Lean はこの区切りを使って名前を扱う。標準ライブラリでは `true` という短い名前も用意されているので、どちらでも書ける (名前の詳しい仕組みは [05_MathematicalTools.lean](#) 7節で見る)。

すべての項は型を持つ。ここで大事な観察が1つある。 **型を表す表記それ自体もまた項である**。実際、`Nat` を `#check` の右に書くと:

```
#check Nat
```

```
Nat : Type
```

`Nat` という表記は、`Type` という型を持つ項なのである (以下ではこのことを、少し縮めて「型も項である」と言う)。

では `Type` は?

```
#check Type
```

```
Type : Type 1
```

`Type` もまた項であり、その型は `Type 1`。

補足: 宇宙の階段

```
#check Type 1
```

```
Type 1 : Type 2
```

`Type`, `Type 1`, `Type 2`, ... の階層を宇宙 (universe) と呼ぶ。ここで `Type 1` の `1` は、`Nat` 型の項ではない。この教材では宇宙には**深入りしない**。

型が項と同じ資格を持つ、というこの一様性が Lean の設計の核で、これがあるから「型を受け取る関数」「型を返す関数」が書ける (3節の `Map` と4節)。

数の演算や、等式も項として書ける。それぞれにどんな型が付くかを見てみよう。

```
#check 3 + 4
```

```
3 + 4 : Nat
```

「`3 + 4` という項は `Nat` 型を持つ」と読む。

```
#check 1 + 1 = 2
```

```
1 + 1 = 2 : Prop
```

今度は、出力の型が `Nat` ではなく `Prop` になった。別の例も見てみよう。

```
#check 2 = 1
```

```
2 = 1 : Prop
```

こちらも型は `Prop` である。ここで現れた `Prop` は、**命題**たちの型である。等式 `1 + 1 = 2` も `2 = 1` も命題なので、`Prop` 型の項になる。`2 = 1` のような**偽の命題**も、命題としては立派な項であることに注意しよう。`#check` は命題の真偽を判定しているわけではない（証明の話は `02_Forall.lean` で扱う）。

補足: Prop 自身の型

`Prop` 自身の型も調べてみよう:

```
#check Prop
```

```
Prop : Type
```

`Prop` 自身もまた項であり、その型は `Type` である。

先取り: 演算と命題の記号

`+` や `=` の仕組みは、必要になるところで少しずつ説明する。`+` は3節で関数として読み、型に応じて演算を選ぶ仕組みは第5章で扱う。次の章では等式の証明を項として使う。不等式の定義や証明の仕組みは、帰納型を学んだあとで見る。今は、それぞれの項にどんな型が付くかを追えばよい。

ここまでで確かめたことは2つである: **項の型はいつでも `#check` で調べられること**、そして**型も項であること**。

`#check` という機能があることから分かるように、Lean には**項を与えると、その型を機械的に計算する仕組み**がある。この仕組みを証明の検証に使うのが、冒頭で述べたこの教材の行き先である。例えば `1 + 1 =`

2 という命題そのものを型として、その型の項を作ることが、この等式の証明を与えることになる。その項が指定された型を持つかを検査すれば、証明の検査になる。この対応を、次の章では単射の合成などの具体例で読む。まずこの章で関数の型を追い、次の `02_Forall.lean` で証明の例を読む。

補足: `Nat` や `Bool` はどこから来たか——Prelude

`import` を1行も書いていないのに `Nat`・`Bool`・`+` が使えるのは、Lean が起動時に **Prelude**——標準ライブラリの核——を自動で読み込んでいるからである。この教材で「標準ライブラリ」と呼ぶものの実体はただの Lean のソースファイルで、`Nat` も `Bool` も、第3章で学ぶ帰納型の仕組みで定義されている。特別なのは「最初から読み込まれている」ことだけである。

⇒ 練習

1. `#check Bool` と `#check Type 2` の表示を予想してから確かめよ。

解答 1

```
#check Bool
```

```
Bool : Type
```

`Bool` は型なので、その型は宇宙 `Type` 。

```
#check Type 2
```

```
Type 2 : Type 3
```

宇宙の階段は1段のぼる: `Type n : Type (n + 1)` 。

2. `#check 3 = 5` と `#check 5 = 5` の表示をそれぞれ予想してから確かめよ。命題の真偽が異なっても、同じ型が付くことを説明せよ。

解答 2

```
#check 3 = 5
```

```
3 = 5 : Prop
```

```
#check 5 = 5
```

```
5 = 5 : Prop
```

$3 = 5$ は偽で $5 = 5$ は真だが、どちらも型は `Prop`。 `#check` は命題の真偽を調べていない——命題を表す項が型 `Prop` を持つことを確認している。

2. def — 新しい項を定義する

```
def 名前 : 型 := 項
```

と書くと、**項に新しい名前を付けて定義**できる。`:=` の右辺には、すでに使える項や、それらを組み合わせた項を書く。定義した後は、左辺の名前自体を項として使える。`def` 全体は、定義を導入するコマンドである。この形の宣言には、終わりを示す特別な記号はいらない。Lean は構文に従って右辺の一つの項を読み取るので、改行しただけで宣言が終わるわけではない。

```
def x : Nat := 2
```

```
#check x
```

```
x : Nat
```

「`x` という `Nat` 型の項を、`2` として定義した」と読む。型は注釈どおり `Nat`。名前 `x` 自体が値を決めるのではなく、右辺の `2` が定義の中身を決めている。

定義した項は、それ自体また項として使える:

```
def y : Nat := x
```

```
#check y
```

```
y : Nat
```

項を扱うコマンドをあと2つ紹介する。`#eval` は項を**計算**して値を表示する:

```
#eval x
```

`#print` は、名前に付けられた**定義そのもの**を表示する:

```
#print x
```

```
def x : Nat :=
2
```

さて、ここからが**この教材の最重要のテーマ**である。Lean は `def` の宣言を受理する前に、**型検査**を行う。`:=` の右の項が、コロンの右に指定した型を持つかを確かめる。指定した型も手がかりとして使い、一致しなければ宣言を受理しない。

この教材では、項 `M` と型 `A` が与えられ、`M : A` が成り立つかを確かめることを**型検査**という。項だけからその型を求めるのが**型の計算**で、`#check` はこれを行う。読者がその計算を自分の手でなぞるのを**推測**と呼ぶ。`def 名前 : 型 := 項` は、Lean に `項 : 型` の型検査を依頼する形である。

読む側の目線では、次の定義の右辺には **Nat 型の項を期待**する:

```
def bad : Nat := true
```

```
error: Type mismatch
  true
has type
  Bool
but is expected to have type
  Nat
```

`#check true` と同じく、`true` の型は `Bool`。Infoview は、要求された `Nat` と合わないと報告している。

コメントと docstring

ソースには、Lean が読み飛ばす**コメント**を書き込める。

```
def x : Nat := 2 -- `--` から行末まではコメント

/- 複数行にわたる
   コメントはこう書く -/
```

この教材の地の文が入っている `/-! ... -/` も、コメントの一種である。

もう1つ、宣言の直前に `/-- ... -/` と書くと、**docstring**（その宣言の説明書き）になる:

```
/-- 最初に定義した数。 -/  
def x : Nat := 2
```

docstring は直後の宣言に結び付けられ、エディタで名前にカーソルを乗せると、型と一緒にこの説明がポップアップで表示される。以後のコードにも、`--` のコメントや docstring で説明を添えていく。

⇒ 練習

3. `def z : Nat := 5` を自分で宣言し、`#check z`・`#eval z`・`#print z` の表示をそれぞれ予想してから確かめよ。

解答 3

```
def z : Nat := 5
```

```
#check z
```

```
z : Nat
```

```
#eval z
```

```
5
```

```
#print z
```

```
def z : Nat :=  
5
```

`#check` は型、`#eval` は計算した値、`#print` は宣言そのものを表示する。

4. 本文の `def x : Nat := 2` が使える状態で、`def oops : Bool := x` は受理されるか。予想してから試し、エラーメッセージを本文の例と見比べよ。別のファイルで試す場合は、先に `def x : Nat := 2` を書くこと。

解答 4

本文の `def x : Nat := 2` が使えることが前提である。この解答ファイルでは `import «01_TypesAndTerms»` によって読み込まれている。別のファイルで単独で試すなら、先にその定義を書く。

受理されない。x の型は `Nat` で、コロンの右の `Bool` と一致しないから型検査が失敗する:

```
def oops : Bool := x
```

実行すると、次のエラーになる:

```
error: Type mismatch
  x
has type
  Nat
but is expected to have type
  Bool
```

本文の `def bad : Nat :=` の例と同じく、「`:=` の右の項の型を計算し、宣言された型と照合する」という手順の破れがそのまま報告されている。

3. 関数

集合 $A \cdot B$ に対して写像全体の集合 $\text{Map}(A, B)$ が定まると同様に、型 $A \cdot B$ に対して「 A から B への関数の型」 $A \rightarrow B$ が定まる。つまり矢印 $\rightarrow$ は、**2つの型から新しい型を1つ作る操作**を表す記号であって、それ自体が具体的な関数を定めるのではない。ただし集合の場合と違い、 $A \rightarrow B$ は写像概念から**定義されたものではない**——関数型は、これ以上さかのぼれない原始的な部品である。

記号の入力のしかた

VS Code の Lean 4 拡張では、`\to` の後に空白か Tab を押すと $\rightarrow$ を入力できる。他の記号も `\` から始まる略記で入力する。記号にマウスを乗せると打ち方を確認できる。以後も初出時に打ち方を添え、一覧を `03_InductiveTypes.lean` の付録に置く。

fun 記法 — 関数を項として書く

関数そのものを書くには、`fun` という記法を使える。引数 `x` が使えるものとして項 `e` を書くと、そこから**一つの項** `fun x => e` を作れる。これは「`x` を受け取って `e` を返す関数」を表す。**`=>` の右側の項 `e` を、関数の本体 (body) と呼ぶ。**

例えば、`x : Nat` を使って書いた項 `x + x` から、関数の項 `fun x => x + x` を作る。本体には、受け取った `x` を使える。

注意 — 名前を書く場所と、項を書く場所。 この形の `x` は引数の名前を書く場所であり、任意の項を入れる場所ではない。名前は `n` などに変えてよく、その場合は本体でその引数を指す名前も揃える。`fun x => x + x` と `fun n => n + n` は同じ関数を表す。一方、`e` の場所には、その引数を使って組み立てた項を書く。

注意 — 記法を作る記号。 `fun` と `=>` は、それ自体が項なのではない。これらは、引数の名前と本体を組み合わせて関数の項を作るための特別な記号である。全体の `fun x => e` が一つの項になる。

この教材の表示設定では、出力中の `=>` が `→` と表示される。同じ関数の書き方である。入力するときも `fun x → e` と書いてよい。なお、`=>` は `fun` 専用ではなく、後で扱う `match` の分岐にも使う。

この項の型は、次のように読む。引数に型を注釈して `fun (x : A) => e` と書けば、`x` の型はその注釈から `A` と分かる。注釈を省略した場合も、関数全体に期待されている型から分かることがある。次に、**`x : A` を前提として本体 `e` の型を調べる**。そのもとで `e : B` ならば、`fun x => e` は関数型 `A → B` の項になる。

```
def double : Nat → Nat := fun n => n + n
```

```
#check double
```

```
double : Nat → Nat
```

表示された型は、注釈した `Nat → Nat` と一致している。定義域の `Nat` が外側から伝わるので、型を書かなかった引数も `n : Nat` と決まる。`fun (n : Nat) => n + n` と書いたのと同じである。この `n` は関数の本体で使える仮引数であり、この定義によって外側にも名前 `n` が導入されるわけではない。

本体の `+` は、ここでは自然数の足し算を表す記法である。次に、**`n : Nat` を前提として、本体 `n + n` の型が `Nat` になるか**を検査する。これが通るので、`fun n => n + n` は注釈どおり `Nat → Nat` 型の項になる。2節と同じく、この照合を経て `def` が受理される。

一致しなければ、もちろん受理されない。本体を `true` にしてみる:

```
def bad2 : Nat → Nat := fun n => true
```

```
error: Type mismatch
  true
has type
  Bool
but is expected to have type
  Nat
```

「本体 `true` の型 `Bool` が、注釈から要求される行き先の型 `Nat` と一致しない」という報告である。

一方、行き先を `Bool` にすれば受理される。これは自然数を受け取り、常に `true` を返す関数である。引数を受け取っても、本体でその名前を使う必要はない。その場合、引数の名前の代わりに `_` と書ける。

```
#check ((fun _ => true) : Nat → Bool)
```

```
fun x ↦ true : Nat → Bool
```

括弧で囲んだ `fun _ => true` 全体に、型 `Nat → Bool` を注釈している。表示では、名前を付けなかった引数に `x` という名前が補われている。

適用の書き方

数学ではふつう $f(x)$ と書く関数適用を、Lean では `f x` と、関数と引数を半角スペースで区切って書く。半角スペース自体は演算ではなく項を区切るもので、項を並べる構文が関数適用を表す。読むときも、関数を表す項のあとにスペースを挟んで引数の項が続いたら、**関数適用**と読む。`double 21` なら、`double` に `21` を渡している。項を書く場所で、関数を表す項のあとに引数の項をこのように並べると、関数適用として読まれる。例えば `double 21` である。数学で「 $f(x)$ 」と書くとき、私たちは f が関数で x がその定義域の元であることを前提している。Lean の `f x` も同じ約束の記法で、**`f` が関数型を持ち、`x` がその定義域の型を持つことを要求する。**

実際 `double 21` と書くと、Lean は2つのことを確かめる:

1. `double` の型は `Nat → Nat` (関数型である) —— 定義域は `Nat`
2. `21` の型は `Nat` —— 定義域と一致する

この2点が通るから `double 21` は**合法的書き方**であり、項 `double 21` の型は矢印の右側の `Nat` になる。この確認こそが、型検査という機能の中身である。一方、`double true` は `true : Bool` が定義域 `Nat` と合わないので、型エラーになる。

```
#check double 21
```

```
double 21 : Nat
```

型が分かったところで、値も予想してから計算してみよう。 `21 + 21` なので `42` のはずだ:

```
#eval double 21
```

```
42
```

一方、`double true` を調べると、引数の型が合わずエラーになる:

```
#check double true
```

```
error: Application type mismatch: The argument
  true
has type
  Bool
but is expected to have type
  Nat
in the application
  double true
```

⇒ 練習

5. 関数 $\text{inc} : \mathbb{N} \rightarrow \mathbb{N}$ を $\text{inc}(n) = n + 1$ で定める。これを `fun` を用いて Lean で定義せよ。 `#check inc` の型と `#eval inc 4` の値を予想してから確かめよ。

解答 5

```
def inc : Nat → Nat := fun n => n + 1
```

```
#check inc
```

```
inc : Nat → Nat
```

```
#eval inc 4
```

```
5
```

6. 関数 $f : \mathbb{N} \rightarrow \mathbb{N}$ を $f(n) = 2n + 3$ で定める。これを `fun` を用いて Lean で定義せよ（掛け算は `*` と書く）。 `#check f` の型と `#eval f 4` の値を予想してから確かめよ。

解答 6

```
def f : Nat → Nat := fun n => 2 * n + 3
```

```
#check f
```

```
f : Nat → Nat
```

```
#eval f 4
```

```
11
```

$2 * 4 + 3 = 11$ 。数学の $2n$ も、Lean では掛け算を明示して $2 * n$ と書く。

表示の読み方: binder 形式

宣言には、引数を左に書く記法もある。次は上の `double` と同じものである:

```
def double' (n : Nat) : Nat := n + n
```

```
#check double'
```

```
double' (n : Nat) : Nat
```

表示が `double' (n : Nat) : Nat` となったことに注意。これは「引数の列をコロンの左に並べ、最後に結果の型を書く」**binder 形式**の表示で、`#check` が関数を表示するときの標準の形である。

<code>double : Nat → Nat</code>	(矢印形式)
<code>double' (n : Nat) : Nat</code>	(binder 形式)

この2つは**同じ型の2通りの表示**である。宣言も `def double : Nat → Nat := fun n => ...` と `def double' (n : Nat) : Nat := ...` の2通りに書ける。binder 形式の宣言は、引数を `fun` で受け取る形の略記である。上の `double` と `double'` は、どちらも定義を展開すると `fun n => n + n` になり、一致するので同じ項として扱われる。以後どちらの表示も出てくるので、読み替えられるようにしておくこと。

🔗 練習

7. 先ほどの $\text{inc}(n) = n + 1$ を、今度は binder 形式で `inc'` という名前で定義せよ。 `#check inc'` の表示と `#eval inc' 4` の値を予想して確かめ、`inc` と比較せよ。

解答 7

```
def inc' (n : Nat) : Nat := n + 1
```

```
#check inc'
```

```
inc' (n : Nat) : Nat
```

```
#eval inc' 4
```

```
5
```

表示は binder 形式になったが、型は `inc` と同じ `Nat → Nat`。値も同じである。

8. 関数 `triple :  $\mathbb{N} \rightarrow \mathbb{N}$` 、`triple( $n$ ) =  $n + n + n$` を binder 形式で定義せよ。 `#check triple` の表示を予想してから確かめよ。

解答 8

```
def triple (n : Nat) : Nat := n + n + n
```

```
#check triple
```

```
triple (n : Nat) : Nat
```

binder 形式のまま表示される。 `triple : Nat → Nat` と読み替えられること。

9. 同様に `$f(n) = 2n + 3$` を、binder 形式で `f'` という名前で定義せよ。 `#check f'` の表示と `#eval f' 4` の値を予想して確かめ、`f` と比較せよ。

解答 9

```
def f' (n : Nat) : Nat := 2 * n + 3
```

```
#check f'
```

```
f' (n : Nat) : Nat
```

```
#eval f' 4
```

f' も $\text{Nat} \rightarrow \text{Nat}$ 型の関数であり、 f と同じ計算をしている。

多変数関数はカーリー化で表す

まず、ふつうの数学の話から始めよう。集合 $X \cdot Y \cdot Z$ に対して、写像全体の集合には次の自然な1対1対応がある:

$$\text{Map}(X \times Y, Z) \cong \text{Map}(X, \text{Map}(Y, Z))$$

左辺の写像 f に、 x を固定して得られる写像「 $y \mapsto f(x, y)$ 」を対応させる。つまり $x \mapsto (y \mapsto f(x, y))$ という右辺の写像にする。この操作を**カーリー化 (currying)**、逆に右辺の写像 g から $(x, y) \mapsto g(x)(y)$ を作る操作を**アンカーリー化 (uncurrying)** と呼ぶ。

Lean では、多変数関数も1引数関数を重ねて表す。2引数なら、1つ目を受け取って**残り1引数の関数を返す**。例えば、数学でいう写像 $\mathbb{N} \times \mathbb{N} \rightarrow \mathbb{N}$ 、 $(a, b) \mapsto a + b$ をカーリー化したものを、`plus : Nat → Nat → Nat` として定義してみよう:

```
def plus : Nat → Nat → Nat := fun a => fun b => a + b
```

読み方の約束を先に押さえる。 $\rightarrow$ は**右結合**である:

`Nat → Nat → Nat` は `Nat → (Nat → Nat)` のこと

(「`Nat` を受け取って、関数 `Nat → Nat` を返す」)。

逆に、適用は**左結合**:

`plus 3 4` は `(plus 3) 4` のこと

(まず `plus 3` で関数を作り、それに `4` を渡す)。この2つの約束がかみ合っているおかげで、カーリー化された関数を括弧なしで自然に読み書きできる。

括弧を左側に付けた `(Nat → Nat) → Nat` は**まったく別の型** (関数を受け取る関数) になることに注意。括弧は読み方が変わらない場合には省略でき、逆に確認のために補ってもよい。

`plus` 自体の型は注釈のとおり:

```
#check plus
```

```
plus : Nat → Nat → Nat
```

では、**plus 3 の型は何か**。表示を見る前に推測しよう。plus の型を右結合で読めば $\text{Nat} \rightarrow (\text{Nat} \rightarrow \text{Nat})$ —— 定義域は Nat 、行き先は $\text{Nat} \rightarrow \text{Nat}$ 。そこに $3 : \text{Nat}$ を渡すのだから、適用の2点確認より $\text{plus } 3 : \text{Nat} \rightarrow \text{Nat}$ のはずである:

```
#check plus 3
```

```
plus 3 : Nat → Nat
```

推測どおり。「1つ渡すと、残り1引数の関数が返る」ことが型にそのまま現れている。

もう1つ渡せば Nat になるはずで:

```
#check plus 3 4
```

```
plus 3 4 : Nat
```

```
#eval plus 3 4
```

```
7
```

🔗 練習

10. 関数 $\text{addThree} : \mathbb{N} \rightarrow \mathbb{N}$ 、 $\text{addThree}(n) = 3 + n$ を、 $\text{plus } 3 : \text{Nat} \rightarrow \text{Nat}$ を使って Lean で定義せよ。 `#check addThree` と `#eval addThree 4` の表示を予想してから確かめよ。

解答 10

```
def addThree : Nat → Nat := plus 3
```

```
#check addThree
```

```
addThree : Nat → Nat
```

```
#eval addThree 4
```

11. 関数 $g : \mathbb{N} \times \mathbb{N} \rightarrow \mathbb{N}$ 、 $g(a, b) = 2a + 3b$ を、カーリー化して Lean で定義せよ。型は `Nat → Nat → Nat` とし、`fun a => fun b => ...` を使うこと。 `#check g` ・ `#check g 2` の型と `#eval g 2 4` の値を予想してから確かめよ。

解答 11

```
def g : Nat → Nat → Nat := fun a => fun b => 2 * a + 3 * b
```

```
#check g
```

```
g : Nat → Nat → Nat
```

```
#check g 2
```

```
g 2 : Nat → Nat
```

1つ目の引数を渡したので、残りの引数を受け取る関数が返る。

```
#eval g 2 4
```

```
16
```

多変数関数の binder 形式

1変数のときと同じく、引数を宣言の左側に並べられる。例えば `plus` は次のようにも定義できる（同じ名前を重ねて定義せず、いずれか1つを使う）：

```
def plus (a : Nat) (b : Nat) : Nat := a + b
```

同じ型の引数は、まとめて `(a b : Nat)` と書いてもよい:

```
def plus (a b : Nat) : Nat := a + b
```

`fun` の引数もまとめられる。`fun a b => a + b` は、`fun a => fun b => a + b` の略記である。まず `a` を受け取り、「`b` を受け取って `a + b` を返す関数」を返している。したがって、もとの定義は次のようにも書ける:

```
def plus : Nat → Nat → Nat := fun a b => a + b
```

3引数以上も同じことの繰り返し。`Nat → Nat → Nat → Nat` は `Nat → (Nat → (Nat → Nat))` と読む。

```
def addMul (a b c : Nat) : Nat := a + b * c
```

```
#check addMul
```

```
addMul (a b c : Nat) : Nat
```

これは `Nat` 型の引数を3つ受け取って `Nat` 型の項を返す、という binder 形式の表示である。矢印形式では `addMul : Nat → Nat → Nat → Nat` と読める。

🔗 練習

12. 先ほどの $g(a, b) = 2a + 3b$ を、binder 形式で2通りに書け。`g'` は `(a : Nat) (b : Nat)` と分け、`g''` は `(a b : Nat)` とまとめること。`#check g'`・`#check g''` の表示と、`#eval g' 2 4`・`#eval g'' 2 4` の値を予想してから確かめよ。

解答 12

```
def g' (a : Nat) (b : Nat) : Nat := 2 * a + 3 * b
```

```
def g'' (a b : Nat) : Nat := 2 * a + 3 * b
```

```
#check g'
```

```
g' (a b : Nat) : Nat
```

```
#check g''
```

```
g'' (a b : Nat) : Nat
```

宣言で型を別々に書いても、`#check` の表示では同じ型の引数がまとめられる。いずれも `Nat → Nat → Nat` 型である。

```
#eval g' 2 4
```

```
16
```

```
#eval g'' 2 4
```

```
16
```

関数を引数として受け取る

数学でいう写像 $\text{Map}(\mathbb{N}, \mathbb{N}) \rightarrow \mathbb{N}$ 、 $F \mapsto F(21)$ を Lean で定義してみよう。引数 `F` の型は、数の型 `Nat` ではなく関数型 `Nat → Nat` になる:

```
def applyTo21 (F : Nat → Nat) : Nat := F 21
```

```
#check applyTo21
```

```
applyTo21 (F : Nat → Nat) : Nat
```

ここでは `F` の型が `Nat → Nat`、最後の `Nat` が結果の型であり、`applyTo21` 全体の型は `(Nat → Nat) → Nat` である。

`applyTo21 double` という項を読んでみよう。定義域は `Nat → Nat` で、`double : Nat → Nat` がちょうど一致する——関数そのものを引数として渡す適用であり、型は `Nat`。値は `double 21`、つまり `42` のはずである:

```
#eval applyTo21 double
```

```
42
```

型の異なる引数の binder 形式

引数の型が異なる場合は、型ごとに括弧を分ける。 $F(n)$ を返す関数なら、`F : Nat → Nat` と `n : Nat` を別々に注釈する:

```
def applyAt (F : Nat → Nat) (n : Nat) : Nat := F n
```

```
#check applyAt
```

```
applyAt (F : Nat → Nat) (n : Nat) : Nat
```

矢印形式では $(\text{Nat} \rightarrow \text{Nat}) \rightarrow \text{Nat} \rightarrow \text{Nat}$ である。 F を1つ渡すと、残りの引数 $n : \text{Nat}$ を受け取る関数が返る。 $(F\ n : \text{Nat})$ とまとめてしまうと両方が Nat 型という指定になるので、ここでは使えない。

練習

13. 写像 $\text{twice} : \text{Map}(\mathbb{N}, \mathbb{N}) \rightarrow \text{Map}(\mathbb{N}, \mathbb{N})$ を、 $F \mapsto (n \mapsto F(F(n)))$ で定める。つまり、 F を同じ入力に2回使うのではなく、1回目の結果にもう一度 F を適用する。これを Lean で `def twice (F : Nat → Nat) : Nat → Nat := ...` と定義せよ。 `#check twice` ・ `#check twice double` の表示と `#eval twice double 3` の値を予想してから確かめよ。

解答 13

```
def twice (F : Nat → Nat) : Nat → Nat := fun n => F (F n)
```

```
#check twice
```

```
twice (F : Nat → Nat) : Nat → Nat
```

矢印形式では $(\text{Nat} \rightarrow \text{Nat}) \rightarrow (\text{Nat} \rightarrow \text{Nat})$ 。数学でいう $\text{Map}(\mathbb{N}, \mathbb{N}) \rightarrow \text{Map}(\mathbb{N}, \mathbb{N})$ に対応している。

$F (F\ n)$ の括弧は、 $F\ n$ 全体を一つの引数として渡すために必要である。省略すると $(F\ F)\ n$ と読まれ、最初の適用で型が合わなくなる。

```
#check twice double
```

```
twice double : Nat → Nat
```

```
#eval twice double 3
```

```
12
```

14. `#eval applyTo21 (plus 100)` の値を予想してから実行せよ。

解答 14

```
#eval applyTo21 (plus 100)
```

```
121
```

applyTo21 F は F 21 だから、plus 100 21 = 121。

定義域や行き先が型でもよい

1節で見たように型も項なので、**型を受け取る関数・型を返す関数**も作れる。例えば `Nat → Type` は、自然数を受け取って型を返す関数の型である。ここでは2つの型 `A · B` を受け取り、関数型 `A → B` を返してみよう:

```
def Map : Type → Type → Type := fun A B => A → B
```

```
#check Map
```

```
Map : Type → Type → Type
```

数学の $\text{Map}(A, B)$ に対応する関数型を返す操作を、Lean の項 `Map` として定義した。 `Map Nat Nat` は定義を展開すると `Nat → Nat` なので、`double` はその項でもある（直後の練習で確かめよう）。型を扱う関数は、4節でも使う。

練習

15. `#check Map Nat Bool` の表示を予想してから確かめよ。また、数学でいう関数 $\text{double} : \mathbb{N} \rightarrow \mathbb{N}$ 、 $\text{double}(n) = n + n$ を、`Map Nat Nat` 型の項としても使えるか考えよ。 `def double2 : Map Nat Nat := double` が受理されるか予想してから試せ。受理されたら、`#check double2` の表示も予想してから確かめよ。

解答 15

```
#check Map Nat Bool
```

```
Map Nat Bool : Type
```

`Map Nat Bool` は計算すると `Nat → Bool` になるが、`#check` が表示するのはまず「型である」ということ (`: Type`)。

```
def double2 : Map Nat Nat := double
```

```
#check double2
```

```
double2 : Map Nat Nat
```

受理される。Map Nat Nat は定義を展開すると $\text{Nat} \rightarrow \text{Nat}$ になり、double の型と一致する。Lean は、このように定義を展開して一致するものを同じ型として扱う。表示には定義で指定した Map Nat Nat が残っていても、型の照合では一致すると判断される。double2 自体も、定義を展開すれば double なので、同じ関数として扱われる。

16. #check Map Nat の表示を予想してから確かめよ (Map に1つだけ渡すと、何が返るか。plus 3 の型を読んだのと同じ手順で考える)。

解答 16

```
#check Map Nat
```

```
Map Nat : Type → Type
```

2引数のうち1つだけ渡したので、「残り1つの型を受け取って型を返す」部分適用。plus 3 : $\text{Nat} \rightarrow \text{Nat}$ と同じ形である。

+ をあらためて

カーリー化を知ったいま、自然数の足し算を表す関数の型は $\text{Nat} \rightarrow \text{Nat} \rightarrow \text{Nat}$ と読める。3 + 4 も、2引数の関数適用を中置の記法で書いたものと考えればよい。ここまで使ってきた自然数の掛け算も同様に、 $\text{Nat} \rightarrow \text{Nat} \rightarrow \text{Nat}$ 型の関数である。

自然数の足し算の関数は Nat.add である。

```
#check Nat.add
```

```
Nat.add : Nat → Nat → Nat
```

```
#eval Nat.add 3 4
```

`3 + 4` は、自然数については `Nat.add 3 4` と同じ計算になる。型に応じて演算を選ぶ仕組みは、[05_MathematicalTools.lean](#) 4節で説明する。

型はどう決まるか——下から計算し、上から期待を伝える

ここまで、項を見るたびに型を確かめてきた。その手順を整理しよう。まず、すでに名前の付いている `double` や `plus` の型は、宣言から分かる。次に、それらを使って作った大きな項の型を、部分項の型から順に求める。

目で追うときは、まず結合規則に従って括弧を補い、どこまでが一つの項かを確認する。次に、いちばん外側の適用を関数側と引数側に分け、それぞれがさらに適用なら同じように分ける。こうして小さな項まで分けたら、その型を手がかりに、適用の型を順に求めて全体へ戻る。ただし、引数の型を省略した `fun` などでは、外側に書かれた型注釈も先に使う。この二つの向きを、以下の例で確かめよう。

適用の型を、内側から求める

適用 `f a` を読んだら、関数の側 `f` と引数の側 `a` に分ける。`f` の型が `A → B` で、`a` の型が `A` と一致すれば、結果の型は `B` になる。関数の定義の中身を実行するのではなく、まず入力と出力の型を読んでいる。

$$\frac{f : A \rightarrow B \quad a : A}{f \ a : B}$$

ここで `A` や `B` が関数型でも、規則は変わらない。`applyTo21 double` なら入力型 `A` は `Nat → Nat`、結果の型 `B` は `Nat`。一方、`plus 3` では入力型が `Nat`、結果の型が `Nat → Nat` である。

複合した項を、小さい照合に分ける

例えば `double (plus 3 4)` を見たら、いちばん外側は `double` の適用である。その引数 `plus 3 4` が `Nat` 型であることを確かめたい。そこで、引数の内側に入り、適用を一つずつ読む。

括弧を補うと `double ((plus 3) 4)` となり、その分解は次の木で表せる。

```
double ((plus 3) 4)
├─ double
└─ (plus 3) 4
    ├─ plus 3
    │   └─ plus
```

```
|   └─ 3
└─ 4
```

枝分かれしている箇所は一つの関数適用で、上の枝が関数、下の枝が引数を表す。木の末端にある名前や数の型から始めて、それらを組み合わせた項の型を順に求める。

1. `3 : Nat`、`4 : Nat`。
2. `plus : Nat → (Nat → Nat)` なので、`plus 3 : Nat → Nat`。
3. その関数に `4 : Nat` を渡すので、`(plus 3) 4 : Nat`。
4. `double : Nat → Nat` にこの `Nat` 型の項を渡すので、全体も `Nat` 型。

このように、**部分項の型を調べ、入力型が合うかを照合し、結果の型を得る**。同じ手順を繰り返すだけで、大きな項の型も確かめられる。

```
#check double (plus 3 4)
```

```
double (plus 3 4) : Nat
```

これは型の確認である。値が14になることを計算する操作は `#eval` が担当する。型を求めるために `3 + 4` やその2倍を計算したわけではない。

期待される型から、関数の本体へ

逆向きの情報も使える。次の宣言では、右辺全体に `Nat → Nat` という型を期待する。

```
def double : Nat → Nat := fun n => n + n
```

期待される型の入力側が `Nat` なので、受け取る `n` の型は `Nat` と決まる。出力側も `Nat` なので、次には本体 `n + n` が `Nat` 型になるかを確認できればよい。**関数全体に求める型から、引数の型と本体に求める型を取り出している。**

ここからは、`n : Nat` を使って本体の型を調べる。自然数の足し算は `Nat` を二つ受け取って `Nat` を返すので、`n + n : Nat` が得られる。要求されていた出力型と一致したから、`fun n => n + n` 全体が `Nat → Nat` 型と確かめられる。

本体を `true` に変えると、ここで得られる型は `Bool` になる。要求された `Nat` と一致しないため、先ほど見た型エラーになる。「どこでどの型が要求され、実際の型が何だったか」を追うと、エラーの意味も読める。

型注釈が、項の読み方を決める

数字も、期待される型を手がかりに読む。型の指定がなければ `2` は自然数として読まれるが、`Int`（整数の型）を指定すれば、整数の項として読まれる。

```
#check 2
```

```
2 : Nat
```

```
#check (2 : Int)
```

```
2 : Int
```

期待される型は、項の内側にも伝わる。次の式では、足し算の結果を `Int` 型と指定しているので、`2` と `3` も整数の項として読まれる。

```
#check (2 + 3 : Int)
```

```
2 + 3 : Int
```

部分項から型を求めることと、期待される型を部分項へ伝えることが組み合わさって、省略された引数の型や演算の読み方が決まる。このように省略などを補う処理を、後でエラボレーションと呼ぶ。補った後も、できた項が宣言された型を持つかをチェックする。

このあとの依存関数や暗黙引数でも、同じ考え方を使う。型を予想するときは、今どの項を見ているか、その項にどの型を期待しているかを分けて追おう。

照合が失敗すれば、その場所がエラーになる。`double` に関数 `plus` を渡してみる:

```
#check double plus
```

```
error: Application type mismatch: The argument
  plus
has type
  Nat → Nat → Nat
but is expected to have type
  Nat
in the application
  double plus
```

「引数の型 `Nat → Nat → Nat` が、要求される型 `Nat` と合わない」と、規則が破れた部分項を名指しで教えてくれる。エラーの下にはさらに `double sorry : Nat` という表示も出る。ここで初めて出てきた `sorry` は、「この部分は未完成」という印の項で、どんな型の項の代わりにも置ける。長い証明の途中で、まだ書けていない部分に置いて残りを先に書けるが、その部分を埋めるまでは証明は完成していない。`#check` は、エラーになった引数を `sorry` で置き換えて、残りの部分の型の計算を続けてくれたのである。自分で書くこともできる。例えば

```
def later : Nat := sorry
```

は受理されるが、`warning: declaration uses `sorry`` という警告が出る。未完成のまま進めても、そのことを Lean は見逃さない。

では `double plus 3 4` と括弧なしで書いたらどうなるか。適用は左結合なのでこれは `((double plus) 3) 4` と読まれ、最初の `double plus` の時点で上と同じエラーになる。`double (plus 3 4)` の括弧は省略できない。

4. 依存関数型 — 入力によって結果の型が変わる

まず集合族で考えよう。集合 A を添字集合とする集合族 $(X_a)_{a \in A}$ に対して、その直積 $\prod_{a \in A} X_a$ の要素は、各 a に X_a の要素を一つずつ指定するものである。これを**依存関数**と思えばよい。値 $f(a)$ の所属先 X_a が、入力 a に依存している。

族が定数、つまりすべての X_a が同じ集合 X ならば、これは普通関数 $A \rightarrow X$ であり、 $\prod_{a \in A} X = \text{Map}(A, X)$ となる。

直和への関数と切断

族が定数でない場合も、直和 $E = \coprod_{a \in A} X_a$ を使えば、普通関数として表せる。この直和の要素は、添字と要素の組 (a, x) ($x \in X_a$) である。

依存関数 f を $s(a) = (a, f(a))$ という関数 $s : A \rightarrow E$ に読み替える。射影 $\pi : E \rightarrow A$ 、 $\pi(a, x) = a$ に対して、これは $\pi \circ s = \text{id}_A$ を満たす。このような s を π の**切断 (section)**と呼ぶ。

逆に、全射 $F : E \rightarrow A$ の切断 s を考えてもよい。 $F \circ s = \text{id}_A$ という条件は、各 a について $s(a) \in F^{-1}(a)$ という条件である。したがって切断とは、ファイバーを集合族 $(F^{-1}(a))_{a \in A}$ と見たときの依存関数である。

型の族と依存関数

Lean では、`B : A → Type` が各 `a : A` に型 `B a` を割り当てる**型の族**を表す。この族の各型から項を一つずつ返す関数の型を**依存関数型**といい、次のように書く：

```
(a : A) → B a
```

集合族の直積 $\prod_{a \in A} B(a)$ に対応する型である。族 B の行き先はいつも `Type` だが、依存関数の結果の型 `B a` は入力 `a` に応じて変わる。結果の型が入力に依存しない特別な場合が、これまでの `A → C` である。

自然数を添字にする型の族 `Fin`

まず、自然数 n に対して型 `Fin n` を対応させる例から始めよう。`Fin n` は、集合 $\{0, 1, \dots, n-1\}$ に対応する型と思ってよい。例えば `Fin 3` には $0 \cdot 1 \cdot 2$ 、`Fin 5` には $0 \cdot 1 \cdot 2 \cdot 3 \cdot 4$ に対応する項がある。

```
#check Fin
```

```
Fin (n : Nat) : Type
```

矢印形式で読むと `Fin : Nat → Type` である。`Fin` 自体は、自然数を受け取って型を返す普通の関数で、**型の族を与える**。次は、その族の各型から、実際に項を選んで返す関数を考えたい。

なぜ `Fin (n + 1)` を使うのか

`Fin 0` には項がない。そのため、すべての自然数 n に対して `Fin n` の項を選ぶ関数は作れない。ここでは添字を一つずらし、いつも項を持つ `Fin (n + 1)` の族を使う。`Fin (n + 1)` は、0から n までの番号の型である。

したがって、作りたい関数の型は次のようになる。

```
(n : Nat) → Fin (n + 1)
```

n を受け取ってから、返すべき項の型が `Fin (n + 1)` に決まる。例えば2を受け取れば `Fin 3`、4を受け取れば `Fin 5` の項を返す。これが、結果の型が入力に依存するということである。

例1: いつも最初の番号を選ぶ

どの `Fin (n + 1)` にも、最初の番号0がある。それを選ぶ依存関数を作ってみよう。

```
def zeroIndex : (n : Nat) → Fin (n + 1) := fun _ => 0
```

```
#check zeroIndex
```

```
zeroIndex (n : Nat) : Fin (n + 1)
```

外側の型注釈から、右辺で受け取る引数の型は `Nat`、返す0の型は `Fin (n + 1)` と決まる。この0は、ここでは `Nat` 型ではなく、指定された `Fin (n + 1)` 型の項として読まれる。数字をその型の項として書ける仕組みは、後で型の定義とともに説明する。

ここでは引数に名前を付けていないが、返す項に求められる型は、その引数の値に応じて変わる。

```
#check zeroIndex 2
```

```
zeroIndex 2 : Fin (2 + 1)
```

```
#check zeroIndex 4
```

```
zeroIndex 4 : Fin (4 + 1)
```

`Fin (2 + 1)` は計算すれば `Fin 3`、`Fin (4 + 1)` は `Fin 5` である。表示に足し算が残っていても、どの入力を結果の型に代入したかが読み取れる。

```
#eval zeroIndex 4
```

```
0
```

表示される値はいつも0でも、返される項の型は入力によって異なる。依存しているのは、まず**値の所属先となる型**である。

例2: 最後の番号を選ぶ

今度は0から `n` までのうち、最後の番号 `n` を選ぼう。標準ライブラリの `Fin.last` は、その番号を `Fin (n + 1)` 型の項として返す関数である。

```
#check Fin.last
```

```
Fin.last (n : Nat) : Fin (n + 1)
```

この関数を使えば、次のように書ける。`Fin.last` がその型の項をどう作るかは、部分型を扱うところで説明する。今は型から入力と出力を読んで使ってみよう。

```
def lastIndex : (n : Nat) → Fin (n + 1) := fun n => Fin.last n
```

```
#check lastIndex 2
```

```
lastIndex 2 : Fin (2 + 1)
```

```
#eval lastIndex 2
```

```
2
```

```
#eval lastIndex 4
```

```
4
```

`zeroIndex` と `lastIndex` は同じ依存関数型を持つ。しかし、前者は最初の番号、後者は最後の番号を選ぶ。型は各入力での所属先を決めるが、その中のどの項を返すかまで決めるわけではない。

例3: いつも1を選べる族にする

いつも番号1を選びたいなら、少なくとも二つの番号を持つ `Fin (n + 2)` の族を使えばよい。 `n = 0` のときでも、結果の型は `Fin 2` になり、番号1を返せる。

```
def oneIndex : (n : Nat) → Fin (n + 2) := fun _ => 1
```

```
#check oneIndex 2
```

```
oneIndex 2 : Fin (2 + 2)
```

```
#eval oneIndex 2
```

```
1
```

ここまでの三つは、いずれも自然数を受け取って、指定された有限の型の項を一つ返す例である。 `Fin` は型の族を与える関数、 `zeroIndex` ・ `lastIndex` ・ `oneIndex` は、その族から項を返す依存関数である。

各集合に恒等写像を割り当てる

次は、受け取る対象を自然数から型に変えてみよう。数学では、各集合 X に恒等写像 $\text{id}_X \in \text{Map}(X, X)$ を対応させる。入力が X なら返す値の所属先は $\text{Map}(X, X)$ なので、これも依存関数と見られる。

Lean では型 α を受け取り、 $\alpha \rightarrow \alpha$ 型の恒等関数を返す。その関数は $a : \alpha$ を受け取って、そのまま a を返せばよい。

```
def idAt ( $\alpha$  : Type) (a :  $\alpha$ ) :  $\alpha$  := a
```

```
#check idAt
```

```
idAt ( $\alpha$  : Type) (a :  $\alpha$ ) :  $\alpha$ 
```

矢印形式では $\text{idAt} : (\alpha : \text{Type}) \rightarrow \alpha \rightarrow \alpha$ と読む。最初に型を一つ渡すと、残りの関数の型が決まる。

```
#check idAt Nat
```

```
idAt Nat : Nat  $\rightarrow$  Nat
```

```
#check idAt Bool
```

```
idAt Bool : Bool  $\rightarrow$  Bool
```

idAt Nat と idAt Bool は、異なる型の上の恒等関数である。 `zeroIndex` では入力に応じて有限の型が変わったが、ここでは返す関数の型が変わる。

```
#eval idAt Nat 42
```

42

$\text{idAt Nat} : \text{Nat} \rightarrow \text{Nat}$ に $42 : \text{Nat}$ を渡している。最初の引数で残りの型が決まり、その型に合う次の引数を渡す、という順序で読める。

依存する適用の規則

$f : (a : A) \rightarrow B\ a$ に $x : A$ を渡すと、結果の型は $B\ x$ になる。入力型が合うことを確かめ、**結果の型**の中にも、渡した対象を代入する。

$$\frac{f : (a : A) \rightarrow B\ a \quad x : A}{f\ x : B\ x}$$

`zeroIndex 2` なら、 $B\ n = \text{Fin}\ (n + 1)$ に2を代入して $\text{Fin}\ (2 + 1)$ 。 idAt Nat なら、 $B\ \alpha = \alpha \rightarrow \alpha$ に Nat を代入して $\text{Nat} \rightarrow \text{Nat}$ となる。入力によらず結果の型が同じ場合が、これまでの普通の関数型 A

→ C である。

⇒ 練習

17. `#check idAt (Nat → Nat)` の表示と、`#eval idAt (Nat → Nat) double 21` の値を予想せよ。

解答 17

```
#check idAt (Nat → Nat)
```

```
idAt (Nat → Nat) : (Nat → Nat) → Nat → Nat
```

$A = \text{Nat} \rightarrow \text{Nat}$ を代入した $A \rightarrow A$ 、つまり $(\text{Nat} \rightarrow \text{Nat}) \rightarrow \text{Nat} \rightarrow \text{Nat}$ 。右結合の約束で外側の括弧は表示されない。

```
#eval idAt (Nat → Nat) double 21
```

```
42
```

`idAt (Nat → Nat) double` は `double` そのもの。

18. 集合 A の要素 a に対して、集合 B 上の定数写像を対応させる `constAt (A B : Type) (a : A) : B → A` を書け。 `#check constAt Nat Bool 5` と `#eval constAt Nat Bool 5 true` を確かめよ。

解答 18

```
def constAt (A B : Type) (a : A) : B → A := fun _ => a
```

```
#check constAt Nat Bool 5
```

```
constAt Nat Bool 5 : Bool → Nat
```

```
#eval constAt Nat Bool 5 true
```

```
5
```

`idAt A` が恒等関数を返すのに対し、`constAt A B a` は定数関数を返す。

型引数を補ってもらう — 明示引数と暗黙引数

`idAt Nat 3` では、型 `Nat` と、その型の項 `3` を順に渡した。しかし、`3` が `Nat` 型だと分かれば、最初の型引数も `Nat` と決められる。このように、ほかの型情報から決まる引数を、Lean に補ってもらうことができる。

型引数は、なくなったのではない

同じ恒等関数を、型引数の括弧だけ変えて定義する。

```
def idImplicit {α : Type} (a : α) : α := a
```

```
#check idImplicit
```

```
idImplicit {α : Type} (a : α) : α
```

`(a : α)` は呼び出す側が通常書く **明示引数**、`{α : Type}` は通常省略して補ってもらう **暗黙引数**である。型と項の二つを受け取る仕組みは変わっていない。呼び出すときに、どちらの引数を書くかが違う。

渡した項の型から補う

`idImplicit 3` と書くと、`3`は明示引数 `a` に渡される。`a` の型は `α` でなければならず、渡した`3`の型は `Nat` なので、Lean は `α := Nat` と補う。その結果、返す項の型も `Nat` になる。

```
#check idImplicit 3
```

```
idImplicit 3 : Nat
```

同様に `true : Bool` を渡せば、`α := Bool` と決まる。「省略したらいつも `Nat`」ということではなく、渡した項の型を手がかりにしている。

```
#check idImplicit true
```

```
idImplicit true : Bool
```

補う型を、こちらから指定する

暗黙引数も、名前を指定して渡せる。`(α := Nat)` は「引数 `α` に `Nat` を渡す」という書き方である。次の二つは、型引数を省略したか明示したかの違いだけである。

```
idImplicit 3
idImplicit (α := Nat) 3
```

また、関数名の前に `@` を付けると、暗黙引数も順に書いて適用できる。この例では `@idImplicit Nat 3` と書く。`idAt Nat 3` と同じ二つの引数が見える。

結果に期待される型から補う

渡す値だけが手がかかりではない。次の定義では、右辺に `Nat → Nat` を期待している。

```
def natId : Nat → Nat := idImplicit

#check natId
```

```
natId : Nat → Nat
```

型引数を補った後の `idImplicit` の型は `α → α` である。これを要求された `Nat → Nat` と照合すると、`α := Nat` と決まる。値の引数をまだ渡していなくても、置かれた場所の型から補える例である。

手がかりが足りない場合

一方、型注釈なしに `def unknownIdentity := idImplicit` とだけ書くと、どの型 `α` の恒等関数を定義したいのかが決まらず、受理されない。結果の型を注釈するか、`idImplicit (α := Nat)` のように引数を指定すればよい。暗黙引数とは、型の情報から補えるところを省略する仕組みである。

省略を補う処理を**エラボレーション**と呼ぶ。補われた引数も含めて完成した項が、要求された型を持つかが検査される。

項を書く場所の `_` は、Lean にその部分の補完を求める記号である。一方、`fun _ => 0` の `_` は、受け取る引数に名前を付けない記号だった。書かれた場所によって役割が違うことに注意しよう。

次の章では、結果の型として具体的な命題を使う。対象ごとに、その対象についての証明を返す依存関数が、全称命題の証明になる。

5. 章末問題

型や値を予想し、本文で学んだ読み方を確かめよう。

⇒ 練習

19. `#eval double (double 5)` の値を予想してから実行せよ。

解答 19

```
#eval double (double 5)
```

```
20
```

内側から: $\text{double } 5 = 10$ 、 $\text{double } 10 = 20$ 。

20. 写像 $\text{thrice} : \text{Map}(\mathbb{N}, \mathbb{N}) \rightarrow \text{Map}(\mathbb{N}, \mathbb{N})$ を、 $F \mapsto (n \mapsto F(F(F(n))))$ で定める。これを Lean で定義し、`#check thrice · #check thrice double` の表示と `#eval thrice double 3` の値を予想してから確かめよ。

解答 20

```
def thrice (F : Nat → Nat) : Nat → Nat := fun n => F (F (F n))
```

```
#check thrice
```

```
thrice (F : Nat → Nat) : Nat → Nat
```

```
#check thrice double
```

```
thrice double : Nat → Nat
```

```
#eval thrice double 3
```

```
24
```

`double` を3回適用するので、 $3 \rightarrow 6 \rightarrow 12 \rightarrow 24$ 。

21. `#check` する前に型を計算せよ: $(3 + 4) * 2$ 、`plus (double 3)`、`applyTo21 (plus 3)`、`fun n : Nat => plus n n`。それから確かめよ。

解答 21

型の計算: $(3 + 4) * 2$ は `Nat`。 `plus (double 3)` は `plus : Nat → Nat → Nat` に第1引数だけ渡したので `Nat → Nat` (部分適用)。 `applyTo21 (plus 3)` は `applyTo21 : (Nat → Nat) → Nat` に関

数を渡し切ったので `Nat`。 `fun n : Nat => plus n n` は `Nat` を受け取って `plus n n : Nat` を返すので `Nat → Nat`。

```
#check (3 + 4) * 2
```

```
(3 + 4) * 2 : Nat
```

```
#check plus (double 3)
```

```
plus (double 3) : Nat → Nat
```

```
#check applyTo21 (plus 3)
```

```
applyTo21 (plus 3) : Nat
```

```
#check fun n : Nat => plus n n
```

```
fun n ↦ plus n n : Nat → Nat
```

22. 次の2つの写像を Lean で定義し、`#check` の表示を予想してから確かめよ:

- `evalAt : Map(N, N) × N → N`、`evalAt(F, n) = F(n)`。カーリー化して定義すること。
`#eval evalAt double 5` の値も予想してから実行せよ。
- `shift : Map(N, N) → Map(N, N)`、`F ↦ (n ↦ F(n + 1))`。関数を受け取って関数を返す関数である。`#eval shift double 3` の値も予想してから実行せよ。

解答 22

```
def evalAt (F : Nat → Nat) (n : Nat) : Nat := F n
```

```
#check evalAt
```

```
evalAt (F : Nat → Nat) (n : Nat) : Nat
```

```
#eval evalAt double 5
```

10

```
def shift (F : Nat → Nat) : Nat → Nat := fun n => F (n + 1)
```

```
#check shift
```

```
shift (F : Nat → Nat) : Nat → Nat
```

```
#eval shift double 3
```

8

shift double は「 n に対して $\text{double } (n + 1)$ を返す関数」。 $n = 3$ なら $\text{double } 4 = 8$ 。

23. 「型から項を書く」練習: $\text{Map}(\mathbb{N}, \mathbb{N}) \rightarrow \mathbb{N}$ という写像を、**中身の違うもので2つ**考えよ（例えば $F \mapsto F(21)$ と $F \mapsto 0$ ）。それらを Lean で `useF1` ・ `useF2` と名付けて定義し、型を予想してから `#check` で確かめよ。どちらも $(\text{Nat} \rightarrow \text{Nat}) \rightarrow \text{Nat}$ 型になるはずである。同じ型に項は何通りもある——型は仕様であって、中身までは決めない。

解答 23

```
def useF1 : (Nat → Nat) → Nat := fun F => F 21
```

```
def useF2 : (Nat → Nat) → Nat := fun _ => 0
```

```
#check useF1
```

```
useF1 : (Nat → Nat) → Nat
```

```
#check useF2
```

```
useF2 : (Nat → Nat) → Nat
```

どちらも同じ型を持つ。型は「関数を受け取って `Nat` を返す」という仕様だけを決めており、返し方（中身）は何通りもある。

24. (発展) 写像 $G : B \rightarrow C$ と $F : A \rightarrow B$ に合成 $G \circ F : A \rightarrow C$ を対応させる写像を考える。型 $A \cdot B \cdot C$ も引数に取り、これを Lean で `compose (A B C : Type) (G : B → C) (F : A → B) : A → C` と定義せよ。 `#check compose` の表示と、 `#eval compose Nat Nat Nat double (fun n => n + 1) 3` の値を予想してから確かめよ。

解答 24

```
def compose (A B C : Type) (G : B → C) (F : A → B) : A → C := fun a => G (F a)

#check compose
```

```
compose (A B C : Type) (G : B → C) (F : A → B) : A → C
```

```
#eval compose Nat Nat Nat double (fun n => n + 1) 3
```

```
8
```

内側から: $F = \text{fun } n \Rightarrow n + 1$ で 3 が 4 に、 $G = \text{double}$ で 8 になる。

25. (発展) `evalAt` と `shift` を一般化し、任意の $A \cdot B$ に対する写像 $\text{Map}(A, B) \times A \rightarrow B$ 、 $(F, a) \mapsto F(a)$ と、写像 $g : A \rightarrow A$ に対する変換 $\text{Map}(A, A) \rightarrow \text{Map}(A, A)$ 、 $F \mapsto (a \mapsto F(g(a)))$ を考える。型や g も引数として受け取るように、Lean で `evalAt' (A B : Type) (F : A → B) (a : A) : B` と `shift' (A : Type) (g : A → A) (F : A → A) : A → A` を定義せよ。両者の `#check` の表示と、 `#eval evalAt' Nat Nat double 5` と `#eval shift' Nat (fun x => x + 1) double 3` の値を予想してから確かめ、元の版と比較せよ。

解答 25

```
def evalAt' (A B : Type) (F : A → B) (a : A) : B := F a

def shift' (A : Type) (g : A → A) (F : A → A) : A → A := fun a => F (g a)

#check evalAt'
```

```
evalAt' (A B : Type) (F : A → B) (a : A) : B
```

```
#check shift'
```

```
shift' (A : Type) (g F : A → A) : A → A
```

```
#eval evalAt' Nat Nat double 5
```

```
10
```

```
#eval shift' Nat (fun x => x + 1) double 3
```

```
8
```

どちらも元の版と同じ値。型を引数にすると、同じ中身が任意の型で使い回せる。

26. 自然数の n 個組を、番号から成分を取り出す関数として表そう。番号の型を `Fin n` とし、型の族 `Tuple` と各成分が7である組を返す `constTuple` を次で定義する。

```
def Tuple : Nat → Type := fun n => Fin n → Nat
def constTuple : (n : Nat) → Tuple n :=
  fun n => fun (_ : Fin n) => 7
```

`Tuple 2` を定義で展開すると、どんな型になるか。 `constTuple 2` の内側の引数と結果の型を答えよ。
`Tuple` と `constTuple` のどちらが型の族で、どちらがその族の項を返す依存関数かも説明せよ。

解答 26

```
def Tuple : Nat → Type := fun n => Fin n → Nat

def constTuple : (n : Nat) → Tuple n :=
  fun n => fun (_ : Fin n) => 7

#check Tuple
#check constTuple
#check constTuple 3
#check constTuple 4
```

```
Tuple : Nat → Type
constTuple (n : Nat) : Tuple n
constTuple 3 : Tuple 3
constTuple 4 : Tuple 4
```

外側の `fun` は回数 n を受け取る。作るべき型 `Tuple n` は `Fin n → Nat` なので、内側の `fun` は `Fin n` の番号を受け取り、自然数7を返す。番号そのものを計算に使わなくても、受け取る番号の型は n によって変わる。

`Tuple 2` は `Fin 2 → Nat`。内側では番号 $i : \text{Fin } 2$ を受け取り、自然数7を返す。族 `Tuple` の行き先は `Type` だが、その型の項 `constTuple 2` の行き先は `Nat` である。

27. n と自然数 a を受け取り、すべての成分が a である n 個組を返す `repeatTuple (n : Nat) (a : Nat) : Tuple n` を書け。

解答 27

```
def repeatTuple (n : Nat) (a : Nat) : Tuple n := fun _ => a
```

```
#check repeatTuple 2 9
```

```
repeatTuple 2 9 : Tuple 2
```

番号を一つ受け取り、どの番号にも a を返す。 n は返す関数の定義域 `Fin n` を決めている。結果の型 `Tuple n` を展開すると `Fin n → Nat` なので、`_` の引数の型はそこから決まり、`fun (_ : Fin n) => a` と型を注釈しても同じである。

`Fin n` 型の引数に数字を書くときの読み方は、[05_MathematicalTools.lean 3節](#)の補足で説明する。

補足: ライブラリにもある型の族

標準環境の `Vector Nat n` は、長さが n と決まった自然数の列の型である。 `Vector Nat : Nat → Type` は、`Tuple` と同じ形の族である。

```
#check Vector Nat 3
```

```
Vector Nat 3 : Type
```

同じ値を指定回数並べる関数が `Vector.replicate` である。自然数の列を作る場合、その型は次の形として読める:

```
(n : Nat) → Nat → Vector Nat n
```

一般には要素の型 α を暗黙引数に持ち、並べる値0の型 `Nat` から α が決まる。

```
def zeros : (n : Nat) → Vector Nat n := fun n => Vector.replicate n 0
```

```
#check zeros 3
```

```
zeros 3 : Vector Nat 3
```

```
#check zeros 4
```

```
zeros 4 : Vector Nat 4
```

ここでは型の変化を確かめればよい。列の内部の構造や表示方法は必要ない。

型と命題 I — ならばと全称量化

この章では、**その命題の証明を与えられたか、証明を書けたか**という視点で数学を読む。命題の真偽を先に判定することや、同じ命題の証明どうしの文面を比べることから、いったん視点を移そう。注目するのは、その主張を支える保証を、実際に与えられたかどうかである。

ここでいう保証は、その命題の**証明書**だと思えばよい。証明がまだ書けていないということと、その命題が偽であるということは区別する。

保証を受け取って、保証を作る

ある前提のもとで結論が成り立つ、という定理を考えよう。これは「前提が正しいという保証を与えてくれたら、結論が正しいという保証を作ります」と言っている、と読める。

この説明のために前提を P 、結論を Q と呼ぶ。「 P ならば Q 」を保証するには、 P の証明書を受け取り、 Q の証明書を返す関数を書けばよい。ここで受け取るものと返すものは、真・偽という値ではなく、それぞれの命題の証明書である。

命題をその証明書の型と思えば、この関数の型は $P \rightarrow Q$ になる。仮定のもとで結論を示す、という証明の形を、入力から出力を作る関数の形として捉えている。

対象ごとに、その対象についての保証を与える

「どんな x に対しても $P(x)$ が成り立つ」を保証する場合はどうだろうか。必要なのは、どの x を渡されても、その x についての命題 $P(x)$ の証明書を返すことである。一つの関数で、すべての対象に対する保証を一斉に与える。

返す証明書の型は $P(x)$ であり、受け取った x に応じて変わる。だから、その保証を与えるものは**依存関数**だと考えられる。前の章で、 n に対して `Fin (n + 1)` の項を返したのと同じ形で、今度は対象ごとの証明を返す。

各 $x \in A$ に命題 $P(x)$ を割り当てるものを**述語**という。命題を型と思えば、述語は A を添字とする**型の族** $(P(x))_{x \in A}$ である。全称命題の証明は、この族の各型 $P(x)$ の項を返す依存関数、すなわち**依存積** $\prod_{x \in A} P(x)$ の項なのである。

人間が納得することと、機械が検査すること

人間が理解したいことは、この読み替えが、数学でしている証明の操作を表しているということである。「ならば」を証明する代わりに保証から保証を作る関数を、全称命題を証明する代わりに対象から保証を作る依存関数を書く。その関数を作れたことで、元の数学の主張を証明したと納得できるかを考えたい。

機械が確かめることは、渡された項が、指定された型を持つかどうかである。この読み替えのもとで書いた証明の項を型検査し、要求された結論の証明になっているかを確かめる。数学の主張をどう表したかを読む

ことと、できた項の型を検査することの、両方が必要になる。

単射の合成、左キャンセル、単調写像の合成、左逆の四つを通して、この対応を見ていこう。

1. 単射の合成 — 対象と仮定を受け取る

写像 $f : A \rightarrow B$ と $g : B \rightarrow C$ がともに単射なら、合成 $g \circ f : A \rightarrow C$ も単射である。まず、ふつうの数学の証明を書いてみよう。

数学の証明を読む

証明。 $x, y \in A$ を任意に取り、 $g(f(x)) = g(f(y))$ と仮定する。 g は単射だから $f(x) = f(y)$ である。さらに f は単射だから $x = y$ となる。したがって、 $g \circ f$ は単射である。■

学生のレポートを採点するつもりで、この証明を読んでみよう。各行で述べたことは、どの仮定から言えるだろうか。最後の結論に必要な対象と仮定が、最初にきちんと取られているだろうか。証明の書き方を「対象を取り、仮定を明示し、それを使って結論を得る」という一定の形にそろえると、仮定の取り忘れや推論の抜けを確認しやすくなる。

対象と仮定を受け取り、保証に名前を付ける

まず、 f の単射性の保証を `hf`、 g の単射性の保証を `hg` と名付ける。次に二つの対象 $x, y \in A$ を任意に取り、等式 $g(f(x)) = g(f(y))$ の保証を `h` と名付ける。Lean の書き方では、次のようになる。

```
h : g (f x) = g (f y)
```

これは等式を表す文字列ではなく、その等式を保証する証明書に `h` という名前を付けている。一般に `h : x = y` と書けば、`x = y` であるという保証を `h` と名付けたことになる。対象に名前を付ける `x : A` と同じ配置で、型の場所に命題を書いている。

この名前を使って証明を読み直そう。`hg` に対象 $f(x), f(y)$ と保証 `h` を渡すと、 $f(x) = f(y)$ の保証が得られる。これを、対象 x, y とともに `hf` に渡すと、 $x = y$ の保証が得られる。「これらを与えられたら、これを作る」という操作に分けると、仮定を使う各行が関数適用として、証明全体が対象と保証を受け取る関数として見えてくる。

命題を型、証明をその型の項として読む

Lean では命題そのものを型とし、その型の項を証明として扱う。自然数の型とその項の関係と、命題とその証明の関係を並べてみよう。 `1 + 1 = 2` の証明を `h` と名付けたとする。

```
Nat : Type
2 : Nat
```

```
(1 + 1 = 2) : Prop
h : 1 + 1 = 2
```

`Prop` は命題たちの型であり、証明 `h` の型は、その証明が保証する命題 `1 + 1 = 2` である。

証明に名前を付けるときも `def 名前 : 型 := 項` を使う。指定する型が証明したい命題で、右辺の項がその証明書になる。

⇒ 練習

28. `#check 3 = 5` と `#check 5 = 5` の表示を予想せよ。 `#check` の結果から、命題が真かどうかを判断できるか。

解答 28

```
#check 3 = 5
```

```
3 = 5 : Prop
```

```
#check 5 = 5
```

```
5 = 5 : Prop
```

どちらも命題なので型は `Prop`。 `#check` は、命題を表す項の型を調べている。真偽や証明の有無を調べているわけではない。

単射性を、全称量化と「ならば」で書く

「どの `x`、`y` についても」は `∀ x y` と書く。 `∀` は `\forall` で入力できる。この `∀` は依存関数型の記法であり、`∀ x : A, B x` は `(x : A) → B x` と同じ型を表す。また、二つの命題の間の `→` を「ならば」と読む。`f` の単射性を表す命題は、次のようになる。

```
∀ x y : A, f x = f y → x = y
```

関数適用は等号より強く結び付き、`f x = f y` は `(f x) = (f y)` と読む。

その証明を `hf` と名付ける。`g` の単射性の証明は `hg` とし、それぞれの型を、数学で使った条件のまま書く。

```
hf : ∀ x y : A, f x = f y → x = y
hg : ∀ u v : B, g u = g v → u = v
```

示したい結論は、合成の単射性である。

```
∀ x y : A, g (f x) = g (f y) → x = y
```

全称量化も「ならば」も関数として扱っている

hf の型は、依存関数型の記法でも書ける。

```
hf : (x : A) → (y : A) → (f x = f y → x = y)
```

最初の二つの引数は対象である。その値 `x`、`y` によって、残りの型に現れる等式も変わる。これが、全称量化の証明が依存関数であるという意味である。

対象を二つ渡した後の `hf x y` は、`f x = f y` の証明を受け取り、`x = y` の証明を返す。ここでは入力も出力も証明だが、関数を適用する規則は前の章と変わらない。

このとき、命題である関数型と、その型の項である関数を区別しよう。

```
(f x = f y → x = y) : Prop
hf x y : f x = f y → x = y
```

上の行は「ならば」で作った命題であり、下の行の `hf x y` が、その命題の証明となる関数である。

hf は、引数を一度に受け取るのではなく、順に受け取る関数として読める。

```
hf : (x : A) → (y : A) → (f x = f y → x = y)
hf x : (y : A) → (f x = f y → x = y)
hf x y : f x = f y → x = y
```

前章のカリー化と同じく、引数を一つ渡すたびに残りの関数が得られる。依存関数なので、そのたびに残りの型にも渡した対象が代入される。

証明を一つの関数の項にする

結論を外側から読むと、まず `x : A` と `y : A`、次に `h : g (f x) = g (f y)` を受け取り、`x = y` の証明を返せばよい。この「受け取る」を、`fun x y h =>` と書く。

では、本体をどう作るか。欲しい結論 `x = y` から考えると、`hf x y` は `f x = f y` の証明を受け取って、その結論の証明を返してくれる。そこで、まず `f x = f y` の証明を作ればよい。

```
hf x y : f x = f y → x = y
```

このために `hg` に渡す対象は、`x`、`y` ではなく、`B` の項である `f x`、`f y` である。 `hg (f x) (f y)` が要求する等式は、ちょうど手元の `h` が保証している。

```
hg (f x) (f y) : g (f x) = g (f y) → f x = f y
```

したがって `hg (f x) (f y) h` を作って `hf x y` に渡すと、本体が完成する。

証明の項は、仮定や既に与えられた定義・補題の名前を参照して組み立てられる。参照先の内容をすべて項の中に書き込む必要はなく、Lean はそれらの宣言も使って型を検査する。

```
def comp_injective (A B C : Type) (f : A → B) (g : B → C)
  (hf : ∀ x y : A, f x = f y → x = y)
  (hg : ∀ u v : B, g u = g v → u = v) :
  ∀ x y : A, g (f x) = g (f y) → x = y :=
fun x y h => hf x y (hg (f x) (f y) h)
```

Lean は、この `def` の宣言を受理するときに証明の型検査を行う。後から `#check comp_injective` と書いたときには、既に検査された定義の型を確認している。

機械が行う型検査を追う

ここまでは結論からさかのぼって証明を組み立てた。今度は完成した項を、内側から型を求めて検査し直そう。右辺全体に期待される型は、結論に書いた次の型である。

```
(x : A) → (y : A) → (g (f x) = g (f y) → x = y)
```

右辺が `fun x y h => ...` なので、機械は `x : A`、`y : A`、`h : g (f x) = g (f y)` を使えるものとして、本体を検査する。本体に要求される型は、最後の結論 `x = y` である。

まず、内側の `hg (f x) (f y)` の型を求める。`hg` の二つの対象に `f x` と `f y` を代入すると、次の型になる。

```
hg (f x) (f y) : g (f x) = g (f y) → f x = f y
```

この関数が要求する入力型は `g (f x) = g (f y)` であり、手元の `h` の型と一致する。したがって、`h` を渡した結果は次の型になる。

```
hg (f x) (f y) h : f x = f y
```

次に外側を見る。hf に x と y を渡すと、次の型になる。

```
hf x y : f x = f y → x = y
```

この入力型 $f x = f y$ は、いま内側で得た証明の型と一致する。よって、その証明を渡した本体全体の型が分かる。

```
hf x y (hg (f x) (f y) h) : x = y
```

本体の型が、要求されていた結論の型と一致した。これで、 x 、 y 、 h を受け取ってその本体を返す `fun x y h => ...` 全体が、指定された全称量化と「ならば」の型を持つと確かめられる。

ここで機械がしているのは、宣言された型への引数の代入と、適用の入力型・出力型の照合である。「 g の単射性を使おう」という証明の発想をもう一度探すのではなく、**書かれた証明の各適用が型に合っているか**を確かめている。

長い項を読むときは、括弧や型注釈を明示し、途中の結果の型を確かめるとよい。この例なら、内側を $(hg (f x) (f y) h : f x = f y)$ と注釈できる。必要なら小さな定義に分けて名前を付ける。

証明を書けるかどうかに注目する

Lean では、**同じ命題の証明どうしは、書き方が違っても同じ項として扱われる**。証明が存在する場合、その命題に対応する型の項は、この意味で一つとみなせる。「ならば」や全称命題の証明である関数も、命題に対応する型の項なので、この扱いを受ける。このため、ここでは**要求された型の関数を書けるかどうか**、すなわち、その命題の証明を与えられるかに注目する。

示すときは、要求される引数を `fun` で受け取る。使うときは、対象や証明を適用で渡す。全称量化と「ならば」が重なった証明も、この二つの操作で読める。

⇒ 練習

29. 写像 $f : \mathbb{N} \rightarrow \mathbb{N}$ が**単調**であるとは、任意の x, y について、 $x \leq y$ ならば $f(x) \leq f(y)$ となることである。 f, g が単調なら $g \circ f$ も単調であることを、`comp_monotone` という名前で証明せよ。単調性の仮定と結論の型を書き、`fun` と仮定の適用だけで証明の項を作ること。 $\leq$ は `\le` で入力する。不等式はいつもの意味で読み、その定義は帰納型のところで扱う。

解答 29

```
def comp_monotone (f g : Nat → Nat)
  (hf : ∀ x y : Nat, x ≤ y → f x ≤ f y)
  (hg : ∀ u v : Nat, u ≤ v → g u ≤ g v) :
```

```

    ∀ x y : Nat, x ≤ y → g (f x) ≤ g (f y) :=
  fun x y h => hg (f x) (f y) (hf x y h)

```

x 、 y と $h : x \leq y$ を受け取り、 $hf\ x\ y\ h : f\ x \leq f\ y$ を作る。これを $hg\ (f\ x)\ (f\ y)$ に渡せばよい。単射の合成とは逆に、もとの不等式から像の不等式へ進む。

2. 左キャンセル — 関数を引数として受け取る

単射 $f : A \rightarrow B$ と、二つの写像 $u, v : X \rightarrow A$ を考える。任意の $t \in X$ に対して $f(u(t)) = f(v(t))$ が成り立つとする。このとき、任意の $t \in X$ に対して $u(t) = v(t)$ が成り立つ。

各点で単射性を使う

証明。 $t \in X$ を任意に取る。仮定から $f(u(t)) = f(v(t))$ であり、 f は単射だから $u(t) = v(t)$ となる。■

仮定と結論はどちらも「すべての t について」という形である。対象 t を受け取り、その点での等式の保証に単射性を適用すればよい。まず、仮定と結論を論理式として書く。

```

(∀ x y : A, f x = f y → x = y) →
  (∀ t : X, f (u t) = f (v t)) → ∀ t : X, u t = v t

```

二つの仮定の保証を hf 、 h と名付けると、使える項の型は次のようになる。

```

hf : ∀ x y : A, f x = f y → x = y
h  : ∀ t : X, f (u t) = f (v t)
h t : f (u t) = f (v t)
hf (u t) (v t) : f (u t) = f (v t) → u t = v t

```

$h\ t$ を最後の関数に渡せば、結論に必要な $u\ t = v\ t$ の証明が得られる。

```

def cancel_pointwise (X A B : Type) (f : A → B)
  (hf : ∀ x y : A, f x = f y → x = y)
  (u v : X → A) (h : ∀ t : X, f (u t) = f (v t)) :
  ∀ t : X, u t = v t :=
  fun t => hf (u t) (v t) (h t)

```

$h\ t$ は、対象 t に対する等式 $f\ (u\ t) = f\ (v\ t)$ の証明である。これを $hf\ (u\ t)\ (v\ t)$ に渡して、 $u\ t = v\ t$ の証明を得る。関数 u と v も引数になったが、証明の操作は先ほどと同じ適用である。

⇒ 練習

30. 単射 $f : A \rightarrow B$ と $u, v : X \rightarrow A$ 、 $h : \forall t : X, f (u t) = f (v t)$ を受け取ったとする。 $\forall t : X, u t = v t$ の証明を書け。 h 全体を単射性に渡すのではなく、どの項を渡せばよいか説明せよ。

解答 30

```
def cancel_pointwise_again (X A B : Type)
  (f : A → B)
  (hf : ∀ x y : A, f x = f y → x = y)
  (u v : X → A) (h : ∀ t : X, f (u t) = f (v t)) :
  ∀ t : X, u t = v t :=
fun t => hf (u t) (v t) (h t)
```

h は全称命題の証明なので、まず対象 t を渡す。得られた $h t : f (u t) = f (v t)$ が、 $hf (u t) (v t)$ の要求する入力である。各点で $u t = v t$ の証明を作り、 $fun t => \dots$ で全体の証明にする。

3. Eq と rfl — 等式の保証を作る

ここまでは、仮定された等式の保証を使ってきた。次に、等式そのものと、その基本の証明をもう少し見ておこう。

等式を表す Eq

$a = b$ は `Eq a b` という項の記法である。例えば自然数については、`Eq` は二つの自然数を受け取って命題を返す関数として使える。

```
#check Eq
```

```
Eq.{u_1} {α : Sort u_1} : α → α → Prop
```

共通の型 α を受け取り、その型の二つの項から命題を作る。 `α := Nat` と指定したときの型も確かめよう。

```
#check (Eq : Nat → Nat → Prop)
```

```
Eq : Nat → Nat → Prop
```

`Eq 2 2` は `2 = 2` という命題、`Eq 2 3` は `2 = 3` という命題である。同じ型の項どうしの等式を扱うので、`Eq` はその共通の型も暗黙引数として受け取る。上の注釈では、それが `Nat` と決まっている。

命題 `Eq a b` 自体と、その保証を区別しよう。`h : Eq a b`、つまり `h : a = b` と書いたとき、`h` が等式の証明である。

補足: Sort と型の範囲

`Sort` は `Prop` と `Type` をまとめて扱うための表記である。`Sort 0` は `Prop`、`Sort (u + 1)` は `Type u` を表す。ここでは `α : Sort u` を「命題も含めて、一般の型を `α` とする」と読めばよい。型の後の `.{u_1}` などは、その階層に関する表示である。

自分自身との等式を保証する `Eq.refl`

等式の基本の証明を作るのが `Eq.refl` である。項 `a` を渡すと、`a = a` の証明を返す。

```
#check Eq.refl
```

```
Eq.refl.{u_1} {α : Sort u_1} (a : α) : a = a
```

型 `α` は暗黙引数、対象 `a` は明示引数である。

```
#check Eq.refl 2
```

```
Eq.refl 2 : 2 = 2
```

`Eq.refl` は等式の証明を作るコンストラクターである。その定義を帰納型として読む方法は後で学ぶが、今は、対象を渡せばその対象自身との等式の証明が得られる、と読めばよい。

`rfl` は、対象を暗黙引数にしたもの

`Eq.refl a` は対象 `a` を明示して `a = a` の証明を作る。`rfl` は、この対象を暗黙引数にしたものである。Lean では、次のように `def` で定義されている。

```
def rfl {α : Sort u} {a : α} : a = a := Eq.refl a
```

ここで注目したいのは `{a : α}`。対象 `a` は、証明を置く場所で期待される型から補われる。`rfl` 自体も普通の項であり、第1章で学んだ暗黙引数の実例になっている。

暗黙引数まで型に残して表示するため、`@rfl` を調べよう。

```
#check @rfl
```

```
@rfl : ∀ {α : Sort u_1} {a : α}, a = a
```

`Eq.refl` の $(a : \alpha)$ に対して、こちらは $\{a : \alpha\}$ である。対象を明示するなら `@rfl Nat 2` と書ける。

計算して一致するときは `rfl`

`rfl` を使うと、期待される等式の型に合わせて対象が補われる。両辺が定義から計算して一致するときに使える。

```
def one_add_one : 1 + 1 = 2 := rfl
```

```
#check one_add_one
```

```
one_add_one : 1 + 1 = 2
```

左辺 `1 + 1` は計算すると `2` になる。そのため、例えば `Eq.refl 2 : 2 = 2` も、ここで要求された型 `1 + 1 = 2` の項として使える。

```
def one_add_one_explicit : 1 + 1 = 2 := Eq.refl 2
```

計算して一致するかどうかは、`Eq.refl` を直接使った場合にも、型の照合で調べられる。`rfl` だけに特別な計算能力があるわけではない。違いは、対象を明示するか、暗黙に補ってもらうかである。

補足: 型の照合に必要な計算

両辺を定義に沿って計算するにも、時間やメモリが必要である。複雑な計算では時間がかかったり、計算資源の上限に達して検査が中断されたりすることがある。Lean には自然数の数値計算を効率よく行う仕組みがあり、負荷は数の大きさだけでなく、行う計算の内容にも依存する。計算が打ち切られたことは、等式が偽であることを意味しないので、エラーメッセージで理由を確かめよう。

🔗 練習

31. `def two_add_three : 2 + 3 = 5 := ...` の右辺を `rfl` で埋めよ。 `rfl` が使える理由を説明せよ。

解答 31

```
def two_add_three : 2 + 3 = 5 := rfl
```

左辺を計算すると 5 になり、右辺と一致するので `rfl` が使える。

32. `def oops : 2 + 2 = 5 := rfl` は受理されるか。受理されないなら、どの二つの項が計算しても一致しないのかを説明せよ。

解答 32

受理されない。2 + 2 を計算した 4 と、右辺の 5 が一致しない。エラーには次の説明が現れる。

```
error: Not a definitional equality: the left-hand side
  2 + 2
is not definitionally equal to the right-hand side
  5
```

一般に、`rfl` が使えないだけで命題が偽だとは判断できない。ここで失敗したのは、計算して両辺を一致させるという確かめ方である。

対象ごとの等式を保証する

対象を受け取って、その対象自身との等式の証明を返すこともできる。`n` を受け取るたびに、返す証明の型が `n = n` に決まる。

```
def all_refl : ∀ n : Nat, n = n :=
  fun n => (rfl : n = n)

#check all_refl
```

```
all_refl (n : Nat) : n = n
```

依存関数型の形では、次のように読める。

```
all_refl : (n : Nat) → n = n
```

入力 `n` によって返す証明の型が変わる。`∀ n : Nat, n = n` の量化部分が、引数 `(n : Nat)` として左側に表示されている。Infoview でもこのような binder 形式が使われる。`∀` の記号が見えなくても、「任意の `n : Nat` について `n = n`」という型は変わっていない。

```
#check all_refl 7
```

```
all_refl 7 : 7 = 7
```

これも全称量化の証明を依存関数として作り、対象を渡して使う例である。次は、このような等式の保証を逆向きに使ったり、二つつないだりする道具を使おう。

⇒ 練習

33. `all_refl 12` の型を予想せよ。さらに `fun n : Nat => all_refl (n + 1)` の型を答えよ。

解答 33

```
#check all_refl 12
```

```
all_refl 12 : 12 = 12
```

```
#check fun n : Nat => all_refl (n + 1)
```

```
fun n => all_refl (n + 1) : ∀ (n : Nat), n + 1 = n + 1
```

`all_refl` に `n + 1` を渡すので、返される証明の型にも `n + 1` が入る。全体は `n` を受け取って、その等式の証明を返す依存関数である。

Eq.symm — 等式の向きを逆にする

```
#check Eq.symm
```

```
Eq.symm.{u} {α : Sort u} {a b : α} (h : a = b) : b = a
```

`h : a = b` があるとき、`Eq.symm h` は `b = a` の証明になる。等式の証明を受け取り、逆向きの等式の証明を返す関数である。一般に、`a = b` の証明をそのまま `b = a` の証明として渡せるわけではない。等式の向きも型に含まれるので、逆向きの証明が必要なときは `Eq.symm` を使う。

```
h : a = b
Eq.symm h : b = a
```

対象 `a` と `b` は暗黙引数であり、渡した `h` の型から補われる。ここでも、第1章で見た暗黙引数の仕組みが使われている。

Eq.trans — 二つの等式をつなぐ

```
#check Eq.trans
```

```
Eq.trans.{u} {α : Sort u} {a b c : α} (h1 : a = b) (h2 : b = c) : a = c
```

`h1 : a = b` と `h2 : b = c` があるとき、`Eq.trans h1 h2` は `a = c` の証明になる。一つ目の右辺と二つ目の左辺が一致することが、二つの証明を渡すための条件である。

```
h1 : a = b
h2 : b = c
Eq.trans h1 h2 : a = c
```

数学で等号を並べて $a = b = c$ と書く操作を、二つの保証から一つの保証を作る関数として表している。三つ以上の等式をつなぐときも、`Eq.trans` の適用を重ねればよい。

congrArg — 両辺に同じ関数を適用する

```
#check congrArg
```

```
congrArg.{u, v} {α : Sort u} {β : Sort v} {a1 a2 : α} (f : α → β) (h : a1 = a2) : f a1 = f a2
```

`f : A → B` と `h : a = b` があるとき、`congrArg f h` は `f a = f b` の証明になる。関数 `f` と等式の証明 `h` を受け取り、像どうしの等式の証明を返す。

```
f : A → B
h : a = b
congrArg f h : f a = f b
```

両辺に同じ関数を適用する、という数学の操作に対応している。この三つの道具は、新しい公理を付け加えたものではなく、`Eq` の帰納型としての仕組みから証明できるものである。次の左逆の例では、これらを組み合わせて証明を作る。

なぜ等式の性質を証明できるのか

等式には対称性や推移性も必要はなはずだが、ここまで出てきたコンストラクターは、反射性 `a = a` を保証する `Eq.refl` だけである。ここから他の性質を証明できるのは、`Eq` を帰納型として定義しているからである。詳しい仕組みは次の章で学ぶが、まず集合の言葉でイメージをつかもう。

集合 X 上の二項関係を $R \subseteq X \times X$ として考える。反射性を満たすには、すべての $x \in X$ について $(x, x) \in R$ でなければならない。したがって、どの反射的な関係も**対角集合**を含む。

$$\Delta_X = \{(x, x) \mid x \in X\}$$

対角集合自身も反射的だから、**反射性を満たす二項関係のうち最小のものは Δ_X** である。これは「同じ要素どうしだけに関係づける」という等号の関係であり、対称性と推移性も満たす。反射性だけを仮定した任意の関係に、対称性や推移性があると言っているわけではない。余分な組を入れない、という最小性が効いている。

関数 $f : X \rightarrow Y$ についても、対角上の組 (x, x) は $(f(x), f(x))$ に移り、これは Δ_Y に属する。したがって、この関係は同じ関数を両辺に適用しても保たれる。いま使った `congrArg` は、この性質の証明である。

帰納型による定義が与えるもの

上の最小性は、集合の言葉によるイメージである。Lean では、`Eq` を帰納型として定義すると、`Eq.refl` で作る場合を調べることで、任意の等式の証明についての主張を示す原理が得られる。この原理が、余分な場合を持たないという最小性に対応する役割を果たす。

例えば対称性を示すとき、`a = b` の証明が `Eq.refl a` である場合を考えればよい。この場合には `b` も `a` なので、逆向きの等式の証明も `Eq.refl a` で作れる。このように、反射性の証明と帰納型の原理を使って、対称性・推移性・関数への適用の性質を証明できる。次の章では、この原理を `match` で使って三つの性質を証明する。

補足: theorem と def

この教材の本文では、証明も `def` で定義している。通常の Lean のコードでは、証明に名前を付けるために `theorem` も使われる。例えば `theorem one_add_one : 1 + 1 = 2 := rfl` という形である。どちらも、右辺の項が指定された命題の型を持つかを検査する。

二つの宣言には違いもある。`def` の本体は型の照合のために展開できるが、`theorem` の証明の本体は、そのように展開して照合する対象にはならない。ここでは、証明が指定された型の項であるという共通の形に注目する。

4. 左逆を持つ写像は単射

写像 $f : A \rightarrow B$ に対し、 $g : B \rightarrow A$ が $g \circ f = \text{id}_A$ を満たすとき、 g を f の **左逆**という。この条件を各点で書くと、どの $x \in A$ に対しても $g(f(x)) = x$ となる。左逆を持つ f は単射であることを示そう。

まず数学の証明を書く

証明。 $x, y \in A$ を任意に取り、 $f(x) = f(y)$ と仮定する。両辺に g を適用すると、 $g(f(x)) = g(f(y))$ となる。 g が左逆であることから $g(f(x)) = x$ 、 $g(f(y)) = y$ なので、

$$x = g(f(x)) = g(f(y)) = y$$

を得る。したがって $x = y$ であり、 f は単射である。■

先ほどの三つの道具を、この証明に当てはめる

受け取る保証を $\text{hgf} : \forall x : A, g (f x) = x$ と $h : f x = f y$ と名付ける。数学の等号の列に必要なのは、次の三つの証明である。

```
Eq.symm (hgf x) : x = g (f x)
congrArg g h : g (f x) = g (f y)
hgf y : g (f y) = y
```

左逆の保証を逆向きにし、像が等しいという保証の両辺に g を適用し、もう一度左逆の保証を使った。これらを `Eq.trans` でつなげば、要求された $x = y$ の証明が得られる。

道具を適用して、証明の項を作る

`Eq.trans` を二回使う。まず後ろの二つの等式をつなぎ、その結果を最初の等式とつなぐ。

```
def injective_of_left_inverse (A B : Type)
  (f : A → B) (g : B → A)
  (hgf : ∀ x : A, g (f x) = x) :
  ∀ x y : A, f x = f y → x = y :=
fun x y h =>
  Eq.trans (Eq.symm (hgf x))
    (Eq.trans (congrArg g h) (hgf y))
```

内側の `Eq.trans` が $g (f x) = y$ を作り、外側の `Eq.trans` が $x = g (f x)$ とそれをつないで $x = y$ を作る。数学の三つの等号に対応する証明を組み合わせ、要求された型の項を得ている。

左逆の条件そのものをこの場で証明したわけではない。その条件の証明を引数 `hgf` として受け取り、単射性の証明を返す関数を作ったのである。

⇒ 練習

34. 左逆の証明で使った `hgf : ∀ x : A, g (f x) = x` と `h : f x = f y` があるとする。 `Eq.symm (hgf x)`、`congrArg g h`、`hgf y` の型を答え、それぞれが $x = g(f(x)) = g(f(y)) = y$ のどの等号に対応するか説明せよ。

解答 34

三つの型は次のとおり。

```
Eq.symm (hgf x) : x = g (f x)
congrArg g h : g (f x) = g (f y)
hgf y : g (f y) = y
```

並べると、数学の証明の三つの等号になる。 `Eq.trans` に渡すときには、一つ目の右辺と二つ目の左辺が一致している。

次の章では帰納型を学び、構成子で項を作り、場合分けで使う方法を増やす。証明検査の全体像は、存在などの証明も読んだ後、[04_Exists.lean 6節](#)で整理する。

5. 章末問題

四つの題材と `rf1` を使って、型と証明を確かめよう。

⇒ 練習 — 単射と左キャンセル

35. 三つの単射 $f : A \rightarrow B$ 、 $g : B \rightarrow C$ 、 $k : C \rightarrow D$ の合成が単射であることを示せ。それぞれの単射性を $\forall x y, f x = f y \rightarrow x = y$ の形で仮定し、 $\forall x y, k (g (f x)) = k (g (f y)) \rightarrow x = y$ の証明を書け。

解答 35

```
def three_comp_injective (A B C D : Type)
  (f : A → B) (g : B → C) (k : C → D)
  (hf : ∀ x y, f x = f y → x = y)
  (hg : ∀ x y, g x = g y → x = y)
  (hk : ∀ x y, k x = k y → x = y) :
  ∀ x y, k (g (f x)) = k (g (f y)) → x = y :=
fun x y h => hf x y (hg (f x) (f y) (hk (g (f x)) (g (f y)) h))
```

最初に hk で $g(f\ x) = g(f\ y)$ 、次に hg で $f\ x = f\ y$ 、最後に hf で $x = y$ を得る。外側の写像の単射性から順に使う。

36. (発展) 単射 $f : A \rightarrow B$ に対して、写像 $L_f : \text{Map}(X, A) \rightarrow \text{Map}(X, B)$ を $L_f(u) = f \circ u$ と定めると、 L_f も単射である。この事実を Lean で形式化せよ。まず $X\ A\ B : \text{Type}$ と $f : A \rightarrow B$ を引数に取り、 L_f を `leftCompose` という名前で定義せよ。次に $hf : \forall\ x\ y : A, f\ x = f\ y \rightarrow x = y$ を仮定し、`leftCompose X A B f` の単射性の証明を `leftCompose_injective` として書け。

ヒント1 — 写像を定義する。 $\text{Map}(X, A)$ に対応する型は $X \rightarrow A$ である。`leftCompose X A B f` の型は $(X \rightarrow A) \rightarrow (X \rightarrow B)$ である。関数 u を受け取り、さらに対象 t を受け取って $f\ (u\ t)$ を返す項を書こう。

ヒント2 — 等式を各点で使う。 u 、 v と、その像が等しいという証明 h を受け取る。関数を対象 t で評価する写像 `fun w : X → B => w t` を `congrArg` に渡すと、 h から $f\ (u\ t) = f\ (v\ t)$ の証明を得られる。それを hf に渡そう。

ヒント3 — 関数どうしの等式に戻す。 各点での等式の証明 $k : \forall\ t : X, u\ t = v\ t$ から、`funext k` : $u = v$ が得られる。これを関数外延性という。`fun t => ...` で各点の証明をまとめ、`funext` に渡そう。

解答 36

```
def leftCompose (X A B : Type) (f : A → B) :  
  (X → A) → (X → B) :=  
  fun u => fun t => f (u t)
```

この写像の単射性を証明する。

```
def leftCompose_injective (X A B : Type) (f : A → B)  
  (hf : ∀ x y : A, f x = f y → x = y) :  
  ∀ u v : X → A,  
    leftCompose X A B f u = leftCompose X A B f v →  
    u = v :=  
  fun u v h =>  
    funext (fun t =>  
      hf (u t) (v t) (congrArg (fun w : X → B => w t) h))
```

`congrArg` で得た証明の型は、定義を展開すると $f\ (u\ t) = f\ (v\ t)$ となる。 hf で $u\ t = v\ t$ を得て、各点の証明を `funext` に渡すと $u = v$ が得られる。

37. $f : \mathbb{N} \rightarrow \mathbb{N}$ について、任意の $u, v : \mathbb{N} \rightarrow \mathbb{N}$ に対する点ごとの左キャンセルを仮定する。この仮定から f の単射性を示せ。 u, v に定数関数を使うとよい。

```

hcancel : ∀ u v : Nat → Nat,
  (∀ t : Nat, f (u t) = f (v t)) → ∀ t : Nat, u t = v t

```

解答 37

```

def injective_of_pointwise_cancel (f : Nat → Nat)
  (hcancel : ∀ u v : Nat → Nat,
    (∀ t : Nat, f (u t) = f (v t)) → ∀ t : Nat, u t = v t) :
  ∀ x y : Nat, f x = f y → x = y :=
  fun x y h => hcancel (fun _ => x) (fun _ => y) (fun _ => h) 0

```

定数関数 `fun _ => x` と `fun _ => y` を渡す。それらに `f` を合成した結果が各点で等しいことは、どの点でも `h` を返して示せる。`hcancel` の結論に対象 `0` を渡せば、`x = y` が得られる。ここでは、前提の証明を返す関数自体も、引数として渡している。

38. 三つの単調写像 $f, g, k : \mathbb{N} \rightarrow \mathbb{N}$ の合成が単調であることを示せ。本文と同じ形で三つの単調性を仮定し、`fun` と適用だけで証明せよ。

解答 38

```

def three_comp_monotone (f g k : Nat → Nat)
  (hf : ∀ x y, x ≤ y → f x ≤ f y)
  (hg : ∀ x y, x ≤ y → g x ≤ g y)
  (hk : ∀ x y, x ≤ y → k x ≤ k y) :
  ∀ x y, x ≤ y → k (g (f x)) ≤ k (g (f y)) :=
  fun x y h => hk (g (f x)) (g (f y)) (hg (f x) (f y) (hf x y h))

```

`hf`、`hg`、`hk` の順に、内側の写像から単調性を使う。それぞれの結果の不等式が、次の単調性の仮定になる。

39. $f : A \rightarrow B$ が単射で、 $g : B \rightarrow A$ が $f(g(y)) = y$ をすべての $y \in B$ で満たすとする。このとき $g(f(x)) = x$ がすべての $x \in A$ で成り立つことを示せ。 f の単射性を、どの二つの対象に適用すればよいか考えること。

解答 39

```

def left_inverse_of_right_inverse (A B : Type) (f : A → B) (g : B → A)
  (hf : ∀ x y : A, f x = f y → x = y)
  (hfg : ∀ y : B, f (g y) = y) :

```

```
∀ x : A, g (f x) = x :=  
fun x => hf (g (f x)) x (hfg (f x))
```

比べたい二つの対象は $g(f\ x)$ と x である。それらに f を適用した結果の等式 $f(g(f\ x)) = f\ x$ は、 $hfg(f\ x)$ が与える。この証明を単射性に渡せばよい。等号の書き換えをせず、二つの仮定の適用で証明できる。

型と項 II — 帰納型

第1・2章では、`fun` で関数を作り、適用でその関数を使った。この章では、**構成子で項を作り**、`match` で**その項を使う**という道具を加える。有限集合、直和、直積、集合族の直和を、同じ仕組みで表していこう。

自然数のように再帰的に作られる型も扱う。最後に、第2章で使った等式と自然数の不等式を帰納型として読み、証明の項をどのように作り、使うのかを確かめる。

1. 構成子と場合分け

`inductive` は、新しい型と、その型の項を作る**構成子**を一緒に導入する。まず、三つの色を表す型を定義しよう。構成子の名前は `型名.構成子名` になる。

```
inductive Signal : Type where
  | red : Signal
  | yellow : Signal
  | green : Signal

#check Signal.red
```

```
Signal.red : Signal
```

この宣言によって、新しい型 `Signal` と三つの項 `Signal.red`、`Signal.yellow`、`Signal.green` が一緒に導入された。それぞれを赤・黄・緑と読むのは人間が与える解釈であり、名前だけで色の性質や信号の動きが決まるわけではない。集合の言葉では、三点集合を作ったと思えばよい。三つの構成子で場合を尽くせること、異なる構成子が等しくないことは、帰納型の仕組みに基づく。「これらで全部」を使うための再帰・帰納法の原理が `Signal.rec` であり、`match` もこの原理に基づく。「互いに別々」であることの証明には `Signal.noConfusion` を使える。後者と同じ仕組みを、否定を扱う第4章で `Bool` を例に見る。

match で項を使う

`match 対象 with` の後に、構成子ごとに `| 構成子 => 返す項` を並べる。`match ... with ...` 全体が一つの項であり、対象に対応する枝の項が結果になる。

```
#eval match Signal.red with
| Signal.red => 0
| Signal.yellow => 1
| Signal.green => 2
```

0

ここでは、どの枝でも `Nat` の項を返しているので、全体の型は `Nat` である。`match`、`with`、`|`、`=>` は、この項を組み立てるための特別な記号である。

関数の本体で場合分けする

関数記法 `fun` と `match` を組み合わせ、次の信号を返す関数を書こう。対象 `s : Signal` を受け取り、その構成子によって返す色を決める。

```
def Signal.next : Signal → Signal := fun s =>
  match s with
  | Signal.red => Signal.green
  | Signal.yellow => Signal.red
  | Signal.green => Signal.yellow

#eval Signal.next Signal.red
```

`Signal.green`

`fun` が入力を受け取り、その本体の `match` が値を選ぶ。有限集合からの写像を、各点の行き先を指定して定義するのと同じ形である。`match` で場合分けする対象や結果の型は、有限集合に限らない。後では自然数についても使う。

🔗 練習

40. `Signal.red`、`Signal.yellow`、`Signal.green` をそれぞれ `0`、`1`、`2` に送る `signalCode : Signal → Nat` を定義せよ。各枝の型を確かめ、`signalCode Signal.yellow` の値を予想せよ。

解答 40

```
def signalCode : Signal → Nat := fun s =>
  match s with
  | Signal.red => 0
  | Signal.yellow => 1
  | Signal.green => 2
```

各枝で `Nat` の項を返す。黄色の場合の値は `1` である。

41. `Signal.next` と逆向きに色を送る `prev : Signal → Signal` を書け。 `prev (Signal.next Signal.red)` の値を確かめよ。

解答 41

```
def prev : Signal → Signal := fun s =>
  match s with
  | Signal.red => Signal.yellow
  | Signal.yellow => Signal.green
  | Signal.green => Signal.red
```

赤を次の色に送ると青、青を逆向きに送ると赤になる。

一点の型と空の型

構成子が一つで引数を取らない型を自作すると、一点集合に対応する。

```
inductive MyUnit : Type where
  | unit : MyUnit
```

構成子を一つも持たない型は、空集合に対応する。

```
inductive MyEmpty : Type
```

空の型にも使い方はある。項を受け取ったとして、場合分けすべき構成子が一つもなければよい。その場合分けを `nomatch` と書く。

```
def emptyToNat (e : MyEmpty) : Nat := nomatch e
```

これは `MyEmpty` の項を作ったのではなく、`MyEmpty → Nat` という関数を作ったのである。

補足: Prelude にある型

標準環境には、同じ形の型 `Unit` と `Empty` が既に用意されている。`Unit` の構成子は `Unit.unit` で、`Empty` には構成子がない。二点の型 `Bool` も、構成子 `Bool.false` と `Bool.true` を持つ帰納型である。これらは標準で読み込まれる `Prelude` に含まれるので、使う側で定義し直す必要はない。

2. 直和と直積

構成子も関数なので、引数を受け取れる。次は「自然数に札を付けたもの」と「真偽値に札を付けたもの」をまとめた型である。構成子の名前を `inn`・`inb` とし、自然数から入れる場合と真偽値から入れる場合を区別する。

```
inductive TaggedSum : Type where
  | inn (n : Nat) : TaggedSum
  | inb (b : Bool) : TaggedSum

#check TaggedSum.inn
```

```
TaggedSum.inn (n : Nat) : TaggedSum
```

構成子 `TaggedSum.inn` は `Nat → TaggedSum` という関数である。集合の言葉では、この型は自然数と真偽値の**直和（非交和）**に対応する。

```
#check TaggedSum.inb
```

```
TaggedSum.inb (b : Bool) : TaggedSum
```

同様に、`TaggedSum.inb` は `Bool → TaggedSum` という関数である。この構成子そのものは関数であり、それに `true : Bool` を渡した `TaggedSum.inb true` は、できあがった `TaggedSum` 型の項である。`true` 自体は `Bool` 型なので、`TaggedSum` 型の項が必要な場所では `TaggedSum.inb true` と書く。この例では、構成子の適用を省略しても自動で補われるわけではない。

中身を受け取る場合分け

`match` の枝で `TaggedSum.inn n` と書くと、その構成子に渡された自然数に `n` と名前を付けられる。返す項では、その名前を使ってよい。

```
def valueOf : TaggedSum → Nat := fun x =>
  match x with
  | TaggedSum.inn n => n
  | TaggedSum.inb _ => 0

#eval valueOf (TaggedSum.inn 7)
```

枝の `TaggedSum.inb _` にある `_` は、その位置にはどの値が来てもよく、その値に名前を付けないことを表す。ここでは `true` と `false` のどちらも同じ枝で扱い、その値を使わずに `0` を返している。直和からの写像は、左側と右側のそれぞれについて行き先を定めることで作れる。

```
#eval valueOf (TaggedSum.inb true)
```

```
0
```

まず `TaggedSum.inb true` という項を作り、それを `valueOf` に渡すので、括弧でまとめている。括弧を外した `valueOf TaggedSum.inb true` は `(valueOf TaggedSum.inb) true` と読まれ、`valueOf` に必要な `TaggedSum` 型の項の代わりに構成子そのものを渡すことになり、型が合わない。

構成子が違う場合と、同じ場合

`TaggedSum.inn n` と `TaggedSum.inb b` が等しくないことも、`TaggedSum.inn n = TaggedSum.inn m` から `n = m` が言えることも、Leanで証明できる。後者を表す補題は、帰納型の宣言に伴って生成される。

```
#check TaggedSum.inn.inj
```

```
TaggedSum.inn.inj {n nt : Nat} : TaggedSum.inn n = TaggedSum.inn nt → n = nt
```

第2章で見た単射性と同じく、像の等式の証明を受け取り、中身の等式の証明を返す関数である。「構成子は単射」と仮定を付け足したのではなく、その証明を帰納型の仕組みから得ている。`TaggedSum.inb` についても同様である。集合の言葉では、この二つの構成子は、それぞれの集合から直和への標準的な単射に対応する。

中身の型をパラメータにする

自然数と真偽値に限定せず、任意の型 `A` と `B` の直和を同じように作れる。

```
inductive MySum (A B : Type) : Type where
| inl (a : A) : MySum A B
| inr (b : B) : MySum A B
```

```
#check MySum.inl
```

```
MySum.inl {A B : Type} (a : A) : MySum A B
```

型を作る `MySum` には `MySum Nat Bool` と二つの型を渡す。構成子の型引数は暗黙引数になっている。例えば `(MySum.inl 3 : MySum Nat Bool)` では、結果の型注釈から `A := Nat`、`B := Bool` が決まる。標準の直和型 `A ⊕ B` と、その構成子 `Sum.inl`・`Sum.inr` も同じ形で使う。

補足: 型をパラメータにした取り出し関数

`valueOf` と同じ形で、一般の直和から項を取り出す関数も書ける。左側ならその項を、右側ならあらかじめ受け取った既定値 `d` を返す。

```
def getLeft {A B : Type} (d : A) : MySum A B → A := fun x =>
  match x with
  | MySum.inl a => a
  | MySum.inr _ => d

#check getLeft
```

```
getLeft {A B : Type} (d : A) : MySum A B → A
```

`A` と `B` は暗黙引数である。次の適用では、`0` と `true` の型からそれぞれ補われる。

```
#eval getLeft 0 (MySum.inr true)
```

```
0
```

⇒ 練習

42. `swapSum : MySum A B → MySum B A` を定義せよ。左側から来た項は右側へ、右側から来た項は左側へ送ること。

解答 42

```
def swapSum {A B : Type} : MySum A B → MySum B A := fun s =>
  match s with
  | MySum.inl a => MySum.inr a
  | MySum.inr b => MySum.inl b
```

例えば左の枝では `a : A`。行き先 `MySum B A` では `A` が右側なので、`MySum.inr` を使う。

組も帰納型で作る

今度は、`A` の項と `B` の項を両方持つ組を作りたい。構成子を一つにし、その構成子に二つの引数を渡す。

```
inductive MyPair (A B : Type) : Type where
  | mk (a : A) (b : B) : MyPair A B

#check (MyPair.mk 3 true : MyPair Nat Bool)
```

```
MyPair.mk 3 true : MyPair Nat Bool
```

直和ではどちらかの構成子を選んだ。直積では、一つの構成子に両方の材料を渡す。型が同じ引数を二つ取る場合も、異なる型の引数を取る場合も、この書き方で扱える。

組を使うときも match

構成子が一つなら、場合分けは一つの枝で済む。第1成分を返す関数を書いてみよう。

```
def pairFirst {A B : Type} : MyPair A B → A := fun p =>
  match p with
  | MyPair.mk a _ => a

#eval pairFirst (MyPair.mk 3 true)
```

```
3
```

標準の直積 `A × B` も `Prod.mk a b` で作り、`match` の `Prod.mk a b` の枝で使える。例えば次は、二つの成分を交換する関数である。

```
def swapPair {A B : Type} : A × B → B × A := fun p =>
  match p with
  | Prod.mk a b => Prod.mk b a
```

⇒ 練習

43. 写像 `f : X → A` と `g : X → B` から、`x` を組 `(f x, g x)` に送る `pairMaps : X → MyPair A B` を定義せよ。型 `X A B` と写像 `f g` も引数に取ること。

解答 43

```
def pairMaps (X A B : Type) (f : X → A) (g : X → B) : X → MyPair A B :=  
  fun x => MyPair.mk (f x) (g x)
```

それぞれ $f\ x : A$ 、 $g\ x : B$ なので、`MyPair.mk` の二つの引数に渡せる。

補足: 帰納型を直和と直積で見る

`MySum A B` では、構成子をどちらか一つ選ぶ。これは直和に対応する。`MyPair A B` では、一つの構成子に A の項と B の項を両方渡す。これは直積に対応する。一般にも、構成子の選択を直和、それぞれの構成子が受け取る材料を直積として読むと、帰納型がどんな項を作るかを見通しやすい。引数の型が前の引数に依存する場合は、次に見る依存する組になる。

3. 依存する組

集合族 $(B(a))_{a \in A}$ の直和 $\bigsqcup_{a \in A} B(a)$ の要素は、添字 a を一つ選び、さらにその集合の要素 $b \in B(a)$ を一つ選んだ組 (a, b) である。第二成分を選ぶ集合は、第一成分の値によって決まる。

第1章で使った `Fin n` を例にしよう。「自然数 n と、 n 未満の番号を一つ選んだ組」を次のように定義できる。

```
inductive Numbered : Type where  
  | mk (n : Nat) (i : Fin n) : Numbered  
  
#check Numbered.mk
```

```
Numbered.mk (n : Nat) (i : Fin n) : Numbered
```

構成子は、 n を受け取ってから **`Fin n` 型の項**を受け取る依存関数である。

依存する組を作る

$n := 3$ を選ぶなら、次に渡す項は `Fin 3` 型でなければならない。

```
def numberedThree : Numbered := Numbered.mk 3 (2 : Fin 3)  
def numberedFive : Numbered := Numbered.mk 5 (4 : Fin 5)  
  
#check Numbered.mk 3
```

```
Numbered.mk 3 : Fin 3 → Numbered
```

二つの組はどちらも `Numbered` 型だが、中に持つ番号の型は違う。`Fin 0` は空なので、第一成分を `0` にした組は作れない。ここでは `Fin n` の内部の定義には立ち入らず、第1章で使った型の族として扱う。

第一成分を取り出す

`Numbered` の構成子是一つなので、使うときも一つの枝でよい。まず、大きさを取り出す関数を書く。

```
def Numbered.size : Numbered → Nat := fun p =>
  match p with
  | Numbered.mk n _ => n

#eval Numbered.size numberedThree
```

```
3
```

この関数の行き先は固定した `Nat` である。枝に入ると `n : Nat` と `i : Fin n` が使えるが、ここでは `n` だけを返している。

第二成分の型も入力から決まる

番号を取り出す関数の行き先は、入力された組の大きさで変わる。したがって、その型にも入力 `p` が現れる。関数全体の型は、次の依存関数型である。

```
Numbered.index : (p : Numbered) → Fin (Numbered.size p)
```

第一成分を返す `Numbered.size : Numbered → Nat` とは異なり、結果の型が入力ごとに変わる。

```
def Numbered.index (p : Numbered) : Fin (Numbered.size p) :=
  match p with
  | Numbered.mk _ i => i

#check Numbered.index numberedThree
```

```
numberedThree.index : Fin numberedThree.size
```

`Numbered.mk n i` の枝では `Numbered.size p` が `n` に計算される。要求される型は `Fin n` になり、手元の `i : Fin n` をそのまま返せる。

一般の依存和

任意の型の族 $B : A \rightarrow \text{Type}$ に対しても、同じ構成子を書ける。

```
inductive FamilyPair (A : Type) (B : A → Type) : Type where
  | mk (a : A) (b : B a) : FamilyPair A B
```

標準の型では、これに対応する依存和を $\text{Sigma } B$ 、または $(a : A) \times B \ a$ と書き、 $\text{Sigma.mk } a \ b$ で項を作る。特に $(n : \text{Nat}) \times \text{Fin } n$ は、自然数 n を一つ選び、続いて **その n に対応する $\text{Fin } n$** の項を選んだ組の型である。第一成分と第二成分の型を独立に決める普通の直積 $\text{Nat} \times B$ との違いは、この依存にある。

```
def numberedSigma : (n : Nat) × Fin n := Sigma.mk 3 (2 : Fin 3)

#check numberedSigma
```

```
numberedSigma : (n : Nat) × Fin n
```

族が定数なら、第二成分の型は第一成分によらない。普通の直積もこの形の特別な場合である。

すべての添字で選ぶか、一つの添字を選ぶか

同じ族 $B : A \rightarrow \text{Type}$ に対して、二通りの項の作り方がある。

型	項を作る	項を使う
依存関数 $(a : A) \rightarrow B \ a$	<code>fun a => ...</code> で、どの a にも答えを用意する	<code>f a</code> と対象を渡す
依存和 $(a : A) \times B \ a$	<code>Sigma.mk a b</code> で、一つの a と $b : B \ a$ をまとめる	<code>match</code> で a と b に名前を付ける

第4章では、型の族を命題の族に置き換えて、全称量化と存在量化の証明を読む。

練習

44. $n : \text{Nat}$ から、大きさが $n + 1$ で番号が最後の n である組を返す `attachLast : Nat → Numbered` を定義せよ。標準の `Fin.last : (n : Nat) → Fin (n + 1)` を使ってよい。
`Numbered.size (attachLast 3)` と `Numbered.index (attachLast 3)` の値も確かめよ。

解答 44

```
def attachLast : Nat → Numbered := fun n =>
  Numbered.mk (n + 1) (Fin.last n)
```

大きさは 4、番号は `Fin 4` の 3 である。構成子の第一引数に `n + 1` を渡すと、第二引数に必要な型が `Fin (n + 1)` と決まる。

4. 自然数と再帰

構成子の引数に、いま定義している型自身を使える。自然数を、零と「一つ次の数を作る操作」で定義してみよう。

```
inductive MyNat : Type where
  | zero : MyNat
  | succ : MyNat → MyNat

#check MyNat.succ (MyNat.succ MyNat.zero)
```

```
MyNat.zero.succ.succ : MyNat
```

表示の `MyNat.zero.succ.succ` は、`MyNat.succ (MyNat.succ MyNat.zero)` のことである。この型の項は、`zero` に `succ` を有限回重ねて作る。標準の `Nat` も同じ形で定義されている。

補足: 集合の方程式としての再帰

集合の言葉では、零のための一点と、次の数を作るためのコピーを合わせるので、 $X \cong 1 \sqcup X$ という形が現れる。ただし、この方程式だけで自然数が決まるわけではない。帰納型では、`zero` から `succ` を有限回使って作ったものですべて、という条件が付く。これが、構成子から生成される最小のもの、という見方である。

自分より小さい項を使う

加法を、第二引数について場合分けして定義しよう。`succ k` の場合は、中に入っていた小さい項 `k` への加法を使う。

```
def myAdd : MyNat → MyNat → MyNat := fun m n =>
  match n with
  | MyNat.zero => m
  | MyNat.succ k => MyNat.succ (myAdd m k)
```

$m + 0 = m$ 、 $m + (k + 1) = (m + k) + 1$ を項の作り方として書いている。再帰呼び出しが構造的に小さい項へ進むことをLeanが確かめる。

```
#reduce myAdd (MyNat.succ MyNat.zero) (MyNat.succ MyNat.zero)
```

```
MyNat.zero.succ.succ
```

補足: 再帰が止まることの検査

同じ入力のまま自分を呼ぶ、次の定義は受理されない。

```
def loop (n : MyNat) : MyNat := loop n
```

再帰呼び出しで引数が変わらず、停止することを確認されないためである。myAdd では `MyNat.succ k` の中の `k` へ進むので、構造が小さくなることを確認できる。Lean は、このように再帰が止まる根拠も検査している。

⇒ 練習

45. `MyNat.zero` を `0` に、`MyNat.succ k` を `toNat k + 1` に送る `toNat : MyNat → Nat` を再帰で定義せよ。

解答 45

```
def toNat : MyNat → Nat := fun n =>
  match n with
  | MyNat.zero => 0
  | MyNat.succ k => toNat k + 1
```

小さい項 `k` への関数の値を使って、`succ k` への値を作っている。

各自然数への証明を再帰で作る

同じ再帰を使い、自然数ごとの等式の証明も作れる。標準の `Nat` で、 $0 + n = n$ を証明しよう。

零の場合は計算で一致する。 $k + 1$ の場合は、 $0 + k = k$ の両辺に一つ加えればよい。この「`k` の場合の証明」が、次のコードの再帰呼び出しである。

```
def zero_add_by_rec : ∀ n : Nat, 0 + n = n := fun n =>
  match n with
```

```
| Nat.zero => rfl
| Nat.succ k => congrArg Nat.succ (zero_add_by_rec k)
```

`congrArg` が返す型は $\text{Nat.succ } (0 + k) = \text{Nat.succ } k$ 。これは $0 + \text{Nat.succ } k = \text{Nat.succ } k$ と計算で一致する。証明を返す依存関数を再帰で作ることが、数学的帰納法による証明になっている。

補足: 定義から計算して一致することと、等式を証明すること

自然数の加法は第二引数について再帰する。そのため $n + 0$ は n に計算される。一方、変数 n に対する $0 + n$ は、 n がどの構成子でできているか分からず、計算を進められない。

```
def add_zero_by_rfl (n : Nat) : n + 0 = n := rfl
```

次の形では、`rfl` だけでは証明できない。

```
def zero_add_bad (n : Nat) : 0 + n = n := rfl
```

そこで本文の `zero_add_by_rec` では、 n について場合分けし、帰納法で等式を証明した。定義を展開して計算すると一致することを**定義的に等しい**という。型の照合ではこの等しさを使えるが、証明された等式がすべて定義的な等しさになるわけではない。

同じ理由で、乗法も $0 * n = 0$ を `rfl` だけで証明できるわけではない。第二引数の作られ方に沿って、証明を再帰で作れる。

```
def zero_mul_by_rec : ∀ n : Nat, 0 * n = 0 := fun n =>
  match n with
  | Nat.zero => rfl
  | Nat.succ k => zero_mul_by_rec k
```

次の数の枝では $0 * \text{Nat.succ } k$ が $0 * k + 0$ 、さらに $0 * k$ に計算されるので、再帰呼び出しで得た証明をそのまま使える。

補足: $2n = n + n$ を定義から証明する

第4章では `Nat.two_mul` を既知の定理として使う。その内容も、再帰と等式の操作から証明できる。数学的帰納法で考えると、零のときは両辺とも零である。 $2 * k = k + k$ を仮定すれば、 $2(k + 1) = 2k + 2 = (k + k) + 2 = (k + 1) + (k + 1)$ となる。

最後の並べ替えには、 $(n + 1) + m = (n + m) + 1$ を使った。これも m に関する帰納法で証明しておこう。

```
def succ_add_from_scratch : ∀ n m : Nat, (n + 1) + m = (n + m) + 1 := fun n m =>
  match m with
```

```
| Nat.zero => rfl
| Nat.succ k => congrArg (fun x => x + 1) (succ_add_from_scratch n k)
```

零の枝は計算で一致する。次の数の枝は、一つ前の等式の両辺に1を加えた証明である。これを使うと、二倍の等式の帰納段階も書ける。

```
def two_mul_from_scratch : ∀ n : Nat, 2 * n = n + n := fun n =>
  match n with
  | Nat.zero => rfl
  | Nat.succ k =>
    Eq.trans (congrArg (fun x => x + 2) (two_mul_from_scratch k))
      (Eq.symm (congrArg (fun x => x + 1) (succ_add_from_scratch k k)))

#check two_mul_from_scratch
```

```
two_mul_from_scratch (n : Nat) : 2 * n = n + n
```

帰納法の仮定は、自分自身への再帰呼び出しで得ている。計算だけで一致しない箇所を、`congrArg` と `Eq.trans` でつないだ。

```
#print axioms two_mul_from_scratch
```

```
'two_mul_from_scratch' does not depend on any axioms
```

「既知の定理」も、定義と計算までたどれば、検査済みの証明の項として与えられている。

5. 等式と自然数の不等式

ここまでは、構成子が `Signal` や `MyNat` という一つの型の項を返していた。次は、構成子が**型の族のどの型に項を作るか**にも注目する。

`Eq a b` は、対象 `a` と `b` ごとに変わる命題である。`Nat.le n m` も、自然数 `n` と `m` ごとに変わる命題である。第2章で見た「述語は型の族」という見方で、これらの族を帰納的に定義する。

依存する組では、第二引数の型に第一引数が現れた。ここではさらに、構成子の**結果の型**にも、引数として受け取った対象が現れる。

Eq の構成子を読む

`a = b` は `Eq a b` の記法である。型引数を明示すると、定義は次の形になる。

```
inductive Eq {A : Sort u} : A → A → Prop where
  | refl (a : A) : Eq a a
```

`Sort u` は `Prop` や `Type` などをもとめて扱う表記で、等式は特定の型に限定されない。唯一の構成子 `Eq.refl` が直接作るのは、両辺が同じ `Eq a a` の証明である。

第2章の「最小の反射的關係」という見方と結び付けると、構成子是对角に証明を作っている。ただしLeanで等式を使う際には、帰納型の除去の原理に従って項を組み立てる。`refl` と `Eq.refl` の関係は、第2章で説明したとおりである。

対称性を自分で証明する

`h : a = b` を受け取り、`b = a` の証明を返す関数を作ろう。等式の構成子は `Eq.refl` の一つなので、その場合を扱う。

```
def eqSymmByMatch {A : Sort u} {a b : A} (h : a = b) : b = a :=
  match h with
  | Eq.refl _ => Eq.refl a
```

この枝では `b` が `a` に特殊化される。返すべき型 `b = a` も `a = a` となるので、`Eq.refl a` を返せる。パターンの `_` は、この構成子の引数が添字から決まるため、名前を付けずに書いている。

単なる構成子の名前の判別だけでなく、**その構成子の結果の型に合わせて、枝の中の型も変わる**。これが、等式の証明を使うときの要点である。

推移性を証明する

次は `a = b` と `b = c` の証明を受け取り、`a = c` の証明を返す。第二の等式について場合分けすればよい。

```
def eqTransByMatch {A : Sort u} {a b c : A}
  (hab : a = b) (hbc : b = c) : a = c :=
  match hbc with
  | Eq.refl _ => hab
```

この枝では `c` が `b` に特殊化され、結論の型が `a = b` になる。そこで、すでに持っている `hab` を返せる。これが第2章で使った `Eq.trans` に相当する証明である。

`congrArg` に相当する関数を作る

等しい二つの対象を同じ関数で送ると、結果も等しい。この証明も、等式の構成子による場合分けで作れる。

```
def congrArgByMatch {A : Sort u} {B : Sort v}
  (f : A → B) {a b : A} (h : a = b) : f a = f b :=
  match h with
  | Eq.refl _ => Eq.refl (f a)
```

枝の中で要求される型は `f a = f a` になる。対称性・推移性・合同性を新しい仮定として加える必要はなく、等式の帰納型の仕組みから証明できた。

```
#print axioms eqSymmByMatch
```

```
'eqSymmByMatch' does not depend on any axioms
```

補足: 等式をもう一つ作る実験

Eq と同じ形の帰納型を自作し、その証明を `match` で使ってみよう。ここでは型の範囲を `Type` に限っている。

```
inductive MyEq {A : Type} (a : A) : A → Prop where
  | refl : MyEq a a

#check MyEq
```

```
MyEq {A : Type} (a : A) : A → Prop
```

対称性の結論は `MyEq b a` だが、構成子の枝では `b` が `a` にそろうため、`MyEq.refl` を返せる。`match` は、構成子がどの添字の型の項を作るかも使って、枝の中で要求する型を決める。

```
def MyEq.symm {A : Type} {a b : A} (h : MyEq a b) : MyEq b a :=
  match h with
  | MyEq.refl => MyEq.refl

def MyEq.trans {A : Type} {a b c : A} (h1 : MyEq a b) (h2 : MyEq b c) : MyEq a c :=
  match h1 with
  | MyEq.refl => h2
```

推移性の枝では、第二の仮定の型が `MyEq a c` になるので、そのまま返せる。標準の等式と自作の等式の間にも、互いの証明を受け取る関数を書ける。

```
def MyEq.toEq {A : Type} {a b : A} (h : MyEq a b) : a = b :=
  match h with
```

```
| MyEq.refl => Eq.refl a
```

```
def MyEq.ofEq {A : Type} {a b : A} (h : a = b) : MyEq a b :=  
  match h with  
  | Eq.refl _ => MyEq.refl
```

いずれも新しい公理を加えず、構成子と場合分けだけで作れた。自作の等式の証明も、標準の等式の証明も、帰納型の同じ原理で扱っている。

自然数の不等式 Nat.le

自然数の $n \leq m$ は `Nat.le n m` の記法である。定義の中心は次の二つの構成子である。

```
inductive Nat.le (n : Nat) : Nat → Prop where  
  | refl : Nat.le n n  
  | step {m} : Nat.le n m → Nat.le n (Nat.succ m)
```

集合の言葉では、これは次の二条件を満たす**最小の二項関係**と考えられる。

- すべての自然数 n について $n \leq n$ が成り立つ。
- $n \leq m$ が成り立てば $n \leq m + 1$ も成り立つ。

第一の条件が `refl`、第二の条件が `step` に対応する。構成子から生成される証明だけを使う、という帰納型の仕組みが、この最小性を表している。

n を固定すると、`Nat.le n` は右辺の自然数を添字に持つ命題の族になる。`refl` は添字 n のところに証明を作り、`step` は添字 m の証明から添字 $m + 1$ の証明を作る。

`Nat.succ m` は $m + 1$ に当たる。自然数を `zero` と `succ` で作ったのと同じように、不等式の証明を `refl` と `step` で作る。

$2 \leq 4$ の証明を作る

まず $2 \leq 2$ の証明を作り、`step` を二回使う。

```
def two_le_four : 2 ≤ 4 := Nat.le.step (Nat.le.step Nat.le.refl)
```

内側から、`Nat.le.refl : 2 ≤ 2`、一回目の `step` の結果が $2 \leq 3$ 、二回目の結果が $2 \leq 4$ である。このように、証明にも組み立て方がある。使える構成子を並べただけでなく、各適用の入力と結果の型が合っていることを確かめよう。

```
#check Nat.le.step two_le_four
```

```
Nat.le.step two_le_four : Nat.le 2 (Nat.succ 4)
```

両辺に一つ加えても順序は保たれる

$n \leq m$ の証明から $n + 1 \leq m + 1$ の証明を作る。数学の言葉で、**不等式の証明の作られ方に関する帰納法**を書いてみよう。

- 出発点 $n \leq n$ から作る場合、結論は $n + 1 \leq n + 1$ なので、反射性から言える。
- $n \leq k$ の証明から一段進んで $n \leq k + 1$ を作った場合を考える。一つ前の証明に対しては帰納法の仮定から $n + 1 \leq k + 1$ が得られる。その右辺をもう一つ増やせば、必要な $n + 1 \leq (k + 1) + 1$ が得られる。

どの証明もこの二通りで作られるので、これで全場合を扱った。一つ前の証明に帰納法の仮定を使う操作を、次のコードでは再帰呼び出しで書く。

```
def succLeSuccByMatch (n m : Nat) (h : n ≤ m) : n + 1 ≤ m + 1 :=  
  match h with  
  | Nat.le.refl => Nat.le.refl  
  | Nat.le.step h' => Nat.le.step (succLeSuccByMatch n _ h')
```

`step` の枝では、 h' は $n \leq k$ の証明、もとの右辺は $k + 1$ である。再帰呼び出しで $n + 1 \leq k + 1$ を得て、`step` で右辺をもう一つ増やす。不等式の証明の構造についての帰納法を、そのまま再帰で書いている。

⇒ 練習

46. $1 \leq 3$ の証明を `Nat.le.refl` と `Nat.le.step` だけで作れ。次に、それを `succLeSuccByMatch` に渡して得られる証明の型を答えよ。

解答 46

```
def one_le_three : 1 ≤ 3 := Nat.le.step (Nat.le.step Nat.le.refl)  
  
def two_le_four_again : 2 ≤ 4 := succLeSuccByMatch 1 3 one_le_three
```

$1 \leq 1$ から二段進む。両辺に一つ加えると、結果の型は $2 \leq 4$ である。

6. 練習 — 章末問題

構成子を使って項を作ることと、`match` で使うことを組み合わせよう。本体を書く前に、受け取る項と返す項の型を確認する。

47. 族 $B : A \rightarrow \text{Type}$ と依存関数 $f : (a : A) \rightarrow B\ a$ がある。各 a に組 $(a, f\ a)$ を対応させる `graphOf` を、`FamilyPair` の構成子で定義せよ。

解答 47

```
def graphOf (A : Type) (B : A → Type) (f : (a : A) → B a) :  
  A → FamilyPair A B := fun a => FamilyPair.mk a (f a)
```

a を渡した後、構成子の第二引数に必要なのは $B\ a$ の項である。関数適用で得た $f\ a : B\ a$ が、その型を持っている。

48. `leAddRight :  $\forall\ n\ k : \text{Nat}, n \leq n + k$` を、 k に関する再帰で証明せよ。零の場合は `Nat.le.refl`、次の数の場合は再帰呼び出しと `Nat.le.step` を使う。

解答 48

```
def leAddRight :  $\forall\ n\ k : \text{Nat}, n \leq n + k$  := fun n k =>  
  match k with  
  | Nat.zero => Nat.le.refl  
  | Nat.succ j => Nat.le.step (leAddRight n j)
```

次の数の枝では $n + \text{Nat.succ}\ j$ が $\text{Nat.succ}\ (n + j)$ に計算される。そのため、 $n \leq n + j$ の証明に `step` を適用すればよい。

49. 自然数 $n\ m$ と、 $h : n = m$ 、 $hn : 0 \leq n$ を受け取り、 $0 \leq m$ を返す `zeroLeOfEq` を定義せよ。 h を `match` で使い、枝の中で要求される型を説明せよ。

解答 49

```
def zeroLeOfEq (n m : Nat) (h : n = m) (hn : 0 ≤ n) : 0 ≤ m :=  
  match h with  
  | Eq.refl _ => hn
```

`Eq.refl` の枝では結論が $0 \leq n$ に特殊化され、 hn をそのまま返せる。

型と命題 II — かつ・または・否定・存在量化

第2章では、全称量化を証明するときに依存関数を作り、仮定として使うときに適用した。第3章では、構成子で項を作り、`match` で使う仕組みを学んだ。この章では、この二つを組み合わせる数学の証明を読む。

中心となる例は全射の合成である。証明を数学の言葉で書き、どこで対象や証明を受け取り、どこで新しい証明を作ったかを確認する。存在を使うもう一つの例として偶数の和を読み、その後、全単射、二つの場合の扱い、否定へ進む。

1. 存在の証明と全射の合成

「条件を満たす対象が存在する」と示すには、対象の一つ挙げ、その対象が条件を満たすことを示す。選んだ対象を、ここでは**証人**と呼ぶことにする。例えば、自然数で `2` に等しいものが存在することは、次のように証明できる。

```
def exists_eq_two : ∃ n : Nat, n = 2 := Exists.intro 2 rfl
```

`∃` は `\exists` で入力する。`Exists.intro` は存在の証明を作る構成子であり、ここでは証人 `2 : Nat` と、その性質の証明 `rfl : 2 = 2` を受け取っている。

第3章の `(n : Nat) × B n` の項が、`n` と `B n` の項からなる依存する組だったことを思い出そう。今度は `B n` の役割を命題 `n = 2` が担い、第二成分としてその命題の証明を渡す。存在の証明も、**対象と、その対象についての保証からなる依存する組**として読める。これは項の作り方の対応であり、Lean の `Exists` と `Sigma` が同じ型だという意味ではない。

任意の自然数の二倍は偶数

自然数 `n` が偶数であることを、`n = 2 * k` となる自然数 `k` が存在することとして定義する。

```
def IsEven (n : Nat) : Prop := ∃ k : Nat, n = 2 * k
```

任意の `n` に対して `2 * n` は偶数である。証人に `n` 自身を選べばよい。

```
def even_two_mul : ∀ n : Nat, IsEven (2 * n) :=  
  fun n => Exists.intro n rfl
```

外側の `fun n => ...` は全称量化の証明を作る。本体の `Exists.intro n rfl` は、受け取った `n` に対する存在の証明を作る。根拠に必要な型は `2 * n = 2 * n` なので、`rfl` が使える。

n + n の場合に必要な等式

`n + n` が偶数であることも、証人に `n` を選んで示せる。今度は根拠として `n + n = 2 * n` の証明が必要である。

```
#check Nat.two_mul
```

```
Nat.two_mul (n : Nat) : 2 * n = n + n
```

```
def even_double : ∀ n : Nat, IsEven (n + n) :=  
  fun n => Exists.intro n (Eq.symm (Nat.two_mul n))
```

`Nat.two_mul n` の等式の向きを `Eq.symm` で逆にして渡した。構成子を適用するときにも、その引数に要求される型を読む手順は変わらない。この算術の定理を再帰から証明する説明は、[03_InductiveTypes.lean 4節](#)にある。

全射どうしの合成は全射

$f : A \rightarrow B$ と $g : B \rightarrow C$ が全射なら、 $g \circ f$ も全射である。まず、数学の証明を読もう。

証明。 $c \in C$ を任意に取る。 g の全射性から、 $g(b) = c$ となる $b \in B$ を取れる。さらに f の全射性から、 $f(a) = b$ となる $a \in A$ を取れる。すると $g(f(a)) = g(b) = c$ である。よって、 c に送られる元として a を取ればよい。■

対象を選ぶ順序に注目しよう。最初に `c` を受け取り、それから `b`、さらに `a` を使っている。二つ目の存在を使う際には、一つ目から得た `b` が必要である。

全射性という保証に名前を付ける

全射であるとは、行き先のどの対象にも、それに送られる元があることである。`f` と `g` の全射性の証明を、それぞれ `hf`、`hg` と名付ける。

```
hf : ∀ b : B, ∃ a : A, f a = b  
hg : ∀ c : C, ∃ b : B, g b = c
```

結論の型は、次のようになる。

```
∀ c : C, ∃ a : A, g (f a) = c
```

外側の全称量化を依存関数型の記法に戻すと、同じ型を次のようにも書ける。

```
hf : (b : B) → (∃ a : A, f a = b)
hg : (c : C) → (∃ b : B, g b = c)
結論 : (c : C) → (∃ a : A, g (f a) = c)
```

受け取る `b` や `c` に応じて、返す存在命題が変わる。外側の全称量化には `fun c => ...` で答える。その本体では、証人 `a` と、`g (f a) = c` の証明を構成子に渡せばよい。

存在の証明から、証人と根拠を使う

`hg` に `c` を適用すると、`hg c : ∃ b : B, g b = c` が得られる。存在の構成子に応じて `match` し、証人と根拠に名前を付ける。

```
match hg c with
| Exists.intro b hb => ...
```

この枝では `b : B` と `hb : g b = c` が使える。次に `hf b` を適用し、その存在の証明を同じように `match` すると、`a : A` と `ha : f a = b` が使える。

ここで選んだ `a` に対して結論の等式を証明し、存在の構成子で包み直す。 `match` の枝から返すのは、最初に求められた存在命題の証明である。

二つの存在を使って、結論の存在を作る

数学の証明と同じ順に、関数適用・場合分け・構成子を組み合わせる。

```
def comp_surjective (A B C : Type) (f : A → B) (g : B → C)
  (hf : ∀ b : B, ∃ a : A, f a = b)
  (hg : ∀ c : C, ∃ b : B, g b = c) :
  ∀ c : C, ∃ a : A, g (f a) = c :=
fun c =>
  match hg c with
  | Exists.intro b hb =>
    match hf b with
    | Exists.intro a ha =>
      Exists.intro a (Eq.trans (congrArg g ha) hb)
```

`hf b` は `b` を取り出した枝の中で使う。そのため `match` も入れ子になる。

最後の構成子に何を渡したか

最後の枝では `ha : f a = b` と `hb : g b = c` が使える。まず、第2章で学んだ等式の操作の型を追う。

```
congrArg g ha : g (f a) = g b
Eq.trans (congrArg g ha) hb : g (f a) = c
```

証人を `a` と決めると、`Exists.intro` の第二引数に必要なのは `g (f a) = c` の証明である。いま作った項がその型を持つので、構成子を適用できる。

```
Exists.intro a (Eq.trans (congrArg g ha) hb) :
  ∃ a : A, g (f a) = c
```

この本体をどの `c` に対しても返すので、外側の `fun` が要求された全称命題の証明になる。

全称量化と存在量化の二つの向き

全射の合成で使った四つの操作を、ここで整理しよう。

命題 (型)	その証明 (項)	作るとき	使うとき
$\forall x : A, P$ <code>x</code>	各 <code>x</code> に <code>P x</code> の項を返す依存関数	<code>fun x => ...</code>	対象を渡して関数適用する
$\exists x : A, P$ <code>x</code>	一つの <code>x</code> と、その <code>P x</code> の証明の組	<code>Exists.intro x</code> <code>h</code>	<code>match</code> で証人と根拠を使う

全称の証明は、型の族の各型に項を返す依存関数である。存在の証明は、依存和の組と対応する形で、一つの対象とその対象についての証明を構成子でまとめる。

全称を使うときは「この対象についての保証を取り出す」。存在を使うときは「保証された対象を一つと、その根拠を使って、続きの証明をする」。どちらも、持っている証明の型に応じた使い方である。

🔗 練習

50. $f : A \rightarrow B$ が右逆 $g : B \rightarrow A$ を持つ、すなわち $hfg : \forall b : B, f (g b) = b$ があるとする。
 $\forall b : B, \exists a : A, f a = b$ を証明する関数 `surjectiveOfRightInverse` を書け。証人には何を選ぶか。

解答 50

```
def surjectiveOfRightInverse (A B : Type) (f : A → B) (g : B → A)
  (hfg : ∀ b : B, f (g b) = b) :
  ∀ b : B, ∃ a : A, f a = b :=
  fun b => Exists.intro (g b) (hfg b)
```

b を受け取り、証人に $g\ b$ を選ぶ。その根拠に要求される型は $f\ (g\ b) = b$ であり、 $hfg\ b$ が与える。

偶数と偶数の和は偶数

証明。 n, m を偶数とする。 $n = 2k$ 、 $m = 2l$ となる自然数 k, l を取れる。すると $n + m = 2k + 2l = 2(k + l)$ だから、 $n + m$ は偶数である。■

仮定に `hn : IsEven n`、`hm : IsEven m` と名前を付ける。まず一つ目の存在の証明を使う。

```
match hn with
| Exists.intro k hk => ...
```

この枝では `k : Nat` と `hk : n = 2 * k` が使える。その中で、二つ目の存在の証明も使う。

```
match hm with
| Exists.intro l hl => ...
```

今度は `l : Nat` と `hl : m = 2 * l` も使える。結論の証人には `k + l` を選ぶ。その根拠として必要なのは `n + m = 2 * (k + l)` の証明である。

等式の連鎖を項で書く

数学の式変形 $n + m = 2k + 2l = 2(k + l)$ を、型を確かめながら組み立てる。まず `hk` と `hl` を順に使って、左端から真ん中への等式を作る。

```
congrArg (fun x => x + m) hk : n + m = 2 * k + m
congrArg (fun x => 2 * k + x) hl : 2 * k + m = 2 * k + 2 * l
```

この二つを `Eq.trans` でつなぐと、`n + m = 2 * k + 2 * l` の証明になる。次に、分配法則の証明として標準環境に用意されている `Nat.mul_add` を使う。

```
#check Nat.mul_add
```

```
Nat.mul_add (n m k : Nat) : n * (m + k) = n * m + n * k
```

この定理も関数として使う。`2`、`k`、`l` を渡すと `2 * (k + l) = 2 * k + 2 * l` の証明が得られる。その向きを `Eq.symm` で逆にし、先ほどの等式に `Eq.trans` でつなぐ。これで数学の等式の連鎖全体が、一つの証明の項になる。

偶数の和の証明を完成させる

証人 `k + l` と、組み立てた等式の証明を `Exists.intro` に渡そう。

```
def isEven_add {n m : Nat} (hn : IsEven n) (hm : IsEven m) : IsEven (n + m) :=
  match hn with
  | Exists.intro k hk =>
    match hm with
    | Exists.intro l hl =>
      Exists.intro (k + l)
      (Eq.trans
        (Eq.trans (congrArg (fun x => x + m) hk)
          (congrArg (fun x => 2 * k + x) hl))
        (Eq.symm (Nat.mul_add 2 k l)))
```

二つの `match` で仮定の存在を使い、最後の `Exists.intro` で結論の存在を作った。構成子の第二引数は長い項だが、部分項の型を追えば、必要な `n + m = 2 * (k + l)` の証明だと確かめられる。

⇒ 練習

51. 任意の自然数 `n` について、`IsEven n → IsEven (n + 2)` を証明せよ。 `2` の偶数性を `Exists.intro` で示し、本文の `isEven_add` を使うこと。

解答 51

```
def isEven_add_two : ∀ n : Nat, IsEven n → IsEven (n + 2) :=
  fun _ hn => isEven_add hn (Exists.intro 1 rfl)
```

`2 = 2 * 1` は計算で一致するので、証人 `1` と `rfl` で `IsEven 2` が示せる。偶数の和についての関数に二つの証明を渡せばよい。

2. かつ — 全単射の合成

写像が全単射であるとは、単射であり、かつ全射であることである。この二つを一緒に保証することを、証明の組として表そう。

```
def IsBijective (A B : Type) (f : A → B) : Prop :=
  (∀ x y : A, f x = f y → x = y) ∧ (∀ b : B, ∃ a : A, f a = b)
```

`∧` は `\and` で入力する。「かつ」の構成子は `And.intro` で、左側と右側の証明を両方受け取る。

全単射性を示すには、単射性と全射性を示して `And.intro` に渡す。全単射性を**使う**には、`match` の `And.intro h_inj h_surj` の枝で、二つの証明に名前を付ける。

全単射どうしの合成は全単射

証明。 f と g が全単射なら、どちらも単射なので $g \circ f$ は単射である。また、どちらも全射なので $g \circ f$ は全射である。よって、合成は全単射である。■

単射性については、第2章と同じように `g` の単射性、次に `f` の単射性を使う。全射性については、この章の `comp_surjective` に必要な仮定を渡す。

`hf` から `hfi` と `hfs`、`hg` から `hgi` と `hgs` を取り出す。`hgi` と `hfi` を順に適用して単射性の証明を作り、全射性の証明と組にする。

全単射の合成の証明項

```
def comp_bijective (A B C : Type) (f : A → B) (g : B → C)
  (hf : IsBijective A B f) (hg : IsBijective B C g) :
  IsBijective A C (fun x => g (f x)) :=
  match hf with
  | And.intro hfi hfs =>
    match hg with
    | And.intro hgi hgs =>
      And.intro (fun x y h => hfi x y (hgi (f x) (f y) h))
        (comp_surjective A B C f g hfs hgs)
```

二つの `match` で仮定を使い、最後の `And.intro` で結論を作った。各成分を取り出す関数 `And.left hf` と `And.right hf` を使う書き方もある。いずれも、二つの証明が入っているという同じ情報を使っている。

⇒ 練習

52. $f : A \rightarrow B$ 、 $g : A \rightarrow C$ と $b : B$ 、 $c : C$ に対して、 $\exists a : A, f a = b \wedge g a = c$ から $\exists a : A, f a = b$ を証明せよ。二種類の構成子を `match` で使い、必要な根拠だけを取り出すこと。

解答 52

```
def existsFirst (A B C : Type) (f : A → B) (g : A → C) (b : B) (c : C)
  (h : ∃ a : A, f a = b ∧ g a = c) : ∃ a : A, f a = b :=
  match h with
  | Exists.intro a habc =>
    match habc with
    | And.intro hab _ => Exists.intro a hab
```

証人 a はそのまま使い、二つの根拠のうち左側の $hab : f\ a = b$ を渡す。

3. または — 二つの場合を扱う

$x = a$ または $x = b$ なら、 $f(x) = f(a)$ または $f(x) = f(b)$ である。これを証明してみよう。`v` は「または」で、`\or` と入力する。

証明。 $x = a$ の場合、等式の両辺に f を適用して $f(x) = f(a)$ を得る。 $x = b$ の場合も同様に $f(x) = f(b)$ を得る。どちらの場合も結論のいずれかが成り立つ。■

「または」を示すには、左の証明を `Or.inl` に渡すか、右の証明を `Or.inr` に渡す。「または」を使うには、どちらの構成子で作られた場合にも答えを用意する。第3章の直和を使う場合分けと同じ形である。

またはの証明を `match` で使う

```
def imageTwoPoints (A B : Type) (f : A → B) (x a b : A)
  (h : x = a ∨ x = b) : f x = f a ∨ f x = f b :=
  match h with
  | Or.inl hxa => Or.inl (congrArg f hxa)
  | Or.inr hxb => Or.inr (congrArg f hxb)
```

左の枝では `hxa : x = a` が使える。`congrArg f hxa : f x = f a` を作り、結論の左側の証明として `Or.inl` に渡す。右の枝では `f x = f b` の証明を `Or.inr` に渡す。

`match` 全体が返す型は、どちらの枝でも同じ `f x = f a ∨ f x = f b` である。

⇒ 練習

53. $f : A \rightarrow B$ を単射とする。 $f\ x = f\ a \vee f\ x = f\ b$ から $x = a \vee x = b$ を示せ。単射性の証明を `hf :  $\forall x\ y : A, f\ x = f\ y \rightarrow x = y$` として受け取ること。

解答 53

```
def preimageTwoPoints (A B : Type) (f : A → B)
  (hf :  $\forall x\ y : A, f\ x = f\ y \rightarrow x = y$ ) (x a b : A)
  (h : f x = f a ∨ f x = f b) : x = a ∨ x = b :=
  match h with
  | Or.inl hxa => Or.inl (hf x a hxa)
  | Or.inr hxb => Or.inr (hf x b hxb)
```

各枝で単射性を適用し、結論の対応する構成子へ証明を渡す。

4. 否定 — 等しいと仮定して矛盾を作る

`False` は、構成子を持たない命題である。第3章で自作した空の型 `MyEmpty` に対応する。`False` の証明を受け取ったとすると、`False.elim` で任意の命題の証明を返せる。

`¬` は `\not` で入力する。「ある命題ではない」とは、その命題の証明を受け取ったら `False` の証明を返せる、という型で表す。例えば $x \neq y$ は $x = y \rightarrow \text{False}$ のことである。

したがって、否定を**示す**ときも関数を作り、否定を**使う**ときも適用する。新しい否定用の構成子を使うのではなく、第2章の「ならば」の読み方を使っている。

単射性と否定を組み合わせる

証明。 f を単射とし、 $x \neq y$ とする。もし $f(x) = f(y)$ なら、単射性から $x = y$ となって仮定に矛盾する。したがって $f(x) \neq f(y)$ である。■

```
def injectivePreservesNe (A B : Type) (f : A → B)
  (hf : ∀ x y : A, f x = f y → x = y)
  (x y : A) (hne : x ≠ y) : f x ≠ f y :=
  fun h => hne (hf x y h)
```

結論の型は $f\ x = f\ y \rightarrow \text{False}$ なので、まず $h : f\ x = f\ y$ を受け取る。 $hf\ x\ y\ h : x = y$ に $hne : x \neq y \rightarrow \text{False}$ を適用すると、本体の型が `False` になる。ここでは二重否定から元の命題を取り出す背理法は使っていない。

⇒ 練習

54. 任意の写像 $f : A \rightarrow B$ について、 $f\ x \neq f\ y$ ならば $x \neq y$ であることを示せ。単射性は仮定しない。 $x = y$ と仮定したとき、`congrArg` で何が得られるか考えよ。

解答 54

```
def neOfImageNe (A B : Type) (f : A → B) (x y : A)
  (hne : f x ≠ f y) : x ≠ y :=
  fun h => hne (congrArg f h)
```

$h : x = y$ から $\text{congrArg}\ f\ h : f\ x = f\ y$ が得られ、それを hne に渡すと `False` が得られる。

帰納型の異なる構成子

第3章で述べた「異なる構成子は等しくない」という性質も、否定の証明として書ける。標準の帰納型 `Bool` に対する `Bool.noConfusion` を使おう。

```
def bool_true_ne_false_by_noConfusion : Bool.true ≠ Bool.false :=
  fun h => Bool.noConfusion h
```

`h : Bool.true = Bool.false` を仮定すると、`Bool.noConfusion h : False` が得られる。この矛盾を返す関数が、不等号 `≠` の証明になっている。この道具を使った別の例を、次の補足で見よう。

補足: 構成子が別々であることと `noConfusion`

`noConfusion` は、帰納型の宣言に伴って作られる道具である。異なる構成子の項は等しくないことと、同じ構成子の項が等しければ中身も等しいことを証明できる。

```
def succ_injective_by_noConfusion {m n : Nat} (h : Nat.succ m = Nat.succ n) : m = n :=
  Nat.noConfusion h (fun h' => h')
```

この例では、中身が等しいという保証 `h' : m = n` を受け取り、そのまま返す関数を渡している。

```
#print axioms Bool.noConfusion
```

```
'Bool.noConfusion' does not depend on any axioms
```

この道具も新たな公理ではなく、帰納型の原理から証明されている。

5. 証明を作る・証明を使う

同じ命題でも、結論として証明する場面と、仮定として使う場面とでは操作が違う。

命題の形	証明するとき	仮定として使うとき
ならば・全称	<code>fun</code> で関数を作る	証明や対象を適用で渡す
存在	<code>Exists.intro</code> で証人と根拠をまとめる	<code>match</code> で証人と根拠を使う
かつ	<code>And.intro</code> で両方の証明をまとめる	<code>match</code> などで両方の証明を使う
または	<code>Or.inl</code> または <code>Or.inr</code> で作る	<code>match</code> で両方の場合を扱う
否定	仮定から <code>False</code> を返す関数を作る	その仮定の証明を適用して <code>False</code> を得る

数学の証明で行う操作を、関数・構成子・場合分けで表した。その読み替えを人間が理解し、完成した項の型を機械が検査する、という第2章の見方につながる。

6. 型検査が証明の検査になる

全称や存在、単射や全射の合成で行った型の照合から、証明検査の意味を整理しよう。

証明は項なのだから、「証明が正しいか」は「項が主張どおりの型を持つか」という **型検査**の問題になる。型検査は機械的な計算であり、Lean はすべての宣言に対してそれを実行している。だから、ファイルがエラーなしに受理された時点で、そこに書かれた全定理の検査は済んでいる。

この主張には但し書きが3つある。第一は、この検査が**宣言済みの公理に相対的**であること。 `axiom bad : 0 = 1` のような宣言もエラーなく受理できてしまうので、「何を公理として使ったか」は `#print axioms` で別に監査する必要がある。第二は、証明を保留する印 `sorry` がエラーではなく**警告**になること（依存すると `#print axioms` に `sorryAx` が現れる）。だから正確には、「エラーがなく、`sorry` の警告もなく、`#print axioms` の結果が納得できる公理だけ」を見る。`06_Topology.lean` の末尾で、主定理に対してこの監査を実行する。第三は、この監査の例を見たあとで述べる。

```
#print axioms comp_surjective
```

```
'comp_surjective' does not depend on any axioms
```

この証明は追加の公理に依存していない。

その検査は、形式化したかったものの検査か

第三の但し書きは、公理や `sorry` の監査を済ませても残る。型検査が確かめるのは「項が主張どおりの型を持つ」ことだけで、**その型が、本当に確かめたかった主張であるか**は型検査の外側にある。問題は3点に分けられる。

1. 形式化の問題。「単射どうしの合成は単射」という気持ちを `$\forall x\ y, g(f\ x) = g(f\ y) \rightarrow x = y$` と書いたとき、式が気持ちを正しく写しているかは、書いた人が判断するほかない。
2. 実装の問題。エラボータやカーネルにバグがあれば、誤った項を受理しうる。カーネルを小さく保ち、独立した検査器で再検査する対策はあるが、ゼロにはならない。
3. 理論の解釈の問題。証明を返す関数の型をなぜ「ならば」や「すべての」と読んでよいのか、型をなぜ集合のように扱ってよいのか。型検査を通った項が一貫した意味を持つ、という仮定がここにある。この仮定が正しければ、矛盾が導けないことも従う。

この教材では以後、これらを認めたうえで「エラーなしに受理された」ことを検査済みと呼ぶ。

証明を考える順序と、出来上がった項を読む順序

証明を考えるときは、まず結論から必要な入力と出力を見る。単射性なら二つの対象と像の等式を受け取り、対象どうしの等式を返す必要がある。その出力を作れる仮定や道具を探し、必要な証明を組み合わせる。

出来上がった項を読むときは、内側の適用から型を追える。単射の合成なら、 $hg(fx)(fy)h : fx = fy$ 、次に $hfxy(hg(fx)(fy)h) : x = y$ である。全射の合成なら、証人 a に対して $g(fa) = c$ の証明を作り、構成子に渡せることを確かめた。どの場合も、部分項の型を求め、次の関数や構成子が要求する入力型と照合している。

一段を飛ばすと型が合わない

単射の合成の本体を $hfxyh$ としてしまうと、受理されない。 $hfxy$ が要求するのは $fx = fy$ の証明だが、 h は $g(fx) = g(fy)$ の証明だからである。紙の証明で「 g の単射性を使う」という一段を飛ばしたことが、引数の型の不一致として検出される。

省略を補う処理と、完成した項の検査

ソースコードから証明の検査までは二段階ある。**エラボレータ**は記法を展開し、暗黙引数や型の情報を補って項を組み立てる。**カーネル**は、その完成した項が指定された型を持つかを検査する。省略をうまく補うことと、出来上がった項が正しいことを確かめることは、役割が異なる。

例えば `Nat.sub_add_cancel` は、自然数の引き算で引く数が大きすぎないという仮定を要する。

```
#check Nat.sub_add_cancel
```

```
Nat.sub_add_cancel {n m : Nat} (h : m ≤ n) : n - m + m = n
```

$n - 1 + 1 = n$ を全自然数で示そうとして、必要な仮定を `_` にするとどうなるか。

```
def sub_one_add_one : ∀ n : Nat, n - 1 + 1 = n :=  
  fun n => Nat.sub_add_cancel _
```

この `_` に必要なのは $1 ≤ n$ の証明であり、エラボレータはそれを補えない。しかも $n = 0$ が反例なので、任意の n に対してこの穴を埋める証明は書けない。一方、その仮定を正しく受け取れば、項を完成させられる。

```
def sub_one_add_one (n : Nat) (h : 1 ≤ n) : n - 1 + 1 = n := Nat.sub_add_cancel h
```

省略を補った後にも、完成した項が宣言の型を持つかが検査される。ただし、その宣言自体が意図した数学の主張を表すかは、人間が読む必要がある。

補足: 理論を解釈する前提

第3点について、Lean の理論（帰納型・宇宙の階層・Prop・後で見る公理を含む）を ZFC に相対的に解釈するモデルは知られている。ただしそのためには、ZFC に到達不能基数を（宇宙の階層の段ごとに1つずつ、可算無限個）加える必要があり、これは ZFC より真に強い仮定である。

7. 練習 — 章末問題

数学の証明を書き、仮定と結論の型を確かめてから、Leanの項にしよう。

55. 三つの全射 $f : A \rightarrow B$ 、 $g : B \rightarrow C$ 、 $k : C \rightarrow D$ の合成が全射であることを証明せよ。本文の `comp_surjective` を二回使う解答を書け。各適用に渡す型と写像を明示すること。

解答 55

```
def threeCompSurjective (A B C D : Type)
  (f : A → B) (g : B → C) (k : C → D)
  (hf : ∀ b, ∃ a, f a = b) (hg : ∀ c, ∃ b, g b = c)
  (hk : ∀ d, ∃ c, k c = d) :
  ∀ d, ∃ a, k (g (f a)) = d :=
  comp_surjective A C D (fun a => g (f a)) k
  (comp_surjective A B C f g hf hg) hk
```

内側の適用は $A \rightarrow C$ の全射性、外側の適用は $A \rightarrow D$ の全射性を証明する。

数学を記述する道具

第3章では、構成子で組を作り、`match` で成分を取り出した。第4章では、同じ仕組みで証明を作ったり使ったりした。この章では、その上に、数学の対象を短く読みやすく書くための道具を加える。

まず、名前の付いた成分をまとめる `structure` と、構成子の略記 `(...)` を扱う。次に、値に証明を添える部分型を、`Fin` の例と結び付けて読む。そこから `class` と `instance`、記法の自作、集合 `Set`、名前空間へ進み、第6章で位相空間を記述するための準備をする。

1. structure — フィールドを持つ型

第3章の `MyPair` は、構成子を一つ持つ帰納型だった。その成分を取り出す `pairFirst` は、`match` の一つの枝で定義した。自然数を二つまとめる型でも、同じように書ける。

```
inductive MyPoint : Type where
  | mk (x y : Nat) : MyPoint

def MyPoint.x : MyPoint → Nat := fun p =>
  match p with
  | MyPoint.mk a _ => a
```

この「構成子一つの帰納型」と「成分を取り出す関数」をまとめて宣言する構文が `structure` である。名前の付いた成分を **フィールド** と呼ぶ。自然数の組を、二つのフィールド `x · y` を持つ型として定義しよう。

```
structure Point : Type where
  x : Nat
  y : Nat
```

`Point` は新しく定義した型であり、先ほどの `MyPoint` とは別の型である。持つデータの形は同じだが、こちらでは構成子と取り出し関数が自動で用意される。

構成子で作り、フィールドを取り出す

構成子 `Point.mk` は二つの自然数を順に受け取る。取り出し関数 `Point.x` は、できた `Point` の項から第一成分を返す。

```
#check Point.mk
#check Point.x
```

```
Point.mk (x y : Nat) : Point
Point.x (self : Point) : Nat
```

矢印形式なら、それぞれ `Nat → Nat → Point` と `Point → Nat` である。 `Point.y : Point → Nat` も同時に定義される。

```
#eval Point.x (Point.mk 1 2)
```

```
1
```

二つの値を渡して作り、そのうち一つを取り出した。構成子と `match` で行っていた操作を、名前の付いた関数で書けるようになった。

ドット記法で関数を適用する

`p : Point` に対して、`Point.x p` は `p.x` とも書ける。Lean は `p` の型から `Point.x` を見つけ、`p` を引数として渡す。

```
#eval (Point.mk 1 2).x
```

```
1
```

したがって `.x` は値を取り出す別の原理ではなく、関数適用の略記である。番号で成分を指定する代わりに、`x · y` のような名前を使って読む。

匿名構成子 — 期待される型から構成子を決める

期待される型が `Point` と分かる位置では、`Point.mk 1 2` を `(1, 2)` と書ける。これを **匿名構成子記法** という。`{ · }` は `\< · \>` で入力できる。

```
def pointFromPair : Point := (1, 2)
def pointFromDot : Point := .mk 1 2

#check ((1, 2) : Point)
```

```
{ x := 1, y := 2 } : Point
```

どちらの定義の右辺も `Point.mk 1 2` と読まれる。`(...)` は期待される型から構成子を選び、`.mk` は期待される型から名前の接頭辞を補う。第1章で見た、周囲から型を伝える仕組みを使っている。

フィールド名を付けて書く

表示された `{ x := 1, y := 2 }` は、フィールド名を付けて構造体の項を書く記法である。これを入力にすることもできる。次の三つは、同じ構成子の適用を表している。

```
def point_pair_eq_mk : ((1, 2) : Point) = Point.mk 1 2 := rfl
def point_fields_eq_mk : ({ x := 1, y := 2 } : Point) = Point.mk 1 2 := rfl
```

匿名構成子記法は `structure` 専用ではない。構成子が一つの型なら、第4章の「かつ」や存在の証明でも使える。

```
def exists_two : ∃ n : Nat, n = 2 := ⟨2, rfl⟩
```

これは `Exists.intro 2 rfl` の略記であり、証人とその根拠を渡す仕組みは同じである。略記を見たら、期待される型と、補われる構成子を確認しよう。

自分で定義した関数もドットで使える

`Point` の項を受け取る関数を `Point.~` という名前で定義すれば、取り出し関数と同じようにドット記法で使える。

```
def Point.swap (p : Point) : Point := ⟨p.y, p.x⟩

#eval (Point.mk 1 2).swap.x
```

2

`p.swap.x` は `Point.x (Point.swap p)` の略記である。二つの成分を交換してから、第一成分を取り出している。この名前の付け方を支える名前空間は、[7節](#)で扱う。

⇒ 練習

56. 型 `A` と二つの写像 `f g : A → Nat` から、各 `a` を組 `(f a, g a)` に送る `pairAt (A : Type) (f g : A → Nat) : A → Point` を定義せよ。 `A = Nat`、`f n = n + 1`、`g n = 2 * n` とし、3 を渡した結果の `x · y` を `#eval` で確かめよ。これは直積へ向かう写像を、成分の写像から作る操作である。

解答 56

```
def pairAt (A : Type) (f g : A → Nat) : A → Point :=
  fun a => Point.mk (f a) (g a)
```

```
#eval (pairAt Nat (fun n => n + 1) (fun n => 2 * n) 3).x
#eval (pairAt Nat (fun n => n + 1) (fun n => 2 * n) 3).y
```

```
4
6
```

同じ入力 3 に `f` と `g` を適用し、それぞれの結果を成分としてまとめている。本体は `(f a, g a)` と書いてもよい。

型をパラメータにする structure

フィールドの型を固定せず、パラメータで受け取ることもできる。第3章の `MyPair A B` と同じ形の組を、`structure` で定義しよう。

```
structure Pair (α β : Type) : Type where
  fst : α
  snd : β

#check Pair.mk
#check Pair.fst
```

```
Pair.mk {α β : Type} (fst : α) (snd : β) : Pair α β
Pair.fst {α β : Type} (self : Pair α β) : α
```

構成子の `α · β` は暗黙引数なので、普通は渡す成分から推測される。自作の `Pair` と `MyPair` は別の型だが、同じ形のデータを持つ。

二つの異なる型の項をまとめる

```
def natBoolPair : Pair Nat Bool := Pair.mk 3 true

#eval natBoolPair.fst
#eval natBoolPair.snd
```

```
3
true
```

匿名構成子なら、この定義の右辺を `(3, true)` と書ける。標準の直積 `α × β`、すなわち `Prod α β` も、同じ形の構造体である。 `Prod.mk` で作り、 `Prod.fst · Prod.snd` (または `.fst · .snd`) で取り出せ

る。

練習

57. `center : Point`、`radius : Nat` を持つ構造体 `Circle` を定義せよ。`makeCircle : Point → Nat → Circle` を構成子名と `{...}` の両方で書き、点 `(1,2)`、半径 `3` の円の `center.x` と `radius` を `#eval` で確かめよ。

解答 57

```
structure Circle : Type where
  center : Point
  radius : Nat

def makeCircle : Point → Nat → Circle := fun p n => Circle.mk p n
```

本体を `fun p n => {p, n}` と書いてもよい。期待される型 `Circle` から、構成子 `Circle.mk` が選ばれる。

```
#eval (makeCircle (Point.mk 1 2) 3).center.x
#eval (makeCircle (Point.mk 1 2) 3).radius
```

```
1
3
```

`center : Point` を取り出してから、その `x : Nat` を取り出している。

2. 依存するフィールド

第3章の `Numbered` は、`n : Nat` と、その `n` に応じた `Fin n` の項を持っていた。`structure` でも、後のフィールドの型に前のフィールドを使える。今度は、型そのものと、その型の項を一つまとめよう。

```
structure PointedType : Type 1 where
  carrier : Type
  point : carrier

#check PointedType.mk
```

```
PointedType.mk (carrier : Type) (point : carrier) : PointedType
```

これは点付き集合 (A, a) に対応する。最初に `carrier` を決めると、次に渡す `point` の型が決まる。`Type` 自体をフィールドに持つので、全体の型は `Type 1` に置いている。

点付き集合を作る

```
def pointedNat : PointedType := ⟨Nat, 0⟩
def pointedBool : PointedType := ⟨Bool, true⟩

#check PointedType.mk Nat
```

```
PointedType.mk Nat : Nat → PointedType
```

第一引数に `Nat` を渡すと、残りの引数は `Nat` の項になった。`⟨Nat, 0⟩` は `PointedType.mk Nat 0`、`⟨Bool, true⟩` は `PointedType.mk Bool true` の略記である。

取り出す項の型も入力に依存する

```
#check PointedType.carrier
#check PointedType.point
```

```
PointedType.carrier (self : PointedType) : Type
PointedType.point (self : PointedType) : self.carrier
```

`p : PointedType` に対して、`p.carrier` は型であり、`p.point` はその型の項である。

`PointedType.point` の結果の型は入力 `p` によって変わる。第3章の `Numbered.index` と同じ依存関数だが、自動で用意されたのである。

```
#reduce pointedNat.point
#reduce pointedBool.point
```

```
0
true
```

⇒ 練習

58. 点付き集合 (A, a) と写像 $f : A \rightarrow A$ から $(A, f(a))$ を作る `mapPointed (p : PointedType) (f : p.carrier → p.carrier) : PointedType` を書け。 `#reduce (mapPointed pointedNat Nat.succ).point` の結果を予想して確かめよ。

解答 58

```
def mapPointed (p : PointedType) (f : p.carrier → p.carrier) : PointedType :=
  PointedType.mk p.carrier (f p.point)

#reduce (mapPointed pointedNat Nat.succ).point
```

1

`p.point : p.carrier` を `f` に渡し、同じ台の型と新しい点を組にして返す。構成子の第二引数に要求される型は、第一引数の `p.carrier` によって決まる。

3. 部分型 — 値と証明の組

後のフィールドの型が前の値に依存するとき、その型が命題でもよい。例えば「偶数である自然数」を持ち歩きたければ、自然数 `n` と `IsEven n` の証明を一緒に持てばよい。こうした値と証明の組を扱うのが **部分型** である。

標準の `Subtype` の定義は、宇宙の指定を省けば次の形になっている。

```
structure Subtype {α : Type} (p : α → Prop) where
  val : α
  property : p val
```

述語 `p` はパラメータであり、値 `val` を決めると、必要な証明の型 `p val` が決まる。 `Subtype p` は `{x // p x}` とも書く。

偶数という条件を付けた型

この章でも単独で使えるよう、偶数性の定義を再掲する。

```
def IsEven (n : Nat) : Prop := ∃ k : Nat, n = 2 * k

def EvenNat : Type := {n : Nat // IsEven n}

def evenFour : EvenNat := Subtype.mk 4 (Exists.intro 2 rfl)

#check evenFour.val
#check evenFour.property
```

```
evenFour.val : Nat
evenFour.property : IsEven evenFour.val
```

`evenFour` は「自然数 `4`」と「`4` は偶数だという証明」の組である。内側の `Exists.intro 2 rfl` は、第4章で学んだ偶数性の証明を作っている。

値と、値についての保証を取り出す

```
#eval evenFour.val
```

```
4
```

`evenFour.property` を使えば、取り出した自然数の偶数性も使える。この具体的な値の和については、証人を `4` として計算で偶数性を確かめられる。

```
def evenFourPlusFour : IsEven (evenFour.val + evenFour.val) :=
  Exists.intro 4 rfl
```

`EvenNat` と `Nat` は別の型である。`evenFour : EvenNat` から自然数を使いたいときは `.val` で取り出す。数学で部分集合を考えるイメージと、Lean の型の区別を分けて読もう。

存在の証明との違い

`∃ n : Nat, IsEven n` の証明も、自然数とその偶数性の証明から作れる。どちらも二つの成分を使うが、全体の型が違う。

- `EvenNat : Type` は、条件を満たす値をデータとして使うための型である。`.val` で自然数を取り出し、計算に使える。
- `(∃ n : Nat, IsEven n) : Prop` は、そのような値が存在するという命題である。第4章のように `match` で証人を使って別の命題を証明できるが、一般にはその証明を場合分けして自然数を返す関数は定義できない。

条件付きの値を計算に渡したいのか、存在することを証明したいのかに応じて使い分ける。

Fin も値に範囲の証明を添えている

第1章から使ってきた `Fin n` の定義は、次の構造体である。

```
structure Fin (n : Nat) where
  val : Nat
```

```
isLt : val < n
```

`Fin n` も「値と、その値についての証明」を持つ。`Subtype` の単なる別名ではなく、同じ考え方で個別に定義された型である。`i : Fin n` からは、`i.val : Nat` と `i.isLt : i.val < n` を取り出せる。

不等式の証明を渡して番号を作る

第3章で見たように、自然数の不等式も証明の型である。ここでは、標準ライブラリの `Nat.lt_succ_self` が `n < n + 1` の証明を与える。

```
#check Nat.lt_succ_self
```

```
Nat.lt_succ_self (n : Nat) : n < n.succ
```

`n.succ` と `n + 1` は計算上等しいので、この証明を `Fin.mk` に渡せる。

```
def lastFromProof : (n : Nat) → Fin (n + 1) :=  
  fun n => Fin.mk n (Nat.lt_succ_self n)
```

```
#check lastFromProof 2  
#eval (lastFromProof 2).val
```

```
lastFromProof 2 : Fin (2 + 1)  
2
```

第1章の依存関数を、構成子まで読む

`lastFromProof` の本体は `(n, Nat.lt_succ_self n)` と略記することもできる。自然数 `n` を受け取り、値 `n` と範囲の証明から `Fin (n + 1)` の項を作っている。第1章でも使った標準の `Fin.last` と同じ「最後の番号」を、今度は中身まで書いた。

```
def lastFromProof_val (n : Nat) : (lastFromProof n).val = n := rfl  
def lastFromProof_eq_last (n : Nat) : lastFromProof n = Fin.last n := rfl
```

入力に応じて結果の型が変わるので、これは依存関数である。その結果として返す `Fin (n + 1)` の項の内部には、値とその値の保証が入っている。型の族に沿って項を与える話と、依存する組の話が、ここでつながった。

補足: Fin の数値リテラル

Fin 3 の項は、自然数の値と、その値が3未満である証明の組である。Nat の項とは別の型の項だが、数字は期待される型に応じて読まれるので、(2 : Fin 3) と書けば Fin 3 の「2番」の項になる。さらに、(5 : Fin 3) も受理される。

```
#eval (5 : Fin 3)
```

```
2
```

正の n に対して、Fin n の数値リテラルは、 n で割った余りとして読まれる。ここでは5を3で割った余りの2が、範囲の証明とともに Fin 3 の項になる。

第1章の章末問題の `repeatTuple (n : Nat) (a : Nat) : Tuple n` でも同じことが起きる。

`repeatTuple 3 4 5` と書くと、最初の3が個数、次の4が各成分の値であり、最後に渡す番号の型は Fin 3 である。したがって数字の5は「2番」として読まれ、その成分の値4が返る。

一方、既に Nat 型と指定した (5 : Nat) を、この番号の引数として渡すと型エラーになる。数字を Fin 3 の項として読めることと、Nat 型の項をそのまま渡せることは別である。数値リテラルの読み方を決める仕組みは、4節の補足で説明する。

⇒ 練習

59. 二つの偶数の和を、偶数という条件を持つ値として返す `addEven (n m : EvenNat) : EvenNat` を書け。ヒント：値は `n.val + m.val` とする。`n.property` と `m.property` を `match` で分解し、得られた証人の和を新しい証人にする。第4章の偶数の和と同じ等式の計算を組み立てよ。 `#eval (addEven evenFour evenFour).val` を確かめよ。

解答 59

```
def addEven (n m : EvenNat) : EvenNat :=
  Subtype.mk (n.val + m.val)
  (match n.property with
  | Exists.intro k hk =>
    match m.property with
    | Exists.intro l hl =>
      Exists.intro (k + l)
      (Eq.trans
        (Eq.trans (congrArg (fun x => x + m.val) hk)
          (congrArg (fun x => 2 * k + x) hl))
        (Eq.symm (Nat.mul_add 2 k l))))
```

```
#eval (addEven evenFour evenFour).val
```

```
8
```

第一成分に和を置くと、第二成分には `IsEven (n.val + m.val)` の証明が要求される。二つの偶数性の証明を分解し、証人の和と等式の証明を組み立てて、この型の項を与えた。

4. class と instance

ここまでは構造体の項を明示して渡していた。`class` を使うと、型ごとに決めた項を `instance` として登録し、必要な場所で Lean に探してもらえる。宣言の形は `structure` と同じである。

2節の `PointedType` は、型とその中の点を一つに束ねた。今度は型 α をパラメータにし、その中の点だけをフィールドにする。

```
class Pointed ( $\alpha$  : Type) : Type where
  point :  $\alpha$ 

#check Pointed
```

```
Pointed ( $\alpha$  : Type) : Type
```

`carrier` がパラメータ α に移り、フィールドは点だけになった。パラメータを取ることは `structure` でもできる。`class` にした意味は、この項を登録して、必要な場所へ自動で渡してもらえることにある。

登録は `instance` 宣言で行う:

```
instance : Pointed Nat where
  point := 0
```

`Pointed  $\alpha$` の項は「 α のどの項を基点と呼ぶかの指定」で、`instance` 宣言によって `Nat` の基点として `0` を登録した。

`instance` は、項を定義すると同時に登録簿へ載せる宣言である。使う側は登録簿から探してもらえるので、宣言の名前を省略できる。実際には Lean が `instPointedNat` のような名前を自動で付けており、`instance myPoint : Pointed Nat where ...` と自分で名前を付けることもできる。

登録簿から探す操作そのものを項として書いたのが `inferInstance` である。型を見ると、インスタンス引数 `[i :  $\alpha$ ]` を受け取ってそのまま返すだけの関数で、「探す」仕事は括弧 `[]` の仕組みがやっていること

が分かる。

これで括弧が3種類そろった。01_TypesAndTerms.lean 4節の () (明示)・{ } (暗黙)に加えて、[inst : C α] が**インスタンス引数**である——他の引数との単一化だけでは決まらず、エラボレータが instance として登録された項の中から探索して埋める。

```
#check @inferInstance
```

```
@inferInstance : {α : Sort u_1} → [i : α] → α
```

頭の @ は「暗黙引数も省略せずに表示・指定する」という印で、@inferInstance は inferInstance の省略なし版である。Sort u は、第3章で Eq の型を読むときに見た、Prop や Type、Type 1 などをもとめて扱う宇宙の表記である。つまりこの関数は、命題も通常の型も対象にできる。

確かめてみる。inferInstance に型 Pointed Nat を指定して、#check で調べる。これは登録済みのインスタンスを探す項であり、新しいインスタンスの登録は行わない。

```
#check (inferInstance : Pointed Nat)
```

```
inferInstance : Pointed Nat
```

Nat には基点が登録してあるので、対応する項が見つかり、型が確認できる。登録していない型では失敗する:

```
#check (inferInstance : Pointed Bool)
```

```
error(lean.synthInstanceFailed): failed to synthesize instance of type class
Pointed Bool
```

インスタンス引数の使いどころ: 「基点が登録されたどんな型でも」働く関数を書ける。pointPair Nat と書くだけで、Pointed Nat の項は登録簿から自動で渡される。

```
def pointPair (α : Type) [Pointed α] : Pair α α := (Pointed.point, Pointed.point)
```

```
#check pointPair
```

```
pointPair (α : Type) [Pointed α] : Pair α α
```

(pointPair Nat).fst の値は、登録簿に載せた point ——つまり 0 ——のはずである:

```
#eval (pointPair Nat).fst
```

```
0
```

自作型にもインスタンスを与える

クラスの効き目は、あとから自分の型を仲間に入れられることにある。1節で作った `Point` に基点（原点）を登録してみる。

```
instance : Pointed Point where  
  point := (0, 0)
```

```
#check pointPair Point
```

```
pointPair Point : Pair Point Point
```

登録した瞬間から、`Pointed` を使う汎用の道具がすべて `Point` でも使えるようになった。

```
#eval (pointPair Point).fst.x
```

```
0
```

インスタンスを受け取る定理

インスタンス引数は、証明を定義する `def` にも書ける。「基点が登録されたどんな型でも成り立つ」一般的な定理を証明してみよう：

```
def pointPair_fst ( $\alpha$  : Type) [Pointed  $\alpha$ ] :  
  (pointPair  $\alpha$ ).fst = Pointed.point := rfl
```

```
#check pointPair_fst
```

```
pointPair_fst ( $\alpha$  : Type) [Pointed  $\alpha$ ] : (pointPair  $\alpha$ ).fst = Pointed.point
```

使うときは `$\alpha$` を指定するだけでよく、基点の指定である `Pointed  $\alpha$` の項は登録簿から自動で供給される。

```
def pointPair_nat_fst : (pointPair Nat).fst = Pointed.point := pointPair_fst Nat
def pointPair_point_fst : (pointPair Point).fst = Pointed.point := pointPair_fst Point
```

積に構造を誘導する

登録簿のもう1つの効き目は、**インスタンスからインスタンスを作れる**ことである。基点付きの型が2つあれば、その積にも「基点の組」という自然な基点が決まる。この「積への構造の誘導」も、前提にインスタンス引数を取る instance として登録できる:

```
instance {α β : Type} [Pointed α] [Pointed β] : Pointed (Pair α β) where
  point := ⟨Pointed.point, Pointed.point⟩

#eval (Pointed.point : Pair Nat Point).snd.x
```

0

`Pair Nat Point` の基点はどこにも直接は登録していないのに、見つかった。探索が**連鎖**するからである: `Pointed (Pair Nat Point)` を探す → 上の誘導 instance が形に合う → その前提 `Pointed Nat` ・ `Pointed Point` をさらに登録簿から探す——と、機械が自動でつないでいく。`07_Exercises.lean` の発展演習で「積空間 $X \times Y$ に位相が自動で載る」のも、これと同じ仕組みである。

記法もクラスで動いている

`01_TypesAndTerms.lean` 3節の先取りで、`+` は `HAdd.hAdd` を使う記法であり、型に応じて演算が選ばれと述べた。同じ型どうしの足し算を指定するために、標準ライブラリにはクラス `Add` が用意されている:

```
class Add (α : Type u) where
  add : α → α → α
```

`Point` に足し算を登録してみる:

```
instance : Add Point where
  add p q := ⟨p.x + q.x, p.y + q.y⟩

#check Point.mk 1 2 + Point.mk 3 4
```

```
{ x := 1, y := 2 } + { x := 3, y := 4 } : Point
```

表示の `{ x := 1, y := 2 }` は `Point.mk 1 2` の別表示である（フィールド名付きの structure リテラル。書くときにも使える）。登録した瞬間から、記法 `+` が `Point` でも通るようになった。`x` 成分は `1 + 3` で `4` のはずである:

```
#eval (Point.mk 1 2 + Point.mk 3 4).x
```

4

補足: `+` と数値リテラルの登録簿

正確に言うと、`+` の読み先は、左右の型が違ってよいクラス `HAdd` の関数 `HAdd.hAdd` である。

「`Add α` があれば `HAdd α α α` にもなる」という橋渡しのインスタンスが標準ライブラリに用意されているので、`Add` を登録するだけで記法まで使えるようになる。登録簿の検索は、このように連鎖する。自然数の場合も、この連鎖の先に `Nat.add` がある。`Nat.add` と、`Nat → Nat → Nat` と注釈した `HAdd.hAdd` の型が同じだから同じ関数になるのではない。自然数用のインスタンスが `Nat.add` を指定しているため、同じ計算になる。

数字のリテラルにも同じ仕組みがある。数字 `2` は `OfNat.ofNat 2` の略記で、クラス `OfNat` の instance が、期待される型ごとに読み方を決めている。[01_TypesAndTerms.lean 3節](#)で `2` が `Nat` とも `Int` とも読まれたのも、[3節](#)の補足で `(5 : Fin 3)` が `3` で割った余りの `2` と読まれたのも、`Nat · Int · Fin n` それぞれの `OfNat` の instance がそう実装されているからである。「記法はクラスで動く」という、この節の主題の一例である。

なお、この教材の `Pointed` は練習用の自作クラスである。標準ライブラリにも同じ形のクラス `Inhabited`（フィールド名は `default`）があり、「少なくとも1つ項を持つ型」の既定値として広く使われている。

補足: 命題の `#eval` と `def` の展開

命題の中には、真偽を計算して判定できるものがある。例えば次は `true` と表示される。

```
#eval 2 < 3
```

```
true
```

このとき `#eval` は、`2 < 3` の真偽を判定するためのインスタンスを探し、その判定を実行している。そのインスタンスの型は `Decidable (2 < 3)` である。`Decidable` は、命題について、証明か否定の証明を持つ判定結果を表すクラスで、自然数の不等式には計算可能な判定の仕組みが用意されている。ところが、この命題に `def` で名前を付けると、同じようには表示できない。

```
def z : Prop := 2 < 3
```

```
#eval z
```

```
error: failed to synthesize  
Decidable z
```

`z` は定義を展開すれば $2 < 3$ であり、命題が変わったわけではない。ここで失敗したのは、`Decidable z` のインスタンスの自動探索である。

定義をどこまで展開するかは、行っている処理によって異なる。通常の型検査では、必要に応じて `def` の中身を展開して型を照合する。例えば次の `def` は受理される。

```
def zFromLt (h : 2 < 3) : z := h
```

`h` の型は $2 < 3$ 、要求される型は `z` だが、定義を展開すれば一致する。一方、インスタンス探索では展開する定義が制限されており、この通常の `def z` は展開されない。そのため、既にある自然数の不等式の判定へ自動ではたどり着けない。「型検査で展開して一致を確かめられること」と「インスタンス探索がその定義を展開して探すこと」は区別して読む必要がある。

この場合は、元の命題の判定を使って `Decidable z` を明示的に登録できる。

```
instance : Decidable z := inferInstanceAs (Decidable (2 < 3))
```

```
#eval z
```

```
true
```

`inferInstanceAs (Decidable (2 < 3))` は、指定した型のインスタンスを探す項である。ここでは探索する型を `Decidable (2 < 3)` と明示したので、標準の判定が見つかる。その項を `Decidable z` として使えるかの型検査では `z` の定義が展開されるため、登録が受理される。以後は `Decidable z` 自体が見つかり、`#eval z` も実行できる。

5. 記法の自作 — `syntax` と `macro_rules`

`06_Topology.lean` は $\cup$ や $\{a \mid p\}$ といった数学記法を自作している。仕組みは:

- `syntax` — 「この書き方を受け付けよ」と構文を追加する
- `macro_rules` — 「その書き方はこの項の略記である」と展開を与える

試しに、1節の `Pair` のための記法を作ってみる。

```
syntax "<" term ", " term ">" : term
```

```
macro_rules
```

```
| `(<$x, $y>) => `(Pair.mk $x $y)
```

`macro_rules` の行はこう読む。`(...) は**構文の断片**を作る引用符、`$x · $y` は受け取った断片をそこへ埋め込む印である。つまりこの行は「`<x, y>` と書かれたら、`Pair.mk x y` と書かれたことにせよ」と読める。

すると `<1, true>` は `Pair.mk 1 true` に展開されてから型検査されるので、型は `Pair Nat Bool` のはずである：

```
#check <1, true>
```

```
{ fst := 1, snd := true } : Pair Nat Bool
```

記法は項に展開されてから型検査されるので、検査の対象はあくまで項のままである。単純な中置・前置の記法には `infixl` や `prefix` という略記もある（次節と `06_Topology.lean` で使用）。

⇒ 練習

60. `<1, true>` にならって、`Point` 用の記法（例えば `⟨x, y⟩`）を `syntax` と `macro_rules` で自作し、`#check ⟨1, 2⟩` で確かめよ。

解答 60

```
syntax "⟨" term ", " term ">" : term
```

```
macro_rules
```

```
| `(<$x, $y>) => `(Point.mk $x $y)
```

```
#check ⟨1, 2⟩
```

```
{ x := 1, y := 2 } : Point
```

読む方向だけの記法なので、表示では展開先の構成子の形が見える。

6. 集合 — Set を自作する

次の章では、集合の上に位相を組み立てる。その土台となる集合をここで作ろう。第2・4章で身につけた証明の書き方を、ここで使ってみる。3節では、述語を満たす値と証明を組にした。ここでは、値を切り出す条件である述語そのものを、一つの集合として扱う。

X の部分集合を1つ指定することは、「各 $a : X$ が入っているかどうか」を決めること——つまり X 上の述語を1つ与えることと同じである。そこで:

X の部分集合の型。実体は述語 $X \rightarrow \text{Prop}$ そのもので、新しいデータは何もない。

```
def Set (X : Type) : Type := X → Prop
```

```
#check Set
```

```
Set (X : Type) : Type
```

01_TypesAndTerms.lean 3節の Map と同じく、「型を受け取って型を返す関数」である。

述語 p を集合とみなすときの「宣言」も1つ用意する。定義上は恒等関数だが、「述語を集合と読み替えました」という意思表示をこの名前が担う:

```
def setOf {X : Type} (p : X → Prop) : Set X := p
```

```
#check setOf
```

```
setOf {X : Type} (p : X → Prop) : Set X
```

数学の内包記法 $\{a \mid p\ a\}$ も、5節の道具でそのまま自作できる:

```
syntax "{ " ident " | " term "}" : term
```

```
macro_rules
```

```
| `({ $x:ident | $p }) => `(setOf fun $x => $p)
```

所属の記号 $\in$ (`\in` と打つ) は、標準ライブラリの記法用クラス `Membership` に instance 登録すると使えるようになる——4節で見た「記法はクラスで動く」の実戦である。中身は「集合 (= 述語) s に点 a を適用する」だけ。まずクラスそのものを見ておく:

```
#check Membership
```

```
Membership.{u, v} (α : outParam (Type u)) (γ : Type v) : Type (max u v)
```

`Membership α γ` は「入れ物 `γ` に要素 `α` が属する」という記法 `∈` のためのクラスである。次の登録は `γ := Set X · α := X` の場合に当たる。

補足: Membership の宇宙変数と outParam

表示は宇宙変数付きだが、この教材の範囲ではどれも `Type` と読んでよい（[01_TypesAndTerms.lean 1節](#)）。`outParam` は instance 探索へのヒントで、「`∈` の右の入れ物の型 `γ` が分かれば、左の要素の型 `α` はそこから自動で決まる」という指定である。

```
instance {X : Type} : Membership X (Set X) := {fun s a => s a}
```

```
#check (1 : Nat) ∈ ({n | n = 1} : Set Nat)
```

```
1 ∈ setOf fun n => n = 1 : Prop
```

型が `Prop` と付いた＝登録に成功している。読むときの注意を2つ。

第一に、ここでは `1 : Nat` と `... : Set Nat` の二つの型注釈で、どの型の話を明示している。数字や内包記法を読むときにも、周囲から伝わる型が手がかりになる。

第二に、表示が `{n | n = 1}` に戻らず `setOf fun n => n = 1` になる理由:自作した記法は**構文解析（読む方向）専用**で、表示（書く方向）の規則までは作っていないからである。以後の表示の枠でも、展開先がそのまま見える。

「`a ∈ s` である」ことの証明は、定義を展開すればただの `s a` である。実際に確かめてみる:

```
def one_mem_singleton : (1 : Nat) ∈ ({n | n = 1} : Set Nat) := rfl
```

`∈` と `setOf` を定義に沿って展開すると、この命題は `1 = 1` そのものになる。だから `rfl` で閉じる——型検査が**定義を展開して**照合してくれるのである。

包含 `⊆` も同じやり方で登録する。「`s` のどの要素も `t` の要素」——`∀` と `→` で書けるようになった言明である:

```
instance {X : Type} : HasSubset (Set X) := {fun s t => ∀ a, a ∈ s → a ∈ t}
```

```
#check ({n | n = 1} : Set Nat) ⊆ {n | n = 2}
```

```
(setOf fun n => n = 1) ⊆ setOf fun n => n = 2 : Prop
```

型は `Prop` —— 包含は命題である。`s ⊆ t` の証明とは、定義から「各点で、`s a` の証明から `t a` の証明を作る関数」に他ならない。

最初の性質を証明してみよう。反射律は「各点で仮定をそのまま返す」だけである：

```
def Set.subset_refl {X : Type} (s : Set X) : s ⊆ s := fun _ ha => ha

#check Set.subset_refl
```

```
Set.subset_refl {X : Type} (s : Set X) : s ⊆ s
```

`s ⊆ s` を展開すれば $\forall a, a \in s \rightarrow a \in s$ である。点 `a` と `ha : a ∈ s` を受け取り、その `ha` を結論の証明として返している。第2章で学んだ、全称量化と含意を関数として読む方法を使った。名前を `Set.~` にしたのはドット記法（1節で見た関数適用の略記）のためで、この命名の仕組みは次節で説明する。

`06_Topology.lean` はこの `Set` を土台に、`n · u` · 補集合 · 像 · 逆像 · 集合族……と道具を足していく。

⇒ 練習

61. 推移律を項で書け（各点で2つの仮定を順に適用する）：

```
def Set.subset_trans {X : Type} {s t u : Set X}
  (hst : s ⊆ t) (htu : t ⊆ u) : s ⊆ u
```

解答 61

```
def Set.subset_trans {X : Type} {s t u : Set X}
  (hst : s ⊆ t) (htu : t ⊆ u) : s ⊆ u :=
  fun a ha => htu a (hst a ha)
```

点 `a` と `ha : a ∈ s` を受け取り、`hst` で `a ∈ t`、続けて `htu` で `a ∈ u` を得る——含意の連鎖と同じ形である。

7. namespace — 名前の接頭辞

1節のドット記法や、前節の `Set.subset_refl` という名前を支えている名前空間の仕組みを見ておく。

`namespace N ... end N` で囲うと、中の宣言の本名に `N.` が付く：

```
namespace Geometry
```

```
def origin : Point := {0, 0}
```

```
#check origin
```

```
Geometry.origin : Point
```

```
end Geometry
```

```
#check Geometry.origin
```

```
Geometry.origin : Point
```

外からはフルネームで呼ぶ。

`p.x` が効くのは、`x` が**型名と同じ名前空間** `Point` に置かれているからである。`06_Topology.lean` では `namespace Set` の中に集合の関数（`Set.ext` など）を置いていく——`s.ext` のようなドット記法が効くのはそのためである。逆に、名前空間の**中**の名前を接頭辞なしで使えるようにする `export N (名前)` という宣言もある（`06_Topology.lean` が `TopologicalSpace.IsOpen` を `IsOpen` と書くために使っている）。

これで `06_Topology.lean` を読む準備が整った。

補足: 短い名前と、型から補う名前

`Bool.true` が構成子の完全な名前である。標準環境では `true` という短い名前も使える。自作した `Point.mk` については、勝手に `mk` という短い名前が使えるようになるわけではない。 `open Point` とすれば、その範囲で `mk` と書ける。

これとは別に、期待される型から名前を補う `.mk` という記法もある。

```
#check (.mk 1 2 : Point)
```

```
#check (.true : Bool)
```

```
{ x := 1, y := 2 } : Point
```

```
true : Bool
```

型が `Point` と分かる位置では `.mk` を `Point.mk` と読める。 `match` の枝で `.mk` と書けるのも、場合分けする項の型が分かるためである。名前空間を開いて短い名前を使うことと、期待される型から名前を補ってもらうことを区別しよう。第3・4章では、構成子を明示するために完全な名前を書いてきた。

8. 練習 — 章末問題

62. 本文で `Pointed Bool` が未登録のため `inferInstance` が失敗する例を見た。 `instance : Pointed Bool where point := false` を登録し、 `#check (inferInstance : Pointed Bool)` と `def pointPair_bool_fst : (pointPair Bool).fst = Pointed.point := pointPair_fst Bool` が通るようになることを確かめよ（登録した瞬間から、一般的な定理も適用できる）。

解答 62

```
instance : Pointed Bool where
  point := false

#check (inferInstance : Pointed Bool)
```

```
inferInstance : Pointed Bool
```

```
def pointPair_bool_fst : (pointPair Bool).fst = Pointed.point := pointPair_fst Bool
```

登録した瞬間から、`inferInstance` の探索も、汎用関数 `pointPair` も、一般的な定理 `pointPair_fst` も、すべて `Bool` で使えるようになる。

63. `Pointed` と同じ手順でマグマを自作する: `class Magma (α : Type) : Type where op : α → α → α` を宣言し、 `instance : Magma Nat where op := Nat.add` を登録して、 `#check (inferInstance : Magma Nat)` が通ることを確かめよ。

解答 63

```
class Magma (α : Type) : Type where
  op : α → α → α

instance : Magma Nat where
  op := Nat.add

#check (inferInstance : Magma Nat)
```

```
inferInstance : Magma Nat
```

手順は `Pointed` とまったく同じ——クラスを宣言し、`instance` で登録簿に載せれば、`inferInstance` が見つける。フィールドが「点」から「二項演算」に変わっただけである。

64. `pointPair` にならって、汎用関数 `opSelf (α : Type) [Magma α] (a : α) : α := Magma.op a a` を書き、`#eval opSelf Nat 3` の値を予想してから確かめよ。

解答 64

```
def opSelf (α : Type) [Magma α] (a : α) : α := Magma.op a a

#eval opSelf Nat 3
```

6

`Nat` の登録簿には `op := Nat.add` を載せたので、`opSelf Nat 3 = 3 + 3`。

65. 本文の「積に構造を誘導する」にならって、成分ごとに演算する `instance {α β : Type} [Magma α] [Magma β] : Magma (Pair α β)` を登録し、`#eval (Magma.op ((1, 2) : Pair Nat Nat) (10, 20)).snd` の値を予想してから確かめよ（探索の連鎖まで含めて、`Pointed` と同じに動く）。

解答 65

```
instance {α β : Type} [Magma α] [Magma β] : Magma (Pair α β) where
  op p q := (Magma.op p.fst q.fst, Magma.op p.snd q.snd)

#eval (Magma.op ((1, 2) : Pair Nat Nat) (10, 20)).snd
```

22

`Magma (Pair Nat Nat)` は直接は登録していないが、誘導 `instance` と `Magma Nat` の連鎖で見つかる。`.snd` は `2 + 20`。

66. `Add` にならって `instance : Mul Point where mul p q := (p.x * q.x, p.y * q.y)` を登録し、`#eval (Point.mk 2 3 * Point.mk 4 5).x` の値を予想してから確かめよ。

解答 66

```
instance : Mul Point where
  mul p q := (p.x * q.x, p.y * q.y)

#eval (Point.mk 2 3 * Point.mk 4 5).x
```

8

x 成分どうしの積 $2 * 4$ 。登録すれば $*$ がそのまま `Point` に使える。

67. `infixl:65 " ⊞ " => Nat.add` で、自然数の足し算に中置記法を与え、`#reduce 1 ⊞ 1` の表示を予想してから確かめよ。

解答 67

```
infixl:65 " ⊞ " => Nat.add
```

```
#reduce 1 ⊞ 1
```

2

中置記法は `Nat.add` の適用に展開され、 $1 + 1$ が計算されている。

68. `def evens : Set Nat := {n | IsEven n}` と宣言せよ (`IsEven` は [04_Exists.lean](#) 1節の述語)。`def four_mem_evens : (4 : Nat) ∈ evens := ⟨2, rfl⟩` が通ることを確かめよ。

解答 68

```
def evens : Set Nat := {n | IsEven n}
```

```
def four_mem_evens : (4 : Nat) ∈ evens := ⟨2, rfl⟩
```

$4 \in \text{evens}$ は定義を展開すると `IsEven 4` すなわち $\exists k, 4 = 2 * k$ 。証人 `2` と `rfl` で作れる。

69. 全体集合 `def allNat : Set Nat := {_n | True}` を定義し、`def subset_allNat : ∀ s : Set Nat, s ⊆ allNat` を書け (各点の証明は `True.intro`。束縛子 `_n` の `_` は「使わない」印である)。

解答 69

```
def allNat : Set Nat := {_n | True}
```

```
def subset_allNat : ∀ s : Set Nat, s ⊆ allNat :=  
  fun _s _a _ha => True.intro
```

どの点でも示すべきは `True` なので、構成子 `True.intro` を返すだけ。使わない引数は `_` 付きの名前にしてある。

70. `def odds : Set Nat := {n | ¬IsEven n}` を宣言し、`#check (3 : Nat) ∈ odds` の表示を予想してから確かめよ。

解答 70

```
def odds : Set Nat := {n | ¬IsEven n}
```

```
#check (3 : Nat) ∈ odds
```

```
3 ∈ odds : Prop
```

集合への所属は命題。中身が `¬...` でも、`∈` の式全体の型は `Prop` である。

71. `namespace Geometry ... end Geometry` をもう一度開いて `unitX : Point := (1, 0)` を追加し、外から `#check Geometry.unitX` では見え、`#check unitX` では見えないことを確かめよ。

解答 71

```
namespace Geometry
```

```
def unitX : Point := (1, 0)
```

```
end Geometry
```

```
#check Geometry.unitX
```

```
Geometry.unitX : Point
```

フルネームなら外から見える。接頭辞なしの `#check unitX` は `Unknown identifier` というエラーになる。

```
import «05_MathematicalTools»
```

位相空間

mathlib を使わず、Lean 4 の標準ライブラリだけで位相空間を組み立てる。

最終目標は「コンパクト空間からハウスドルフ空間への連続全単射は同相写像である」。そのために必要なものを、必要になった順に足していく。

数学でいう「集合」には、`05_MathematicalTools.lean` で作った `Set` を使う（このファイルはそれを import している）。

この定理を信じるには何を信じればよいか

末尾の定理 `Homeomorph.ofContinuousBijective` を信じるのに必要なのは、次の3つだけである。

1. Lean の**カーネル**が正しいこと。エラボレータやタクティクがどれだけ複雑でも、最後に項を検査するのはカーネルの小さな規則集だけである（`04_Exists.lean` の6節）
2. このファイルに書いた**定義**が、意図した数学的概念を写していること
3. 末尾の `#print axioms` に表示される3つの公理

証明そのものは信じる必要がない。途中の議論がどれだけ長く込み入っていても、型検査を通った時点で検査は済んでいる。

機械が直接保証してくれるのは、「1 を信頼したうえで、書かれた証明が主張どおりの型を持つ」ことと、「3 にどの公理が入っているかの列挙」である。1 と 3 は信頼の**基盤**として明示的に引き受けるものであり、2 は人間が読んで監査するしかない。定義が間違っていれば、そこから証明した定理は意図と違うことを言っている。だからこのファイルでは、すべての定義を1ファイルに収めて目視で監査できる分量に保ち（mathlib を使わないのはこのためでもある）、主要な定義にはよく知る例・事実がそこから出ることを確かめる「定義の確認」を添えてある。

読み方: 主要な宣言のあとの `#check`

主要な名前付きの定義・定理の直後に `#check 名前` を置き、その表示を `01_TypesAndTerms.lean` と同じ Infoview 風の枠で書き添えてある。`01_TypesAndTerms.lean` 以来の読み方——表示を見る前に「いま何がどんな型で手に入るはずか」を**予想**してから確かめる——を、ここでも続けてほしい。

表示は `名前 (x : A) (y : B) : C` という「引数の列：結果の型」の形をとる。これは関数型 `名前 : A → B → C` と**同じ型の別表示**である。`def f (n : α) : β := ...` と `def f : α → β := fun n => ...` が同じ宣言の2通りの書き方である（`01_TypesAndTerms.lean` 3節）のに対応して、表示もこの2つの形を行き来す

る。括弧 () { } の違いは [01_TypesAndTerms.lean](#) 4節、[] は [05_MathematicalTools.lean](#) 4節。

1. タクティク — 証明のもう1つの書き方

`02_Forall.lean` ・ `04_Exists.lean` では、証明はすべて項として直接書いた。Lean にはもう1つの流儀があり、`by` に続けて、項を組み立てる**命令**（タクティク）の列を書く。例えば、「かつ」の左右を交換する証明は、項で次のように書ける。

```
def swapAnd {p q : Prop} : p ∧ q → q ∧ p :=  
  fun h => And.intro h.right h.left
```

同じ主張をタクティクで書いてみよう。

```
theorem swapAnd' {p q : Prop} : p ∧ q → q ∧ p := by  
  intro h          -- `fun h =>` に対応  
  exact (h.right, h.left) -- この項をそのまま置く  
  
#check swapAnd'
```

```
swapAnd' {p q : Prop} : p ∧ q → q ∧ p
```

`intro` は `fun`、`exact` は項そのものに対応していて、どちらの流儀でも、最終的に出来上がるのは**同じ型を持つ項**である（字面まで同じになるとは限らない）。つまりタクティクは証明そのものではなく、**項を組み立てるためのメタなプログラム**であり、`by` ブロックは実行されると項を生成する。検査されるのはあくまで生成された項のほう。生成結果は `#print` で見られる:

```
#print swapAnd'
```

```
theorem swapAnd' : ∀ {p q : Prop}, p ∧ q → q ∧ p :=  
  fun {p q} h => (h.right, h.left)
```

タクティクが生成した項は、上で手書きした `swapAnd` と同じものである。

だから [04_Exists.lean](#) 6節の話はそのまま当てはまる: タクティクで書いた証明も、出来上がった項が型検査を通過することで検証される。

このファイルは証明が長いので、タクティク主体で書く。出てくるものの早見表:

- `intro h` — 仮定を1つ取り込む (`fun h =>`)
- `exact e` — 項 `e` でゴールを閉じる

- `refine (..., ?_, ?_)` — 項の形を先に与え、埋め残す部分を `?_` の穴にして続きのゴールとする
- `apply f` — 結論から逆向きに `f` を適用し、引数をゴールとして残す
- `constructor` — ゴールの型の構成子を適用する
- `cases h with ...` — 帰納型の項 `h` を場合分けする (`match` に対応)
- `cases i using Fin.cases` — どの場合分けのしかたを使うかを名指しする
- `rw [heq]` — 等式 `heq` でゴールを書き換える
- `rw [h] at hmem` — ゴールではなく**仮定** `hmem` の側を書き換える
- `show t` — ゴールを定義上等しい形 `t` に読み替える
- `have h : t := e` — 補助的な項に名前を付けて続ける
- `by_cases h : p` — `p` が成り立つ場合と否定の場合に分ける (古典論理)
- `·` — 場合分けなどで生じた**サブゴールごとの証明の区切り**
- `t1; t2` — `t1` を実行してから、続けて `t2` を実行する
- `h ▸ e` — 等式 `h` で `e` の型を書き換える (`rw` の項版。タクティクではなく項の記法)

初出の箇所にも、それぞれ短い説明を添えてある。

また、タクティクで書いた証明にはそれぞれ、**同じ命題の項スタイルの証明**を対にして並べる (本文の証明の直後の `example`)。どちらの流儀でも最終的に同じ型の項に行き着くことの、実地の見比べである。

⇒ 練習

72. 次の「または」の左右を交換する証明をタクティク (`intro · cases · exact`) で書き直し、`#print` で生成された項と見比べよ。

```
def swapOr {p q : Prop} : p v q → q v p := fun h =>
  match h with
  | Or.inl hp => Or.inr hp
  | Or.inr hq => Or.inl hq
```

字面は一致しない—— `cases` は `match` ではなく `Or.casesOn` を直接置くからである。 **それでも型は同じ** であり、検査されるのはその型だけである、というのが本節の要点である。

解答 72

```
theorem swapOrTac {p q : Prop} : p v q → q v p := by
  intro h
  cases h with
  | inl hp => exact Or.inr hp
  | inr hq => exact Or.inl hq
```

```
#print swapOrTac
```

```
theorem swapOrTac :  $\forall$  {p q : Prop}, p  $\vee$  q  $\rightarrow$  q  $\vee$  p :=  
fun {p q} h  $\mapsto$  Or.casesOn (motive := fun t  $\mapsto$  h = t  $\rightarrow$  q  $\vee$  p) h (fun hp h  $\mapsto$  Or.inr hp)  
(fun hq h  $\mapsto$  Or.inl hq) (Eq.refl h)
```

04_Exists.lean の swapOr (match で書いた項) と字面は一致しない——cases は Or.casesOn を直接置くからである。それでも型は同じ $p \vee q \rightarrow q \vee p$ であり、検査されるのはその型だけである。

73. theorem idTac {p : Prop} : p $\rightarrow$ p を intro $\cdot$ exact で書き、#print idTac の表示を予想してから確かめよ (こちらは手書きと同じ字面に戻る)。

解答 73

```
theorem idTac {p : Prop} : p  $\rightarrow$  p := by  
  intro h  
  exact h  
  
#print idTac
```

```
theorem idTac :  $\forall$  {p : Prop}, p  $\rightarrow$  p :=  
fun {p} h  $\mapsto$  h
```

こちらは手書きの fun h => h と同じ字面に戻る。

2. 集合

集合 Set (実体は述語 $\alpha \rightarrow \text{Prop}$) と、内包記法 {a | p a} $\cdot$ 所属 $\in$ $\cdot$ 包含 $\subseteq$ は 05_MathematicalTools.lean 6節で作った。ここではその上に、残りの道具——n $\cdot$ u $\cdot$ $\emptyset$ $\cdot$ 補集合 $\cdot$ 像 $\cdot$ 逆像 $\cdot$ 集合族 $\cdot$ 有限性 $\cdot$ 外延性——を積んでいく。

```
-- ここから `end Set` までの宣言には接頭辞 `Set.` が付く (`05_MathematicalTools.lean` 7節)  
namespace Set  
  
-- 共通の引数の前置き。以後の宣言が `α` を使うと、自動で引数に取り込まれる  
variable {α : Type}
```

$n \cup \emptyset$ の記法も、[05_MathematicalTools.lean](#) 6節の $\in \cdot \subseteq$ と同じやり方——標準ライブラリの記法用クラスへの `instance` 登録——で使えるようにする。`{...}` はクラスの構成子にフィールドの中身を渡す書き方である。

共通部分 `s n t`: 両方に属する点の全体（「かつ」）。

```
instance : Inter (Set  $\alpha$ ) := {fun s t => {a | a  $\in$  s  $\wedge$  a  $\in$  t}}
```

```
#check ({n | n = 1} : Set Nat) n {n | n = 2}
```

```
(setOf fun n => n = 1) n setOf fun n => n = 2 : Set Nat
```

型は `Set Nat`（命題でなく集合）

合併 `s u t`: どちらかに属する点の全体（「または」）。

```
instance : Union (Set  $\alpha$ ) := {fun s t => {a | a  $\in$  s  $\vee$  a  $\in$  t}}
```

```
#check ({n | n = 1} : Set Nat) u {n | n = 2}
```

```
(setOf fun n => n = 1) u setOf fun n => n = 2 : Set Nat
```

型は `Set Nat`

空集合 `∅`: どの点も属さない集合。中身は「つねに `False` を返す述語」。

```
instance : EmptyCollection (Set  $\alpha$ ) := {_a | False}
```

```
#check (∅ : Set Nat)
```

```
∅ : Set Nat
```

補足: `instance` の内部名

`instance` は名前のない宣言だが、実際には Lean が自動で命名している。上の3つは `Set.instInter` のような名前になる（`set_option pp.explicit true` で項を表示すると中に見える）。`instance memInst : Membership  $\alpha$  (Set  $\alpha$ ) := ...` と自分で名前を付けてもよい。

全体集合。どの点も属する集合（つねに `True` を返す述語）。束縛子を `_a` としたのは、述語の中で点を使わないから（`_` は「使わない」印）。

```
def univ : Set α := {_a | True}
```

```
#check univ
```

```
Set.univ {α : Type} : Set α
```

namespace の中で宣言したのでフルネームは `Set.univ`。明示引数はなく、暗黙の α は使う場面の期待される型から決まる。

```
-- 確認: `0 ∈ univ` は定義を展開すると命題 `True` そのもの。  
-- `trivial : True` がちょうどその型を持つので、これで閉じる  
example : (0 : Nat) ∈ (univ : Set Nat) := trivial
```

補集合 $s^c : s$ に属さない点の全体。 $\notin$ は $\neg(a \in s)$ の略記。

```
def compl (s : Set α) : Set α := {a | a ∉ s}
```

```
#check compl
```

```
Set.compl {α : Type} (s : Set α) : Set α
```

集合を受け取って集合を返す。 `compl : Set α → Set α` と同じこと。

後置記法 s^c 。 `max` は最も強い結合を表す（結合の強さは後でまとめて確認する）。

```
postfix:max "c" => Set.compl
```

像。 $f '' s$ は s の点を f で送った先の全体。「どこかの $a \in s$ から来た」を $\exists$ で言う。

```
def image {β : Type} (f : α → β) (s : Set α) : Set β := {b | ∃ a, a ∈ s ∧ f a = b}
```

```
#check image
```

```
Set.image {α β : Type} (f : α → β) (s : Set α) : Set β
```

明示引数が2つ並ぶカリー化された関数。 `image : (α → β) → Set α → Set β` と読み替えられる。

中置記法 $f '' s$ 。数字 `80` は結合の強さ。

```
infixl:80 " '' " => Set.image
```

「集合の集まり」 $S : \text{Set} (\text{Set } \alpha)$ に属する集合すべての合併。位相の公理でいう「任意個の合併」を、添字を使わずにこれで表す。

```
def sUnion (S : Set (Set α)) : Set α := {a | ∃ s, s ∈ S ∧ a ∈ s}
```

```
#check sUnion
```

```
Set.sUnion {α : Type} (S : Set (Set α)) : Set α
```

引数の型が $\text{Set} (\text{Set } \alpha)$ = 「集合の集合」であることに注意。

前置記法 $\bigcup_0 S$ 。

```
prefix:110 "∪₀ " => Set.sUnion
```

添字づけられた集合族 $U : I \rightarrow \text{Set } \alpha$ の合併。族とは「添字を受け取って集合を返す関数」である（[01_TypesAndTerms.lean](#) 4節の型の族の仲間）。`sUnion` が「集合の集まり」を受け取るのに対し、こちらは添字でパラメータづけた族を受け取る。

```
def iUnion {I : Type} (U : I → Set α) : Set α := {a | ∃ i, a ∈ U i}
```

```
#check iUnion
```

```
Set.iUnion {α I : Type} (U : I → Set α) : Set α
```

暗黙引数が α と I の2つ。どちらも U の型から決まるので書かずに済む。

添字を $J \subseteq I$ に制限した合併。「部分族の合併」を表す。

```
def biUnion {I : Type} (J : Set I) (U : I → Set α) : Set α := {a | ∃ i, i ∈ J ∧ a ∈ U i}
```

```
#check biUnion
```

```
Set.biUnion {α I : Type} (J : Set I) (U : I → Set α) : Set α
```

逆像。 $f^{-1} s$ は、 f で送ると s に入る点の全体。

```
def preimage {β : Type} (f : α → β) (s : Set β) : Set α := {a | f a ∈ s}
```

```
#check preimage
```

```
Set.preimage {α β : Type} (f : α → β) (s : Set β) : Set α
```

`image` と見比べると `Set β → Set α` で、集合の移動が `f` と逆向き。

中置記法 `f-1 s`。

```
infixl:80 " ^-1 " => Set.preimage
```

-- 確認: 逆像も定義どおりに展開される。両辺は計算 (関数適用の簡約) で一致するので `rfl`

```
example : (fun n : Nat => n + 1) ^-1 {m | m = 3} = {n | n + 1 = 3} := rfl
```

括弧を省いた記法は結合の強さに従って読まれる。細かい順位は次の補足で確かめる。

補足: 記法の結合の強さ

`postfix:max` や `infixl:80` の数字は結合の強さで、大きいほど強く結合する。ここまでの記法と標準ライブラリの `n (70) · u (65)` の間には

```
^ (max) > ' ' と ^-1 (80) > n (70) > u (65)
```

の関係がある。次の等式は「左辺を規則どおりに読むと右辺になる」ことの確認で、両辺が**同じ項**に構文解析されるからこそ `rfl` で通る。

```
example (f : Nat → Nat) (s t : Set Nat) : f ' ' s n t = (f ' ' s) n t := rfl
```

```
example (s t : Set Nat) : s n tc = s n (tc) := rfl
```

```
example (s t u : Set Nat) : s n t u u = (s n t) u u := rfl
```

有限集合: ある `n` について、`Fin n` からの写像で `s` の点をすべて拾えること。

`Fin n` は `0, 1, ..., n-1` のちょうど `n` 個からなる型なので、「`Fin n` で番号づけられる」がそのまま「点が有限個しかない」を意味する。

`f` は `s` の外の点を拾ってもよい (`Fin n` と `s` の全単射までは要求しない)。「高々 `n` 個」で十分であり、こうしておくで部分集合の有限性がただちに従う。

書き方を2つ: `∃ (n : Nat) (f : Fin n → α), P` は `∃ n, ∃ f, P` の略記、`∀ a ∈ s, P a` は `∀ a, a ∈ s → P a` の略記である (どちらも `#check` の表示では展開された形で見える)。

```
def Finite (s : Set α) : Prop := ∃ (n : Nat) (f : Fin n → α), ∀ a ∈ s, ∃ i, f i = a
```

```
#check Finite
```

```
Set.Finite {α : Type} (s : Set α) : Prop
```

結果が `Prop`。つまりこれは集合の性質（集合を受け取って命題を返す述語）。

空集合は有限。 `n = 0` とし、拾う関数には `Fin.elim0`（`Fin 0` は空の型なので、そこからはどこへでも関数が作れる）を渡す。 `{..., ..., ...}` は `∃` を示す形（[04_Exists.lean](#) 1節）。

```
theorem Finite.empty : (∅ : Set α).Finite :=
  (0, Fin.elim0, fun _ ha => False.elim ha)

#check Finite.empty
```

```
Set.Finite.empty {α : Type} : ∅.Finite
```

仮定なしの定理。 `∅.Finite` はドット記法の表示で `Set.Finite ∅` のこと。定理の `#check` は「証明済みの命題」を型として見せてくれる。

外延性

属する要素が一致する集合は等しい——関数の外延性 `funext` と命題の外延性 `propext` から従う（集合を関数として定義したことの代金をここで払う）。この場面に特殊化した型は

```
funext : (∀ a, s a = t a) → s = t
propext : (a ∈ s ↔ a ∈ t) → (a ∈ s) = (a ∈ t)
```

で、下の証明項 `funext fun a => propext (h a)` は、この2つを型どおりに組み合わせただけである。以後、集合の等式を示すときは `apply Set.ext` で「要素ごとの同値」に還元する。

外延性: 属する要素が一致する集合は等しい。

```
theorem ext {s t : Set α} (h : ∀ a, a ∈ s ↔ a ∈ t) : s = t :=
  funext fun a => propext (h a)

#check ext
```

```
Set.ext {α : Type} {s t : Set α} (h : ∀ (a : α), a ∈ s ↔ a ∈ t) : s = t
```

定理の型は「仮定 → 結論」の関数型。仮定 `h` を渡すと結論 `s = t` の証明が返る。`s t` が暗黙なのは、`h` の型に現れるので自動で決まるから。

二重補集合。ここで初めて古典論理を使う: `Classical.byContradiction : (¬p → False) → p` は背理法そのものである。

```
theorem compl_compl (s : Set α) : scc = s :=
  ext fun _ => ⟨fun h => Classical.byContradiction h, fun h hn => hn h⟩

#check compl_compl
```

```
Set.compl_compl {α : Type} (s : Set α) : scc = s
```

どの集合 `s` にも適用できる等式。 $\forall s, \dots$ と書くのと `(s : Set α)` を引数に取るのは同じこと (`02_Forall.lean` 1節: $\forall$ は依存関数型の記法)。

2つの合併は「2つだけからなる族」の合併に書き直せる。 `{u | u = s ∨ u = t}` は要素が `s` と `t` の2つ (だけ) の集合族。位相の公理は族の合併の形で書くので、この橋渡しを補題に切り出しておく (位相の節の `isOpen_union` で使う)。

証明中の `have ⟨u, hu, hau⟩ := ha` はここが初出の構文で、「`ha` ($\exists$ の証明) を **分解して名前を付ける**」と読む——`match` (`04_Exists.lean` 1節) の1ケース版の略記であり、タクティクとしても項としても同じ形で書ける。分解を伴わない `have h : 主張 := 根拠` (名前を付けて続きで使う) もこの先で頻用する。`h ▶ hau` は「等式 `h` で `hau` の型を書き換える」記法。

```
theorem union_eq_sUnion (s t : Set α) : s ∪ t = ∪₀ {u | u = s ∨ u = t} := by
  apply ext
  intro a
  constructor
  · intro ha
    cases ha with
    | inl h => exact ⟨s, Or.inl rfl, h⟩
    | inr h => exact ⟨t, Or.inr rfl, h⟩
  · intro ha
    have ⟨u, hu, hau⟩ := ha
    cases hu with
    | inl h => exact Or.inl (h ▶ hau)
    | inr h => exact Or.inr (h ▶ hau)
```

-- 同じ命題を項で直接書くと、こうなる (タクティクと項の見比べ) :

```
example (s t : Set α) : s ∪ t = ∪₀ {u | u = s ∨ u = t} :=
  ext fun _ =>
    ⟨fun ha =>
      match ha with
      | Or.inl h => ⟨s, Or.inl rfl, h⟩
      | Or.inr h => ⟨t, Or.inr rfl, h⟩,
      fun ⟨_, hu, hau⟩ =>
```

```
match hu with
| Or.inl h => Or.inl (h ▶ hau)
| Or.inr h => Or.inr (h ▶ hau))
```

```
#check union_eq_sUnion
```

```
Set.union_eq_sUnion {α : Type} (s t : Set α) : s ∪ t = ∪₀ setOf fun u => u = s ∨ u = t
```

内包記法は表示では展開先の `setOf` の形で見える（前述）。

有限個の共通部分

`Fin n` で番号づけられた集合たちの共通部分。`n` についての再帰で定義する。ハウスドルフ側の証明で、有限部分被覆から近傍を1つ作るのに使う。

`n` 個の集合 `W 0, ..., W (n-1)` の共通部分。`Nat` の構造にそったパターンマッチ（[03_InductiveTypes.lean 4節](#)）で定義する: `0` 個なら `univ`、`n + 1` 個なら「先頭 `W 0`」と「残り `n` 個の共通部分」の `n`。 `fun i => W i.succ` は添字を1つずらして「残りの族」を作っている。

```
def interFin : (n : Nat) → (Fin n → Set α) → Set α := fun n W =>
  match n with
  | 0 => univ
  | n + 1 => W 0 n interFin n fun i => W i.succ

#check interFin
```

```
Set.interFin {α : Type} (n : Nat) : (Fin n → Set α) → Set α
```

表示に binder 形式（`(n : Nat)`）と矢印形式（`→`）が**混ざっている**が、どちらも同じ関数型の表示にすぎない。全体としては `interFin : (n : Nat) → (Fin n → Set α) → Set α` という2引数関数である。

すべての `W i` に入る点は共通部分に入る。定義と同じ再帰の形で証明を書く——**再帰で書いた証明が数学的帰納法**である。`n = 0` の場合のゴールは `a ∈ univ` すなわち `True` なので `trivial`。

```
theorem mem_interFin : ∀ (n : Nat) (W : Fin n → Set α) (a : α), (∀ i, a ∈ W i) →
  a ∈ interFin n W := fun n W a h =>
  match n with
  | 0 => trivial
  | n + 1 => ⟨h 0, mem_interFin n (fun i => W i.succ) a fun i => h i.succ⟩

#check mem_interFin
```

```
Set.mem_interFin {α : Type} (n : Nat) (W : Fin n → Set α) (a : α) : (∀ (i : Fin n), a ∈ W i) → a ∈ interFin n W
```

引数3つのあと、「仮定 → 結論」。長い表示は適当な位置で折り返されるが意味は変わらない。

共通部分に入る点は、すべての $W\ i$ に入る（逆向き）。 $n = 0$ では示すべき添字がそもそもない（ $Fin\ 0$ は空）ので `Fin.elim0`。 $n + 1$ では添字 i を `Fin.cases` で「先頭か、後続か」に場合分けする（`Fin.cases` は、場合分けを項として書くための標準ライブラリの関数）。

```
theorem interFin_mem : ∀ (n : Nat) (W : Fin n → Set α) (a : α), a ∈ interFin n W →
  ∀ i, a ∈ W i := fun n W a h i =>
  match n with
  | 0 => Fin.elim0 i
  | n + 1 => Fin.cases h.left (fun j => interFin_mem n (fun k => W k.succ) a h.right j) i

#check interFin_mem
```

```
Set.interFin_mem {α : Type} (n : Nat) (W : Fin n → Set α) (a : α) : a ∈ interFin n W → ∀
  (i : Fin n), a ∈ W i
```

`mem_interFin` と仮定・結論が入れ替わっている。2つ合わせて同値。

```
end Set
```

⇒ 練習

74. `example : (2 : Nat) ∈ ({n | n < 5} : Set Nat)` を証明せよ。ヒント: $\in$ と `setOf` を展開すればゴールは $2 < 5$ 、すなわち $3 \leq 5$ —— [03_InductiveTypes.lean 5節](#)の構成子 `Nat.le.step`・`Nat.le.refl` で書ける。

解答 74

```
example : (2 : Nat) ∈ ({n | n < 5} : Set Nat) :=
  Nat.le.step (Nat.le.step Nat.le.refl)
```

$\in$ と `setOf` を展開するとゴールは $2 < 5$ 、すなわち $2 + 1 \leq 5$ 。`Nat.le.refl` : $3 \leq 3$ から `step` を2回で $3 \leq 5$ に届く（[03_InductiveTypes.lean 5節](#)）。

75. `#print axioms Set.compl_compl` の結果を予想してから確かめよ（背理法を使った証明だった）。

解答 75

```
#print axioms Set.compl_compl
```

```
'Set.compl_compl' depends on axioms: [propext, Classical.choice, Quot.sound]
```

背理法 `Classical.byContradiction` を使ったので、`Classical.choice` への依存が現れる (`propext` ・ `Quot.sound` はその道連れ)。

3. 全単射

目標の「連続全単射」を述べるために要る。 `Function.Injective` ($\forall a\ b, f\ a = f\ b \rightarrow a = b$ 。括弧 λ は暗黙引数 $\{ \}$ の変種で、この教材では $\forall a\ b, f\ a = f\ b \rightarrow a = b$ と読んでよい) と `Function.Surjective` ($\forall b, \exists a, f\ a = b$) は標準ライブラリにあるので、全単射だけを定義する。

全単射: 単射かつ全射。2つの性質を名前付きフィールドに束ねる。中身が命題だけなので `structure ... : Prop` にできる。

```
structure Function.Bijjective {α β : Type} (f : α → β) : Prop where
  /-- 単射性: 送り先が同じなら元も同じ。 -/
  injective : Function.Injective f
  /-- 全射性: どの点にも、そこへ送られてくる元がある。 -/
  surjective : Function.Surjective f

#check Function.Bijjective
```

```
Function.Bijjective {α β : Type} (f : α → β) : Prop
```

写像の性質 (写像を受け取って命題を返す)。

補足: なぜ $\wedge$ でなく `structure` なのか

`Function.Injective f` $\wedge$ `Function.Surjective f` と束ねても内容は同じだが、ここでは `structure` にする。 `hbij.injective`、`hbij.surjective` のように、性質の名前で取り出せるので、読み手に意図が伝わる。フィールドごとに `docstring` を付けられ、フィールドが増えたり順序が変わったりしても使う側の記述が壊れにくい、という利点もある。

⇒ 練習

76. 恒等写像が全単射であることを `example : Function.Bijective (fun n : Nat => n)` を証明せよ (`injective` は仮定をそのまま返し、`surjective` は証人 `b` と `rfl`)。

解答 76

```
example : Function.Bijective (fun n : Nat => n) :=  
  {fun _ _ h => h, fun b => {b, rfl}}
```

単射性: 仮定 $h : a_1 = a_2$ がそのままゴール (恒等写像なので送り先の等式=元の等式)。先頭の `fun _ _` は2点 $a_1 a_2$ のぶんの束縛である。全射性: 点 b への証人は b 自身、根拠は `rfl`。

$\bigcup i, U_i$ で族全体の合併を表す (`syntax · macro_rules` の仕組みは [05_MathematicalTools.lean](#) 5節)。

```
syntax:110 "⋃ " ident ", " term : term
```

$\bigcup i \in J, U_i$ で添字を J に制限した合併を表す。

```
syntax:110 "⋃ " ident " ∈ " term:110 ", " term : term
```

macro_rules

```
| `(⋃ $i, $U) => `(Set.iUnion fun $i => $U)  
| `(⋃ $i ∈ $J, $U) => `(Set.biUnion $J fun $i => $U)
```

注意: この $\bigcup$ も `setOf` と同じく読む方向だけの記法なので、`#check` の表示では展開先の `Set.iUnion fun i => ... / J.biUnion fun i => ...` が見える。同様に、 $\forall a \in s, P$ は $\forall (a : X), a \in s \rightarrow P$ に、 $a \notin K$ は $\neg a \in K$ に開かれて表示される。以後の表示の枠は、この展開後の形で書いてある。

4. 位相空間

開集合が何であるかを指定するデータ `IsOpen` と、それが満たすべき3つの公理。

位相空間の構造。「どの部分集合を開と呼ぶか」のデータ `IsOpen` と、それが満たすべき3公理を `class` で束ねる (データ+性質という構成は [05_MathematicalTools.lean](#) 1節の `structure` ・ [05_MathematicalTools.lean](#) 4節の `class` と同じ)。

`class` にしたので、以後 `[TopologicalSpace X]` と角括弧で書くだけで「 X に載っている位相」がインスタンス引数として暗黙に渡る。

フィールドは `def` と同じく「引数をコロンの左に書く」形で宣言できる（[01_TypesAndTerms.lean](#) 3節の `binder` 形式）。`∀` と `→` を並べて書いても同じ型である。

```
class TopologicalSpace (X : Type) where
  /-- その集合が開集合であるという述語。 -/
  IsOpen (s : Set X) : Prop
  /-- 全体集合は開。 -/
  isOpen_univ : IsOpen Set.univ
  /-- 2つの開集合の共通部分は開。 -/
  isOpen_inter (s t : Set X) (hs : IsOpen s) (ht : IsOpen t) : IsOpen (s ∩ t)
  /-- 開集合をいくつ集めて合併しても開。 -/
  isOpen_sUnion (S : Set (Set X)) (h : ∀ s ∈ S, IsOpen s) : IsOpen (⋃₀ S)

#check TopologicalSpace
```

```
TopologicalSpace (X : Type) : Type
```

クラスも「型を受け取って型を返す関数」。`TopologicalSpace X` は「`X` 上の位相全体の型」で、その項1つが位相1つに当たる。

補足: 開集合系を「集合の集合」と見る

主フィールドの型 `Set X → Prop` は `Set (Set X)` と同じもの——つまり `IsOpen` は「部分集合の集まり」 $P(P(X))$ の元である。「開集合系を指定する」という日常の言い方が、そのまま型に写っている。

```
-- `export` で `TopologicalSpace.IsOpen` などを接頭辞なしの `IsOpen` で書けるようにする
export TopologicalSpace (IsOpen isOpen_univ isOpen_inter isOpen_sUnion)

-- 以後 `X` は位相空間: 位相はインスタンス引数として各宣言に暗黙に付く
variable {X : Type} [TopologicalSpace X]
```

空集合が開であることは公理に含めなくてよい。空な集合族の合併が空集合だから、`isOpen_sUnion` から従う。

証明は `by` のタクティク（[06_Topology.lean](#) 1節）。まず `⋃₀ ∅ = ∅` を `Set.ext` で示し、`rw [← h]` で等式 `h` を右辺から左辺の向きに使うってゴールを書き換える。

```
theorem isOpen_empty : IsOpen (∅ : Set X) := by
  have h : (⋃₀ (∅ : Set (Set X))) = (∅ : Set X) := by
    apply Set.ext
    intro a
    constructor
```

```

· intro ⟨_, hs, _⟩
  exact False.elim hs
· intro ha
  exact False.elim ha
rw [← h]
exact isOpen_sUnion _ fun _ hs => False.elim hs

```

```
#check isOpen_empty
```

```
isOpen_empty {X : Type} [TopologicalSpace X] : IsOpen ∅
```

`[TopologicalSpace X]` がインスタンス引数として表示に現れる。結論の `IsOpen ∅` がどの位相の話かは、この引数が決めている。

同じ証明を項で直接書くと、次のようになる。`have h : ... := ...;` の並びがタクティクの `have` に、`h ▶ e` が `rw [← h]; exact e` に対応する (`h ▶ e` は、等式 `h` を使って `e` の型を書き換える項の記法である)。

```

example : IsOpen (∅ : Set X) :=
  have h : (∅ : Set (Set X)) = (∅ : Set X) :=
    Set.ext fun _ => ⟨fun ⟨_, hs, _⟩ => False.elim hs, fun ha => False.elim ha⟩
  h ▶ isOpen_sUnion _ fun _ hs => False.elim hs

```

2つの開集合の合併も開。公理は「集合族の合併」の形なので、まず補題 `Set.union_eq_sUnion` で合併を族の合併に書き換えてから公理を適用する。族の各要素は `s` か `t` のどちらかなので、場合分けしてそれぞれの開性を使う。

```

theorem isOpen_union {s t : Set X} (hs : IsOpen s) (ht : IsOpen t) : IsOpen (s u t) := by
  rw [Set.union_eq_sUnion]
  refine isOpen_sUnion _ fun u hu => ?_
  cases hu with
  | inl h =>
    rw [h]
    exact hs
  | inr h =>
    rw [h]
    exact ht

#check isOpen_union

```

```

isOpen_union {X : Type} [TopologicalSpace X] {s t : Set X} (hs : IsOpen s) (ht : IsOpen t)
: IsOpen (s u t)

```

明示引数は証明2つだけ。s t は hs ht の型から決まるので暗黙。

```
-- こちらも項で、`match` の場合分けが `cases` に当たる
example {s t : Set X} (hs : IsOpen s) (ht : IsOpen t) : IsOpen (s ∪ t) :=
  (Set.union_eq_sUnion s t) ▸ isOpen_sUnion _ fun _u hu =>
    match hu with
    | Or.inl h => h ▸ hs
    | Or.inr h => h ▸ ht
```

有限個の開集合の共通部分は開。n についての再帰で、公理の「2つの共通部分」を繰り返す。

無限個の共通部分では成り立たないことに注意（例えば実数直線で $n \mapsto (-1/n, 1/n)$ は開でない）。有限性がここで効く。

```
theorem isOpen_interFin : ∀ (n : Nat) (W : Fin n → Set X), (∀ i, IsOpen (W i)) →
  IsOpen (Set.interFin n W) := fun n W h =>
  match n with
  | 0 => isOpen_univ
  | n + 1 => isOpen_inter _ _ (h 0) (isOpen_interFin n (fun i => W i.succ) fun i => h
    i.succ)

#check isOpen_interFin
```

```
isOpen_interFin {X : Type} [TopologicalSpace X] (n : Nat) (W : Fin n → Set X) :
  (∀ (i : Fin n), IsOpen (W i)) → IsOpen (Set.interFin n W)
```

閉集合: 補集合が開。原始概念は開集合だけなので、閉はこれで定義するほかない。

```
def IsClosed (s : Set X) : Prop := IsOpen sᶜ

#check IsClosed
```

```
IsClosed {X : Type} [TopologicalSpace X] (s : Set X) : Prop
```

IsOpen と同じ形の、集合の性質。

定義の確認

型検査が見ているのは「IsOpen が3公理を満たす」ことだけで、TopologicalSpace という定義が数学の位相空間の定義を写しているかどうかは機械では確かめられない（冒頭の2）。その代わりに、よく知る例が定義を満たすことを見ておく。

離散位相: すべての部分集合が開。どんな型にも入る、いちばん簡単な位相。 `where` 構文でクラスの各フィールドを埋めて、`TopologicalSpace X` の項を作る。公理側はすべて `True` の証明なので `trivial` で済む。頭の `@[reducible]` は属性 (宣言に付ける印) で、「この定義は必要に応じていつでも展開してよい」という指定。 `07_Exercises.lean` の instance 探索で効いてくるだけなので、ここでは気にしなくてよい。

```
@[reducible] def discrete (X : Type) : TopologicalSpace X where
  IsOpen _ := True
  isOpen_univ := trivial
  isOpen_inter := fun _ _ _ => trivial
  isOpen_sUnion := fun _ _ => trivial

#check discrete
```

```
discrete (X : Type) : TopologicalSpace X
```

型を受け取って位相そのもの (`TopologicalSpace X` の項) を返す関数。

⇒ 練習

77. `example : (discrete Nat).IsOpen {n | n = 0}` を証明せよ (離散位相では、どの部分集合の開性も `True` —— 証明は `trivial`)。

解答 77

```
example : (discrete Nat).IsOpen {n | n = 0} := trivial
```

離散位相の `IsOpen` は定義上どの集合でも `True`。証明は構成子 `trivial`。

5. 連続写像

位相空間の間の写像が連続であることを、開集合だけを使って定義する。写像の向きと逆に、行き先の開集合を引き戻して考えるのがポイント。

```
variable {Y : Type} [TopologicalSpace Y] {Z : Type} [TopologicalSpace Z]
```

連続写像: 開集合の逆像がつねに開。

```
def Continuous (f : X → Y) : Prop := ∀ s, IsOpen s → IsOpen (f ⁻¹ s)
```

```
#check Continuous
```

```
Continuous {X : Type} [TopologicalSpace X] {Y : Type} [TopologicalSpace Y] (f : X → Y) :  
Prop
```

両側の空間それぞれにインスタンス引数が付く。「`f` が連続」という命題は、実は「どの位相に関してか」を2つ暗黙に抱えている。

定義の確認

連続の定義についても、よく知る事実がここから出ることを見ておく。どちらも証明は1行で、逆像が定義上ぴったり重なることを使うだけ。

恒等写像は連続。示すべきは `IsOpen ((fun x => x) ⁻¹' s)` だが、`(fun x => x) ⁻¹' s` は定義を展開すると `s` そのもの。だから仮定 `hs : IsOpen s` が、そのまま型の合う項になる。

```
theorem continuous_id : Continuous (fun x : X => x) :=  
  fun _ hs => hs  
  
#check continuous_id
```

```
continuous_id {X : Type} [TopologicalSpace X] : Continuous fun x ↦ x
```

仮定なしの定理。結論の中の `fun x ↦ x` が恒等写像。

連続写像の合成は連続。`(g ∘ f) ⁻¹' s` が `f ⁻¹' (g ⁻¹' s)` と定義上等しいので、引き戻しを2回続けるだけ。名前を `Continuous.comp` としたので、`hg.comp hf` とドット記法で使える ([05_MathematicalTools.lean](#) 7節)。

```
theorem Continuous.comp {g : Y → Z} {f : X → Y} (hg : Continuous g) (hf : Continuous f) :  
  Continuous (fun x => g (f x)) :=  
  fun s hs => hf _ (hg s hs)  
  
#check Continuous.comp
```

```
Continuous.comp {X : Type} [TopologicalSpace X] {Y : Type} [TopologicalSpace Y] {Z : Type}  
[TopologicalSpace Z]  
  {g : Y → Z} {f : X → Y} (hg : Continuous g) (hf : Continuous f) : Continuous fun x ↦ g  
  (f x)
```

空間3つぶんのインスタンス引数が並ぶ。明示引数は証明 `hg hf` の2つで、先に適用される `f` の連続性が後ろに来る（合成の記法 `g ∘ f` と同じ順）。

⇒ 練習

78. `#check @Continuous` の表示を予想してから確かめよ（2つの空間の位相が、どの種類の括弧で並ぶか）。

解答 78

```
#check @Continuous
```

```
@Continuous : {X : Type} → [TopologicalSpace X] → {Y : Type} → [TopologicalSpace Y] →  
(X → Y) → Prop
```

@ 付きの表示では、暗黙引数も含めた引数の列が `→` でつながった 1 本の関数型として見える。2つの空間の位相は、どちらも角括弧 `[TopologicalSpace X] · [TopologicalSpace Y]` ——インスタンス引数——で並ぶ。

6. ハウスドルフ空間

「異なる2点は開集合で見分けられる」という条件。

ハウスドルフ空間であるという性質。フィールドが命題1つだけの `class : Prop` で、データは持たない。位相を前提にするので `[TopologicalSpace X]` を引数に取る。

空間ごとに決まる常置的な性質なので `class` にしてある: 以後の定理は `[Hausdorff Y]` と書くだけで、その証明が登録簿から自動で供給される。

```
class Hausdorff (X : Type) [TopologicalSpace X] : Prop where  
  /-- 異なる2点は、交わらない開集合で分離できる。 -/  
  separate : ∀ x y : X, x ≠ y →  
    ∃ U V : Set X, IsOpen U ∧ IsOpen V ∧ x ∈ U ∧ y ∈ V ∧ U ∩ V = ∅  
  
#check Hausdorff
```

```
Hausdorff (X : Type) [TopologicalSpace X] : Prop
```

`TopologicalSpace X : Type`（データ）と違い、こちらは結果が `Prop`（性質）。

7. コンパクト

「どんな開被覆にも有限部分被覆がある」という条件。

被覆は添字づけられた族 $U : I \rightarrow \text{Set } X$ で表し、部分被覆は添字の部分集合 $J \subseteq I$ を取ることで表す。 J が有限であることは `Set.Finite` (`Fin n` で番号づけられること) で表す。

コンパクト集合。定義を読み下すと: 任意の添字型 I (暗黙引数) と開集合族 $U : I \rightarrow \text{Set } X$ について、 U が K を覆うなら、**添字の有限部分集合** J に間引いても K を覆える。

```
def IsCompact (K : Set X) : Prop :=
  ∀ {I : Type} (U : I → Set X), (∀ i, IsOpen (U i)) → K ⊆ (⋃ i, U i) →
    ∃ J : Set I, J.Finite ∧ K ⊆ (⋃ i ∈ J, U i)
```

```
#check IsCompact
```

```
IsCompact {X : Type} [TopologicalSpace X] (K : Set X) : Prop
```

表示は短い、定義の中身は上のおり $\forall$ が3つ重なった命題である。

補足: IsCompact を class にしない理由

Hausdorff と違って、こちらは class にしない。IsCompact は個々の部分集合ごとの述語で、 $hK : \text{IsCompact } K$ というふつうの仮定として渡したり、補題の**結論**として新しく作ったりする対象だからである (集合ごとに登録簿へ載せる運用はできない)。空間全体の性質である CompactSpace (下) は Hausdorff と同様 class にする。

空間そのものがコンパクトであること: 全体集合 `univ` がコンパクト集合である。Hausdorff と同じく、命題1つだけの `class : Prop`。

```
class CompactSpace (X : Type) [TopologicalSpace X] : Prop where
  isCompact_univ : IsCompact (Set.univ : Set X)
```

```
#check CompactSpace
```

```
CompactSpace (X : Type) [TopologicalSpace X] : Prop
```

8. 補題1: コンパクト集合の連続像はコンパクト

方針: 与えられた $f : K \rightarrow Y$ の開被覆 U に対して

1. 被覆を f で引き戻して K の開被覆 $f^{-1} \cup i$ を作る (`Set.subset_preimage_iUnion`)
2. K のコンパクト性で、有限部分被覆の添字 J を取る
3. 同じ J が $f^{-1} K$ の有限部分被覆を与える (`Set.image_subset_biUnion`)

1 と 3 は位相と無関係な、像と逆像についての一般的な補題である。有限性は受け取った J をそのまま使い回すだけなので、`Fin n` を開ける必要はない。

補足: lemma という宣言名

なお `mathlib` では、細かい補題を `theorem` の同義語 `lemma` で宣言する慣例があるが、`lemma` は `core Lean` にはない (`mathlib` がマクロで定義している)。この教材では一貫して `theorem` を使う。

像が族で覆われるなら、元の集合は逆像の族で覆われる (方針の 1)。位相と無関係な補題なので、節の変数 $X Y$ (位相つき) ではなく新しい型変数 $\alpha \beta$ で述べる。

```
theorem Set.subset_preimage_iUnion {α β : Type} {f : α → β} {K : Set α} {I : Type}
  {U : I → Set β} (h : f '' K ⊆ ⋃ i, U i) : K ⊆ ⋃ i, f ⁻¹ U i := by
  intro x hx
  have ⟨i, hi⟩ := h (f x) ⟨x, hx, rfl⟩
  exact ⟨i, hi⟩

#check Set.subset_preimage_iUnion
```

```
Set.subset_preimage_iUnion {α β : Type} {f : α → β} {K : Set α} {I : Type} {U : I → Set β}
  (h : f '' K ⊆ Set.iUnion fun i => U i) : K ⊆ Set.iUnion fun i => f ⁻¹ U i
```

位相のインスタンス引数が付いていない＝純粋に集合の補題である。

```
-- 項で書くと1行になる。`x ∈ ⋃ i, f ⁻¹ U i` と `f x ∈ ⋃ i, U i` は定義を
-- 展開すると同じ命題なので、`h` を適用した結果がそのまま答えになる
example {α β : Type} {f : α → β} {K : Set α} {I : Type}
  {U : I → Set β} (h : f '' K ⊆ ⋃ i, U i) : K ⊆ ⋃ i, f ⁻¹ U i :=
  fun x hx => h (f x) ⟨x, hx, rfl⟩
```

逆像の部分族で覆われるなら、像は同じ添字の部分族で覆われる (方針の 3)。証明中の `have ⟨x, hx, hf x⟩ := ...` は $\exists$ (依存和) の分解 ([04_Exists.lean](#) 1 節)。

```
theorem Set.image_subset_biUnion {α β : Type} {f : α → β} {K : Set α} {I : Type}
  {J : Set I} {U : I → Set β} (h : K ⊆ ⋃ i ∈ J, f ⁻¹ U i) : f '' K ⊆ ⋃ i ∈ J, U i := by
  intro b hb
  have ⟨x, hx, hf x⟩ := hb
  have ⟨i, hiJ, hxi⟩ := h x hx
```

```
exact ⟨i, hiJ, hfx ▶ hxi⟩
```

```
#check Set.image_subset_biUnion
```

```
Set.image_subset_biUnion {α β : Type} {f : α → β} {K : Set α} {I : Type} {J : Set I} {U : I → Set β}
(h : K ⊆ J.biUnion fun i => f ⁻¹' U i) : f '' K ⊆ J.biUnion fun i => U i
```

-- 項で。タクティク版の `have ⟨...⟩ :=` による分解が `match` に当たる

```
example {α β : Type} {f : α → β} {K : Set α} {I : Type}
{J : Set I} {U : I → Set β} (h : K ⊆ ⋃ i ∈ J, f ⁻¹' U i) : f '' K ⊆ ⋃ i ∈ J, U i :=
fun _b hb =>
  match hb with
  | ⟨x, hx, hfx⟩ =>
    match h x hx with
    | ⟨i, hiJ, hxi⟩ => ⟨i, hiJ, hfx ▶ hxi⟩
```

補題1: コンパクト集合の連続像はコンパクト。方針の3歩をそのまま並べる。下の `#check` を見る前に、この主張が「引数の列：結果」としてどう並ぶかを予想してみてほしい。名前が `IsCompact.image` なので、`hK : IsCompact K` に対し `hK.image hf` と書ける。

```
theorem IsCompact.image {K : Set X} (hK : IsCompact K) {f : X → Y} (hf : Continuous f) :
  IsCompact (f '' K) := by
  intro I U hU hcov
  have ⟨J, hJ, hsub⟩ := hK (fun i => f ⁻¹' U i) (fun i => hf _ (hU i))
    (Set.subset_preimage_iUnion hcov)
  exact ⟨J, hJ, Set.image_subset_biUnion hsub⟩

#check IsCompact.image
```

```
IsCompact.image {X : Type} [TopologicalSpace X] {Y : Type} [TopologicalSpace Y] {K : Set X} (hK : IsCompact K)
{f : X → Y} (hf : Continuous f) : IsCompact (f '' K)
```

数学の主張「 K コンパクト、 f 連続 $\Rightarrow f(K)$ コンパクト」が、そのまま「証明を2つ受け取って証明を返す関数」の型になっている。

-- 項で。`intro` が `fun` に、`have` の分解が `match` に、`exact` が組の構成に対応する

```
example {K : Set X} (hK : IsCompact K) {f : X → Y} (hf : Continuous f) :
  IsCompact (f '' K) :=
fun U hU hcov =>
  match hK (fun i => f ⁻¹' U i) (fun i => hf _ (hU i))
```

```
(Set.subset_preimage_iUnion hcov) with
| (J, hJ, hsub) => (J, hJ, Set.image_subset_biUnion hsub)
```

⇒ 練習

79. `#print axioms Set.subset_preimage_iUnion` の結果を予想してから確かめよ（包含の付け替えだけの証明に、公理は要るだろうか）。

解答 79

```
#print axioms Set.subset_preimage_iUnion
```

```
'Set.subset_preimage_iUnion' does not depend on any axioms
```

包含の付け替えだけの証明なので、公理は1つも要らない。

9. 補題2: ハウスドルフ空間のコンパクト集合は閉

ここが証明の山場で、有限性を実際に使うのもここだけ。方針:

1. 「各点が開近傍ごとに入っている集合は開」という一般論を用意する（`isOpen_of_nhds`）
2. K の外の点 y に、 K と交わらない開近傍 W を作る（`IsCompact.exists_disjoint_nhds`）
3. 1 に 2 を渡せば K^c が開、すなわち K は閉

2 の中身: K の各点 x は y とハウスドルフ性で分離できる。分離する開集合の組を点ごとに1つ選ぶのではなく、「 y を分離する開集合の組」の全体を添字型にして K を覆う。必要なデータ（ y 側の開集合と分離性の証明）が添字自身に抱き合わされているので、選択公理を使わずに済む。この「分離データの束」をまず名前付きの structure として定義してから、補題に進む。コンパクト性で組を有限個に間引き、その V 側の有限交叉（`interFin`）を W とする。無限個の共通部分では開とは限らないので、有限に減らせたことが本質的に効いている。

各点が開近傍ごとに入っている集合は開（方針の 1）。 s は「 s に含まれる開集合すべて」の合併に等しくなるので、合併の公理が使える。

```
theorem isOpen_of_nhds {s : Set X} (h : ∀ a ∈ s, ∃ W, IsOpen W ∧ a ∈ W ∧ W ⊆ s) :
  IsOpen s := by
  have heq : s = ⋃₀ {W | IsOpen W ∧ W ⊆ s} := by
    apply Set.ext
    intro a
    constructor
```

```

· intro ha
  have ⟨W, hW, haW, hWs⟩ := h a ha
  exact ⟨W, ⟨hW, hWs⟩, haW⟩
· intro ⟨W, hW, haW⟩
  exact hW.right a haW
rw [heq]
exact isOpen_sUnion _ fun _ hW => hW.left

```

```
#check isOpen_of_nhds
```

```

isOpen_of_nhds {X : Type} [TopologicalSpace X] {s : Set X} (h : ∀ (a : X), a ∈ s → ∃ W,
  IsOpen W ∧ a ∈ W ∧ W ⊆ s) :
  IsOpen s

```

```

-- 項で、`rw [heq]` が `heq ▸` に当たる
example {s : Set X} (h : ∀ a ∈ s, ∃ W, IsOpen W ∧ a ∈ W ∧ W ⊆ s) :
  IsOpen s :=
  have heq : s = ⋃ {W | IsOpen W ∧ W ⊆ s} :=
  Set.ext fun a =>
    { fun ha =>
      match h a ha with
      | ⟨W, hW, haW, hWs⟩ => ⟨W, ⟨hW, hWs⟩, haW⟩,
      fun ⟨_, hW, haW⟩ => hW.right a haW }
  heq ▸ isOpen_sUnion _ fun _ hW => hW.left

```

「点 y を分離する開集合の組」を名前付きで束ねた structure (`Function.Bijective` と同じ理由で、 $\wedge$ のネストではなくフィールド名を選ぶ)。`left` は K を覆う側として使い、`right` は y を含み、2つは交わらない。フィールドの型が前のフィールドに依存している ([05_MathematicalTools.lean 2節](#)) ことにも注意。

K はどこにも現れない: この structure は「点 y を分離する開集合の組」だけを束ねたもので、 K との関係 (`left` たちが K を覆うこと) は使う側の補題で述べる。定義は必要最小限の材料だけを持つのがよい。

```

structure SeparatingPair (y : Y) where
  /-- 分離の `K` 側の開集合。 -/
  left : Set Y
  /-- 分離の `y` 側の開集合。 -/
  right : Set Y
  isOpen_left : IsOpen left
  isOpen_right : IsOpen right
  mem_right : y ∈ right
  disjoint : left ∩ right = ∅

```

```
#check SeparatingPair
```

```
SeparatingPair {Y : Type} [TopologicalSpace Y] (y : Y) : Type
```

点ごとに「分離データ」の型が決まる。項は6つのフィールド（開集合2つ+証明4つ）の束で、`s.left` や `s.disjoint` で取り出す。

コンパクト集合 `K` の外の点は、`K` と交わらない開近傍を持つ（方針の 2）。

被覆の添字型として、いま定義した `SeparatingPair y` をそのまま使う。つまり添字は「開集合の組」と「それが `y` を分離しているという証明」の抱き合わせであり、被覆の各成分が自分の `right` と分離性を持参してくる。被覆であること自体は、型注釈付きの `hcov` として先に切り出す——型を書いておけばそこから読める、という本教材の方法の実演である（`⋃` 記法は束縛子に型注釈を付けられないので、ここは展開先の `Set.iUnion` を直接書く）。

```
theorem IsCompact.exists_disjoint_nhds [Hausdorff Y] {K : Set Y} (hK : IsCompact K)
  {y : Y} (hy : y ∉ K) : ∃ W, IsOpen W ∧ y ∈ W ∧ ∀ a ∈ W, a ∉ K := by
-- `K` のどの点 `x` も `y` と分離できる。分離データ自身を添字と思えば `K` は覆われる
have hcov : K ⊆ Set.iUnion fun i : SeparatingPair y => i.left := fun x hx => by
  have hne : x ≠ y := fun h => hy (h ▸ hx)
  have ⟨U, V, hU, hV, hU, hV, hUV⟩ := Hausdorff.separate x y hne
  exact ⟨⟨U, V, hU, hV, hU, hV, hUV⟩, hxU⟩
-- コンパクト性で有限個に間引く（最初のラムダの束縛子に添字型を書いて伝える）
have ⟨J, hJ, hsub⟩ := hK (fun i : SeparatingPair y => i.left) (fun i => i.isOpen_left)
hcov
-- `J` は有限なので `Fin n` で番号づけ、対応する `right` 側の有限交叉を `W` とする
have ⟨n, g, hg⟩ := hJ
refine ⟨Set.interFin n fun k => (g k).right, ?_, ?_, ?_⟩
· -- 有限個の開集合の共通部分は開
  exact isOpen_interFin n _ fun k => (g k).isOpen_right
· -- `y` はどの `right` にも入っている
  exact Set.mem_interFin n _ y fun k => (g k).mem_right
· -- `W` が `K` と交わったとすると、ある組の `left` と `right` の両方に入る点ができて矛盾
  intro a ha haK
  have ⟨i, hiJ, hai⟩ := hsub a haK
  have ⟨k, hk⟩ := hg i hiJ
  have hav : a ∈ (g k).right := Set.interFin_mem n _ a ha k
  have hau : a ∈ (g k).left := by rw [hk]; exact hai
  have hmem : a ∈ (g k).left n (g k).right := ⟨hau, hav⟩
  rw [(g k).disjoint] at hmem
  exact hmem

#check IsCompact.exists_disjoint_nhds
```

```
IsCompact.exists_disjoint_nhds {Y : Type} [TopologicalSpace Y] [Hausdorff Y] {K : Set Y}
(hK : IsCompact K) {y : Y}
(hy : ¬y ∈ K) : ∃ W, IsOpen W ∧ y ∈ W ∧ ∀ (a : Y), a ∈ W → ¬a ∈ K
```

```
-- 確認: この補題は選択公理どころか `Classical.choice` にも依存していない
-- (`propext` のみ)。データを添字に抱き合わせた効果がここに現れている。
#print axioms IsCompact.exists_disjoint_nhds
```

```
'IsCompact.exists_disjoint_nhds' depends on axioms: [propext]
```

```
-- 山場の補題も項で書ける。`by` の各行がどの項に写るか、対照しながら読んでほしい
-- (`refine {_, ?_, ?_, ?_}` は ` $\langle \dots, \dots, \dots, \dots \rangle$ ` の直接の構成に、
-- 最後の矛盾は `disjoint` による ` $a \in \emptyset$  すなわち `False` への書き換えになる)
example [Hausdorff Y] {K : Set Y} (hK : IsCompact K)
{y : Y} (hy : y ∉ K) : ∃ W, IsOpen W ∧ y ∈ W ∧ ∀ a ∈ W, a ∉ K :=
have hcov : K ⊆ Set.iUnion fun i : SeparatingPair y => i.left := fun x hx =>
  have hne : x ≠ y := fun h => hy (h ▸ hx)
  match Hausdorff.separate x y hne with
  | ⟨U, V, hU, hV, hxU, hyV, hUV⟩ => ⟨⟨U, V, hU, hV, hyV, hUV⟩, hxU⟩
match hK (fun i : SeparatingPair y => i.left) (fun i => i.isOpen_left) hcov with
| ⟨_, hJ, hsub⟩ =>
  match hJ with
  | ⟨n, g, hg⟩ =>
    ⟨Set.interFin n fun k => (g k).right,
      isOpen_interFin n _ fun k => (g k).isOpen_right,
      Set.mem_interFin n _ y fun k => (g k).mem_right,
      fun a ha haK =>
        match hsub a haK with
        | ⟨i, hiJ, hai⟩ =>
          match hg i hiJ with
          | ⟨k, hk⟩ =>
            have hav : a ∈ (g k).right := Set.interFin_mem n _ a ha k
            have hau : a ∈ (g k).left := hk ▸ hai
            have hmem : a ∈ (g k).left n (g k).right := ⟨hau, hav⟩
            ((g k).disjoint ▸ hmem : a ∈ (∅ : Set Y))
```

補題2: ハウスドルフ空間のコンパクト集合は閉。上の2つの補題を合わせるだけ。

```
theorem IsCompact.isClosed [Hausdorff Y] {K : Set Y} (hK : IsCompact K) : IsClosed K := by
  show IsOpen (Kᶜ : Set Y)
  refine isOpen_of_nhds fun y hy => ?_
  have ⟨W, hW, hyW, hWK⟩ := hK.exists_disjoint_nhds hy
```

```
exact ⟨W, hW, hyW, hWK⟩
```

```
#check IsCompact.isClosed
```

```
IsCompact.isClosed {Y : Type} [TopologicalSpace Y] [Hausdorff Y] {K : Set Y} (hK :  
IsCompact K) : IsClosed K
```

「空間がハウスドルフ」という仮定は `[Hausdorff Y]` というインスタンス引数の形で現れる（証明も登録簿から探される）。

```
-- 項ではここまで縮む。`show` は不要（`IsClosed K` と `IsOpen K` は定義上同じ命題）で、  
-- 補題が返す  $\exists$  の中身が `isOpen_of_nhds` の要求とそのまま一致している  
example [Hausdorff Y] {K : Set Y} (hK : IsCompact K) : IsClosed K :=  
  isOpen_of_nhds fun _ hy => hK.exists_disjoint_nhds hy
```

10. 補題3: コンパクト空間の閉集合はコンパクト

方針: C の開被覆 U に対して

1. 各成分に開集合 C^c を足した族 $U \cup C^c$ は空間全体を覆う
(`Set.univ_subset_iUnion_union_compl`)
2. 空間のコンパクト性で、有限部分被覆の添字 J を取る
3. C の点は C^c 側には入らないので、 U だけの部分被覆に戻せる (`Set.subset_biUnion_of_compl`)

1 と 3 は位相と無関係な集合の補題である。添字集合を増やさずに済ませるため、 I が空の場合だけ主証明で先に片付ける。

被覆の各成分に C^c を足せば空間全体を覆う（方針の 1）。 I が空でないという仮定は、 C の外の点をどれかの成分に割り当てるために要る。`Nonempty I` は「 I の項が少なくとも1つある」という命題で、構成子は `{i}`、使うときは `have {i} := hI` と分解する（証明の中でそうしている）。

```
theorem Set.univ_subset_iUnion_union_compl {α : Type} {C : Set α} {I : Type} {U : I → Set  
α}  
  (hcov : C ⊆ ∪ i, U i) (hI : Nonempty I) :  
  (Set.univ : Set α) ⊆ ∪ i, U i ∪ C^c := by  
  intro x _  
  by_cases hx : x ∈ C  
  · have {i, hi} := hcov x hx  
    exact ⟨i, Or.inl hi⟩  
  · have {i} := hI  
    exact ⟨i, Or.inr hx⟩
```

```
#check Set.univ_subset_iUnion_union_compl
```

```
Set.univ_subset_iUnion_union_compl {α : Type} {C : Set α} {I : Type} {U : I → Set α} (hcov : C ⊆ Set.iUnion fun i ↦ U i) (hI : Nonempty I) : Set.univ ⊆ Set.iUnion fun i ↦ U i ∪ Cᶜ
```

-- 項で、`by\_cases` の正体は排中律 `Classical.em` による場合分けである

```
example {α : Type} {C : Set α} {I : Type} {U : I → Set α} (hcov : C ⊆ ∪ i, U i) (hI : Nonempty I) : (Set.univ : Set α) ⊆ ∪ i, U i ∪ Cᶜ := fun x _ => (Classical.em (x ∈ C)).elim (fun hx => match hcov x hx with | ⟨i, hi⟩ => ⟨i, Or.inl hi⟩) (fun hx => match hI with | ⟨i⟩ => ⟨i, Or.inr hx⟩)
```

全体が $\bigcup i \cup C^c$ たちで覆われるなら、 C は U だけで覆われる（方針の 3）。

```
theorem Set.subset_biUnion_of_compl {α : Type} {C : Set α} {I : Type} {J : Set I} {U : I → Set α} (hsub : (Set.univ : Set α) ⊆ ∪ i ∈ J, U i ∪ Cᶜ) : C ⊆ ∪ i ∈ J, U i := by intro x hx have ⟨i, hiJ, hi⟩ := hsub x trivial cases hi with | inl h => exact ⟨i, hiJ, h⟩ | inr h => exact (h hx).elim #check Set.subset_biUnion_of_compl
```

```
Set.subset_biUnion_of_compl {α : Type} {C : Set α} {I : Type} {J : Set I} {U : I → Set α} (hsub : Set.univ ⊆ J.biUnion fun i ↦ U i ∪ Cᶜ) : C ⊆ J.biUnion fun i ↦ U i
```

-- 項で、`cases hi with` の場合分けは、`match` の入れ子パターンでも書ける

```
example {α : Type} {C : Set α} {I : Type} {J : Set I} {U : I → Set α} (hsub : (Set.univ : Set α) ⊆ ∪ i ∈ J, U i ∪ Cᶜ) : C ⊆ ∪ i ∈ J, U i := fun x hx => match hsub x trivial with | ⟨i, hiJ, Or.inl h⟩ => ⟨i, hiJ, h⟩ | ⟨_, _, Or.inr h⟩ => (h hx).elim
```

補題3: コンパクト空間の閉集合はコンパクト。 `by_cases` は「成り立つ場合」と「成り立たない場合」の古典論理による場合分け。

```
theorem IsClosed.isCompact [CompactSpace X] {C : Set X} (hC : IsClosed C) : IsCompact C :=
by
  intro I U hU hcov
  by_cases hI : Nonempty I
  · have ⟨J, hJ, hsub⟩ :=
      CompactSpace.isCompact_univ (fun i => U i ∪ Cᶜ)
        (fun i => isOpen_union (hU i) hC)
        (Set.univ_subset_iUnion_union_compl hcov hI)
    exact ⟨J, hJ, Set.subset_biUnion_of_compl hsub⟩
  · -- `I` が空なら `C` も空で、空な部分被覆でよい
    refine ⟨∅, Set.Finite.empty, ?_⟩
    intro x hx
    have ⟨i, _⟩ := hcov x hx
    exact (hI (i)).elim

#check IsClosed.isCompact
```

```
IsClosed.isCompact {X : Type} [TopologicalSpace X] [CompactSpace X] {C : Set X} (hC :
IsClosed C) : IsCompact C
```

補題2と対をなす形。こちらの空間側の仮定は `[CompactSpace X]`。

```
-- 項で。暗黙引数 `I` を場合分けで使うので、`fun {I} U ...` と名前を付けて受ける
example [CompactSpace X] {C : Set X} (hC : IsClosed C) : IsCompact C :=
  fun {I} U hU hcov =>
    (Classical.em (Nonempty I)).elim
      (fun hI =>
        match CompactSpace.isCompact_univ (fun i => U i ∪ Cᶜ)
          (fun i => isOpen_union (hU i) hC)
          (Set.univ_subset_iUnion_union_compl hcov hI) with
        | ⟨J, hJ, hsub⟩ => ⟨J, hJ, Set.subset_biUnion_of_compl hsub⟩)
      (fun hI =>
        ⟨∅, Set.Finite.empty, fun x hx =>
          match hcov x hx with
          | ⟨i, _⟩ => (hI (i)).elim))
```

11. 目標: コンパクトからハウスドルフへの連続全単射は同相

同相写像とは、連続な全単射であって逆写像も連続なもの。一般には逆写像の連続性は自動ではないが、定義域がコンパクトで終域がハウスドルフならそれが従う、というのがここでの定理。

方針: 示すべきは「 X の開集合 s について $g^{-1} s$ が開」。

1. 両側逆写像では「 g による逆像」=「 f による像」(`Set.preimage_eq_image`)。これを s^c に使うと、 $g^{-1} s$ は $f^{-1} s^c$ の補集合だと分かる
2. 開集合の補集合は閉 (`isClosed_compl`)
3. 補題3 → 補題1 → 補題2 の連鎖で「 s^c は閉 → s^c はコンパクト → $f^{-1} s^c$ はコンパクト → $f^{-1} s^c$ は閉」
4. 閉の定義は「補集合が開」だから、その補集合である $g^{-1} s$ は開

同相写像。互いに逆な連続写像の組を `structure` で束ねる: データ2つ (`toFun`, `invFun`) と、それらが満たすべき性質4つ。 `Homeomorph X Y : Type` は「 X と Y の間の同相写像」**全体の型**であり、 $X \cong Y$ という命題ではなく、同相写像そのものをデータとして持ち歩く。

```
structure Homeomorph (X Y : Type) [TopologicalSpace X] [TopologicalSpace Y] where
  toFun : X → Y
  invFun : Y → X
  left_inv : ∀ x, invFun (toFun x) = x
  right_inv : ∀ y, toFun (invFun y) = y
  continuous_toFun : Continuous toFun
  continuous_invFun : Continuous invFun

#check Homeomorph
```

```
Homeomorph (X Y : Type) [TopologicalSpace X] [TopologicalSpace Y] : Type
```

結果が `Prop` でなく `Type`。「同相である」という命題ではなく、同相写像というデータの型である。

両側逆写像をもつ写像では、 g による逆像は f による像に等しい (方針の 1)。位相と無関係な、集合の補題である。全単射のとき「逆像で考えても像で考えても同じ」という日常的な言い替えの正体がこれ。

```
theorem Set.preimage_eq_image {α β : Type} {f : α → β} {g : β → α}
  (hgf : ∀ x, g (f x) = x) (hfg : ∀ y, f (g y) = y) (s : Set α) :
  g⁻¹ s = f⁻¹ s := by
  apply Set.ext
  intro y
  constructor
  · -- `g y ∈ s` なら、`y` は `s` の点 `g y` の像 (`f (g y) = y`)
    intro hy
```

```

exact ⟨g y, hy, hfg y⟩
· -- `y = f x` (`x ∈ s`) なら、`g y = g (f x) = x ∈ s`
intro ⟨x, hx, hfx⟩
show g y ∈ s
rw [← hfx, hgf]
exact hx

```

```

#check Set.preimage_eq_image

```

```

Set.preimage_eq_image {α β : Type} {f : α → β} {g : β → α} (hgf : ∀ (x : α), g (f x) = x)
(hfg : ∀ (y : β), f (g y) = y)
(s : Set α) : g -1 s = f ' s

```

```

-- 項で、`rw [← hfx, hgf]` の2回の書き換えが、`>` 2回に分かれる
example {α β : Type} {f : α → β} {g : β → α}
(hgf : ∀ x, g (f x) = x) (hfg : ∀ y, f (g y) = y) (s : Set α) :
g -1 s = f ' s :=
Set.ext fun y =>
  ⟨fun hy => ⟨g y, hy, hfg y⟩,
  fun ⟨x, hx, hfx⟩ =>
    have h1 : g (f x) ∈ s := (hgf x).symm ▸ hx
    (hfx ▸ h1 : g y ∈ s)⟩

```

開集合の補集合は閉（方針の 2）。閉の定義は「補集合が開」なので、二重補集合 $s^{cc} = s$ に帰着する。

```

theorem isClosed_compl {s : Set X} (hs : IsOpen s) : IsClosed sc := by
  show IsOpen (scc : Set X)
  rw [Set.compl_compl]
  exact hs

#check isClosed_compl

```

```

isClosed_compl {X : Type} [TopologicalSpace X] {s : Set X} (hs : IsOpen s) : IsClosed sc

```

```

-- 項で、`rw [Set.compl_compl]` が `>` に、`show` が型注釈に当たる
example {s : Set X} (hs : IsOpen s) : IsClosed sc :=
  ((Set.compl_compl s).symm ▸ hs : IsOpen scc)

```

逆写像の連続性。これが定理の中身: g が連続写像 f の両側逆写像なら、 g も連続。前半の4行が方針の 3（補題3 → 補題1 → 補題2 をドット記法でつなぐ）、後半が方針の 1・4 による書き換えである。

仮定を「 f は全単射」ではなく「両側逆写像 g がある」という形にしたのは、結論が g の連続性——特定の写像 g についての主張——だからである。全単射から逆写像を作る作業（選択公理が要る）は最後の `Homeomorph.ofContinuousBijective` に分離しており、この定理自体は選択公理なしで証明できる。

```

theorem continuous_invFun [CompactSpace X] [Hausdorff Y]
  {f : X → Y} (hf : Continuous f) {g : Y → X}
  (hgf : ∀ x, g (f x) = x) (hfg : ∀ y, f (g y) = y) : Continuous g := by
intro s hs
-- `sᶜ` は閉 → コンパクト → その像はコンパクト → その像は閉
have hcC : IsClosed (sᶜ : Set X) := isClosed_compl hs
have hcpt : IsCompact (sᶜ : Set X) := hcC.isCompact
have himg : IsCompact (f '' (sᶜ : Set X)) := hcpt.image hf
have hcl : IsClosed (f '' (sᶜ : Set X)) := himg.isClosed
-- `g⁻¹ s` は `f '' sᶜ` の補集合 (`preimage_eq_image` を `sᶜ` に適用して補集合を取る。
-- `g⁻¹ sᶜ` と `(g⁻¹ s)ᶜ` は定義上同じ集合であることを使っている)
have heq : g⁻¹ s = (f '' (sᶜ : Set X))ᶜ := by
  rw [← Set.preimage_eq_image hgf hfg (sᶜ : Set X)]
  exact (Set.compl_compl _).symm
rw [heq]
exact hcl

#check continuous_invFun

```

```

continuous_invFun {X : Type} [TopologicalSpace X] {Y : Type} [TopologicalSpace Y]
[CompactSpace X] [Hausdorff Y]
  {f : X → Y} (hf : Continuous f) {g : Y → X} (hgf : ∀ (x : X), g (f x) = x) (hfg : ∀ (y :
Y), f (g y) = y) :
  Continuous g

```

長いが、左から順に「空間の条件（インスタンス引数4つ）→ f の連続性 → g が両側逆写像であること2つ → 結論 g の連続性」と読み下せる。

```

-- 定理の中身も項で、`have` の連鎖はそのまま項の `have` になり、
-- `rw` だけが `▷` と `congrArg`（等式の両辺の補集合を取る）に置き換わる
example [CompactSpace X] [Hausdorff Y]
  {f : X → Y} (hf : Continuous f) {g : Y → X}
  (hgf : ∀ x, g (f x) = x) (hfg : ∀ y, f (g y) = y) : Continuous g :=
fun s hs =>
  have hcC : IsClosed (sᶜ : Set X) := isClosed_compl hs
  have hcpt : IsCompact (sᶜ : Set X) := hcC.isCompact
  have himg : IsCompact (f '' (sᶜ : Set X)) := hcpt.image hf
  have hcl : IsClosed (f '' (sᶜ : Set X)) := himg.isClosed
  have h1 : g⁻¹ (sᶜ : Set X) = f '' (sᶜ : Set X) :=

```

```

Set.preimage_eq_image hgf hfg _
have heq : g -1 s = (f -1 (sc : Set X))c :=
  (Set.compl_compl (g -1 s)).symm.trans (congrArg (·c) h1)
heq ▸ hcl

```

目標の定理: コンパクト空間からハウスドルフ空間への連続全単射は同相写像。(#check の表示を見る前に、主定理の型——空間の条件・ `f` の条件・結論——がどんな引数の列になるか、自分で予想してから確かめてほしい。)

where 構文で `Homeomorph` の6フィールドを埋める。逆写像は、全射性 `hbij.surjective` の「存在する」から `Classical.choose` で1つ選んで作る。存在の証明から値を取り出す操作は、一般には存在の証明の `match` だけでは定義できない (05_MathematicalTools.lean 3節で部分型と比較した)。この選択は、公理 `Classical.choice` に依存する。そのため、この定義には `noncomputable` (この関数は計算はできないが、項としては正当) という印を付ける。

最後のフィールドで使う `_root_` は「トップレベルの名前」の明示。いま埋めているフィールド自身が `continuous_invFun` という同名なので、外の定理の方を指すために付けている。

```

noncomputable def Homeomorph.ofContinuousBijjective [CompactSpace X] [Hausdorff Y]
  (f : X → Y) (hf : Continuous f) (hbij : Function.Bijjective f) : Homeomorph X Y where
toFun := f
invFun := fun y => Classical.choose (hbij.surjective y)
left_inv := fun x => hbij.injective (Classical.choose_spec (hbij.surjective (f x)))
right_inv := fun y => Classical.choose_spec (hbij.surjective y)
continuous_toFun := hf
continuous_invFun :=
  _root_.continuous_invFun hf
  (fun x => hbij.injective (Classical.choose_spec (hbij.surjective (f x))))
  (fun y => Classical.choose_spec (hbij.surjective y))

#check Homeomorph.ofContinuousBijjective

```

```

Homeomorph.ofContinuousBijjective {X : Type} [TopologicalSpace X] {Y : Type}
[TopologicalSpace Y] [CompactSpace X]
[Hausdorff Y] (f : X → Y) (hf : Continuous f) (hbij : Function.Bijjective f) : Homeomorph
X Y

```

この型そのものが主定理の主張である——「コンパクト空間 X からハウスドルフ空間 Y への写像 f が連続かつ全単射なら、 X と Y の同相写像 (というデータ) が得られる」。

使った公理の確認

冒頭の「信じるもの」の3がこれで、`sorry` は使っていない。

ここに形式化の利点がもう1つ現れている: **どの定理がどの公理に依存するかが、機械的に確かめられる形で明示される**。実際、この教材の範囲でも `#print axioms` を当てると

- 補題1 `IsCompact.image` は公理に**一切**依存しない
- 補題2の核 `IsCompact.exists_disjoint_nhds` は `propext` のみ (途中の `#print axioms` で確認したとおり、選択公理なしで証明できた)
- `Classical.choice` が入る経路は3つだけ:
 1. `Set.compl_compl` の背理法 (`Classical.byContradiction`)
 2. 補題3まわりの `by_cases` (`x ∈ C` と `Nonempty I`)。任意の述語への排中律
 3. 主定理での逆写像の構成 (`Classical.choose`。「存在する」からデータを取り出す、`choice` のもっとも直接的な使用)

という勾配が観察できる。紙の証明では「どこで選択公理を使ったか」は注意深く読まないと分からないが、ここでは検査器が正確に教えてくれる。

なお `propext` は集合を述語として定義したことから、`Quot.sound` は `Set.ext` の中の `funext` (Lean では定理) から来ている。

```
#print axioms Homeomorph.ofContinuousBijjective
```

```
'Homeomorph.ofContinuousBijjective' depends on axioms: [propext, Classical.choice, Quot.sound]
```

補足: 排中律から選択公理への依存経路

`by_cases` は排中律を使う。Lean では排中律は公理として直接追加されたものではなく、Diaconescu の定理によって `Classical.choice` から導かれた**定理**である。したがって `by_cases` を使った証明の公理一覧にも `Classical.choice` が現れる。

⇒ 練習

80. `#print axioms isOpen_empty` の結果を予想してから確かめよ。 `Classical.choice` は入るだろうか (合併の公理と `Set.ext` だけで示した証明だった)。

解答 80

```
#print axioms isOpen_empty
```

```
'isOpen_empty' depends on axioms: [propext, Quot.sound]
```

`Set.ext`（中身は `funext` と `propext`）の使用で `propext` と `Quot.sound`（`funext` の道連れ）が入るが、`Classical.choice` は入らない——背理法なしの証明だったことが、ここからも確認できる。

81. `discrete Bool` を使って、`TopologicalSpace Bool` の項を1つ `example` で書け。

解答 81

```
example : TopologicalSpace Bool := discrete Bool
```

`discrete` は「型を受け取って位相そのものを返す関数」なので、`Bool` に適用するだけで項が得られる。

82. `IsCompact.isClosed` の型で、「空間がハウスドルフ」という仮定が3種類の括弧のどれで現れるか予想してから、本文の表示の枠で確かめよ。

解答 82

角括弧（インスタンス引数）で現れる:

```
IsCompact.isClosed {Y : Type} [TopologicalSpace Y] [Hausdorff Y] {K : Set Y} (hK : IsCompact K) : IsClosed K
```

「空間がハウスドルフ」は `[Hausdorff Y]`。位相と同じく空間に常置される性質なので、登録簿から自動で供給される形になっている。波括弧は型や集合の暗黙引数、丸括弧は明示的に渡す証明 `hK` である。

余力があれば、発展演習 `07_Exercises.lean` へ（解答は `07_ExercisesSol.lean`）。位相空間の圏と自由忘却随伴を組み立て、このファイルの主定理のハウスドルフという仮定が外せないことを反例で確かめるほか、位相空間の別定義（閉集合系・閉包・近傍系・ネット）の等価性や、部分空間・積・直和・商をまとめて生む誘導位相とその普遍性を扱う。

import «06_Topology»

発展演習: 位相空間の圏と自由忘却随伴

06_Topology.lean まで読み終えた人のための演習問題。題材は次の5つ。

- 密着位相・離散位相と、その普遍性
- 位相空間の圏と、離散・密着・忘却の3つの関手
- 自由忘却随伴 「離散 $\dashv$ 忘却 $\dashv$ 密着」
- 位相空間の別定義（閉集合系・閉包作用素・近傍系・ネット）と等価性（Part 5）
- 誘導位相（始位相・終位相）と部分空間・積・直和・商、その普遍性（Part 6）

最後に、これらを使って **06_Topology.lean** の主定理のハウスドルフという仮定が外せないことを反例で確かめる。

進め方

- `sorry` と書かれた場所を自分の証明・構成で置き換える。`sorry` が残っているとビルド時に警告が出るので、消えたら完成。
- 「主張も自分で書く」問題は、コメントの指示に従って宣言ごと自分で書く。
- 構造の定義（`Category` など）は与えてある。ただし `Functor` のように「まず自分で設計してから見比べよ」とある定義は、先に紙に書いてみることを勧める。
- 解答は `07_ExercisesSol.lean` にある。
- 各宣言の直後の `#check` には、`06_Topology.lean` と同じ流儀で「表示」と「読み」を添えてある。

記法の準備

`06_Topology.lean` の `Continuous` は位相をインスタンス引数で受け取るので、1つの型に別の位相を載せて論じるには `@` で位相を明示する。例えば `@Continuous X (discrete X) Y tY f` は「 X に離散位相を入れたとき f は連続」。

Part 1: 密着位相

離散位相（`06_Topology.lean` の `discrete`、すべての部分集合が開）の対極として、開集合が $\emptyset$ と `univ` しかない位相を**密着位相**という。

問題1: 密着位相が位相の3公理を満たすことを示せ。

ヒント: `isOpen_sUnion` では「`S` のどれかが `univ` か、全部 $\emptyset$ か」で場合分けする (`by_cases`)。

```
@[reducible] def indiscrete (X : Type) : TopologicalSpace X where
  isOpen s := s =  $\emptyset$   $\vee$  s = Set.univ
  isOpen_univ := sorry
  isOpen_inter := sorry
  isOpen_sUnion := sorry

#check indiscrete
-- 表示: `indiscrete (X : Type) : TopologicalSpace X`
-- 読み: `discrete` と同じ形—型を受け取って位相そのものを返す関数。
```

問題2: 離散位相からの写像はすべて連続であることを示せ。

離散位相では何もかも開なので、逆像の開性はただで手に入る。

```
theorem continuous_from_discrete {X Y : Type} [tY : TopologicalSpace Y] (f : X → Y) :
  @Continuous X (discrete X) Y tY f :=
  sorry

#check continuous_from_discrete
-- 表示: `continuous_from_discrete {X Y : Type} [tY : TopologicalSpace Y] (f : X → Y) :
--      Continuous f`
-- 読み: 表示では暗黙・インスタンス引数が省かれるので、結論がただの `Continuous f` に
-- 見える。実際の主張は `@Continuous X (discrete X) Y tY f` で、`X` 側が離散位相で
-- あることは表示からは読めない。`@` 付きの主張は `#check` の表示だけでは
-- 区別できないことに注意。
```

問題3: 密着位相への写像はすべて連続であることを示せ。

ヒント: 逆像を取る前に開集合は $\emptyset$ か `univ` しかない。 `f-1  $\emptyset$` と `f-1 Set.univ` は定義上それぞれ $\emptyset$ と `univ` に等しい。

```
theorem continuous_to_indiscrete {X Y : Type} [tX : TopologicalSpace X] (f : X → Y) :
  @Continuous X tX Y (indiscrete Y) f :=
  sorry

#check continuous_to_indiscrete
-- 表示: `continuous_to_indiscrete {X Y : Type} [tX : TopologicalSpace X] (f : X → Y) :
--      Continuous f`
-- 読み: 同様に、実際の主張は `@Continuous X tX Y (indiscrete Y) f` (`Y` 側が密着位相)。
```

-- 問題4 (主張も自分で書く):

-- 「密着位相を入れた `Bool` はハウスドルフでない」を形式化し、証明せよ。

-- ヒント: 否定 `¬p` は `p → False` (`!04_Exists.lean` 4節)。

```
-- `true` と `false` を分離する開集合の組が取れたとして、矛盾を導く。  
-- （ここに theorem を書く）
```

Part 2: 圏と関手

「対象」と「射」、恒等射と合成、そして3つの法則。圏の定義は次のとおり与える。 `Obj` と `Hom` がデータ、`id_comp` 以下が性質という構成は、`TopologicalSpace` の定義（`IsOpen` がデータ、3公理が性質）と同じ形をしている。

なお `Functor` という名前は標準ライブラリと衝突するので、この部の定義は名前空間 `Cat` に入れる。

```
namespace Cat  
  
structure Category where  
  /-- 対象の集まり。`Type 1` なのは、対象として `Type` の全体を入れたいから。 -/  
  Obj : Type 1  
  /-- 対象 `A` から `B` への射の型。 -/  
  Hom : Obj → Obj → Type  
  /-- 恒等射。 -/  
  id : (A : Obj) → Hom A A  
  /-- 合成は図式順: `comp f g` は「`f` してから `g`」。 -/  
  comp : {A B C : Obj} → Hom A B → Hom B C → Hom A C  
  /-- 恒等射は合成の左単位。 -/  
  id_comp : ∀ {A B : Obj} (f : Hom A B), comp (id A) f = f  
  /-- 恒等射は合成の右単位。 -/  
  comp_id : ∀ {A B : Obj} (f : Hom A B), comp f (id B) = f  
  /-- 合成の結合律。 -/  
  assoc : ∀ {A B C D : Obj} (f : Hom A B) (g : Hom B C) (h : Hom C D),  
    comp (comp f g) h = comp f (comp g h)
```

#check Category

```
-- 表示: `Cat.Category : Type 2`  
-- 読み: フィールド `Obj` が `Type 1` の項なので、`Category` 自身は `Type 2` の項。  
-- `01_TypesAndTerms.lean` 1節の宇宙の階段（`Type : Type 1 : Type 2`）が実際に必要になる場面である。
```

問題5: 型の圏を作れ。対象は型、射は関数。

法則の証明はすべて `rfl` で済むはず。なぜ済むのかも考えてみる（関数の合成は定義上結合的である）。

```
@[reducible] def TypeCat : Category :=  
  sorry
```

```
#check TypeCat
-- 表示: `Cat.TypeCat : Category`
-- 読み: 「圏」という型の項が1つ手に入った。
```

位相空間の圏の対象: 台の型と、その上の位相を束ねたもの。

`attribute [instance] TopSpace.str` としておくと、`A : TopSpace` に対して `A.carrier` 上の位相が自動で見つかるようになり、`Continuous f` とだけ書ける。

```
structure TopSpace : Type 1 where
  /-- 台の型。 -/
  carrier : Type
  /-- 台の上の位相。 -/
  str : TopologicalSpace carrier

#check TopSpace
-- 表示: `Cat.TopSpace : Type 1`
-- 読み: 台の型 (`Type` の項) をフィールドに含むので、`Type` でなく `Type 1` に住む。

attribute [instance] TopSpace.str
```

問題6: 位相空間の圏を作れ。対象は `TopSpace`、射は「関数と、その連続性の証明の組」`{ f : A.carrier → B.carrier // Continuous f }`。

これは [05_MathematicalTools.lean](#) 3節で見た部分型 (Subtype) である。ヒント: 恒等射は `continuous_id`、合成は `Continuous.comp`。射の等しさは `Subtype.ext` (第一成分が等しければ等しい) で示す。

```
@[reducible] def TopCat : Category :=
  sorry

#check TopCat
-- 表示: `Cat.TopCat : Category`
```

関手

圏から圏への「構造を保つ対応」。対象の対応 `obj` と射の対応 `map` からなり、恒等射と合成を保つ。まず自分でこの `structure` を設計してから、下と見比べよ。

```
structure Functor (C D : Category) where
  /-- 対象の対応。 -/
  obj : C.Obj → D.Obj
```

```

/-- 射の対応。行き先の型が `obj` に依存していることに注意（依存関数型）。 -/
map : {A B : C.Obj} → C.Hom A B → D.Hom (obj A) (obj B)
/-- 恒等射を恒等射に写す。 -/
map_id : ∀ A : C.Obj, map (C.id A) = D.id (obj A)
/-- 合成を合成に写す。 -/
map_comp : ∀ {A B E : C.Obj} (f : C.Hom A B) (g : C.Hom B E),
  map (C.comp f g) = D.comp (map f) (map g)

```

#check Functor

```

-- 表示: `Cat.Functor (C D : Category) : Type 1`
-- 読み: 2つの圏を受け取って「その間の関手全体の型」を返す。

```

問題7: 忘却関手 `U : 06_Topology → Type` を作れ。位相を忘れて台の型だけを残す。

```

@[reducible] def forgetful : Functor TopCat TypeCat :=
  sorry

```

#check forgetful

```

-- 表示: `Cat.forgetful : Functor TopCat TypeCat`

```

問題8: 離散関手 `D : Type → 06_Topology` を作れ。型に離散位相を載せる。射の行き先（任意の写像が連続になること）は問題2で作った。

ヒント: 行き先の位相はインスタンス探索では見つからないので、`continuous_from_discrete (tY := discrete _) f` のように名前付き引数で明示する。

```

@[reducible] def discreteFunctor : Functor TypeCat TopCat :=
  sorry

```

#check discreteFunctor

```

-- 表示: `Cat.discreteFunctor : Functor TypeCat TopCat`

```

問題9: 密着関手 `I : Type → 06_Topology` を作れ。型に密着位相を載せる。射の行き先は問題3で作った。

```

@[reducible] def indiscreteFunctor : Functor TypeCat TopCat :=
  sorry

```

#check indiscreteFunctor

```

-- 表示: `Cat.indiscreteFunctor : Functor TypeCat TopCat`

```

Part 3: 随伴

関手 $F : C \rightarrow D$ と $G : D \rightarrow C$ が随伴 $F \dashv G$ であるとは、 hom 集合の間に自然な全単射

$$\text{Hom}_D(F A, B) \simeq \text{Hom}_C(A, G B)$$

があること。全単射は `06_Topology.lean` の `Homeomorph` と同じ形の `structure` で表す。「自然」の意味は `Adjunction` の2つの等式（`f` を前に合成してから対応させても、対応させてから合成しても同じ、およびその `h` 版）。

```
structure Equiv (α β : Type) where
  /-- 順方向の写像。 -/
  toFun : α → β
  /-- 逆方向の写像。 -/
  invFun : β → α
  /-- 往復すると戻る（`α` 側）。 -/
  left_inv : ∀ a, invFun (toFun a) = a
  /-- 往復すると戻る（`β` 側）。 -/
  right_inv : ∀ b, toFun (invFun b) = b

#check Equiv
-- 表示: `Cat.Equiv (α β : Type) : Type`

structure Adjunction {C D : Category} (F : Functor C D) (G : Functor D C) where
  /-- hom 集合の全単射 `Hom_D(F A, B) ≃ Hom_C(A, G B)`。 -/
  homEquiv : (A : C.Obj) → (B : D.Obj) → Equiv (D.Hom (F.obj A) B) (C.Hom A (G.obj B))
  /-- 自然性（`A` 側）：射 `f` を先に合成してから対応させても、
  対応させてから合成しても同じ。 -/
  naturality_left :
    ∀ {A' A : C.Obj} {B : D.Obj} (f : C.Hom A' A) (g : D.Hom (F.obj A) B),
      (homEquiv A' B).toFun (D.comp (F.map f) g) = C.comp f ((homEquiv A B).toFun g)
  /-- 自然性（`B` 側）。 -/
  naturality_right :
    ∀ {A : C.Obj} {B B' : D.Obj} (g : D.Hom (F.obj A) B) (h : D.Hom B B'),
      (homEquiv A B').toFun (D.comp g h) = C.comp ((homEquiv A B).toFun g) (G.map h)

#check Adjunction
-- 表示: `Cat.Adjunction {C D : Category} (F : Functor C D) (G : Functor D C) : Type 1`
-- 読み: 圏 `C` `D` は関手 `F` `G` の型から決まるので暗黙引数。
```

問題10: 随伴「離散 $\dashv$ 忘却」を作れ。

$\text{Hom}_{\text{Top}}(D X, A) \simeq \text{Hom}_{\text{Type}}(X, U A)$: 左辺の射は「連続な写像」、右辺の射は「ただの写像」だが、離散位相からはどんな写像も連続（問題2）なので、この対応の実体は「同じ関数を右へ左へ読み替えるだけ」で

ある。自然性も含め、ほとんどの証明が `rfl` か `Subtype.ext rfl` で済むはず。

```
def discreteAdj : Adjunction discreteFunctor forgetful :=
  sorry

#check discreteAdj
-- 表示: `Cat.discreteAdj : Adjunction discreteFunctor forgetful`
-- 読み: 型がそのまま「離散  $\rightarrow$  忘却」と読める。
```

問題11: 随伴「忘却 $\rightarrow$ 密着」を作れ。問題10と対称的。

これで 離散 $\rightarrow$ 忘却 $\rightarrow$ 密着 という3連随伴ができたことになる。離散位相が「もっとも細かい位相」、密着位相が「もっとも粗い位相」であることの圏論的な言い換えである。

```
def indiscreteAdj : Adjunction forgetful indiscreteFunctor :=
  sorry

#check indiscreteAdj
-- 表示: `Cat.indiscreteAdj : Adjunction forgetful indiscreteFunctor`
-- 読み: こちらは「忘却  $\rightarrow$  密着」。合わせて 離散  $\rightarrow$  忘却  $\rightarrow$  密着。

end Cat
```

Part 4 (発展) : 反例 — 主定理の仮定は外せない

`06_Topology.lean` の主定理は「コンパクト空間からハウスドルフ空間への連続全単射は同相」。ハウスドルフという仮定を外すと成り立たないことを、ここまでの道具で確かめる。

反例は恒等写像 `id : (Bool, 離散)  $\rightarrow$  (Bool, 密着)`。これは連続（問題3）かつ全単射で、定義域はコンパクト（問題12）だが、逆写像（これも `id`）は連続でない（問題13）。つまり同相ではない。主定理と見比べると、足りないのは終域のハウスドルフ性だけである（問題4）。

```
-- 問題12（主張も自分で書く）：
-- 「離散位相を入れた `Bool` はコンパクト」を形式化し、証明せよ。
-- ヒント：開被覆から `true` を覆う番号と `false` を覆う番号を1つずつ取り、
-- その2つからなる添字集合を使う。有限性の証人は `Fin 2` からの関数で作る。
-- `CompactSpace.mk` のインスタンス引数も `@` で明示する必要がある。
-- （ここに theorem を書く）

-- 問題13（主張も自分で書く）：
-- 「恒等写像 `(Bool, 密着)  $\rightarrow$  (Bool, 離散)` は連続でない」を形式化し、証明せよ。
-- ヒント：離散側で開集合 `{b | b = true}` を考える。その逆像は自分自身だが、
```

```
-- 密着位相の開集合は `∅` と `univ` だけなので、どちらの場合も矛盾する。
-- （ここに theorem を書く）
```

Part 5（さらに発展）：位相空間の別定義と等価性

`TopologicalSpace`（開集合系）は位相の唯一の定義ではない。この Part では

- A: 閉集合系（`ClosedTopology`）
- B: Kuratowski の閉包作用素（`KuratowskiClosure`）
- C: 近傍系（`NeighborhoodSystem`）
- D: ネットによる閉集合の特徴づけ（こちらは公理化ではなく特徴づけ）

を形式化し、A～C が開集合系と等価であることを、Part 3 で作った `Cat.Equiv` を「構造どうしの全単射」として使って示す。

等価性の証明には「同じ `IsOpen` を持つ2つの `TopologicalSpace` は等しい」という外延性補題が要る。証明フィールドは `proof irrelevance` により等しさに影響しないので、述語フィールドの一致だけから構造全体の等しさが出る（下の `ext'`。この形の補題は各構造に対して与えておく）。

この Part の `#check` は表示の注記を省く（読み方はこれまでと同じ）。

集合族の共通部分。 `sUnion` の双対。

```
def Set.sInter {α : Type} (S : Set (Set α)) : Set α := {a | ∀ s ∈ S, a ∈ s}
```

前置記法 `⋂ S`。

```
prefix:110 "⋂ " => Set.sInter
```

包含の反対称性。集合の等式を示す基本手段（以下で多用する）。

```
theorem Set.subset_antisymm {α : Type} {s t : Set α} (h1 : s ⊆ t) (h2 : t ⊆ s) : s = t :=
  Set.ext fun a => ⟨fun ha => h1 a ha, fun ha => h2 a ha⟩
```

同じ `IsOpen` を持つ位相は等しい。 `cases` で両者を分解すると、述語の一致を代入した残りは証明フィールドだけになり、 `proof irrelevance` により `rfl` で閉じる。

```
theorem TopologicalSpace.ext' {X : Type} {t1 t2 : TopologicalSpace X}
  (h : t1.IsOpen = t2.IsOpen) : t1 = t2 := by
  cases t1; cases t2; cases h; rfl
```

問題14: 補集合の基本法則を示せ（ド・モルガンを含む6本）。 `compl_inter` と `compl_sInter` には古典論理（`by_cases` か `Classical.byContradiction`）が要る。どこで要るのか意識しながら書くこと。

```

theorem Set.compl_univ {α : Type} : (Set.univ : Set α)c = ∅ := sorry

theorem Set.compl_empty {α : Type} : (∅ : Set α)c = Set.univ := sorry

theorem Set.compl_union {α : Type} (s t : Set α) : (s ∪ t)c = sc ∩ tc := sorry

theorem Set.compl_inter {α : Type} (s t : Set α) : (s ∩ t)c = sc ∪ tc := sorry

theorem Set.compl_sUnion {α : Type} (S : Set (Set α)) :
  (∪₀ S)c = ∩₀ (Set.compl '' S) := sorry

theorem Set.compl_sInter {α : Type} (S : Set (Set α)) :
  (∩₀ S)c = ∪₀ (Set.compl '' S) := sorry

```

A: 閉集合系

閉集合系による位相の公理化。開集合系の公理の双対になっている。

```

structure ClosedTopology (X : Type) where
  /-- その集合が閉集合であるという述語。 -/
  IsClosed (s : Set X) : Prop
  /-- 空集合は閉。 -/
  isClosed_empty : IsClosed ∅
  /-- 2つの閉集合の合併は閉。 -/
  isClosed_union (s t : Set X) (hs : IsClosed s) (ht : IsClosed t) : IsClosed (s ∪ t)
  /-- 閉集合をいくつ集めて共通部分を取っても閉。 -/
  isClosed_sInter (S : Set (Set X)) (h : ∀ s ∈ S, IsClosed s) : IsClosed (∩₀ S)

theorem ClosedTopology.ext' {X : Type} {c₁ c₂ : ClosedTopology X}
  (h : c₁.IsClosed = c₂.IsClosed) : c₁ = c₂ := by
  cases c₁; cases c₂; cases h; rfl

```

問題15: 閉集合系から開集合系を作れ（開 = 補集合が閉）。問題14を使う。

```

@[reducible] def ClosedTopology.toTop {X : Type} (c : ClosedTopology X) : TopologicalSpace
X where
  IsOpen s := c.IsClosed sc
  isOpen_univ := sorry
  isOpen_inter := sorry
  isOpen_sUnion := sorry

```

問題16: 開集合系から閉集合系を作れ (閉 = 補集合が開)。

```
def TopologicalSpace.toClosed {X : Type} (t : TopologicalSpace X) : ClosedTopology X where
  IsClosed s := t.IsOpen sᶜ
  isClosed_empty := sorry
  isClosed_union := sorry
  isClosed_sInter := sorry
```

問題17: 往復が恒等であることを示し、`Cat.Equiv` にまとめよ。ヒント: `ext'` → `funext` → `show` で二重補集合の形に読み替えて `compl_compl`。

```
theorem toTop_toClosed {X : Type} (t : TopologicalSpace X) : t.toClosed.toTop = t := sorry

theorem toClosed_toTop {X : Type} (c : ClosedTopology X) : c.toTop.toClosed = c := sorry

def topEquivClosed (X : Type) : Cat.Equiv (TopologicalSpace X) (ClosedTopology X) := sorry
```

B: Kuratowski の閉包作用素

Kuratowski の閉包公理。「閉包」の振る舞いだけで位相を決める。

```
structure KuratowskiClosure (X : Type) where
  /-- 閉包作用素。 -/
  cl (s : Set X) : Set X
  /-- 空集合の閉包は空。 -/
  cl_empty : cl ∅ = ∅
  /-- 拡大性: もとの集合を含む。 -/
  subset_cl (s : Set X) : s ⊆ cl s
  /-- 有限合併の保存。 -/
  cl_union (s t : Set X) : cl (s ∪ t) = cl s ∪ cl t
  /-- 冪等性: 2回閉包しても変わらない。 -/
  cl_cl (s : Set X) : cl (cl s) = cl s

theorem KuratowskiClosure.ext' {X : Type} {k₁ k₂ : KuratowskiClosure X}
  (h : k₁.cl = k₂.cl) : k₁ = k₂ := by
  cases k₁; cases k₂; cases h; rfl
```

位相から定める閉包: s を含む閉集合すべての共通部分。

```
def TopologicalSpace.closure {X : Type} (t : TopologicalSpace X) (s : Set X) : Set X :=
  ⋂₀ {u | t.IsOpen uᶜ ∧ s ⊆ u}
```

問題18: 閉包の基本性質4本を示せ。 `closure_isClosed` には問題14の `compl_sInter` が要る。

```
theorem TopologicalSpace.subset_closure {X : Type} (t : TopologicalSpace X) (s : Set X) :  
  s ⊆ t.closure s := sorry
```

```
theorem TopologicalSpace.closure_min {X : Type} (t : TopologicalSpace X) {s u : Set X}  
  (hu : t.IsOpen uc) (hsu : s ⊆ u) : t.closure s ⊆ u := sorry
```

```
theorem TopologicalSpace.closure_isClosed {X : Type} (t : TopologicalSpace X) (s : Set X)  
  :  
    t.IsOpen (t.closure s)c := sorry
```

```
theorem TopologicalSpace.closure_mono {X : Type} (t : TopologicalSpace X) {s u : Set X}  
  (h : s ⊆ u) : t.closure s ⊆ t.closure u := sorry
```

問題19: 閉包公理から単調性を導け。使ってよいのは `cl_union` だけ。ヒント: $s \subseteq t$ なら $s \cup t = t$ 。

```
theorem KuratowskiClosure.mono {X : Type} (k : KuratowskiClosure X) {s t : Set X}  
  (h : s ⊆ t) : k.cl s ⊆ k.cl t := sorry
```

問題20: 閉包作用素から開集合系を作れ（開 = 補集合が `cl` の不動点）。 `isOpen_sUnion` が山場: `subset_antisymm` と `mono` を使う。

```
@[reducible] def KuratowskiClosure.toTop {X : Type} (k : KuratowskiClosure X) :  
  TopologicalSpace X where  
    IsOpen s := k.cl sc = sc  
    isOpen_univ := sorry  
    isOpen_inter := sorry  
    isOpen_sUnion := sorry
```

問題21: 位相から閉包作用素を作れ。問題18の4本を使う。

```
def TopologicalSpace.toKur {X : Type} (t : TopologicalSpace X) : KuratowskiClosure X where  
  cl := t.closure  
  cl_empty := sorry  
  subset_cl := sorry  
  cl_union := sorry  
  cl_cl := sorry
```

問題22: 往復が恒等であることを示し、 `Cat.Equiv` にまとめよ。

```
theorem toKur_toTop {X : Type} (t : TopologicalSpace X) : t.toKur.toTop = t := sorry
```

```

theorem toTop_toKur {X : Type} (k : KuratowskiClosure X) : k.toTop.toKur = k := sorry

def topEquivKur (X : Type) : Cat.Equiv (TopologicalSpace X) (KuratowskiClosure X) := sorry

```

C: 近傍系

近傍系による位相の公理化（ハウスドルフ流）。

```

structure NeighborhoodSystem (X : Type) where
  /-- 各点に、その「近傍」の族を割り当てる。 -/
  N (x : X) : Set (Set X)
  /-- 全体集合はどの点の近傍でもある。 -/
  univ_mem (x : X) : Set.univ ∈ N x
  /-- 近傍はその点を含む。 -/
  mem_of (x : X) (U : Set X) (h : U ∈ N x) : x ∈ U
  /-- 近傍を含む集合は近傍。 -/
  superset (x : X) (U V : Set X) (hU : U ∈ N x) (hUV : U ⊆ V) : V ∈ N x
  /-- 2つの近傍の共通部分は近傍。 -/
  inter (x : X) (U V : Set X) (hU : U ∈ N x) (hV : V ∈ N x) : U ∩ V ∈ N x
  /-- 近傍 `U` の中には「その各点にとっても `U` が近傍」となる近傍 `V` がある。 -/
  interior (x : X) (U : Set X) (h : U ∈ N x) : ∃ V ∈ N x, ∀ y ∈ V, U ∈ N y

theorem NeighborhoodSystem.ext' {X : Type} {n1 n2 : NeighborhoodSystem X}
  (h : n1.N = n2.N) : n1 = n2 := by
  cases n1; cases n2; cases h; rfl

```

問題23: 近傍系から開集合系を作れ（開 = 自分の各点の近傍である集合）。

```

@[reducible] def NeighborhoodSystem.toTop {X : Type} (n : NeighborhoodSystem X) :
TopologicalSpace X where
  IsOpen s := ∀ x ∈ s, s ∈ n.N x
  isOpen_univ := sorry
  isOpen_inter := sorry
  isOpen_sUnion := sorry

```

問題24: 開集合系から近傍系を作れ（近傍 = 開集合を間に挟んで点を含む集合）。

```

def TopologicalSpace.toNbhd {X : Type} (t : TopologicalSpace X) : NeighborhoodSystem X
where
  N x := {U | ∃ V, t.IsOpen V ∧ x ∈ V ∧ V ⊆ U}
  univ_mem := sorry
  mem_of := sorry
  superset := sorry

```

```
inter := sorry
interior := sorry
```

問題25: 往復が恒等であることを示し、`Cat.Equiv` にまとめよ。片方は `06_Topology.lean` の `isOpen_of_nhds` がそのまま効く。もう片方の鍵は $W := \{y \mid U \in \mathcal{N} y\}$ が開になること (`interior` 公理を使う)。

```
theorem toNbhd_toTop {X : Type} (t : TopologicalSpace X) : t.toNbhd.toTop = t := sorry

theorem toTop_toNbhd {X : Type} (n : NeighborhoodSystem X) : n.toTop.toNbhd = n := sorry

def topEquivNbhd (X : Type) : Cat.Equiv (TopologicalSpace X) (NeighborhoodSystem X) :=
  sorry
```

D: ネットによる閉集合の特徴づけ

有向な添字: 空でなく、どの2元にも上界がある。

```
structure DirectedIndex : Type 1 where
  /-- 添字の型。 -/
  ι : Type
  /-- 「後」の関係。 -/
  le : ι → ι → Prop
  /-- 空でない。 -/
  inhabited : Nonempty ι
  /-- どの2元にも共通の「後」がある。 -/
  upper (i j : ι) : ∃ k, le i k ∧ le j k
```

ネット $\text{net} : D.\iota \rightarrow X$ が点 a に収束する: a のどの開近傍にも、ある添字から先はすべて入る。

```
def Converges {X : Type} (t : TopologicalSpace X) (D : DirectedIndex)
  (net : D.ι → X) (a : X) : Prop :=
  ∀ U, t.IsOpen U → a ∈ U → ∃ d, ∀ e, D.le d e → net e ∈ U
```

問題26: 閉集合の中のネットの極限は、その閉集合に入ること示せ。ヒント: `upper d d` で「d 以後」の添字を1つ取る。

```
theorem nets_of_isClosed {X : Type} (t : TopologicalSpace X) {s : Set X}
  (hs : t.IsOpen sᶜ) (D : DirectedIndex) (net : D.ι → X)
  (hnet : ∀ e, net e ∈ s) {a : X} (hconv : Converges t D net a) : a ∈ s := sorry
```

問題27: 逆——中のネットの極限で閉じている集合は閉であることを示せ。

方針: `isOpen_of_nhds` に持ち込む。 $a \in s^\circ$ の開近傍が取れないと仮定すると、 a のどの開近傍も s と交わる。そこで添字型を「 a の開近傍の全体」(部分型)とし、包含の逆向きで有向にして、各近傍から s の点を `Classical.choose` で選んでネットを作ると、それは a に収束し、仮定から $a \in s$ となって矛盾する。 `06_Topology.lean` の補題2 (選択を避けた) と対照的に、ここでは選択が本質的に要る。できたら `#print axioms` で確認せよ。

```
theorem isClosed_of_nets {X : Type} (t : TopologicalSpace X) {s : Set X}
  (h : ∀ (D : DirectedIndex) (net : D.ι → X), (∀ e, net e ∈ s) →
    ∀ a, Converges t D net a → a ∈ s) :
  t.IsOpen s∘ := sorry
```

Part 6: 誘導位相 — 位相を写像に沿って移す

ここまで位相は空間ごとに個別に与えてきた (`discrete`・`indiscrete`、`06_Topology.lean` の `[TopologicalSpace X]`)。この Part では、すでに位相を持つ空間から、写像に沿って新しい空間へ位相を移す一般的な方法を2つ作る。

- **終位相** (押し出し) : 写像の族 $f_i : Y_i \rightarrow X$ の行き先 X に位相を入れる。「すべての f_i が連続になる最も細かい位相」。
- **始位相** (引き戻し) : 写像の族 $f_i : X \rightarrow Y_i$ の出発点 X に位相を入れる。「すべての f_i が連続になる最も粗い位相」。

部分空間・積・直和・商という日常的な構成は、すべてこの2つの特殊化として一挙に手に入る。さらに「どんな写像が連続になるか」の判定法 (普遍性) も、族の一般論として一度証明すれば全構成で使い回せる。

なお、商位相の例では商型 `Quot` を使う。`05_MathematicalTools.lean` の末尾で「商型はこの教材では使わない」と述べたのは本編 (Intro・CH・06_Topology) の話で、発展演習のこの Part で初めて登場する。

押し出しから: 逆像は集合演算と交換する

「 $f^{-1} s$ が開なら s を開と呼ぶ」——これが押し出しの定義になる。位相の公理を満たすのは、逆像が集合演算と交換するからである。 `univ` と `n` については、両辺が定義上同じ集合になる:

```
example {α β : Type} (f : α → β) : f-1 (Set.univ : Set β) = Set.univ := rfl

example {α β : Type} (f : α → β) (s t : Set β) :
  f-1 (s ∩ t) = f-1 s ∩ f-1 t := rfl
```

問題28: 族の合併との交換は `rfl` では済まない ($\exists$ の組み替えが要る) ので、補題として示せ。

`Set.preimage f-1 S` は「 S に属する各集合の逆像を集めた族」(Part 5 の `Set.compl-1 S` と同じ書

き方)。

```
theorem Set.preimage_sUnion {α β : Type} (f : α → β) (S : Set (Set β)) :  
  f ⁻¹' (⋃₀ S) = ⋃₀ (Set.preimage f ' S) := sorry
```

問題29: 押し出し位相 (終位相の1本版) を完成させよ。 `isOpen_univ` ・ `isOpen_inter` は上の `rfl` 級の交換のおかげで、 `tX` の対応する公理を**そのまま**返せば通る。 `isOpen_sUnion` は `show tX.isOpen (f ⁻¹' (⋃₀ S))` で目標を読み替えてから問題28で書き換える。

```
@[reducible] def TopologicalSpace.coinduced {X Y : Type} (tX : TopologicalSpace X)  
  (f : X → Y) : TopologicalSpace Y where  
  isOpen s := tX.isOpen (f ⁻¹' s)  
  isOpen_univ := sorry  
  isOpen_inter := sorry  
  isOpen_sUnion := sorry  
  
#check TopologicalSpace.coinduced  
-- 表示: `TopologicalSpace.coinduced {X Y : Type} (tX : TopologicalSpace X) (f : X → Y) :  
--       TopologicalSpace Y`  
-- 読み: 位相と写像を受け取り、行き先の上の位相を返す。`tX.coinduced f` と  
--       ドット記法で使う。
```

問題30: 終位相 (族版) を完成させよ。「族のすべての `f i` について逆像が開」。証明は1本版と同じ形を `∀ i` の下で繰り返すだけである。添字ごとに空間 `Y i` が違ってよい (依存関数型の族) ことに注意。

```
@[reducible] def finalTopology {I : Type} {Y : I → Type} {X : Type}  
  (tY : (i : I) → TopologicalSpace (Y i)) (f : (i : I) → Y i → X) :  
  TopologicalSpace X where  
  isOpen s := ∀ i, (tY i).isOpen (f i ⁻¹' s)  
  isOpen_univ := sorry  
  isOpen_inter := sorry  
  isOpen_sUnion := sorry  
  
#check finalTopology  
-- 表示: `finalTopology {I : Type} {Y : I → Type} {X : Type}  
--       (tY : (i : I) → TopologicalSpace (Y i)) (f : (i : I) → Y i → X) :  
--       TopologicalSpace X`
```

引き戻しの困難と、上からの定義

引き戻し側を同じ発想で「`f i ⁻¹' u` (`u` は `Y i` の開集合) の形の集合を開とする」と定めたいが、**族ではこれは位相にならない**。別々の添字から来た逆像の共通部分 `f i ⁻¹' u ∩ f j ⁻¹' v` が、また逆像

の形になる保証がないからである（1本なら $f^{-1} u \cap f^{-1} v = f^{-1} (u \cap v)$ で閉じるが、族では閉じない）。

教科書はここで「逆像たちから**生成される**位相」（有限交叉の合併を全部足したもの）を取る。同じものを、ここでは**上から**定義する：

s が開 $\Leftrightarrow$ すべての f_i を連続にするどの位相 t' でも s は開

つまり「候補すべての共通部分」である。生成の組み合わせ論を経由しないので、公理の検証は拍子抜けするほど短い——各公理は、任意に与えられた t' の対応する公理をそのまま返すだけで済む。代わりに開集合の**具体形**は定義から直接は読めなくなるが、それは後の問題36で（部分空間の場合に）取り出す。

問題31: 始位相（族版）を完成させよ。各フィールドは1行で書ける。

```
@[reducible] def initialTopology {I : Type} {Y : I → Type} {X : Type}
  (tY : (i : I) → TopologicalSpace (Y i)) (f : (i : I) → X → Y i) :
  TopologicalSpace X where
  IsOpen s := ∀ t' : TopologicalSpace X,
    (∀ i u, (tY i).IsOpen u → t'.IsOpen (f i ⁻¹ u)) → t'.IsOpen s
  isOpen_univ := sorry
  isOpen_inter := sorry
  isOpen_sUnion := sorry

#check initialTopology
-- 表示: `initialTopology {I : Type} {Y : I → Type} {X : Type}
--       (tY : (i : I) → TopologicalSpace (Y i)) (f : (i : I) → X → Y i) :
--       TopologicalSpace X`
-- 読み: `finalTopology` と見比べると、族の向きが `Y i → X` から `X → Y i` に
-- 反転している。それだけで定義の中身はこれほど変わる。
```

定義の確認: 連続性と極値性

「すべての f_i が連続になる最も細かい／粗い位相」という言明を、2定理ずつに分けて確かめる。異なる位相を比べるので `@Continuous` で位相を明示する（冒頭「記法の補足」参照）。

問題32(1): 終位相のもとで各 f_i は連続であることを示せ。定義を展開すると、連続性の言明そのものが現れる。

```
theorem continuous_toFinal {I : Type} {Y : I → Type} {X : Type}
  (tY : (i : I) → TopologicalSpace (Y i)) (f : (i : I) → Y i → X) (i : I) :
  @Continuous (Y i) (tY i) X (finalTopology tY f) (f i) := sorry
```

問題32(2): 終位相は最も細かい:すべての f_i を連続にする位相 t' の開集合は、終位相でも開であることを示せ。

```
theorem final_finetest {I : Type} {Y : I → Type} {X : Type}
  {tY : (i : I) → TopologicalSpace (Y i)} {f : (i : I) → Y i → X}
  {t' : TopologicalSpace X} (h : ∀ i, @Continuous (Y i) (tY i) X t' (f i)) :
  ∀ s, t'.IsOpen s → (finalTopology tY f).IsOpen s := sorry
```

問題33(1): 始位相のもとで各 f_i は連続であることを示せ。「どの位相でも開」という条件の中に、ちょうど $f_i^{-1} u$ の開性が入っている。

```
theorem continuous_fromInitial {I : Type} {Y : I → Type} {X : Type}
  (tY : (i : I) → TopologicalSpace (Y i)) (f : (i : I) → X → Y i) (i : I) :
  @Continuous X (initialTopology tY f) (Y i) (tY i) (f i) := sorry
```

問題33(2): 始位相は最も粗い:始位相の開集合は、すべての f_i を連続にするどの位相 t' でも開であることを示せ。

```
theorem initial_coarsest {I : Type} {Y : I → Type} {X : Type}
  {tY : (i : I) → TopologicalSpace (Y i)} {f : (i : I) → X → Y i}
  {t' : TopologicalSpace X} (h : ∀ i, @Continuous X t' (Y i) (tY i) (f i)) :
  ∀ s, (initialTopology tY f).IsOpen s → t'.IsOpen s := sorry
```

普遍性

構成した位相の使い道は、次の**普遍性**に集約される。

- 終位相からの写像 $g : X \rightarrow Z$ が連続 $\iff$ 各合成 $g \circ f_i$ が連続
- 始位相への写像 $g : Z \rightarrow X$ が連続 $\iff$ 各合成 $f_i \circ g$ が連続

写像の連続性が、合成の連続性だけで機械的に判定できるようになる。

問題34(1): 押し出し (1本版) の普遍性。実は**証明を書く必要がない**:両辺を定義に沿って展開すると同じ命題に行き着く (何で閉じられるか考えよ)。($(g \circ f)^{-1} s$ と $f^{-1} (g^{-1} s)$) が定義上同じ集合であることが効いている。

```
theorem continuous_fromCoinduced_iff {X Y Z : Type} (tX : TopologicalSpace X)
  (tZ : TopologicalSpace Z) (f : X → Y) (g : Y → Z) :
  @Continuous Y (tX.coinduced f) Z tZ g  $\iff$  @Continuous X tX Z tZ (fun x => g (f x)) :=
  sorry
```

問題34(2): 終位相 (族版) の普遍性を示せ。ヒント: $\forall$ の順序の入れ替えだけ。

```

theorem continuous_fromFinal_iff {I : Type} {Y : I → Type} {X Z : Type}
  (tY : (i : I) → TopologicalSpace (Y i)) (f : (i : I) → Y i → X)
  (tZ : TopologicalSpace Z) (g : X → Z) :
  @Continuous X (finalTopology tY f) Z tZ g ↔
  ∀ i, @Continuous (Y i) (tY i) Z tZ (fun y => g (f i y)) := sorry

```

問題35: 始位相の普遍性を示せ。逆向き（各合成が連続 $\Rightarrow$ g が連続）が山場。ヒント: 始位相の開集合 s の定義は「…するどの位相でも開」だった。そこに「 tZ を g で押し出した位相」を食わせると、ちょうど欲しい $tZ.IsOpen (g^{-1} s)$ が出てくる。押し出しが引き戻しの普遍性を支える、この Part の白眉である。

```

theorem continuous_toInitial_iff {I : Type} {Y : I → Type} {X Z : Type}
  (tY : (i : I) → TopologicalSpace (Y i)) (f : (i : I) → X → Y i)
  (tZ : TopologicalSpace Z) (g : Z → X) :
  @Continuous Z tZ X (initialTopology tY f) g ↔
  ∀ i, @Continuous Z tZ (Y i) (tY i) (fun z => f i (g z)) := sorry

```

部分空間・積・直和・商

4つの日常的な構成を、始位相・終位相の特殊化として `instance` 登録する（ここは問題ではなく、以下すべて与える）。登録した瞬間から、`Subtype p` や $X \times Y$ と書くだけで位相が自動で載るようになる——[05_MathematicalTools.lean](#) 4節で見た `class` の効き目である。

1本の写像に沿った引き戻し。族版の `I := Unit`（添字が1点）への特殊化。

```

@[reducible] def TopologicalSpace.induced {X Y : Type} (f : X → Y)
  (t : TopologicalSpace Y) : TopologicalSpace X :=
  initialTopology (Y := fun _ : Unit => Y) (fun _ => t) (fun _ => f)

```

部分空間位相: 包含写像 `Subtype.val` に沿った引き戻し。[05_MathematicalTools.lean](#) 3節の部分型が、晴れて**部分空間**になる。

```

instance instTopSubtype {X : Type} [tX : TopologicalSpace X] (p : X → Prop) :
  TopologicalSpace (Subtype p) :=
  tX.induced Subtype.val

```

積位相（一般の直積）: 射影の族 `fun g => g i` の始位相。族版の真価が出る例。

```

instance instTopPi {I : Type} {Y : I → Type} [tY : (i : I) → TopologicalSpace (Y i)] :
  TopologicalSpace ((i : I) → Y i) :=
  initialTopology tY fun i g => g i

```

二項の積位相: 射影2本を `I := Bool` の族とみなした始位相。添字によって行き先の型が違う (`true` なら `X`、`false` なら `Y`) ので、族を書くのに `match` による場合分け——依存関数型——が要る。

```
instance instTopProd {X Y : Type} [t₁ : TopologicalSpace X] [t₂ : TopologicalSpace Y] :
  TopologicalSpace (X × Y) :=
  initialTopology (Y := fun b => cond b X Y)
  (fun b => match b with | true => t₁ | false => t₂)
  (fun b => match b with | true => Prod.fst | false => Prod.snd)
```

直和位相: 包含の族 `Sigma.mk i : Y i → (j : I) × Y j` の終位相。

```
instance instTopSigma {I : Type} {Y : I → Type} [tY : (i : I) → TopologicalSpace (Y i)] :
  TopologicalSpace ((i : I) × Y i) :=
  finalTopology tY fun i => Sigma.mk i
```

商位相: 商への射影 `Quot.mk r` に沿った押し出し。 `Quot r` は、型 `X` を関係 `r` で「貼り合わせた」商型である。

```
instance instTopQuot {X : Type} [tX : TopologicalSpace X] {r : X → X → Prop} :
  TopologicalSpace (Quot r) :=
  tX.coinduced (Quot.mk r)

-- 定義の確認: 直和への包含は連続 (問題32(1)の特殊化がそのまま通る)
example {I : Type} {Y : I → Type} [tY : (i : I) → TopologicalSpace (Y i)] (i : I) :
  Continuous (Sigma.mk i : Y i → (j : I) × Y j) :=
  continuous_toFinal tY (fun i => Sigma.mk i) i

-- 定義の確認: 二項積の第一射影は連続 (問題33(1)を `i := true` で使う。
-- 族を明示して渡すと、`match` が `true` の枝に簡約されて `Prod.fst` になる)
example {X Y : Type} [t₁ : TopologicalSpace X] [t₂ : TopologicalSpace Y] :
  Continuous (@Prod.fst X Y) :=
  continuous_fromInitial (Y := fun b => cond b X Y)
  (fun b => match b with | true => t₁ | false => t₂)
  (fun b => match b with | true => Prod.fst | false => Prod.snd) true
```

問題36: 部分空間位相の開集合の具体形を取り出せ: `s` が開 $\iff$ ある開集合 `u` の逆像 `Subtype.val ⁻¹' u` に等しい。

$\iff$ は定義どおり。 $\Rightarrow$ が本番で、「`val ⁻¹' u` (`u` 開) の形の集合の全体」が **位相を成す**ことを示し、それを始位相の定義に食わせる (問題35と同じ発想)。合併のケースでは、各集合ごとに `u` を選ぶと選択公理が要りそうになるが、「逆像が `S` に入るような開集合を**全部**集めた合併」を証人にすれば選択なしで済む。

```

theorem isOpen_subtype_iff {X : Type} [tX : TopologicalSpace X] {p : X → Prop}
  {s : Set (Subtype p)} :
  IsOpen s ↔ ∃ u, IsOpen u ∧ s = Subtype.val-1 u := sorry

```

問題37(1): 射影（直積の各点での値を取る写像）は連続であることを示せ。ヒント: 問題33(1)の特殊化。

```

theorem continuous_apply {I : Type} {Y : I → Type} [tY : (i : I) → TopologicalSpace (Y i)]
  (i : I) : Continuous fun g : (j : I) → Y j => g i := sorry

```

問題37(2): 直積への写像の連続性は成分ごとに調べればよいことを示せ。ヒント: 問題35の特殊化。「各点収束位相」とも呼ばれる積位相の使い勝手は、すべてこの1行に集約される。

```

theorem continuous_pi_iff {I : Type} {Y : I → Type} [tY : (i : I) → TopologicalSpace (Y i)]
  {Z : Type} [tZ : TopologicalSpace Z] (g : Z → (i : I) → Y i) :
  Continuous g ↔ ∀ i, Continuous fun z => g z i := sorry

```

問題38(1): 商への射影は連続であることを示せ（押し出しの定義そのもの）。

```

theorem continuous_quotMk {X : Type} [tX : TopologicalSpace X] (r : X → X → Prop) :
  Continuous (Quot.mk r) := sorry

```

問題38(2): 商からの写像の連続性は、持ち上げる前の写像で調べればよいことを示せ。ヒント: `Quot.lift` $g \circ \text{Quot.mk } r = g$ は商型の計算規則により定義上の等式なので、問題34(1)がそのまま効く。

```

theorem continuous_quotLift {X Z : Type} [tX : TopologicalSpace X] [tZ : TopologicalSpace Z]
  {r : X → X → Prop} {g : X → Z} (hg : ∀ a b, r a b → g a = g b) :
  Continuous (Quot.lift g hg) ↔ Continuous g := sorry

```

問題39(1): 空族 ($I := \text{Empty}$) の始位相は密着位相であることを示せ。「どの位相でも開」な集合は $\emptyset$ と univ しかない（片方向は、密着位相自身を定義に食わせると分かる）。Part 1 の `indiscrete` が、誘導位相の退化した場合として回収される。

```

theorem initialTopology_empty {X : Type} {Y : Empty → Type}
  (tY : (i : Empty) → TopologicalSpace (Y i)) (f : (i : Empty) → X → Y i) :
  initialTopology tY f = indiscrete X := sorry

```

問題39(2): 空族の終位相は離散位相であることを示せ（条件が空なので何でも開）。Part 3 の随伴の言葉でいえば、離散・密着が「もっとも細かい／粗い」ことと、空族の終位相・始位相であることは、同じ事実の2つの言い方である。

```

theorem finalTopology_empty {X : Type} {Y : Empty → Type}
  (tY : (i : Empty) → TopologicalSpace (Y i)) (f : (i : Empty) → Y i → X) :
  finalTopology tY f = discrete X := sorry

```

商写像 — 主定理の再訪

終域の位相が押し出し位相に一致する連続全射を**商写像**と呼ぶ。商写像 f に対しては、問題34(1)の普遍性がそのまま「 f を通した連続性判定」になるので、これは「終域が商位相だと思ってよい写像」ということである。

一般の連続全射は商写像とは限らない (Part 4 の恒等写像 $(\text{Bool}, \text{密着}) \rightarrow (\text{Bool}, \text{離散})$ の逆向き $(\text{Bool}, \text{離散}) \rightarrow (\text{Bool}, \text{密着})$ が反例: 押し出しは離散になるが終域は密着)。ところが、`06_Topology.lean` の主定理とまったく同じ仮定——定義域がコンパクトで終域がハウスドルフ——の下では、連続全射は必ず商写像になる。証明も主定理の核 (補題1~3の連鎖) の再利用である。

問題40: 全射なら「逆像を取ってから像を取る」と元に戻ることを示せ ($\supseteq$ 向きで全射性を使う)。

```

theorem Set.image_preimage_of_surjective {α β : Type} {f : α → β}
  (hf : Function.Surjective f) (u : Set β) : f '' (f ⁻¹' u) = u := sorry

```

問題41: コンパクト空間からハウスドルフ空間への連続全射は商写像であることを示せ。

方針: 位相の等式は `TopologicalSpace.ext'` (Part 5) で開集合の一致に還元する。 $\Leftarrow$ 向きは連続性そのもの。 $\Rightarrow$ 向き ($f^{-1}' s$ が開 $\Rightarrow s$ が開) は `06_Topology.lean` の主定理の核と同じ連鎖: 補集合が閉 $\rightarrow$ コンパクト $\rightarrow$ 像がコンパクト $\rightarrow$ 像が閉、で s^c の閉性を出す。問題40で $f '' (f^{-1}' s^c) = s^c$ と書き換えるところで全射性が効く。

```

theorem coinduced_eq_of_surjective {X Y : Type}
  [tX : TopologicalSpace X] [tY : TopologicalSpace Y] [CompactSpace X] [Hausdorff Y]
  {f : X → Y} (hf : Continuous f) (hsurj : Function.Surjective f) :
  tX.coinduced f = tY := sorry

```

問題42 (発展): 主定理の核 `continuous_invFun` を、問題41から再証明せよ。

方針: g の連続性の主張に現れる位相 tY を、問題41の等式で $tX.coinduced f$ に書き換える (`rw [← ...]`)。すると問題34(1)の普遍性により、 g の連続性は合成 `fun x => g (f x)` の連続性に落ち、それは $hg f$ により恒等写像だから `continuous_id` で閉じる。 `06_Topology.lean` では1ページかかった証明が、商写像の言葉では4行になる。

```

example {X Y : Type} [tX : TopologicalSpace X] [tY : TopologicalSpace Y]
  [CompactSpace X] [Hausdorff Y] {f : X → Y} (hf : Continuous f) {g : Y → X}
  (hgf : ∀ x, g (f x) = x) (hfg : ∀ y, f (g y) = y) : Continuous g := sorry

```

```
-- できたら確認: 問題41は Classical.choice に依存する (補題2・3の by_cases 経由) が、  
-- 主定理と違って Classical.choose による「構成」はない  
-- #print axioms coinduced_eq_of_surjective
```

```
import «06_Topology»
```

発展演習: 実数

`06_Topology.lean` まで読み終えた人のための演習問題集 (`07_Exercises.lean` とは独立に読める)。題材は**実数**で、「実数の公理」から出発して、四則演算と順序の計算規則、絶対値、アルキメデス性、実数の位相と連続関数、そして閉区間のコンパクト性 (Heine–Borel)・連結性・ルベーグ数の補題までを証明する。

実数を構成せずに仮定する

実数を有理数から構成する (デデキント切断やコーシー列) ことはしない。代わりに、実数が満たすべき性質——**完備順序体**の公理——を `class` として書き、「この公理を満たす型 `R`」を変数として受け取る。以下の定理はすべて「任意の完備順序体 `R` について」の形をしている。

公理を `axiom` で宣言するのではなく `class` の引数として受け取るので、`#print axioms` に新しい公理は現れない (末尾で確かめる)。何を仮定したかは、定理の型の引数 `[CompleteOrderedField R]` として見える。

構成

- Part A: 実数の公理
- Part B: 足し算と掛け算
- Part C: 順序
- Part D: 絶対値と最小値
- Part E: 完備性
- Part F: 位相と連続性
- Part G: 閉区間

Part B～D は公理から計算規則を導く部分で、1 問 1 問は短い。Part E で初めて完備性 (上限の存在) を使う。山場は Part G の 3 つの定理である。

進め方

`07_Exercises.lean` と同じく、`sorry` を自分の証明で置き換える。解答は `08_RealSol.lean` にある。与えてある宣言 (`sorry` のないもの) も、問題を解くときに使ってよい。

Part A: 実数の公理

完備順序体の公理を並べる。4 つのグループからなる。

- **可換体**: 足し算と掛け算の結合律・可換律、 0 と 1 、符号反転 $-a$ 、分配法則、 $0 \neq 1$ 、 0 でない数の逆数 a^{-1}
- **全順序**: $\leq$ の反射律・推移律・反対称律と、どの 2 数も比べられること。 $<$ は $\leq$ から $a \leq b \wedge \neg b \leq a$ として定まる
- **演算と順序の両立**: 両辺に同じ数を足しても $\leq$ は保たれる。0 以上の数の積は 0 以上
- **完備性**: 空でなく上に有界な集合には、最小の上界（上限）がある

Add R ・ Mul R など extends しているのは、 $a + b \cdot a * b \cdot a^{-1} \cdot a \leq b$ といった記法を使うためである（記法が class で動く仕組みは [05_MathematicalTools.lean 4節](#)）。引き算 $a - b$ と割り算 a / b は公理に含めず、 $a + -b \cdot a * b^{-1}$ として定義する。数 2 は $1 + 1$ と定義する。

完備順序体の公理。

```
class CompleteOrderedField (R : Type) extends Add R, Mul R, Neg R, Inv R, Zero R, One R,
  LE R, LT R where
  add_assoc : ∀ a b c : R, a + b + c = a + (b + c)
  add_comm : ∀ a b : R, a + b = b + a
  zero_add : ∀ a : R, 0 + a = a
  neg_add_cancel : ∀ a : R, -a + a = 0
  mul_assoc : ∀ a b c : R, a * b * c = a * (b * c)
  mul_comm : ∀ a b : R, a * b = b * a
  one_mul : ∀ a : R, 1 * a = a
  mul_add : ∀ a b c : R, a * (b + c) = a * b + a * c
  zero_ne_one : (0 : R) ≠ 1
  mul_inv_cancel : ∀ a : R, a ≠ 0 → a * a⁻¹ = 1
  le_refl : ∀ a : R, a ≤ a
  le_trans : ∀ a b c : R, a ≤ b → b ≤ c → a ≤ c
  le_antisymm : ∀ a b : R, a ≤ b → b ≤ a → a = b
  le_total : ∀ a b : R, a ≤ b ∨ b ≤ a
  lt_iff_le_not_le : ∀ a b : R, a < b ↔ a ≤ b ∧ ¬ b ≤ a
  add_le_add_left : ∀ a b : R, a ≤ b → ∀ c, c + a ≤ c + b
  mul_nonneg : ∀ a b : R, 0 ≤ a → 0 ≤ b → 0 ≤ a * b
  exists_lub : ∀ S : Set R, (∃ x, x ∈ S) → (∃ M, ∀ x ∈ S, x ≤ M) →
    ∃ s, (∀ x ∈ S, x ≤ s) ∧ ∀ M, (∀ x ∈ S, x ≤ M) → s ≤ M

namespace CompleteOrderedField

variable {R : Type} [CompleteOrderedField R]

instance : Sub R := ⟨fun a b => a + -b⟩
```

```

instance : Div R := {fun a b => a * b⁻¹}
instance : OfNat R 2 := {1 + 1}

instance : Std.Associative (α := R) (· + ·) := {add_assoc}
instance : Std.Commutative (α := R) (· + ·) := {add_comm}
instance : Std.Associative (α := R) (· * ·) := {mul_assoc}
instance : Std.Commutative (α := R) (· * ·) := {mul_comm}

theorem sub_def (a b : R) : a - b = a + -b := rfl

theorem div_def (a b : R) : a / b = a * b⁻¹ := rfl

theorem two_def : (2 : R) = 1 + 1 := rfl

```

Part B: 足し算と掛け算

$a * 0 = 0$ や $-(-a) = a$ のような「当たり前」の規則も、公理には入っていないので証明が要る。ここでは、以後の計算で使う規則を公理から導く。

証明はほとんどが `rw`（書き換え）の連鎖である。足し算の項の順序や括弧の付け方を入れ替えるだけの等式は、`ac_rfl` で閉じられる（下の補足）。

補足: `ac_rfl` が閉じる等式

`ac_rfl` は、結合律と可換律だけで移り合う 2 つの式が等しいことを示すタクティクである。たとえば $a + (b + c) + d = d + c + (a + b)$ を閉じる。使うには、演算が結合的・可換であることを `Std.Associative` ・ `Std.Commutative` の instance として登録しておく（Part A の末尾でしている）。`ac_rfl` が扱うのは項の並べ替えだけで、 $a + -a = 0$ のような打ち消しや、分配法則は扱わない。それらは `rw` で公理や補題を使って書き換える。

```

theorem add_zero (a : R) : a + 0 = a := by rw [add_comm, zero_add]

theorem add_neg_cancel (a : R) : a + -a = 0 := by rw [add_comm, neg_add_cancel]

```

問題1: 足し算は左から簡約できることを示せ。

ヒント: `congrArg (fun x => -a + x) h` で両辺の左に $-a$ を足し、`add_assoc` ・ `neg_add_cancel` ・ `zero_add` で書き換える。

```

theorem add_left_cancel {a b c : R} (h : a + b = a + c) : b = c :=
  sorry

```

```

theorem add_right_cancel {a b c : R} (h : a + c = b + c) : a = b := by
  rw [add_comm a, add_comm b] at h
  exact add_left_cancel h

```

```

theorem neg_eq_of_add_eq_zero {a b : R} (h : a + b = 0) : -a = b := by
  apply add_left_cancel (a := a)
  rw [add_neg_cancel, h]

```

問題2: $-(-a) = a$ を示せ。

ヒント: 与えてある `neg_eq_of_add_eq_zero` に、公理 `neg_add_cancel a` を渡す。

```

theorem neg_neg (a : R) : -(-a) = a :=
  sorry

theorem neg_zero : -(0 : R) = 0 := neg_eq_of_add_eq_zero (add_zero 0)

```

問題3: $-(a + b) = -a + -b$ を示せ。

ヒント: `neg_eq_of_add_eq_zero` を使い、 $a + b + (-a + -b) = 0$ を示す。 $(a + -a) + (b + -b)$ への並べ替えは `ac_refl` で。

```

theorem neg_add (a b : R) : -(a + b) = -a + -b :=
  sorry

theorem sub_self (a : R) : a - a = 0 := add_neg_cancel a

theorem sub_zero (a : R) : a - 0 = a := by rw [sub_def, neg_zero, add_zero]

theorem zero_sub (a : R) : 0 - a = -a := by rw [sub_def, zero_add]

theorem sub_add_cancel (a b : R) : a - b + b = a := by
  rw [sub_def, add_assoc, neg_add_cancel, add_zero]

theorem add_sub_cancel (a b : R) : a + b - b = a := by
  rw [sub_def, add_assoc, add_neg_cancel, add_zero]

theorem sub_eq_zero {a b : R} : a - b = 0 ↔ a = b := by
  constructor
  · intro h
    have h' : a - b + b = 0 + b := congrArg (· + b) h
    rw [sub_add_cancel, zero_add] at h'
    exact h'
  · intro h; rw [h, sub_self]

```

```

theorem neg_sub (a b : R) : -(a - b) = b - a := by
  rw [sub_def, neg_add, neg_neg, sub_def, add_comm]

theorem sub_add_sub_cancel (a b c : R) : a - b + (b - c) = a - c := by
  rw [sub_def, sub_def, sub_def, add_assoc, ← add_assoc (-b), neg_add_cancel, zero_add]

theorem add_sub_add_right (a b c : R) : a + c - (b + c) = a - b := by
  rw [sub_def, sub_def, neg_add]
  calc a + c + (-b + -c) = a + -b + (c + -c) := by ac_rfl
  _ = a + -b := by rw [add_neg_cancel, add_zero]

theorem mul_one (a : R) : a * 1 = a := by rw [mul_comm, one_mul]

theorem add_mul (a b c : R) : (a + b) * c = a * c + b * c := by
  rw [mul_comm, mul_add, mul_comm c, mul_comm c]

```

問題4: $a * 0 = 0$ を示せ。

ヒント: $a * 0 + a * 0 = a * (0 + 0) = a * 0 + 0$ なので、問題1で $a * 0$ を簡約する。

```

theorem mul_zero (a : R) : a * 0 = 0 :=
  sorry

theorem zero_mul (a : R) : 0 * a = 0 := by rw [mul_comm, mul_zero]

```

問題5: $a * -b = -(a * b)$ を示せ。

ヒント: `neg_eq_of_add_eq_zero` を使う。 $a * b + a * -b = a * (b + -b)$ (分配法則) $= a * 0$ 。

```

theorem mul_neg (a b : R) : a * -b = -(a * b) :=
  sorry

theorem neg_mul (a b : R) : -a * b = -(a * b) := by
  rw [mul_comm, mul_neg, mul_comm]

theorem neg_mul_neg (a b : R) : -a * -b = a * b := by
  rw [neg_mul, mul_neg, neg_neg]

theorem mul_sub (a b c : R) : a * (b - c) = a * b - a * c := by
  rw [sub_def, sub_def, mul_add, mul_neg]

theorem sub_mul (a b c : R) : (a - b) * c = a * c - b * c := by
  rw [mul_comm, mul_sub, mul_comm c, mul_comm c]

```

```
theorem two_mul (a : R) : 2 * a = a + a := by rw [two_def, add_mul, one_mul]
```

問題6: 積が 0 なら、どちらかが 0 であることを示せ。

ヒント: `by_cases ha : a = 0` で分ける。`a ≠ 0` なら `b = a-1 * a * b = a-1 * (a * b) = 0`。

```
theorem mul_eq_zero {a b : R} (h : a * b = 0) : a = 0 ∨ b = 0 :=  
  sorry
```

```
theorem inv_mul_cancel {a : R} (h : a ≠ 0) : a-1 * a = 1 := by  
  rw [mul_comm, mul_inv_cancel a h]
```

```
theorem div_mul_cancel {a b : R} (h : b ≠ 0) : a / b * b = a := by  
  rw [div_def, mul_assoc, inv_mul_cancel h, mul_one]
```

```
theorem mul_div_cancel {a b : R} (h : b ≠ 0) : a * b / b = a := by  
  rw [div_def, mul_assoc, mul_inv_cancel b h, mul_one]
```

```
theorem mul_div_cancel_left {c : R} (e : R) (h : c ≠ 0) : c * (e / c) = e := by  
  rw [mul_comm, div_mul_cancel h]
```

```
theorem inv_ne_zero {a : R} (h : a ≠ 0) : a-1 ≠ 0 := by  
  intro h'  
  have := mul_inv_cancel a h  
  rw [h', mul_zero] at this  
  exact zero_ne_one this
```

Part C: 順序

順序の公理は `≤` について述べてあるので、`<` についての規則はそこから導く。`0 < 1` も公理ではなく定理である（問題12）。

不等式をいくつもつなぐ議論は `calc` で書くと読みやすい。`≤` と `<` を混ぜてつなぐには、推移律を `Trans` の instance として登録しておく必要がある（この Part で与えている）。

```
theorem lt_irrefl (a : R) : ¬ a < a := fun h => ((lt_iff_le_not_le a a).mp h).2 (le_refl a)
```

```
theorem le_of_lt {a b : R} (h : a < b) : a ≤ b := ((lt_iff_le_not_le a b).mp h).1
```

問題7: `a ≤ b` でないことと `b < a` は同値であることを示せ。

ヒント: $\rightarrow$ は `le_total a b` で場合分けし、公理 `lt_iff_le_not_le` で `<` を組み立てる。

```
theorem not_le {a b : R} :  $\neg a \leq b \leftrightarrow b < a :=$   
  sorry
```

```
theorem not_lt {a b : R} :  $\neg a < b \leftrightarrow b \leq a :=$  by  
  constructor  
  · intro h  
    rcases le_total b a with h' | h'  
    · exact h'  
    · exact Classical.byContradiction fun h'' => h (not_le.mp h'')  
  · intro h h'  
    exact not_le.mpr h' h
```

```
theorem lt_of_le_of_lt {a b c : R} (h1 :  $a \leq b$ ) (h2 :  $b < c$ ) :  $a < c :=$   
  not_le.mp fun h => not_le.mpr h2 (le_trans c a b h h1)
```

```
theorem lt_of_lt_of_le {a b c : R} (h1 :  $a < b$ ) (h2 :  $b \leq c$ ) :  $a < c :=$   
  not_le.mp fun h => not_le.mpr h1 (le_trans b c a h2 h)
```

```
theorem lt_trans {a b c : R} (h1 :  $a < b$ ) (h2 :  $b < c$ ) :  $a < c :=$   
  lt_of_le_of_lt (le_of_lt h1) h2
```

```
instance : Trans (α := R) (β := R) (γ := R) ( $\cdot \leq \cdot$ ) ( $\cdot \leq \cdot$ ) ( $\cdot \leq \cdot$ ) := ⟨le_trans _ _ _⟩
```

```
instance : Trans (α := R) (β := R) (γ := R) ( $\cdot \leq \cdot$ ) ( $\cdot < \cdot$ ) ( $\cdot < \cdot$ ) := ⟨lt_of_le_of_lt⟩
```

```
instance : Trans (α := R) (β := R) (γ := R) ( $\cdot < \cdot$ ) ( $\cdot \leq \cdot$ ) ( $\cdot < \cdot$ ) := ⟨lt_of_lt_of_le⟩
```

```
instance : Trans (α := R) (β := R) (γ := R) ( $\cdot < \cdot$ ) ( $\cdot < \cdot$ ) ( $\cdot < \cdot$ ) := ⟨lt_trans⟩
```

```
theorem ne_of_lt {a b : R} (h :  $a < b$ ) :  $a \neq b :=$  fun e => lt_irrefl b (e ▸ h)
```

```
theorem lt_of_le_of_ne {a b : R} (h1 :  $a \leq b$ ) (h2 :  $a \neq b$ ) :  $a < b :=$   
  not_le.mp fun h => h2 (le_antisymm a b h1 h)
```

```
theorem lt_or_le (a b : R) :  $a < b \vee b \leq a :=$  by  
  by_cases h :  $b \leq a$   
  · exact .inr h  
  · exact .inl (not_le.mp h)
```

```
theorem add_le_add_right {a b : R} (h :  $a \leq b$ ) (c : R) :  $a + c \leq b + c :=$  by  
  rw [add_comm a, add_comm b]; exact add_le_add_left a b h c
```

```
theorem le_of_add_le_add_left {a b c : R} (h :  $c + a \leq c + b$ ) :  $a \leq b :=$  by  
  have := add_le_add_left _ _ h (-c)
```

```
rw [← add_assoc, ← add_assoc, neg_add_cancel, zero_add, zero_add] at this
exact this
```

問題8: 両辺に同じ数を足しても $<$ は保たれることを示せ。

ヒント: 問題7で $<$ を「 $\leq$ でない」に言い換える。 $c + b \leq c + a$ なら、与えてある `le_of_add_le_add_left` で $b \leq a$ となり矛盾する。

```
theorem add_lt_add_left {a b : R} (h : a < b) (c : R) : c + a < c + b :=
  sorry

theorem add_lt_add_right {a b : R} (h : a < b) (c : R) : a + c < b + c := by
  rw [add_comm a, add_comm b]; exact add_lt_add_left h c

theorem add_le_add {a b c d : R} (h1 : a ≤ b) (h2 : c ≤ d) : a + c ≤ b + d :=
  le_trans _ _ _ (add_le_add_right h1 c) (add_le_add_left _ _ h2 b)

theorem add_lt_add {a b c d : R} (h1 : a < b) (h2 : c < d) : a + c < b + d :=
  lt_trans (add_lt_add_right h1 c) (add_lt_add_left h2 b)

theorem add_lt_add_of_lt_of_le {a b c d : R} (h1 : a < b) (h2 : c ≤ d) : a + c < b + d :=
  lt_of_lt_of_le (add_lt_add_right h1 c) (add_le_add_left _ _ h2 b)
```

問題9: $0 \leq b - a$ と $a \leq b$ は同値であることを示せ。

ヒント: 両向きとも `add_le_add_right` で両辺に a (または $-a$) を足し、`sub_add_cancel`・`add_neg_cancel` で整理する。

```
theorem sub_nonneg {a b : R} : 0 ≤ b - a ↔ a ≤ b :=
  sorry

theorem sub_pos {a b : R} : 0 < b - a ↔ a < b := by
  constructor
  · intro h; exact not_le.mp fun h' => not_le.mpr h (by
    have := add_le_add_right h' (-a); rwa [add_neg_cancel] at this)
  · intro h; exact not_le.mp fun h' => not_le.mpr h (by
    have := add_le_add_right h' a; rwa [zero_add, sub_add_cancel] at this)
```

問題10: 符号を反転すると $\leq$ の向きが逆になることを示せ。

ヒント: $a \leq b$ の両辺に $-a$ を足し、さらに $-b$ を足す (`add_le_add_right` を2回)。

```
theorem neg_le_neg {a b : R} (h : a ≤ b) : -b ≤ -a :=
  sorry
```

```

theorem neg_lt_neg {a b : R} (h : a < b) : -b < -a :=
  not_le.mp fun h' => not_le.mpr h (by have := neg_le_neg h'; rwa [neg_neg, neg_neg] at
this)

theorem neg_nonneg {a : R} : 0 ≤ -a ↔ a ≤ 0 := by
  constructor
  · intro h; have := neg_le_neg h; rwa [neg_neg, neg_zero] at this
  · intro h; have := neg_le_neg h; rwa [neg_zero] at this

theorem neg_pos {a : R} : 0 < -a ↔ a < 0 := by
  constructor
  · intro h; have := neg_lt_neg h; rwa [neg_neg, neg_zero] at this
  · intro h; have := neg_lt_neg h; rwa [neg_zero] at this

theorem add_pos {a b : R} (ha : 0 < a) (hb : 0 < b) : 0 < a + b := by
  have := add_lt_add ha hb; rwa [add_zero] at this

theorem add_nonneg {a b : R} (ha : 0 ≤ a) (hb : 0 ≤ b) : 0 ≤ a + b := by
  have := add_le_add ha hb; rwa [add_zero] at this

theorem lt_add_of_pos_right (a : R) {b : R} (h : 0 < b) : a < a + b := by
  have := add_lt_add_left h a; rwa [add_zero] at this

theorem sub_lt_self (a : R) {b : R} (h : 0 < b) : a - b < a := by
  have := add_lt_add_left (neg_lt_neg h) a
  rwa [neg_zero, add_zero] at this

```

問題11: 正の数どうしの積は正であることを示せ。

ヒント: `mul_nonneg` で $0 \leq a * b$ 。 $0 \neq a * b$ は問題6から。 `lt_of_le_of_ne` でまとめる。

```

theorem mul_pos {a b : R} (ha : 0 < a) (hb : 0 < b) : 0 < a * b :=
  sorry

theorem mul_le_mul_of_nonneg_left {a b c : R} (h : a ≤ b) (hc : 0 ≤ c) : c * a ≤ c * b :=
  by
    have := mul_nonneg c (b - a) hc (sub_nonneg.mpr h)
    rw [mul_sub] at this
    exact sub_nonneg.mp this

theorem mul_le_mul_of_nonneg_right {a b c : R} (h : a ≤ b) (hc : 0 ≤ c) : a * c ≤ b * c :=
  by
    rw [mul_comm a, mul_comm b]; exact mul_le_mul_of_nonneg_left h hc

theorem mul_lt_mul_of_pos_left {a b c : R} (h : a < b) (hc : 0 < c) : c * a < c * b := by

```

```

have := mul_pos hc (sub_pos.mpr h)
rw [mul_sub] at this
exact sub_pos.mpr this

theorem mul_self_nonneg (a : R) : 0 ≤ a * a := by
  rcases le_total 0 a with h | h
  · exact mul_nonneg a a h h
  · have := mul_nonneg (-a) (-a) (neg_nonneg.mpr h) (neg_nonneg.mpr h)
    rwa [neg_mul_neg] at this

```

問題12: $0 < 1$ を示せ。

ヒント: 与えてある `mul_self_nonneg 1` で $0 \leq 1 * 1 = 1$ 。等しくないことは公理 `zero_ne_one`。

```

theorem zero_lt_one : (0 : R) < 1 :=
  sorry

theorem zero_lt_two : (0 : R) < 2 := add_pos zero_lt_one zero_lt_one

```

問題13: 正の数の逆数は正であることを示せ。

ヒント: 問題7で背理法の形にする。 $a^{-1} \leq 0$ なら、 a を掛けて (`mul_le_mul_of_nonneg_left`) $1 \leq 0$ となり、問題12に反する。

```

theorem inv_pos {a : R} (h : 0 < a) : 0 < a⁻¹ :=
  sorry

theorem div_pos {a b : R} (ha : 0 < a) (hb : 0 < b) : 0 < a / b := mul_pos ha (inv_pos hb)

theorem two_ne_zero : (2 : R) ≠ 0 := (ne_of_lt zero_lt_two).symm

theorem add_halves (a : R) : a / 2 + a / 2 = a := by
  rw [← two_mul, mul_comm, div_mul_cancel two_ne_zero]

theorem half_pos {a : R} (h : 0 < a) : 0 < a / 2 := div_pos h zero_lt_two

```

問題14: 正の数の半分は、もとの数より小さいことを示せ。

ヒント: $a / 2 < a / 2 + a / 2 = a$ (`lt_add_of_pos_right` · `half_pos` · `add_halves`)。

```

theorem half_lt_self {a : R} (h : 0 < a) : a / 2 < a :=
  sorry

```

```

theorem half_add_lt {c ε : R} (h : 0 < ε) : c + ε / 2 < c + ε :=
  add_lt_add_left (half_lt_self h) c

```

Part D: 絶対値と最小値

絶対値 `abs a` は、 $0 \leq a$ なら `a`、そうでなければ `-a` と定義する。「 $0 \leq a$ かどうか」で場合分けする `if` を書くには、その命題が決定可能である必要があり、ここでは排中律 (`open Classical`) でそれを与える。そのため `abs` は `noncomputable` になる。最小値 `min a b` も同様である。

この Part の中心は三角不等式（問題16）で、以後 ϵ -論法のたびに使う。

```

open Classical in

```

絶対値。

```

noncomputable def abs (a : R) : R := if 0 ≤ a then a else -a

```

```

theorem abs_of_nonneg {a : R} (h : 0 ≤ a) : abs a = a := by
  rw [abs, if_pos h]

```

```

theorem abs_of_neg {a : R} (h : a < 0) : abs a = -a := by
  rw [abs, if_neg (not_le.mpr h)]

```

```

theorem abs_zero : abs (0 : R) = 0 := abs_of_nonneg (le_refl 0)

```

```

theorem abs_of_nonpos {a : R} (h : a ≤ 0) : abs a = -a := by
  rcases lt_or_le a 0 with h' | h'
  · exact abs_of_neg h'
  · have : a = 0 := le_antisymm _ _ h h'
    rw [this, abs_zero, neg_zero]

```

```

theorem abs_nonneg (a : R) : 0 ≤ abs a := by
  rcases lt_or_le a 0 with h | h
  · rw [abs_of_neg h]; exact le_of_lt (neg_pos.mpr h)
  · rw [abs_of_nonneg h]; exact h

```

```

theorem le_abs_self (a : R) : a ≤ abs a := by
  rcases lt_or_le a 0 with h | h
  · rw [abs_of_neg h]; exact le_trans _ _ _ (le_of_lt h) (le_of_lt (neg_pos.mpr h))
  · rw [abs_of_nonneg h]; exact le_refl a

```

```

theorem abs_neg (a : R) : abs (-a) = abs a := by
  rcases lt_or_le a 0 with h | h

```

```

· rw [abs_of_neg h, abs_of_nonneg (le_of_lt (neg_pos.mpr h))]
· rcases lt_or_le 0 a with h' | h'
  · rw [abs_of_nonneg h, abs_of_neg (neg_pos.mp (by rwa [neg_neg]))]
    exact neg_neg a
  · have : a = 0 := le_antisymm _ _ h' h
    rw [this, neg_zero]

```

```

theorem neg_abs_le (a : R) : -abs a ≤ a := by
  have := le_abs_self (-a)
  rw [abs_neg] at this
  have := neg_le_neg this
  rwa [neg_neg] at this

```

```

theorem abs_sub_comm (a b : R) : abs (a - b) = abs (b - a) := by
  rw [← neg_sub, abs_neg]

```

問題15: $|a| < b$ と $-b < a < b$ は同値であることを示せ。

ヒント: $\rightarrow$ は `le_abs_self` ・ `neg_abs_le` 。 $\leftarrow$ は `lt_or_le a 0` で a の符号により場合分けする。

```

theorem abs_lt {a b : R} : abs a < b ↔ -b < a ∧ a < b :=
  sorry

```

問題16: 三角不等式 $|a + b| \leq |a| + |b|$ を示せ。

ヒント: $a + b$ の符号で場合分けする。負なら $|a + b| = -a + -b$ で、 $-a \leq |a|$ は `neg_abs_le` から出る。

```

theorem abs_add (a b : R) : abs (a + b) ≤ abs a + abs b :=
  sorry

```

```

theorem abs_sub_le (a b c : R) : abs (a - c) ≤ abs (a - b) + abs (b - c) := by
  rw [← sub_add_sub_cancel a b c]
  exact abs_add _ _

```

```

theorem abs_sub_self (a : R) : abs (a - a) = 0 := by rw [sub_self, abs_zero]

```

```

theorem abs_le_abs_add_abs_sub (a b : R) : abs a ≤ abs b + abs (a - b) := by
  have := abs_add b (a - b)
  rwa [sub_def, ← add_assoc, add_comm b a, add_assoc, add_neg_cancel, add_zero] at this

```

```

theorem abs_sub_lt_of {c y ε : R} (h₁ : c - ε < y) (h₂ : y < c + ε) : abs (y - c) < ε :=
  by
    refine abs_lt.mpr {?, ?_}
    · have := add_lt_add_right h₁ (-c)

```

```

    rw [sub_def, add_comm c, add_assoc, add_neg_cancel, add_zero] at this
    exact this
  · have := add_lt_add_right h₂ (-c)
    rw [add_comm c, add_assoc, add_neg_cancel, add_zero] at this
    exact this

```

```

theorem mul_nonpos_of_nonneg_of_nonpos {a b : R} (ha : 0 ≤ a) (hb : b ≤ 0) : a * b ≤ 0 :=
by
  have := mul_nonneg a (-b) ha (neg_nonneg.mpr hb)
  rw [mul_neg] at this
  exact neg_nonneg.mp this

```

問題17: $|a * b| = |a| * |b|$ を示せ。

ヒント: `le_total 0 a` と `le_total 0 b` で 4 通りに分ける。 `abs_of_nonneg` ・ `abs_of_nonpos` ・ `mul_nonpos_of_nonneg_of_nonpos` ・ `neg_mul_neg` を使う。

```

theorem abs_mul (a b : R) : abs (a * b) = abs a * abs b :=
sorry

```

```

open Classical in
noncomputable def min (a b : R) : R := if a ≤ b then a else b

```

```

theorem min_le_left (a b : R) : min a b ≤ a := by
  unfold min
  by_cases h : a ≤ b
  · rw [if_pos h]; exact le_refl a
  · rw [if_neg h]; exact le_of_lt (not_le.mp h)

```

```

theorem min_le_right (a b : R) : min a b ≤ b := by
  unfold min
  by_cases h : a ≤ b
  · rw [if_pos h]; exact h
  · rw [if_neg h]; exact le_refl b

```

```

theorem lt_min {a b c : R} (hb : a < b) (hc : a < c) : a < min b c := by
  unfold min
  by_cases h : b ≤ c
  · rw [if_pos h]; exact hb
  · rw [if_neg h]; exact hc

```

Part E: 完備性

Part B～D で使った公理は、有理数でも成り立つ。実数を有理数から区別するのは完備性（上限の存在）だけである。この Part では、完備性から **アルキメデス性**（どの実数よりも大きい自然数がある）と、**整数部分**（どの実数も、ある整数 n について $n \leq x < n + 1$ を満たす）を導く。

そのために、自然数と整数を実数に埋め込む関数 `natCast` ・ `intCast` を定義する。 `intCast` が足し算を保つこと（`intCast_add`）などの計算は与えてある。

問題18: 上限より少しでも小さい値の上には、集合の元があることを示せ。

ヒント: 背理法（`Classical.byContradiction`）。どの元も $s - \varepsilon$ 以下なら、 $s - \varepsilon$ は上界なので $s \leq s - \varepsilon$ となり、`sub_lt_self` に反する。

```
theorem exists_mem_gt_of_lub {S : Set R} {s : R}
  (hlub : ∀ M, (∀ x ∈ S, x ≤ M) → s ≤ M) {ε : R} (hε : 0 < ε) :
  ∃ x, x ∈ S ∧ s - ε < x :=
  sorry
```

自然数の埋め込み。

```
def natCast : Nat → R
| 0 => 0
| n + 1 => natCast n + 1

theorem natCast_zero : (natCast 0 : R) = 0 := rfl

theorem natCast_succ (n : Nat) : (natCast (n + 1) : R) = natCast n + 1 := rfl

theorem natCast_nonneg : ∀ n : Nat, (0 : R) ≤ natCast n
| 0 => le_refl 0
| n + 1 => add_nonneg (natCast_nonneg n) (le_of_lt zero_lt_one)

theorem natCast_add (m : Nat) : ∀ n : Nat, (natCast (m + n) : R) = natCast m + natCast n
| 0 => (add_zero _).symm
| n + 1 => by
  rw [← Nat.add_assoc, natCast_succ, natCast_succ, natCast_add m n, add_assoc]
```

問題19: **アルキメデス性**: どの実数よりも大きい自然数があることを示せ。

ヒント: 背理法。自然数の像 $\{y \mid \exists n, y = \text{natCast } n\}$ が x で押さえられるなら上限 s がある。問題18で $s - 1 < \text{natCast } n$ となる n を取ると、`natCast (n + 1)` が s を超える。

```

theorem exists_nat_gt (x : R) :  $\exists n : \text{Nat}, x < \text{natCast } n :=$ 
  sorry

```

整数の埋め込み。

```

def intCast : Int → R
| Int.ofNat n => natCast n
| Int.negSucc n => -natCast (n + 1)

theorem intCast_natCast (n : Nat) : (intCast (n : Int) : R) = natCast n := rfl

theorem intCast_zero : (intCast 0 : R) = 0 := rfl

theorem intCast_sub_one :  $\forall k : \text{Int}, (\text{intCast } (k - 1) : R) = \text{intCast } k - 1$ 
| Int.ofNat 0 => by
  rw [show Int.ofNat 0 - 1 = Int.negSucc 0 from rfl]
  show -(natCast 1 : R) = natCast 0 - 1
  rw [natCast_succ, natCast_zero, zero_add, zero_sub]
| Int.ofNat (n + 1) => by
  rw [show Int.ofNat (n + 1) - 1 = Int.ofNat n by simp only [Int.ofNat_eq_natCast];
omega]
  show (natCast n : R) = natCast (n + 1) - 1
  rw [natCast_succ, add_sub_cancel]
| Int.negSucc n => by
  rw [show Int.negSucc n - 1 = Int.negSucc (n + 1) by simp only [Int.negSucc_eq];
omega]
  show -(natCast (n + 1 + 1) : R) = -natCast (n + 1) - 1
  rw [natCast_succ (n + 1), neg_add]; rfl

theorem intCast_add_one (k : Int) : (intCast (k + 1) : R) = intCast k + 1 := by
  have h := intCast_sub_one (R := R) (k + 1)
  rw [Int.add_sub_cancel] at h
  rw [h, sub_add_cancel]

theorem intCast_add_natCast (k : Int) :  $\forall n : \text{Nat},$ 
  (intCast (k + n) : R) = intCast k + natCast n
| 0 => by rw [Int.natCast_zero, Int.add_zero]; exact (add_zero _).symm
| n + 1 => by
  rw [Int.natCast_add, Int.natCast_one, ← Int.add_assoc, intCast_add_one,
intCast_add_natCast k n, natCast_succ, add_assoc]

theorem intCast_neg_natCast_add (N : Nat) : (intCast -(N : Int) : R) + natCast N = 0 :=
by
  rw [← intCast_add_natCast, Int.add_left_neg]; rfl

```

```

theorem intCast_sub_natCast (k : Int) : ∀ n : Nat,
  (intCast (k - n) : R) = intCast k - natCast n
| 0 => by rw [Int.natCast_zero, Int.sub_zero, natCast_zero, sub_zero]
| n + 1 => by
  rw [show k - ((n + 1 : Nat) : Int) = k - (n : Int) - 1 by omega, intCast_sub_one,
    intCast_sub_natCast k n, natCast_succ, sub_def, sub_def, sub_def, neg_add,
    add_assoc]

```

整数の足し算は実数の足し算に写る。

```

theorem intCast_add (m : Int) : ∀ n : Int, (intCast (m + n) : R) = intCast m + intCast n
| Int.ofNat k => intCast_add_natCast m k
| Int.negSucc k => by
  rw [show m + Int.negSucc k = m - ((k + 1 : Nat) : Int) by simp only
[Int.negSucc_eq]; omega,
    intCast_sub_natCast]
  rfl

theorem intCast_neg (m : Int) : (intCast (-m) : R) = -intCast m := by
  apply Eq.symm
  apply neg_eq_of_add_eq_zero
  rw [← intCast_add, Int.add_right_neg]; rfl

```

問題20: (発展) **整数部分**: どの実数 x にも、 $n \leq x < n + 1$ となる整数 n があることを示せ。

ヒント: 問題19を x と $-x$ に使い、自然数 $N \cdot M$ で $-N \leq x < M$ とする。「 $a \leq x < a + m$ なら $a + k \leq x < a + k + 1$ となる自然数 k がある」を m の帰納法で示し、 $a = -N$ 、 $m = N + M$ に使う。
`intCast_add_natCast` ・ `intCast_neg_natCast_add` が使える。

```

theorem exists_int_floor (x : R) : ∃ n : Int, intCast n ≤ x ∧ x < intCast n + 1 :=
  sorry

```

Part F: 位相と連続性

「どの点の周りにも ε -近傍が収まる」集合を開集合として、 $\mathbb{R}$ に位相を入れる (`06_Topology.lean` の `TopologicalSpace` の instance)。以後、 $\mathbb{R}$ を値に取る関数の連続性は、`06_Topology.lean` の定義 (開集合の逆像が開) の意味である。

その定義のままでは扱いにくいので、 ε -論法の形の判定法 (問題23) を用意する。これを使って、連続関数の和・積が連続であることを示す。位相空間 X は一般のもので、 $\mathbb{R}$ 自身に限らない。

問題21: 実数の位相: どの点のまわりにも ε -近傍が収まる集合を開集合とする。位相の公理を満たすことを示せ。

ヒント: 共通部分では、2つの ε の小さいほう (`min · lt_min · min_le_left`) を取る。合併では、点が属する1つの集合の ε をそのまま使う。

```
instance : TopologicalSpace R where
  IsOpen s := ∀ x ∈ s, ∃ ε, 0 < ε ∧ ∀ y, abs (y - x) < ε → y ∈ s
  isOpen_univ := sorry
  isOpen_inter := sorry
  isOpen_sUnion := sorry

theorem isOpen_iff {s : Set R} :
  IsOpen s ↔ ∀ x ∈ s, ∃ ε, 0 < ε ∧ ∀ y, abs (y - x) < ε → y ∈ s := Iff.rfl
```

ε -近傍 (开区間)。

```
def ball (x ε : R) : Set R := {y | abs (y - x) < ε}
```

問題22: ε -近傍は開集合であることを示せ。

ヒント: 点 y のまわりには、半径 $\varepsilon - |y - x|$ の近傍が収まる。三角不等式 `abs_sub_le` を使う。

```
theorem isOpen_ball (x ε : R) : IsOpen (ball x ε) :=
  sorry

theorem mem_ball_self {x ε : R} (h : 0 < ε) : x ∈ ball x ε := by
  show abs (x - x) < ε
  rw [abs_sub_self]; exact h

section Continuity

variable {X : Type} [TopologicalSpace X]
```

問題23: 実数値関数の連続性は、各点で「値が ε 以内に収まる開近傍がある」で判定できることを示せ。

ヒント: `f-1 s` を「`f-1 s` に含まれる開集合全体の合併」と書き直し (`Set.ext`)、`isOpen_sUnion` を使う。

```
theorem continuous_of_forall {f : X → R}
  (h : ∀ x ε, 0 < ε → ∃ U : Set X, IsOpen U ∧ x ∈ U ∧ ∀ y ∈ U, abs (f y - f x) < ε) :
  Continuous f :=
  sorry
```

```

theorem forall_of_continuous {f : X → R} (hf : Continuous f) (x : X) {ε : R} (hε : 0 < ε)
:
  ∃ U : Set X, IsOpen U ∧ x ∈ U ∧ ∀ y ∈ U, abs (f y - f x) < ε :=
  (f ⁻¹' ball (f x) ε, hf _ (isOpen_ball _ _), mem_ball_self hε, fun _ hy => hy)

theorem continuous_id' : Continuous (fun x : R => x) :=
  continuous_of_forall fun x ε hε => (ball x ε, isOpen_ball x ε, mem_ball_self hε, fun _
  hy => hy)

theorem continuous_const (c : R) : Continuous (fun _ : X => c) :=
  continuous_of_forall fun _ _ hε =>
  (Set.univ, isOpen_univ, trivial, fun _ _ => by rw [abs_sub_self]; exact hε)

```

問題24: 連続関数の和は連続であることを示せ。

ヒント: 問題23を使う。f と g に $\varepsilon / 2$ で forall_of_continuous を使い、2つの開近傍の共通部分を取る。差を $(f y - f x) + (g y - g x)$ に並べ替え (ac_rfl)、三角不等式と add_halves。

```

theorem continuous_add {f g : X → R} (hf : Continuous f) (hg : Continuous g) :
  Continuous (fun x => f x + g x) :=
  sorry

theorem continuous_neg {f : X → R} (hf : Continuous f) : Continuous (fun x => -f x) := by
  apply continuous_of_forall
  intro x ε hε
  obtain (U, hU, hxU, hfU) := forall_of_continuous hf x hε
  refine (U, hU, hxU, fun y hy => ?_)
  have e : -f y - -f x = -(f y - f x) := by rw [sub_def, sub_def, neg_add]
  rw [e, abs_neg]; exact hfU y hy

theorem continuous_sub {f g : X → R} (hf : Continuous f) (hg : Continuous g) :
  Continuous (fun x => f x - g x) := continuous_add hf (continuous_neg hg)

```

問題25: (発展) 連続関数の積は連続であることを示せ。

ヒント: $f y * g y - f x * g x = f y * (g y - g x) + g x * (f y - f x)$ と分ける。f の近傍では値の差を1未満にも抑えて $|f y| \leq |f x| + 1$ とし、各項が $\varepsilon / 2$ 未満になるよう、g の差を $\varepsilon / 2 / (|f x| + 1)$ 未満、f の差を $\varepsilon / 2 / (|g x| + 1)$ 未満に取る。

```

theorem continuous_mul {f g : X → R} (hf : Continuous f) (hg : Continuous g) :
  Continuous (fun x => f x * g x) :=
  sorry

end Continuity

```

Part G: 閉区間

閉区間 $[a, b]$ について、3 つの定理を示す。どれも Part E の上限の議論で証明する。

- **Heine–Borel** (問題27) : 閉区間はコンパクト。 `06_Topology.lean` の `IsCompact` が、ここで初めて具体的な空間の具体的な集合に対して成り立つ
- **連結性** (問題28) : 閉区間は、交わらない 2 つの開集合に分けられない
- **ルベーグ数の補題** (問題30) : 閉区間を開集合で覆うと、ある $\delta > 0$ について、区間のどの点のまわりの δ -近傍も、覆う開集合のどれか 1 つに収まる

3 つの証明は同じ型をしている。「 a から x まで性質が成り立つ」ような x の集合の上限 c を取り、 c の近くの様子を見て、 c が区間の右端 b まで届くことを示す。

閉区間 $[a, b]$ 。

```
def Icc (a b : R) : Set R := {x | a ≤ x ∧ x ≤ b}
```

問題26: 有限集合に 1 点を足しても有限であることを示せ。

ヒント: J を拾う $f : \text{Fin } n \rightarrow I$ に、最後の番号 n で i を拾うよう 1 つ足す: `fun k => if h : k.val < n then f (k.val, h) else i`。

```
theorem finite_insert {I : Type} {J : Set I} (hJ : J.Finite) (i : I) :  
  ({j | j ∈ J ∨ j = i} : Set I).Finite :=  
  sorry  
  
theorem finite_singleton {I : Type} (i : I) : ({j | j = i} : Set I).Finite :=  
  (1, fun _ => i, fun _ hj => (<{0, Nat.zero_lt_one}, hj.symm))
```

問題27: **Heine–Borel**: 閉区間はコンパクトであることを示せ。

ヒント: $[a, x]$ が有限個で覆える $x \in [a, b]$ の集合の上限を c とする。 c を含む開集合 U_{i_1} の ε -近傍と、 $c - \varepsilon$ より大きいその集合の元 x (問題18) を取ると、 $[a, c + \varepsilon/2]$ まだが i_1 を足して覆える (問題26)。 $b \leq c + \varepsilon/2$ なら終わり、そうでなければ $c + \varepsilon/2$ が上限を超えて矛盾する。

```
theorem isCompact_Icc (a b : R) : IsCompact (Icc a b) :=  
  sorry
```

問題28: **閉区間の連結性**: 閉区間を、互いに交わらない 2 つの開集合で分けることはできないことを示せ。 a を含む側が区間全体を含む。

ヒント: 問題27と同じ形。 $[a, x] \subseteq U$ となる x の集合の上限 c を取る。まず $c \in U$ を示す ($c \in V$ なら、 c に近いその集合の元が U と V の両方に入る)。あとは問題27と同じく $c + \varepsilon/2$ を見る。

```

theorem Icc_connected {a b : R} {U V : Set R} (hU : IsOpen U) (hV : IsOpen V)
  (hcov :  $\forall x \in \text{Icc } a \ b, x \in U \vee x \in V$ ) (hdisj :  $\forall x \in \text{Icc } a \ b, x \in U \rightarrow x \in V \rightarrow \text{False}$ )
  (hab :  $a \leq b$ ) (ha :  $a \in U$ ) :  $\forall x \in \text{Icc } a \ b, x \in U :=$ 
sorry

```

有限個の実数の最小値 (0 個なら 1)。

```

noncomputable def finMin : (n : Nat) → (Fin n → R) → R
| 0, _ => 1
| n + 1, g => min (g 0) (finMin n fun k => g k.succ)

theorem finMin_pos :  $\forall (n : \text{Nat}) (g : \text{Fin } n \rightarrow \mathbb{R}), (\forall k, 0 < g \ k) \rightarrow 0 < \text{finMin } n \ g$ 
| 0, _, _ => zero_lt_one
| n + 1, _, h => lt_min (h 0) (finMin_pos n _ fun k => h k.succ)

```

問題29: 有限個の数の最小値は、そのどれ以下でもあることを示せ。

ヒント: n についての再帰。 k は `cases k using Fin.cases` で `0` と `succ` に分ける。

```

theorem finMin_le :  $\forall (n : \text{Nat}) (g : \text{Fin } n \rightarrow \mathbb{R}) (k : \text{Fin } n), \text{finMin } n \ g \leq g \ k :=$ 
sorry

```

問題30: **ルベグ数の補題**: 閉区間を覆う開集合族には、ある $\delta > 0$ があって、区間のどの点のまわりの δ -近傍も、族のどれか 1 つに収まることを示せ。

ヒント: 区間の各点 p に、ある U_i に収まる近傍の半径 r_p を選び (`Classical.choose`)、半径 $r_p / 2$ の近傍で区間を覆う。問題27で有限個に減らし、それらの半径の最小値 (`finMin`) を δ とする。最後は三角不等式で $|y - p| < r_p$ を示す。

```

theorem lebesgue {a b : R} {I : Type} (U : I → Set R) (hU :  $\forall i, \text{IsOpen } (U \ i)$ )
  (hcov :  $\text{Icc } a \ b \subseteq \bigcup_i U \ i$ ) :
   $\exists \delta, 0 < \delta \wedge \forall x \in \text{Icc } a \ b, \exists i, \forall y, \text{abs } (y - x) < \delta \rightarrow y \in U \ i :=$ 
sorry

end CompleteOrderedField

```

解答 (`08_RealSol.lean`) では、ルベグ数の補題が依存する公理は `propext` ・ `Classical.choice` ・ `Quot.sound` の 3 つだけである。実数の公理は `axiom` ではなく `class` の引数なので、ここには現れない。問題を解き終えたら、次の出力に `sorryAx` が残っていないことを確かめよ。

```

#print axioms CompleteOrderedField.lebesgue

```

```
import «06_Topology»
```

発展演習: Ascoli の定理（ブルバキ流）

06_Topology.lean まで読み終えた人のための、もう1つの演習問題集（07_Exercises.lean とは独立に読める）。題材は関数空間のコンパクト性の定理 **Ascoli の定理**である。

古典版とブルバキ版

古典的な Arzelà–Ascoli の定理は「 $C([a, b])$ の部分集合が相対コンパクトであるための必要十分条件は、一様有界かつ同程度連続（等連続）であること」と述べられ、証明には点列と対角線論法を使う。

ブルバキ（『位相』第X章 §2）はこれを次の2段に分解した。

1. **Théorème 1**: 等連続な関数族の上では、各点収束と一様収束が（コンパクト集合上で）一致する
2. **Théorème 2 (Ascoli)**: 等連続で、各点での値の集合が全有界（précompact）な族は、一様収束について全有界である。コンパクト性はその系として出る

点列も対角線論法も使わない。本質は 1——「等連続なら、有限個の点での値を見れば全体が一様に制御できる」——であり、2 はそこから有限被覆を掛け合わせるだけで出る。この演習ではこの分解に沿って進む。

この演習での取捨

ブルバキは一様構造をフィルターで定義し、 $\mathcal{C}$ -収束（部分集合族 $\mathcal{C}$ 上での一様収束）の一般論の中で定理を述べる。mathlib の `Mathlib.Topology.UniformSpace.Ascoli` もほぼそのまま形式化している。この演習では次のように絞る。

- 一様構造は**近縁（entourage）の集合族**として直接公理化する（標準形の5公理）
- 関数空間は「コンパクト空間 X から一様空間 Y への写像全体」に **全体での一様収束**の一様構造を入れたものだけを扱う
- Théorème 1 は「一様構造の一致」という抽象的な形ではなく、**有限点制御補題**（Part C）という具体的な形で証明する
- 有限性は `List` で扱う（06_Topology.lean の `Set.Finite` との橋渡しは与える）

構成

- Part A: 一様構造と、そこから定まる位相

- Part B: 一様収束の一様構造と等連続性
- Part C: 核心補題 (Théorème 1 の有限版)
- Part D: 全有界性と主定理 (全有界版)
- Part E: 反例——等連続性の仮定は外せない
- Part F: 逆向き——全有界なら等連続
- Part G: フィルターと Zorn の補題 (コンパクト版の準備)
- Part H: 主定理のコンパクト版

Part A~D が本体で、ここまでに選択公理による「選択」は有限点制御補題の近傍の選択 (`Classical.choose`) しか使わない。Part G・H は重い。一般の一様空間で「全有界 $\Rightarrow$ (完備なら) コンパクト」を示すには **超フィルター補題** が要り、これは選択公理より真に弱い、選択公理なしの集合論 (ZF) では証明できない。core Lean には Zorn の補題がないので、Part G で自作する。

進め方

`07_Exercises.lean` と同じく、`sorry` を自分の証明で置き換える。解答は `09_AscoliSol.lean` にある。与えてある宣言 (`sorry` のないもの) も、問題を解くときに使ってよい。

Part A: 一様構造

まず集合の道具を少し足す。`compRel V W` は関係の合成 (`a` から `V` で `z` へ、`z` から `W` で `b` へ行ける)、`swapRel V` は逆関係である。リストと有限性の橋渡し2つも与える。

```
namespace Set

variable {α : Type}
```

2つの集合が互いに含め合えば等しい。

```
theorem subset_antisymm {s t : Set α} (h₁ : s ⊆ t) (h₂ : t ⊆ s) : s = t :=
  ext fun a => ⟨h₁ a, h₂ a⟩
```

関係の合成 `V ∘ W`。

```
def compRel (V W : Set (α × α)) : Set (α × α) := {p | ∃ z, (p.1, z) ∈ V ∧ (z, p.2) ∈ W}
```

逆関係 `V-1`。

```
def swapRel (V : Set (α × α)) : Set (α × α) := {p | (p.2, p.1) ∈ V}
```

リストに載っている元の全体は有限集合。

```
theorem finite_of_list (L : List α) : Set.Finite {a | a ∈ L} :=
  ⟨L.length, L.get, fun _ ha => List.get_of_mem ha⟩
```

有限集合の元はすべて、あるリストに載せられる。

```
theorem Finite.exists_list {s : Set α} (hs : s.Finite) : ∃ L : List α, ∀ a ∈ s, a ∈ L :=
  have ⟨_, f, hf⟩ := hs
  ⟨List.ofFn f, fun a ha => List.mem_ofFn.mpr (hf a ha)⟩

end Set
```

一様構造は「2点が V -近い」という関係 $V \subseteq \alpha \times \alpha$ たち（近縁）を指定する。距離空間なら $V_\epsilon = \{(a, b) \mid d(a, b) < \epsilon\}$ を含む関係が近縁である。5つの公理はそれぞれ、距離の性質の抽象化になっている。

- `univ_mem`・`mono`・`inter_mem`: 近縁の全体はフィルターをなす（ ϵ の取り方の自由度）
- `refl`: 近縁は対角線を含む（ $d(a, a) = 0$ ）
- `symm`: 近縁の逆も近縁（ $d(a, b) = d(b, a)$ ）
- `comp`: どの近縁 V にも、 $W \circ W \subseteq V$ となる近縁 W がある（三角不等式、 $\epsilon/2$ ）

一様構造（標準形）。

```
class UniformSpace (α : Type) where
  /-- 近縁の全体。 -/
  Entourage : Set (Set (α × α))
  /-- 全体の関係は近縁。 -/
  univ_mem : Set.univ ∈ Entourage
  /-- 近縁より大きい関係は近縁。 -/
  mono : ∀ {V W : Set (α × α)}, V ∈ Entourage → V ⊆ W → W ∈ Entourage
  /-- 2つの近縁の共通部分は近縁。 -/
  inter_mem : ∀ {V W : Set (α × α)}, V ∈ Entourage → W ∈ Entourage → V ∩ W ∈ Entourage
  /-- 近縁は対角線を含む。 -/
  refl : ∀ {V : Set (α × α)}, V ∈ Entourage → ∀ a, (a, a) ∈ V
  /-- 近縁の逆関係は近縁。 -/
  symm : ∀ {V : Set (α × α)}, V ∈ Entourage → Set.swapRel V ∈ Entourage
  /-- 「半分」の近縁がある。 -/
  comp : ∀ {V : Set (α × α)}, V ∈ Entourage → ∃ W, W ∈ Entourage ∧ Set.compRel W W ⊆ V
```

$\mathcal{U} \alpha$ で α の近縁の全体を表す（mathlib と同じ記号）。

```
notation "⌢ " α:α => @UniformSpace.Entourage α _
```

```
namespace UniformSpace
```

```
variable {α : Type} [UniformSpace α]
```

問題1: 対称な「半分」の近縁を作れ。古典的な議論の「 $\epsilon/2$ を取る」に当たる。

ヒント: `comp` で $W \circ W \subseteq V$ となる W を取り、`W n swapRel W` を使う。

```
theorem exists_half {V : Set (α × α)} (hV : V ∈ ⌢ α) :  
  ∃ W ∈ ⌢ α, (∀ a b, (a, b) ∈ W → (b, a) ∈ W) ∧  
    ∀ a b c, (a, b) ∈ W → (b, c) ∈ W → (a, c) ∈ V :=  
sorry
```

問題2: 対称な「3分の1」の近縁を作れ。これが Part C・F の「 $\epsilon/3$ 論法」の道具になる。

ヒント: 問題1を2回使う (V の半分 W_1 、 W_1 の半分 W_2)。 $(c, d) \in W_2$ から $(c, d) \in W_1$ を出すには、 $(d, d) \in W_2$ (`refl`) をつなげばよい。

```
theorem exists_third {V : Set (α × α)} (hV : V ∈ ⌢ α) :  
  ∃ W ∈ ⌢ α, (∀ a b, (a, b) ∈ W → (b, a) ∈ W) ∧  
    ∀ a b c d, (a, b) ∈ W → (b, c) ∈ W → (c, d) ∈ W → (a, d) ∈ V :=  
sorry
```

問題3: 一様構造から位相が定まることを示せ。 s が開 $\iff s$ のどの点 a にも、 a の「 V -球」 $\{b \mid (a, b) \in V\}$ が s に収まる近縁 V がある。

距離空間で「どの点にも、その点を中心とする ϵ -球が収まる」とするのと同じ定義である。 `instance` にしたので、以後一様空間には自動でこの位相が載る。

```
instance toTopologicalSpace : TopologicalSpace α where  
  isOpen s := ∀ a ∈ s, ∃ V ∈ ⌢ α, ∀ b, (a, b) ∈ V → b ∈ s  
  isOpen_univ := sorry  
  isOpen_inter := sorry  
  isOpen_sUnion := sorry
```

問題4: V -球そのものは開とは限らないが、中心を含む開集合を内側に含むことを示せ。

ヒント: $O = \{y \mid \exists W \in \mathcal{U} \alpha, \forall z, (y, z) \in W \rightarrow (a, z) \in V\}$ (y の周りに V -球に収まる近縁がある点の全体) が求めるもの。開であることは、 W の半分 W' (`comp`) を y の近くの点の証人に使えば示せる。

```

theorem exists_open_ball {V : Set (α × α)} (hV : V ∈  $\mathcal{U}$  α) (a : α) :
  ∃ 0 : Set α, IsOpen 0 ∧ a ∈ 0 ∧ ∀ b ∈ 0, (a, b) ∈ V :=
  sorry

end UniformSpace

open UniformSpace

```

Part B: 一様収束の一様構造と等連続性

写像 $f, g : X \rightarrow Y$ が「一様に V -近い」とは、すべての x で $(f\ x, g\ x) \in V$ となること。この関係 $\text{unifRel } V$ を含む関係を近縁とするのが、**一様収束の一様構造**である。Part A の位相と組み合わせると、 $X \rightarrow Y$ には一様収束の位相が自動で載る。

```

section FunctionSpace

variable {X Y : Type} [UniformSpace Y]

```

近縁 V から作る「一様に V -近い」写像の組の全体。

```

def unifRel (V : Set (Y × Y)) : Set ((X → Y) × (X → Y)) := {p | ∀ x, (p.1 x, p.2 x) ∈ V}

```

問題5: 一様収束の一様構造が公理を満たすことを示せ。

Entourage を展開すると $\exists V \in \mathcal{U} Y, \text{unifRel } V \subseteq \mathcal{V}$ なので、各公理は Y 側の同じ公理から出る。 comp では Y 側の半分 W に対する $\text{unifRel } W$ を使う。

```

instance uniformFun : UniformSpace (X → Y) where
  Entourage := { $\mathcal{V}$  | ∃ V ∈  $\mathcal{U}$  Y, unifRel V ⊆  $\mathcal{V}$ }
  univ_mem := sorry
  mono := sorry
  inter_mem := sorry
  refl := sorry
  symm := sorry
  comp := sorry

variable [TopologicalSpace X]

```

等連続性は、族 H のすべての写像について、連続性の U が f によらず共通に取れることをいう。定義域 X は位相空間でよい（一様構造は要らない）。

点 x での等連続性: どの近縁 V にも x の開近傍 U があって、 H のどの f についても U の点での値が $f x$ と V -近い。

```
def EquicontinuousAt (H : Set (X → Y)) (x : X) : Prop :=
  ∀ V ∈ ℱ Y, ∃ U : Set X, IsOpen U ∧ x ∈ U ∧ ∀ f ∈ H, ∀ x' ∈ U, (f x, f x') ∈ V
```

等連続性: すべての点で等連続。

```
def Equicontinuous (H : Set (X → Y)) : Prop := ∀ x, EquicontinuousAt H x
```

問題6: 等連続な族の部分族は等連続であることを示せ。

```
theorem Equicontinuous.mono {H H' : Set (X → Y)} (hH : Equicontinuous H) (h : H' ≤ H) :
  Equicontinuous H' :=
  sorry
```

問題7: 連続写像1つだけからなる族は等連続であることを示せ。等連続性が連続性の「族版」であることの確認である。

ヒント: 問題4で $f x$ の周りの開集合 U を取り、その逆像 $f^{-1} U$ を V にする。

```
theorem equicontinuous_singleton {f : X → Y} (hf : Continuous f) :
  Equicontinuous {g | g = f} :=
  sorry
```

Part C: 核心補題 (Théorème 1 の有限版)

ブルバキの Théorème 1 は「等連続集合の上では、各点収束の一樣構造とコンパクト収束の一樣構造が一致する」と述べる。 X がコンパクトなときにこれを具体的に言い直したのが次の補題である:

> 等連続族 H と近縁 V に対して、有限個の点 P と近縁 W があって、> H の2つの写像が P の各点で W -近いなら、**すべての点で** V -近い。

「一樣に近い」かどうか、有限個の点での値だけで判定できる。これが Ascoli の定理の心臓部で、証明は古典的な $\varepsilon/3$ 論法そのものである。

問題8: 有限点制御補題を示せ。

方針:

- 問題2で V の対称な3分の1 W を取る
- 各点 x で等連続性を W に使い、開近傍 $U x$ を `Classical.choose` で選ぶ

3. U は X を覆うので、コンパクト性 (`CompactSpace.isCompact_univ`) で有限個の添字の集合 J に間引き、`Set.Finite.exists_list` でリスト P にする
4. 任意の x は、ある $y \in J$ について $x \in U y$ 。すると $(f x, f y) \cdot (f y, g y) \cdot (g y, g x)$ がどれも W に入り、3つつなげて $(f x, g x) \in V$

```

theorem Equicontinuous.finite_control [CompactSpace X] {H : Set (X → Y)}
  (hH : Equicontinuous H) {V : Set (Y × Y)} (hV : V ∈  $\mathcal{U}$  Y) :
  ∃ W ∈  $\mathcal{U}$  Y, ∃ P : List X, ∀ f ∈ H, ∀ g ∈ H,
    (∀ p ∈ P, (f p, g p) ∈ W) → ∀ x, (f x, g x) ∈ V :=
sorry

end FunctionSpace

```

Part D: 全有界性と主定理

集合 A が V -小さいとは、 A のどの2点も V -近いこと。集合 s が**全有界**（ブルバキの *précompact*）とは、どの近縁 V についても有限個の V -小さい集合で覆えること。距離空間なら「どの ε についても直径 ε 以下の有限個の集合で覆える」である。

```

section TotallyBounded

```

```

variable {α : Type}

```

A は V -小さい。

```

def Small (V : Set (α × α)) (A : Set α) : Prop := ∀ a ∈ A, ∀ b ∈ A, (a, b) ∈ V

```

全有界: どの近縁 V についても、 V -小さい集合の有限リストで覆える。

```

def TotallyBounded [UniformSpace α] (s : Set α) : Prop :=
  ∀ V ∈  $\mathcal{U}$  α, ∃ L : List (Set α), (∀ A ∈ L, Small V A) ∧ ∀ a ∈ s, ∃ A ∈ L, a ∈ A

```

2つの有限被覆 $L \cdot M$ の**共通細分**—— $A \cap B$ ($A \in L$ 、 $B \in M$) の全体——も有限被覆である。個数で言えば $|L| \times |M|$ 個だが、`Fin` の積の番号づけを避けて `List.flatMap` で作る。この組合せ論の部分は与える。

2つの有限被覆の共通細分。

```

def List.interCover (L M : List (Set α)) : List (Set α) :=
  L.flatMap fun A => M.map fun B => A ∩ B

```

共通細分の各片は $A \cap B$ の形。

```
theorem List.mem_interCover {L M : List (Set  $\alpha$ )} {C : Set  $\alpha$ } (h : C ∈ List.interCover L M) :  
  ∃ A ∈ L, ∃ B ∈ M, C = A ∩ B := by  
  have ⟨A, hA, hC⟩ := List.mem_flatMap.mp h  
  have ⟨B, hB, hAB⟩ := List.mem_map.mp hC  
  exact ⟨A, hA, B, hB, hAB.symm⟩
```

L と M がともに s を覆うなら、共通細分も s を覆う。

```
theorem List.interCover_covers {s : Set  $\alpha$ } {L M : List (Set  $\alpha$ )}  
  (hL : ∀ a ∈ s, ∃ A ∈ L, a ∈ A) (hM : ∀ a ∈ s, ∃ B ∈ M, a ∈ B) :  
  ∀ a ∈ s, ∃ C ∈ List.interCover L M, a ∈ C := by  
  intro a ha  
  have ⟨A, hA, haA⟩ := hL a ha  
  have ⟨B, hB, haB⟩ := hM a ha  
  exact ⟨A ∩ B, List.mem_flatMap.mpr ⟨A, hA, List.mem_map_of_mem hB⟩, haA, haB⟩
```

問題9: 全有界な集合の部分集合は全有界であることを示せ。

```
theorem TotallyBounded.subset [UniformSpace  $\alpha$ ] {s t : Set  $\alpha$ } (hs : TotallyBounded s)  
  (h : t ⊆ s) : TotallyBounded t :=  
  sorry  
  
variable {X Y : Type} [UniformSpace Y]
```

問題10: 各点で値の集合が全有界なら、有限個の点 P を固定するごとに、 H を「 P の各点で W -近い」写像の集まり有限個で覆えることを示せ。

P についての帰納法（再帰）で示す。

- $P = []$: 条件は空なので、 $[\text{Set.univ}]$ 1つで覆える
- $P = x :: P' : P'$ についての被覆 C と、 x での値の W -小さい被覆 L を取り、 L の各片の逆像 $(\text{fun } f \Rightarrow f\ x)^{-1} A$ のリストと C の共通細分を取る

ブルバキの証明では、ここが「有限個の全有界集合の積は全有界」に当たる。

```
theorem finite_points_cover {H : Set (X → Y)}  
  (hpt : ∀ x, TotallyBounded ((fun f => f x) '' H)) {W : Set (Y × Y)} (hW : W ∈  $\mathcal{U}$  Y) :  
  ∀ P : List X, ∃ C : List (Set (X → Y)),  
    (∀ S ∈ C, ∀ f ∈ S, ∀ g ∈ S, ∀ p ∈ P, (f p, g p) ∈ W) ∧ ∀ f ∈ H, ∃ S ∈ C, f ∈ S :=  
  sorry
```

```
variable [TopologicalSpace X]
```

問題11 (主定理・Ascoli の定理、全有界版) :コンパクト空間 X から一様空間 Y への写像の族 H が等連続で、各点での値の集合が全有界なら、 H は一様収束について全有界であることを示せ。

方針: 近縁 $\mathcal{V} \supseteq \text{unifRel } V$ を取る。問題8で V に対する W と有限個の点 P を取り、問題10で P についての被覆 $\mathcal{C}$ を作る。各片を $S \cap H$ に取り替えれば、問題8により $\mathcal{V}$ -小さくなる (問題8は H の写像にしか使えないので $\cap H$ が要る)。

```
theorem ascoli [CompactSpace X] {H : Set (X → Y)} (hH : Equicontinuous H)
  (hpt : ∀ x, TotallyBounded ((fun f => f x) '' H)) : TotallyBounded H :=
  sorry

end TotallyBounded
```

Part E: 反例——等連続性の仮定は外せない

X を「自然数に無限遠点を1つ付け加えた空間」(1点コンパクト化)、 Y を Bool とし、 H を「点 n だけで true になる関数 δ_n 」の全体とする。値は Bool にしかならないので各点での全有界性はただで成り立つが、 H は無限遠点で等連続でなく、実際に一様収束について全有界でない。

Option Nat の none を無限遠点とする。 none を含む開集合は、ある番号から先の自然数をすべて含むもの (補集合が有限であることに相当する) とする。

```
namespace Counterexample
```

1点コンパクト化 Option Nat の位相 (与える)。

```
instance : TopologicalSpace (Option Nat) where
  isOpen s := none ∈ s → ∃ N, ∀ n, N ≤ n → some n ∈ s
  isOpen_univ := fun _ => ⟨0, fun _ _ => trivial⟩
  isOpen_inter := by
    intro s t hs ht ⟨h₁, h₂⟩
    have ⟨N₁, hN₁⟩ := hs h₁
    have ⟨N₂, hN₂⟩ := ht h₂
    exact ⟨N₁ + N₂, fun n hn =>
      ⟨hN₁ n (Nat.le_trans (Nat.le_add_right N₁ N₂) hn),
        hN₂ n (Nat.le_trans (Nat.le_add_left N₂ N₁) hn)⟩⟩
  isOpen_sUnion := by
    intro S hS ⟨s, hsS, hs⟩
```

```

have ⟨N, hN⟩ := hS s hsS hs
exact ⟨N, fun n hn => ⟨s, hsS, hN n hn⟩⟩

```

問題12: `Option Nat` はコンパクトであることを示せ。

ヒント: `none` を覆う $\bigcup i_0$ は、ある N 以上の `some n` をすべて含む。残りの `some 0, ..., some (N-1)` を覆う添字を1つずつ `Classical.choose` で選び、 i_0 と合わせて $\text{Fin } (N + 1) \rightarrow I$ で番号づける (match で 0 と $k + 1$ に分ける)。

```

instance : CompactSpace (Option Nat) where
  isCompact_univ := sorry

```

対角線。

```

def diagonal (α : Type) : Set (α × α) := {p | p.1 = p.2}

```

離散一様構造: 対角線を含む関係がすべて近縁 (与える)。

```

@[reducible] def discreteUniformity (α : Type) : UniformSpace α where
  Entourage := {V | ∀ a, (a, a) ∈ V}
  univ_mem := fun _ => trivial
  mono := fun hV h a => h _ (hV a)
  inter_mem := fun hV hW a => ⟨hV a, hW a⟩
  refl := fun hV => hV
  symm := fun hV a => hV a
  comp := fun hV =>
    (diagonal α, fun _ => rfl, fun ⟨a, b⟩ ⟨z, (h₁ : a = z), (h₂ : z = b)⟩ => by
      rw [← h₂, ← h₁]
      exact hV a)

instance : UniformSpace Bool := discreteUniformity Bool

```

点 n だけで `true` になる関数。無限遠点では `false`。

```

def δ (n : Nat) : Option Nat → Bool
| none => false
| some m => decide (m = n)

```

反例の族。

```

def H : Set (Option Nat → Bool) := {f | ∃ n, f = δ n}

```

問題13: `Bool` の部分集合はすべて全有界であることを示せ。

ヒント: `[{b | b = true}, {b | b = false}]` で覆う。各片は1点なので、どの近縁についても小さい。

```
theorem totallyBounded_bool (s : Set Bool) : TotallyBounded s :=  
  sorry
```

問題14: `H` は無限遠点で等連続でないことを示せ。

ヒント: 近縁として対角線 `diagonal Bool` を使う。 `none` の開近傍は十分大きい `some N` を含むが、 `δ N none = false` と `δ N (some N) = true` は近くない。

```
theorem not_equicontinuousAt : ¬ EquicontinuousAt H none :=  
  sorry
```

問題15: `unifRel (diagonal Bool)` について小さい集合は `δ` を高々1つしか含まない。そこで、そのような集合の有限リストに対し、どこまで先に行ってもどの片にも入らない `δ n` があることを示せ（鳩の巣原理の代わり）。

ヒント: リストについての帰納法。 `A :: L` では、 `L` を避ける `δ n` ($N \leq n$) と、さらに `L` を避ける `δ m` ($n + 1 \leq m$) を取る。 `A` が両方を含むことはない（点 `some n` での値が `true` と `false` で異なる）。

```
theorem exists_avoid (L : List (Set (Option Nat → Bool)))  
  (hL : ∀ A ∈ L, Small (unifRel (diagonal Bool)) A) :  
  ∀ N, ∃ n, N ≤ n ∧ ∀ A ∈ L, δ n ∉ A :=  
  sorry
```

問題16: `H` は一様収束について全有界でないことを示せ。問題12～14と合わせると、主定理の「等連続」を外すと結論が成り立たないことがわかる。

```
theorem not_totallyBounded : ¬ TotallyBounded H :=  
  sorry  
  
end Counterexample
```

Part F: 逆向き——全有界なら等連続

ブルバキの Théorème 2 は必要十分条件である。逆向き（Arzelà が示した向き）は `x` のコンパクト性も要らない。ただし `H` の各写像が連続であることは仮定する。

```
section Converse
```

```
variable {X Y : Type} [UniformSpace Y]
```

問題17: 一様収束について全有界な族は、各点での値の集合も全有界であることを示せ。

ヒント: `unifRel V` についての被覆の各片の、`x` での値の像を並べる。

```
theorem TotallyBounded.eval {H : Set (X → Y)} (hH : TotallyBounded H) (x : X) :
  TotallyBounded ((fun f => f x) '' H) :=
  sorry

variable [TopologicalSpace X]
```

問題18: 連続写像からなる、一様収束について全有界な族は等連続であることを示せ。

方針: 3分の1の近縁 `W` を取り、`unifRel W` について `H` を有限個の小さい片で覆う。被覆のリストについての帰納法で、「どの片 `S` についても、`S n H` の写像がすべて `U` 上で `V`-振れに収まる」開近傍 `U` を作る。片 `S` が `H` の元 `g` を含むなら、`g` の連続性で近傍を縮め、`(f x, g x) · (g x, g x') · (g x', f x')` の3つをつなぐ（含まないなら縮めなくてよい）。

```
theorem TotallyBounded.equicontinuous {H : Set (X → Y)} (hc : ∀ f ∈ H, Continuous f)
  (hH : TotallyBounded H) : Equicontinuous H :=
  sorry

end Converse
```

Part G: フィルターと Zorn の補題

コンパクト版の証明には、コンパクト性をフィルターで言い換える必要がある。フィルターは「十分大きい集合」の集まりで、一様構造の近縁の全体（Part A の `univ_mem` ・ `mono` ・ `inter_mem`）がまさにその例になっている。

フィルター。

```
structure Filter (α : Type) where
  /-- 属する集合の全体。 -/
  sets : Set (Set α)
  /-- 全体集合は属する。 -/
  univ_mem : Set.univ ∈ sets
  /-- 属する集合より大きい集合は属する。 -/
  mono : ∀ {s t : Set α}, s ∈ sets → s ≤ t → t ∈ sets
  /-- 2つの共通部分も属する。 -/
  inter_mem : ∀ {s t : Set α}, s ∈ sets → t ∈ sets → s n t ∈ sets
```

```
namespace Filter
```

```
variable {α β : Type}
```

$s \in F$ で $s \in F.\text{sets}$ を表す。

```
instance : Membership (Set α) (Filter α) := {fun F s => s ∈ F.sets}
```

超フィルター: 空集合を含まず、どの集合もそれ自身か補集合のどちらかが属する。

```
structure IsUltra (F : Filter α) : Prop where
  /-- 空集合は属さない。 -/
  empty_not_mem : (∅ : Set α) ∉ F
  /-- どの集合も、それ自身か補集合が属する。 -/
  mem_or_compl_mem : ∀ s : Set α, s ∈ F ∨ sc ∈ F
```

超フィルターに属さない集合の補集合は属する。

```
theorem IsUltra.compl_mem {F : Filter α} (hF : F.IsUltra) {s : Set α} (hs : s ∉ F) : sc ∈ F :=
  (hF.mem_or_compl_mem s).resolve_left hs
```

空集合を含まないフィルターの元は空でない。

```
theorem nonempty_of_mem {F : Filter α} (hF : (∅ : Set α) ∉ F) {s : Set α} (hs : s ∈ F) :
  ∃ a, a ∈ s :=
  Classical.byContradiction fun h => hF (F.mono hs fun a ha => h (a, ha))
```

像フィルター: f による逆像が F に属する集合の全体。

```
def map (f : α → β) (F : Filter α) : Filter β where
  sets := {s | f-1 s ∈ F}
  univ_mem := F.univ_mem
  mono := fun hs h => F.mono hs fun a ha => h (f a) ha
  inter_mem := fun hs ht => F.inter_mem hs ht
```

問題19: 超フィルターの像は超フィルターであることを示せ。

ヒント: $f^{-1} \emptyset$ と $\emptyset$ 、 $f^{-1} s^c$ と $(f^{-1} s)^c$ は定義上等しい。

```
theorem IsUltra.map {F : Filter α} (hF : F.IsUltra) (f : α → β) : (F.map f).IsUltra :=
  sorry
```

問題20: 有限個（リスト `L` で添字づけた）の元の共通部分もフィルターに属することを示せ。リストについての再帰で示す。

```
theorem mem_list_inter (F : Filter α) {I : Type} (s : I → Set α) :  
  ∀ L : List I, (∀ i ∈ L, s i ∈ F) → {a | ∀ i ∈ L, a ∈ s i} ∈ F :=  
  sorry  
  
end Filter
```

Zorn の補題（包含順序の集合族版）

示したいのは次の形である（ β は任意の型、 $\mathcal{S}$ はその部分集合の族）：

> $\mathcal{S}$ の空でない鎖の合併がつねに $\mathcal{S}$ に属するなら、 $\mathcal{S}$ のどの元 A についても、 A を含む $\mathcal{S}$ の極大元 M がある。

証明は Bourbaki-Witt の不動点定理の方法による。選択公理で「真に大きい元があればそれを1つ選ぶ」写像 `next` を作り、 A から `next` と鎖の合併で到達できる集合の全体（塔）を帰納的に定義する。塔が鎖であることを示せば、塔全体の合併 M が `next M = M` を満たし、それが極大であることを意味する。

```
namespace Zorn  
  
variable {β : Type}
```

鎖: どの2つも包含で比較できる集合族。

```
def IsChain (c : Set (Set β)) : Prop := ∀ s ∈ c, ∀ t ∈ c, s ⊆ t ∨ t ⊆ s  
  
open Classical in
```

S より真に大きい $\mathcal{S}$ の元があればそれを1つ選び、なければ S 自身を返す。

```
noncomputable def next (S : Set (Set β)) (S : Set β) : Set β :=  
  if h : ∃ T, T ∈ S ∧ S ⊆ T ∧ T ≠ S then Classical.choose h else S
```

塔: A から出発して、`next` と「空でない鎖の合併」で到達できる集合。塔に属することを帰納的な述語として定義する（`Tower S A` の構成子が塔の閉包規則）。

```
inductive Tower (S : Set (Set β)) (A : Set β) : Set β → Prop  
| base : Tower S A A  
| next {S : Set β} : Tower S A S → Tower S A (next S S)  
| sup (c : Set (Set β)) : (∀ S ∈ c, Tower S A S) → IsChain c → (∪ S, S ∈ c) →
```

Tower $\mathcal{A}$ ($\bigcup_0 c$)

```
variable { $\mathcal{A}$  : Set (Set  $\beta$ )} {A : Set  $\beta$ }
```

`next` は大きくする (与える)。

```
theorem subset_next (S : Set  $\beta$ ) : S  $\subseteq$  next  $\mathcal{A}$  S := by
  unfold next
  split
  · next h => exact (Classical.choose_spec h).2.1
  · exact Set.subset_refl S
```

`next` は $\mathcal{A}$ から出ない (与える)。

```
theorem next_mem {S : Set  $\beta$ } (hS : S  $\in \mathcal{A}$ ) : next  $\mathcal{A}$  S  $\in \mathcal{A}$  := by
  unfold next
  split
  · next h => exact (Classical.choose_spec h).1
  · exact hS
```

問題21: 塔の元は $\mathcal{A}$ に属し、 A を含むことを示せ。塔についての帰納法 (`induction hS with`) で、構成子 `base` $\cdot$ `next` $\cdot$ `sup` ごとに示す。

```
theorem Tower.mem (hchain :  $\forall c, c \subseteq \mathcal{A} \rightarrow \text{IsChain } c \rightarrow (\exists S, S \in c) \rightarrow \bigcup_0 c \in \mathcal{A}$ )
  (hA : A  $\in \mathcal{A}$ ) {S : Set  $\beta$ } (hS : Tower  $\mathcal{A}$  S) : S  $\in \mathcal{A} \wedge A \subseteq S$  :=
  sorry
```

極点: 自分より真に小さい塔の元 T について、`next T` も自分に収まる。

```
def Extreme ( $\mathcal{A}$  : Set (Set  $\beta$ )) (A C : Set  $\beta$ ) : Prop :=
   $\forall T, \text{Tower } \mathcal{A} T \rightarrow T \subseteq C \rightarrow T \neq C \rightarrow \text{next } \mathcal{A} T \subseteq C$ 
```

問題22: 塔の元 C が極点なら、どの塔の元 T も $T \subseteq C$ か $\text{next } C \subseteq T$ であることを示せ。

ヒント: T についての帰納法。

- `base`: 問題21で $A \subseteq C$
- `next T`: 帰納法の仮定で場合分けし、 $T \subseteq C$ の場合はさらに $T = C$ かどうかで分ける ($T \neq C$ なら極点の定義が使える)
- `sup c`: 「 c のすべてが C に含まれる」かどうかで分ける

```

theorem Extreme.dichotomy (hchain :  $\forall c, c \subseteq \mathfrak{F} \rightarrow \text{IsChain } c \rightarrow (\exists S, S \in c) \rightarrow \bigcup_0 c \in \mathfrak{F}$ )
  (hA :  $A \in \mathfrak{F}$ ) {C : Set  $\beta$ } (hCT : Tower  $\mathfrak{F} A C$ ) (hC : Extreme  $\mathfrak{F} A C$ ) {T : Set  $\beta$ }
  (hT : Tower  $\mathfrak{F} A T$ ) :  $T \subseteq C \vee \text{next } \mathfrak{F} C \subseteq T :=$ 
sorry

```

問題23: 塔の元はすべて極点であることを示せ。

ヒント: C についての帰納法。

- **base**: A より真に小さい塔の元はない
- **next** C : 問題22を C に使う
- **sup** c : $T \neq \bigcup_0 c$ から、 T に含まれない $S \in c$ が取れる。問題22を S に使うと $T \subseteq S$ (かつ $T \neq S$) となり、 S の極点性が使える

```

theorem Tower.extreme (hchain :  $\forall c, c \subseteq \mathfrak{F} \rightarrow \text{IsChain } c \rightarrow (\exists S, S \in c) \rightarrow \bigcup_0 c \in \mathfrak{F}$ )
  (hA :  $A \in \mathfrak{F}$ ) {C : Set  $\beta$ } (hC : Tower  $\mathfrak{F} A C$ ) : Extreme  $\mathfrak{F} A C :=$ 
sorry

```

問題24: 塔は鎖であることを示せ (問題22・23から直ちに出る)。

```

theorem Tower.isChain (hchain :  $\forall c, c \subseteq \mathfrak{F} \rightarrow \text{IsChain } c \rightarrow (\exists S, S \in c) \rightarrow \bigcup_0 c \in \mathfrak{F}$ )
  (hA :  $A \in \mathfrak{F}$ ) : IsChain {S | Tower  $\mathfrak{F} A S$ } :=
sorry

```

問題25 (Zorn の補題) : 極大元の存在を示せ。

方針: 塔全体の合併 M は (問題24により) それ自身塔の元なので、**next** M も塔の元で **next** $M \subseteq M$ 。 M より真に大きい $N \in \mathfrak{F}$ があったとすると、**next** の定義により **next** M は M より真に大きい元になり、矛盾する (**dif_pos** で **next** の **if** を開く)。

```

theorem exists_maximal (hchain :  $\forall c, c \subseteq \mathfrak{F} \rightarrow \text{IsChain } c \rightarrow (\exists S, S \in c) \rightarrow \bigcup_0 c \in \mathfrak{F}$ )
  (hA :  $A \in \mathfrak{F}$ ) :  $\exists M, M \in \mathfrak{F} \wedge A \subseteq M \wedge \forall N, N \in \mathfrak{F} \rightarrow M \subseteq N \rightarrow N = M :=$ 
sorry

```

end Zorn

namespace Filter

variable { α : Type}

問題26 (超フィルター補題) : 空集合を含まないフィルターは超フィルターに延長できることを示せ。

方針: 「空集合を含まないフィルターの `sets`」の全体 $P : \text{Set} (\text{Set} (\text{Set } \alpha))$ に Zorn の補題を使う (鎖の合併もフィルターの3条件を満たし、空集合を含まない)。極大元 M が超フィルターでないとして、 $s \notin M \cdot s^c \notin M$ となる s を取ると、 $\{t \mid \exists m \in M, m \cap s \subseteq t\}$ が M より真に大きい P の元になり、極大性に反する。

```
theorem exists_ultra (F : Filter α) (hF : (∅ : Set α) ∉ F) :
  ∃ G : Filter α, G.IsUltra ∧ F.sets ⊆ G.sets :=
  sorry

end Filter
```

Part H: 主定理のコンパクト版

フィルター F が点 a に収束するとは、 a のどの開近傍も F に属すること。コンパクト性は超フィルターの収束で特徴づけられる (問題27・28)。一様空間では、全有界な集合を含む超フィルターはコーシーになる (問題31)。あとは、コーシーで各点収束するフィルターが一様収束すること (問題32) を示せば、主定理 (全有界版) からコンパクト版が出る。

section Compact

フィルターの収束。

```
def Filter.ConvergesTo {X : Type} [TopologicalSpace X] (F : Filter X) (a : X) : Prop :=
  ∀ U, IsOpen U → a ∈ U → U ∈ F

variable {X : Type} [TopologicalSpace X]
```

問題27: K を含む超フィルターがすべて K の点に収束するなら、 K はコンパクトであることを示せ。

方針: 開被覆 U が有限部分被覆を持たないと仮定する。 $\{s \mid \exists L : \text{List } I, \{a \mid a \in K \wedge \forall i \in L, a \notin U_i\} \subseteq s\}$ は空集合を含まないフィルターになる (空集合を含むなら L が有限部分被覆を与える。 `Set.finite_of_list` を使う)。これを超フィルターに延長して極限 $a \in K$ を取ると、 $a \in U_i$ なる i について U_i とその補集合の両方が属してしまう。

```
theorem isCompact_of_ultra {K : Set X}
  (h : ∀ F : Filter X, F.IsUltra → K ∈ F → ∃ a, a ∈ K ∧ F.ConvergesTo a) : IsCompact K
:=
  sorry
```

問題28: コンパクト集合を含む超フィルターは、その集合の点に収束することを示せ。

方針: 収束先がないと仮定すると、 K の各点 a に F に属さない開近傍が取れる（添字型は部分型 $\{a // a \in K\}$ ）。有限部分被覆を取ると、各片の補集合は F に属し（超フィルター）、問題20でその共通部分と K の共通部分も属するが、それは空である。

```
theorem IsCompact.ultra_converges {K : Set X} (hK : IsCompact K) {F : Filter X}
  (hF : F.IsUltra) (hKF : K ∈ F) : ∃ a, a ∈ K ∧ F.ConvergesTo a :=
  sorry
```

問題29: 超フィルターに属する集合が有限個の集合で覆われるなら、そのどれかが属することを示せ。

ヒント: リストについての再帰。先頭 A が属さなければ A^c が属するので、 $s \cap A^c$ を残りのリストで覆う。

```
theorem Filter.IsUltra.mem_of_cover {α : Type} {F : Filter α} (hF : F.IsUltra) :
  ∀ (L : List (Set α)) (s : Set α), s ∈ F → (∀ a ∈ s, ∃ A ∈ L, a ∈ A) → ∃ A ∈ L, A ∈ F
:=
  sorry

variable {α : Type} [UniformSpace α]
```

問題30: 一様空間のコンパクト集合は全有界であることを示せ。

ヒント: 対称な半分 w （問題1）を取り、各点 a の周りに w -球に収まる開集合（問題4）を選んで K を覆う。有限部分被覆の各片は V -小さい。

```
theorem IsCompact.totallyBounded {K : Set α} (hK : IsCompact K) : TotallyBounded K :=
  sorry
```

コーシーフィルター: どの近縁 V についても、 V -小さい集合が属する。

```
def Filter.Cauchy (F : Filter α) : Prop := ∀ V ∈ U α, ∃ A, A ∈ F ∧ Small V A
```

問題31: 全有界な集合を含む超フィルターはコーシーであることを示せ（問題29を使う）。

```
theorem Filter.IsUltra.cauchy {F : Filter α} (hF : F.IsUltra) {s : Set α}
  (hs : TotallyBounded s) (hsF : s ∈ F) : F.Cauchy :=
  sorry

end Compact

section AscoliCompact
```

```
variable {X Y : Type} [UniformSpace Y]
```

問題32: コーシーフィルターが各点で φ に収束するなら、一様収束の位相で φ に収束することを示せ。

方針: 開集合 $\mathcal{O} \ni \varphi$ は $\text{unifRel } V$ -球を含む。 V の半分 W を取り、 $\text{unifRel } W$ -小さい $A \in \mathcal{F}$ を取れば $A \subseteq \mathcal{O}$ となる。実際 $f \in A$ と点 x について、 φx の近くに値をとる $g \in A$ が ($\mathcal{F}$ が空集合を含まないので) 取れ、 $(\varphi x, g x) \in W$ と $(g x, f x) \in W$ をつなげばよい。

```
theorem Filter.Cauchy.convergesTo_of_pointwise {F : Filter (X → Y)} (hF : F.Cauchy)
  (hne : (∅ : Set (X → Y)) ∉ F) (φ : X → Y)
  (hφ : ∀ x, (F.map fun f => f x).ConvergesTo (φ x)) : F.ConvergesTo φ :=
  sorry
```

```
variable [TopologicalSpace X]
```

問題33 (主定理・Ascoli の定理、コンパクト版) : コンパクト空間 X から一様空間 Y への写像の族 H が等連続で、各点での値の集合がコンパクト集合に含まれ (相対コンパクト)、 H が一様収束の位相で閉なら、 H は一様収束の位相でコンパクトであることを示せ。

方針: 問題27に持ち込む。 H を含む超フィルター $\mathcal{F}$ は、主定理 (問題11) と問題30・31によりコーシー。各点 x での像フィルターは超フィルター (問題19) で、コンパクト集合を含むので収束する (問題28)。その極限を並べた φ に $\mathcal{F}$ は一様収束し (問題32)、 H が閉なので $\varphi \in H$ 。

```
theorem ascoli_compact [CompactSpace X] {H : Set (X → Y)} (hH : Equicontinuous H)
  (hpt : ∀ x, ∃ K : Set Y, IsCompact K ∧ (fun f => f x) '' H ⊆ K) (hcl : IsClosed H) :
  IsCompact H :=
  sorry
```

```
end AscoliCompact
```

```
-- できたら確認: 主定理 (全有界版・コンパクト版) が使う公理
-- #print axioms ascoli
-- #print axioms ascoli_compact
```

```
import «08_RealSol»
import «09_AscoliSol»
```

発展演習: 実数版 Arzelà–Ascoli の定理

`08_Real.lean` (実数) と `09_Ascoli.lean` (ブルバキ流 Ascoli の定理) の続き。抽象的な Ascoli の定理から、解析の講義でおなじみの形を導く:

> **Arzelà–Ascoli の定理**: 閉区間 $[a, b]$ 上の実数値関数の族が 一様有界かつ一様等連続なら、その族のどの関数列も、一様収束する部分列を持つ。

何が新しく要るか

`09_Ascoli.lean` の主定理 (コンパクト版 `ascoli_compact`) を実数に使うには、仮定を1つずつ実数の言葉で確かめればよい。Ascoli の議論そのものはやり直さない。

- 実数に一様構造を入れる (Part A)。 $|a - b| < \varepsilon$ で近縁を作る
- 各点での値の集合がコンパクト集合に入る——一様有界なら $[-M, M]$ に入り、それがコンパクトであることは `08_Real.lean` の **Heine–Borel** (`isCompact_Icc`) である
- 等連続性を ε -論法の形で言い直す (Part C)

これで「閉集合版」(Part D) がすぐに出る。そこから、閉包を取って「相対コンパクト版」(Part F)、さらに集積点と部分列を作って「点列版」(Part G) へ進み、最後に定義域を閉区間にする (Part H)。逆向き (Part E) も `09_Ascoli.lean` の結果から出る。

構成

- Part A: 実数の一様構造
- Part B: 2つの位相の一致
- Part C: 用語の対応 (等連続性・一様有界性)
- Part D: 実数版 Ascoli (閉集合版)
- Part E: 逆向き——コンパクトなら一様有界かつ等連続
- Part F: 相対コンパクト版 (閉包)
- Part G: 点列版 (一様収束する部分列)
- Part H: 閉区間上の関数 (古典的な Arzelà–Ascoli の定理)

進め方

これまでと同じく、`sorry` を自分の証明で置き換える。解答は `10_AscoliRealSol.lean` にある。このファイルは `08_Real.lean` ・ `09_Ascoli.lean` の解答を import しているので、それらを解き終えていなくても、そこで示した定理をすべて使える。

```
open CompleteOrderedField UniformSpace

variable {R : Type} [CompleteOrderedField R]
```

Part A: 実数の一様構造

2点 a, b が「 ε より近い」という関係 $\{(a, b) \mid |a - b| < \varepsilon\}$ を含む関係を、近縁とする。一様構造の5公理は、`08_Real.lean` の計算規則から出る。とくに `comp`（「半分」の近縁）は三角不等式 `abs_sub_le` と `add_halves` による $\varepsilon/2$ 論法である。

問題1: 実数の一様構造が公理を満たすことを示せ。

ヒント: `inter_mem` では2つの ε の小さいほう (`CompleteOrderedField.min` ・ `lt_min`) を取る。 `comp` では $\{p \mid \text{abs } (p.1 - p.2) < \varepsilon / 2\}$ を「半分」にする。

```
instance realUniformSpace : UniformSpace R where
  Entourage := {V | ∃ ε, 0 < ε ∧ ∀ a b : R, abs (a - b) < ε → (a, b) ∈ V}
  univ_mem := sorry
  mono := sorry
  inter_mem := sorry
  refl := sorry
  symm := sorry
  comp := sorry
```

近縁 $\{(a, b) \mid |a - b| < \varepsilon\}$ 。

```
def distLt (ε : R) : Set (R × R) := {p | abs (p.1 - p.2) < ε}

theorem distLt_mem {ε : R} (hε : 0 < ε) : distLt ε ∈  $\mathcal{U}$  R := (ε, hε, fun _ _ h => h)
```

Part B: 2つの位相の一致

この時点で、実数には位相が2つ載っている。

- `08_Real.lean` で入れた ε -位相 `CompleteOrderedField.instTopologicalSpace`

- `09_Ascoli.lean` の instance `UniformSpace.toTopologicalSpace` が、Part A の一様構造から作る位相

どちらも「どの点の周りにも ε -近傍が収まる集合が開」という同じ定義に見えるが、定義の字面が違う（片方は近縁を経由する）ので、Lean にとっては別の項である。 `IsCompact (Icc a b)` と書くとインスタンス探索は後者を選び、`08_Real.lean` の `isCompact_Icc a b`（前者についての定理）はそのままでは型が合わない。

そこで、2つの位相が等しいことを示し（問題2）、その等式で定理を運ぶ（問題3）。

補足（初読は飛ばしてよい）：インスタンスの菱形

同じ型に同じクラスのインスタンスが2通りの経路で作られる状況を、**菱形**（diamond）と呼ぶ。mathlib では、一様構造のクラスが位相そのものをフィールドとして持ち、実数の一様構造を作るときに「その位相は既存の位相と等しい」ことの証明を添える、という設計で菱形を避けている。ここでは `09_Ascoli.lean` の定義を変えずに、等式を1本示して済ませる。

開集合が一致する位相は等しい。フィールド `IsOpen` 以外は命題なので、`IsOpen` が関数として等しければ構造全体が等しい。

```
theorem TopologicalSpace.ext_iff_isOpen {X : Type} {t1 t2 : TopologicalSpace X}
  (h : ∀ s, t1.IsOpen s ↔ t2.IsOpen s) : t1 = t2 := by
  cases t1
  cases t2
  congr
  funext s
  exact propext (h s)
```

問題2: 一様構造から定まる位相は、`08_Real.lean` の ε -位相と等しいことを示せ。

ヒント: `TopologicalSpace.ext_iff_isOpen` で開集合の同値に還元する。 $|y - x|$ と $|x - y|$ の入れ替えに `abs_sub_comm` を使う。

```
theorem uniform_topology_eq :
  (UniformSpace.toTopologicalSpace : TopologicalSpace R) =
    CompleteOrderedField.instTopologicalSpace :=
  sorry
```

問題3: Heine–Borel を、一様構造の位相についての主張に運べ。

ヒント: `rw [uniform_topology_eq]` でゴールの中の位相を書き換えてから、`isCompact_Icc`。

```
theorem isCompact_Icc' (a b : R) : IsCompact (Icc a b) :=
  sorry
```

Part C: 用語の対応

解析でふつうに使う ε -論法の形の定義を用意し、`09_Ascoli.lean` の定義と対応させる。定義域 X はここでは一般の位相空間である。

```
section Dictionary

variable {X : Type} [TopologicalSpace X]
```

ε -論法での等連続性: どの点 x とどの $\varepsilon > 0$ にも x の開近傍 U があって、族のどの関数も U の上で $f x$ から ε 以上ずれない。

```
def EquicontinuousR (H : Set (X → R)) : Prop :=
  ∀ x ε, 0 < ε → ∃ U : Set X, IsOpen U ∧ x ∈ U ∧ ∀ f ∈ H, ∀ y ∈ U, abs (f y - f x) < ε
```

一様有界: 族のすべての関数の値が、共通の M で抑えられる。

```
def UniformlyBounded (H : Set (X → R)) : Prop := ∃ M, ∀ f ∈ H, ∀ x, abs (f x) ≤ M
```

問題4: ε -論法の等連続性と、近縁による等連続性 (`09_Ascoli.lean`) は同値であることを示せ。

ヒント: $\Rightarrow$ では近縁 V に含まれる ε -関係を使い、 $\Leftarrow$ では近縁 `distLt ε` を使う。

```
theorem equicontinuousR_iff {H : Set (X → R)} : EquicontinuousR H ↔ Equicontinuous H :=
  sorry
```

問題5: $|a| \leq M$ なら $a \in [-M, M]$ であることを示せ (`neg_abs_le · le_abs_self`)。

```
theorem mem_Icc_of_abs_le {a M : R} (h : abs a ≤ M) : a ∈ Icc (-M) M :=
  sorry

omit [TopologicalSpace X] in
```

問題6: 一様有界なら、各点での値の集合はコンパクト集合 ($[-M, M]$) に含まれることを示せ。

```
theorem UniformlyBounded.pointwise {H : Set (X → R)} (h : UniformlyBounded H) (x : X) :
  ∃ K : Set R, IsCompact K ∧ (fun f => f x) '' H ⊆ K :=
```

sorry

Part D: 実数版 Ascoli (閉集合版)

問題7 (実数版 Ascoli の定理、閉集合版) : コンパクト空間上の実数値関数の族が等連続・一様有界で、一様収束の位相で閉なら、コンパクトであることを示せ。 `09_Ascoli.lean` の `ascoli_compact` に、問題4・6を渡すだけである。

```
theorem arzela_ascoli_closed [CompactSpace X] {H : Set (X → ℝ)} (hH : EquicontinuousR H)
  (hb : UniformlyBounded H) (hcl : IsClosed H) : IsCompact H :=
```

sorry

Part E: 逆向き——コンパクトなら一様有界かつ等連続

等連続性は `09_Ascoli.lean` の問題30 (コンパクト $\Rightarrow$ 全有界) と問題18 (全有界 $\Rightarrow$ 等連続) から出る。一様有界性のために、「有限個の有界な集合の合併は有界」を用意する。上界の最大値を取る代わりに、上界の絶対値の和を取ると、場合分けが減る。

問題8: 有限個の有界な集合で覆われる集合は有界であることを示せ。「 y は M で抑えられる」を一般の述語 `B y M` (M について単調) として述べておき、実数 (問題9) と関数 (問題11) の両方に使う。

ヒント: リストについての再帰。 `A :: L` では、`s` のうち `A` に入らない点の集合に再帰し、2つの上界 `M1 · M2` から `abs M1 + abs M2` を作る。

```
theorem bounded_of_cover {β : Type} (B : β → ℝ → Prop)
  (hB : ∀ y M M', B y M → M ≤ M' → B y M') {s : Set β} :
  ∀ L : List (Set β), (∀ A ∈ L, ∃ M, ∀ y ∈ A, y ∈ s → B y M) →
  (∀ y ∈ s, ∃ A ∈ L, y ∈ A) → ∃ M, ∀ y ∈ s, B y M :=
```

sorry

問題9: 実数の全有界集合は有界であることを示せ。

ヒント: `distLt 1` について小さい集合で覆い、問題8を使う。`s` の点 `a` を含む片は `|a| + 1` で抑えられる (`abs_le_abs_add_abs_sub`)。

```
theorem TotallyBounded.bounded {s : Set ℝ} (hs : TotallyBounded s) :
  ∃ M, ∀ y ∈ s, abs y ≤ M :=
```

sorry

問題10: コンパクト空間上の連続関数は有界であることを示せ。

ヒント: 値の集合 `f '' univ` はコンパクト (`IsCompact.image`) なので全有界 (`IsCompact.totallyBounded`)。

```
theorem bounded_of_continuous [CompactSpace X] {f : X → ℝ} (hf : Continuous f) :  
  ∃ M, ∀ x, abs (f x) ≤ M :=  
sorry
```

問題11 (逆向き): 連続関数からなる、一様収束の位相でコンパクトな族は、一様有界かつ等連続であることを示せ。

ヒント: 一様有界性は、`unifRel (distLt 1)` について小さい片で族を覆い、問題8を使う。族の関数 `g` を含む片は、`g` の上界 (問題10) に `1` を足した数で抑えられる。

```
theorem arzela_ascoli_converse [CompactSpace X] {H : Set (X → ℝ)}  
  (hc : ∀ f ∈ H, Continuous f) (hH : IsCompact H) :  
  UniformlyBounded H ∧ EquicontinuousR H :=  
sorry
```

Part F: 相対コンパクト版 (閉包)

Part D の「閉」の仮定を外すため、族の閉包を取る。閉包も一様有界・等連続であることを示せば、Part D から閉包がコンパクト、つまり族が相対コンパクトであることがわかる。

閉包: どの開近傍も `s` と交わる点の全体。

```
def closure {α : Type} [TopologicalSpace α] (s : Set α) : Set α :=  
  {a | ∀ U, IsOpen U → a ∈ U → ∃ b, b ∈ U ∧ b ∈ s}
```

問題12: 集合は閉包に含まれることを示せ。

```
theorem subset_closure {α : Type} [TopologicalSpace α] (s : Set α) : s ⊆ closure s :=  
sorry
```

問題13: 閉包は閉集合であることを示せ。

ヒント: `isOpen_of_nhds` (`06_Topology.lean`)。閉包の外の点 `a` には `s` と交わらない開近傍 `U` があり、`U` の点はどれも閉包の外にある。

```
theorem isClosed_closure {α : Type} [TopologicalSpace α] (s : Set α) : IsClosed (closure s) :=  
sorry
```

閉包の点の近くには、元の集合の点がある（一様収束の意味で）。

```
theorem exists_near_of_mem_closure {X : Type} {H : Set (X → ℝ)} {f : X → ℝ}
  (hf : f ∈ closure H) {ε : ℝ} (hε : 0 < ε) : ∃ g, g ∈ H ∧ ∀ x, abs (f x - g x) < ε :=
by
  have hV : unifRel (distLt ε) ∈  $\mathcal{U}$  (X → ℝ) := ⟨_, distLt_mem hε, fun _ h => h⟩
  have ⟨0, h0, hf0, h0V⟩ := exists_open_ball hV f
  have ⟨g, hg0, hgH⟩ := hf 0 h0 hf0
  exact ⟨g, hgH, h0V g hg0⟩

omit [TopologicalSpace X] in
```

問題14: 一様有界な族の閉包は一様有界であることを示せ（上界は $M + 1$ にしてよい）。

```
theorem UniformlyBounded.closure {H : Set (X → ℝ)} (h : UniformlyBounded H) :
  UniformlyBounded (closure H) :=
sorry
```

$a < \varepsilon/4 \cdot b < \varepsilon/2 \cdot c < \varepsilon/4$ なら $a + b + c < \varepsilon$ 。

```
theorem add_three_lt {a b c ε : ℝ} (ha : a < ε / 2 / 2) (hb : b < ε / 2) (hc : c < ε / 2 / 2) :
  a + b + c < ε := by
  have h1 := add_lt_add ha hc
  rw [add_halves] at h1
  have h2 := add_lt_add h1 hb
  rw [add_halves] at h2
  have e : a + b + c = a + c + b := by ac_rfl
  rw [e]
  exact h2
```

問題15: 等連続な族の閉包は等連続であることを示せ。

ヒント: $\varepsilon/4 \cdot \varepsilon/2 \cdot \varepsilon/4$ の3分割（`add_three_lt`）。閉包の関数 f に一様に $\varepsilon/4$ 近い族の関数 g を取り、 $|f y - f x| \leq |f y - g y| + |g y - g x| + |g x - f x|$ とする。

```
theorem EquicontinuousR.closure {H : Set (X → ℝ)} (h : EquicontinuousR H) :
  EquicontinuousR (closure H) :=
sorry
```

問題16（実数版 Ascoli の定理、相対コンパクト版）：コンパクト空間上の実数値関数の族が等連続・一様有界なら、その閉包はコンパクトであることを示せ。

```

theorem arzela_ascoli_closure [CompactSpace X] {H : Set (X → ℝ)} (hH : EquicontinuousR H)
  (hb : UniformlyBounded H) : IsCompact (closure H) :=
  sorry

```

Part G: 点列版（一様収束する部分列）

コンパクト性から点列版へ進む。2段に分ける。

1. コンパクト集合の中の点列は**集積点** g を持つ: g のどの開近傍にも、どこまで先に行っても点列の項が入る（問題17）
2. 近傍を $1/(k+1)$ -球と小さくしながら、その中に入る項を順に選ぶと、 g に一様収束する部分列ができる（問題19）

2 で $1/(k+1) \rightarrow 0$ を使うところで、`08_Real.lean` の**アルキメデス性** (`exists_nat_gt`) が効く。逆数の計算を避けるため、 $1/(k+1)$ -球を $\{(a, b) \mid |a - b| * (k + 1) < 1\}$ の形で表す。部分列の添字 φ は「 $\varphi \ k < \varphi \ (k + 1)$ 」を満たす関数として表す。

問題17: コンパクト集合の中の点列は集積点を持つことを示せ。

方針: 背理法。 K のどの点 g にも、ある番号 $N \ g$ から先の項が入らない開近傍 $U \ g$ がある（添字型は部分型 $\{g \mid g \in K\}$ ）。有限部分被覆の番号の和 N を取ると、項 $u \ N$ はどの $U \ g$ にも入らないはずだが、 K は覆われているので矛盾する。番号の和は $(P.map \ N).sum$ 、和が各項以上であることはリストについての帰納法で示す。

```

theorem IsCompact.exists_clusterPt {α : Type} [TopologicalSpace α] {K : Set α}
  (hK : IsCompact K) {u : Nat → α} (hu : ∀ n, u n ∈ K) :
  ∃ g, g ∈ K ∧ ∀ U, IsOpen U → g ∈ U → ∀ N, ∃ n, N ≤ n ∧ u n ∈ U :=
  sorry

```

問題18: $\varepsilon^{-1} < n$ かつ $|d| * n < 1$ なら $|d| < \varepsilon$ であることを示せ。

ヒント: 背理法。 $\varepsilon \leq |d|$ なら $1 = \varepsilon * \varepsilon^{-1} < \varepsilon * n \leq |d| * n < 1$ となる。

```

theorem lt_of_mul_natCast_lt_one {d ε n : ℝ} (hε : 0 < ε) (hn : ε⁻¹ < n)
  (h : abs d * n < 1) : abs d < ε :=
  sorry

theorem natCast_succ_pos (k : Nat) : (0 : ℝ) < natCast (k + 1) := by
  rw [natCast_succ]
  exact lt_of_le_of_lt (natCast_nonneg k) (lt_add_of_pos_right _ zero_lt_one)

theorem natCast_le_natCast {m n : Nat} (h : m ≤ n) : (natCast m : ℝ) ≤ natCast n := by
  have ⟨k, hk⟩ := Nat.exists_eq_add_of_le h

```

```

rw [hk, natCast_add]
have := add_le_add_left _ _ (natCast_nonneg k) (natCast m : R)
rw [add_zero] at this
exact this

```

近縁 $\{(a, b) \mid |a - b| * (k + 1) < 1\}$ 。

```

def distInv (k : Nat) : Set (R × R) := {p | abs (p.1 - p.2) * natCast (k + 1) < 1}

theorem distInv_mem (k : Nat) : distInv (R := R) k ∈  $\mathcal{U}$  R := by
  have hk := natCast_succ_pos (R := R) k
  refine ⟨(natCast (k + 1))-1, inv_pos hk, fun a b h => ?_⟩
  have := mul_lt_mul_of_pos_left h hk
  rw [mul_comm (natCast (k + 1)) (natCast (k + 1))-1, inv_mul_cancel (ne_of_lt hk).symm,
    mul_comm] at this
  exact this

```

問題19: 点列の集積点 g に一様収束する部分列を作れ。

方針:

1. 各 k と N について、 N 以後の項で、 g から一様に $\text{distInv } k$ -近いものがある ($\text{unifRel } (\text{distInv } k)$ について exists_open_ball で開近傍を取り、集積点の性質を使う)
2. その項の番号を $c \ k \ N$ と Classical.choose で選び、 Nat.rec で $\varphi \ 0 = c \ 0 \ 0$ 、 $\varphi \ (k + 1) = c \ (k + 1) \ (\varphi \ k + 1)$ と再帰的に定める
3. $\varepsilon > 0$ にはアルキメデス性で $\varepsilon^{-1} < \text{natCast } N$ となる N を取り、問題18を使う

```

theorem exists_subseq_tendsto {X : Type} {u : Nat → X → R} {g : X → R}
  (hg : ∀ U, IsOpen U → g ∈ U → ∀ N, ∃ n, N ≤ n ∧ u n ∈ U) :
  ∃ φ : Nat → Nat, (∀ k, φ k < φ (k + 1)) ∧
  ∀ ε, 0 < ε → ∃ N, ∀ k, N ≤ k → ∀ x, abs (u (φ k) x - g x) < ε :=
  sorry

```

問題20 (実数版 Ascoli の定理、点列版) : コンパクト空間上の等連続・一様有界な関数族のどの関数列も、一様収束する部分列を持ち、その極限は連続であることを示せ。

ヒント: 閉包 (問題16) の中で集積点を取り (問題17)、問題19。極限の連続性は、閉包の等連続性 (問題15) から出る。

```

theorem arzela_ascoli_seq [CompactSpace X] {H : Set (X → R)} (hH : EquicontinuousR H)
  (hb : UniformlyBounded H) {u : Nat → X → R} (hu : ∀ n, u n ∈ H) :
  ∃ φ : Nat → Nat, (∀ k, φ k < φ (k + 1)) ∧ ∃ g : X → R,
  (∀ x ε, 0 < ε → ∃ U : Set X, IsOpen U ∧ x ∈ U ∧ ∀ y ∈ U, abs (g y - g x) < ε) ∧
  ∀ ε, 0 < ε → ∃ N, ∀ k, N ≤ k → ∀ x, abs (u (φ k) x - g x) < ε :=

```

sorry

end Dictionary

Part H: 閉区間上の関数（古典的な Arzelà–Ascoli の定理）

最後に、定義域を閉区間 $[a, b]$ にする。閉区間は $\mathbb{R}$ の部分集合 $\text{Icc } a \ b$ だが、関数の定義域には型が要るので、部分型 $\{x \ // \ x \in \text{Icc } a \ b\}$ を使い、部分空間位相を入れる。

部分空間位相: 全体の開集合で切り取った集合を開とする。

```
instance subspaceTopology {α : Type} [TopologicalSpace α] (p : α → Prop) :
  TopologicalSpace (Subtype p) where
  IsOpen s := ∃ U : Set α, IsOpen U ∧ ∀ x : Subtype p, x ∈ s ↔ x.1 ∈ U
  isOpen_univ := ⟨Set.univ, isOpen_univ, fun _ => ⟨fun _ => trivial, fun _ => trivial⟩⟩
  isOpen_inter := fun _ _ ⟨U, hU, hsU⟩ ⟨V, hV, htV⟩ =>
    ⟨U ∩ V, isOpen_inter _ _ hU hV, fun x =>
      ⟨fun ⟨h₁, h₂⟩ => ⟨(hsU x).mp h₁, (htV x).mp h₂⟩,
        fun ⟨h₁, h₂⟩ => ⟨(hsU x).mpr h₁, (htV x).mpr h₂⟩⟩⟩
  isOpen_sUnion := fun S hS =>
    ⟨⋃₀ {U | IsOpen U ∧ ∀ x : Subtype p, x.1 ∈ U → x ∈ ⋃₀ S},
      isOpen_sUnion _ fun _ hU => hU.1, fun x =>
        ⟨fun ⟨s, hsS, hxs⟩ =>
            have ⟨U, hU, hsU⟩ := hsS s hsS
            ⟨U, ⟨hU, fun y hy => ⟨s, hsS, (hsU y).mpr hy⟩⟩, (hsU x).mp hxs⟩,
          fun ⟨_, ⟨_, hU⟩, hxU⟩ => hU x hxU⟩⟩
```

問題21: 閉区間（部分空間）はコンパクト空間であることを示せ。

ヒント: 部分空間の開被覆 W_i の各片を切り取る全体の開集合 U_i を `Classical.choose` で選び、 U が $\text{Icc } a \ b$ を覆うことから問題3で有限個に減らす。

```
instance (a b : ℝ) : CompactSpace {x // x ∈ Icc a b} where
  isCompact_univ := sorry
```

問題22: 古典的な一様等連続性（ δ が点によらない）から、等連続性が出ることを示せ。

ヒント: 点 x の開近傍として $\{y \mid |y - x| < \delta\}$ を取る。これは $\text{ball } x \ \delta$ を切り取った集合で、 $\text{ball } x \ \delta$ の開性は `08_Real.lean` の `isOpen_ball`（問題2で位相を書き換えてから使う）。

```
theorem equicontinuousR_of_uniform {a b : ℝ} {H : Set ({x // x ∈ Icc a b} → ℝ)}
  (h : ∀ ε, 0 < ε → ∃ δ, 0 < δ ∧ ∀ f ∈ H, ∀ x y : {x // x ∈ Icc a b},
    abs (y.1 - x.1) < δ → abs (f y - f x) < ε) :
```

EquicontinuousR H :=

sorry

問題23（古典的な Arzelà–Ascoli の定理）：閉区間上の実数値関数の族が一様有界かつ一様等連続なら、そのどの関数列も一様収束する部分列を持つことを示せ（問題20・21・22）。

```
theorem arzela_ascoli {a b : ℝ} {H : Set ({x // x ∈ Icc a b} → ℝ)}
  (hb : ∃ M, ∀ f ∈ H, ∀ x, abs (f x) ≤ M)
  (heq : ∀ ε, 0 < ε → ∃ δ, 0 < δ ∧ ∀ f ∈ H, ∀ x y : {x // x ∈ Icc a b},
    abs (y.1 - x.1) < δ → abs (f y - f x) < ε)
  {u : ℕ → {x // x ∈ Icc a b} → ℝ} (hu : ∀ n, u n ∈ H) :
  ∃ φ : ℕ → ℕ, (∀ k, φ k < φ (k + 1)) ∧ ∃ g : {x // x ∈ Icc a b} → ℝ,
    ∀ ε, 0 < ε → ∃ N, ∀ k, N ≤ k → ∀ x, abs (u (φ k) x - g x) < ε :=
```

sorry

解答（10_AscoliRealSol.lean）では、古典的な Arzelà–Ascoli の定理が依存する公理は `propext` ・ `Classical.choice` ・ `Quot.sound` の3つだけである。実数の公理は `class` の引数なので現れない。`09_Ascoli.lean` で自作した Zorn の補題（超フィルター補題）を経由していることは、公理の一覧からは見えない——それは公理ではなく、証明された定理だからである。問題を解き終えたら、次の出力に `sorryAx` が残っていないことを確かめよ。

```
#print axioms arzela_ascoli
```

発展演習: グラフの被覆

06_Topology.lean まで読み終えた人のための演習問題集 (07_Exercises.lean · 08_Real.lean · 09_Ascoli.lean · 10_AscoliReal.lean とは独立に読める)。題材は被覆と基本群で、ゴールは「円周の基本群は整数の群 $\mathbb{Z}$ と同型」の離散版である。

位相空間の被覆の証明の骨組み

被覆 $p: \mathbb{R} \rightarrow S^1$ を使って $\pi_1(S^1) \cong \mathbb{Z}$ を示す古典的な議論は、次の段を踏む。

- 局所自明性:** 被覆とは、底空間の各点のまわりで、上の空間が底の開集合のコピーの直和になっている写像である
- 道の持ち上げ:** 底の道は、始点の持ち上げを決めれば上へ一意に持ち上がる。区間 $[0, 1]$ のコンパクト性 (ルベグ数) で道を局所自明な小片に切って示す
- ホモトピーの持ち上げ:** ホモトピックな道の持ち上げは、同じ点に着く
- モノドロミー:** 2 · 3 から、ループのホモトピー類がファイバー (1点の逆像) に作用する
- 計算:** 上の空間 $\mathbb{R}$ が単連結なので、この作用によってループの類が整数と対応する

グラフ版: 解析を抜いた骨組み

この演習では、同じ骨組みをグラフの上で組み立てる。位相は一切出てこない。

位相空間	グラフ
道 $[0, 1] \rightarrow X$	辺のリスト (隣接条件つき)
ホモトピー	行って戻る辺の対 $e \bar{e}$ の挿入・削除
局所自明性	各頂点から出る辺 (star) の上の全単射
区間のコンパクト性とルベグ数	リストの長さに関する帰納法
$\mathbb{R}$	直線グラフ (頂点は整数)
S^1	花束 (頂点1つ・ループ1本)
$\mathbb{R}$ は単連結	直線グラフには、行って戻る以外の閉じた道がない

段 2 の解析（区間のコンパクト性）は、グラフではリストの帰納法に置き換わる。一方で段 4・5 の議論は、位相空間版とほぼ同じ形のまま残る。グラフ版を組み立てると、被覆の理論のどこが解析で、どこが解析によらない部分かが分かれて見える。

構成

- Part A: Serre グラフと道
- Part B: ホモトピーと正規形定理
- Part C: π_1 亜群
- Part D: 被覆と持ち上げ
- Part E: モノドロミー
- Part F: $\pi_1(\text{花束}) \simeq \mathbb{Z}$

山場は Part B の正規形定理（問題11）と Part D の持ち上げ（問題20～23）である。Part F はそれまでの結果をすべて使う。

進め方

`07_Exercises.lean` ・ `09_Ascoli.lean` と同じく、`sorry` を自分の証明で置き換える。解答は `11_CoveringSol.lean` にある。与えてある宣言（`sorry` のないもの）も、問題を解くときに使ってよい。`06_Topology.lean` からは、全単射 `Function.Bijective` だけを使う。

Part A: Serre グラフと道

Serre 流のグラフは、頂点の型 `V`、向きつきの辺の型 `E`、各辺の始点 `init`、辺の向きを反対にする反転 `bar` からなる。無向の辺 1 本を、互いに反転しあう向きつきの辺 2 本 `e · e̅` で表す。こうしておく、道を「向きつきの辺の列」として一様に扱え、辺の終点も「反転した辺の始点」として定義できる。

辺のリストを語と呼ぶ。語が `v` から `w` への道であることを、添字つき帰納型 `Wf G v l w` で表す。`nil v` は `v` にとどまる長さ 0 の道、`cons e h` は `e` の終点から出る道 `h` の前に `e` をつないだ道である。

Serre 流のグラフ。辺は向きつきで、反転 `bar` は不動点のない対合。無向の辺 1 本を、互いに反転しあう向きつきの辺 2 本 `e · e̅` で表す。

```
structure SGraph where
  V : Type
  E : Type
  init : E → V
  bar : E → E
  bar_bar : ∀ e, bar (bar e) = e
```

```
bar_ne : ∀ e, bar e ≠ e
```

```
namespace SGraph
```

```
variable {G : SGraph}
```

辺の終点 = 反転した辺の始点。

```
def term (G : SGraph) (e : G.E) : G.V := G.init (G.bar e)
```

問題1: 反転した辺の終点は、元の辺の始点であることを示せ。

ヒント: `term` を展開して `bar_bar` で書き換える。

```
theorem term_bar (e : G.E) : G.term (G.bar e) = G.init e :=  
  sorry
```

語 (辺のリスト) が `v` から `w` への道であること: 隣接条件。

```
inductive Wf (G : SGraph) : G.V → List G.E → G.V → Prop  
| nil (v : G.V) : Wf G v [] v  
| cons (e : G.E) {l : List G.E} {w : G.V} :  
  Wf G (G.term e) l w → Wf G (G.init e) (e :: l) w
```

逆転補題: 空の語の道は動かない。

```
theorem Wf.nil_inv {v w : G.V} (h : Wf G v [] w) : v = w := by  
  cases h with  
| nil => rfl
```

逆転補題: `e :: l` が道なら、始点は `e` の始点で、残りも道。

```
theorem Wf.cons_inv {v w : G.V} {e : G.E} {l : List G.E} (h : Wf G v (e :: l) w) :  
  v = G.init e ∧ Wf G (G.term e) l w := by  
  cases h with  
| cons _ h' => exact ⟨rfl, h'⟩
```

補足: 逆転補題を与えておく理由

道 `h : Wf G v (e :: l) w` を `cases h` で分解すると、Lean は添字の等式 `v = G.init e` を解いて変数 `v` を消そうとする。添字の片方が変数ならこれは解けるが、両辺がどちらも関数の適用になると解けない。たとえば `e :: e :: l` の形の道を2段続けて `cases` で分解すると、2段目で `G.init`

($G.\text{bar } e$) (e の終点) と $G.\text{init } e'$ (次の辺 e' の始点) を等しいとおく必要が生じ、依存消去の失敗 (Dependent elimination failed) というエラーになる。

逆転補題 Wf.cons_inv は、添字を変数 v のままにして分解し、解けない等式を **結論の一部** $v = G.\text{init } e$ として返す。こうすると等式は証明の中で rw に使えるただの仮定になる。以下の問題では、道の分解には cases ではなくこの 2 つの補題を使うとよい。

問題2: 道の接続は道であることを示せ。

ヒント: 1 つ目の道についての場合分けで再帰する。 nil なら 2 つ目の道そのもの、 $\text{cons } e \ h_1$ なら $\text{.cons } e \ (\text{Wf.append } h_1 \ h_2)$ 。

```
theorem Wf.append {v u w : G.V} : ∀ {l₁ l₂ : List G.E},
  Wf G v l₁ u → Wf G u l₂ w → Wf G v (l₁ ++ l₂) w :=
  sorry
```

問題3: 道の終点は、始点と語で決まることを示せ。

ヒント: 1 つ目の道で場合分けし、2 つ目の道は逆転補題で分解して再帰する。

```
theorem Wf.end_unique : ∀ {y w₁ w₂ : G.V} {m : List G.E},
  Wf G y m w₁ → Wf G y m w₂ → w₁ = w₂ :=
  sorry
```

問題4: 接続の形の語が道なら、途中の点で 2 つの道に分かれることを示せ。

ヒント: l_1 についての再帰。 $e :: l_1$ の場合は Wf.cons_inv で分解して再帰し、始点を rw で合わせてから $\text{Wf.cons } e$ をつける。

```
theorem Wf.append_split : ∀ {v w : G.V} (l₁ : List G.E) {l₂ : List G.E},
  Wf G v (l₁ ++ l₂) w → ∃ u, Wf G v l₁ u ∧ Wf G u l₂ w :=
  sorry
```

語の反転: 各辺を反転して逆順に並べる。道を逆向きにたどることに当たる。

```
def revWord (G : SGraph) (l : List G.E) : List G.E := (l.map G.bar).reverse

theorem revWord_cons (e : G.E) (l : List G.E) :
  revWord G (e :: l) = revWord G l ++ [G.bar e] := by
  simp [revWord]

theorem revWord_append (l l' : List G.E) :
```

```

    revWord G (l ++ l') = revWord G l' ++ revWord G l := by
simp [revWord]

```

問題5: 道を逆向きにたどった語も道であることを示せ。

ヒント: 道についての再帰。 `cons e h` の場合は `revWord_cons` で書き換え、 `[G.bar e]` が `G.term e` から `G.init e` への道であること（問題1）と問題2を使う。

```

theorem Wf.revWord : ∀ {v w : G.V} {l : List G.E},
  Wf G v l w → Wf G w (SGraph.revWord G l) v :=
  sorry

end SGraph

```

例を 2 つ用意しておく。**花束**は頂点 1 つ・ループ 1 本のグラフで、円周の離散版に当たる。**直線グラフ**は頂点が整数で、`i` と `i + 1` を辺でつないだグラフで、実数直線の離散版に当たる。Part D で、直線グラフから花束への「向きだけを見る」射が被覆になることを示す。

花束（頂点1つ・ループ1本）。円周の離散版。向き付きの辺は `true`（正の向き）と `false`（逆向き）の2本。

```

def bouquet : SGraph where
  V := Unit
  E := Bool
  init _ := ()
  bar := Bool.not
  bar_bar := by decide
  bar_ne := by decide

instance : DecidableEq bouquet.E := inferInstanceAs (DecidableEq Bool)

```

直線グラフ。実数直線の離散版。頂点は整数、辺 `(i, true)` は `i` から `i + 1` へ、その反転 `(i, false)` は `i + 1` から `i` へ向かう。

```

def zcover : SGraph where
  V := Int
  E := Int × Bool
  init e := cond e.2 e.1 (e.1 + 1)
  bar e := (e.1, !e.2)
  bar_bar e := by cases e with | mk i b => simp
  bar_ne := by
    intro e h

```

```
cases e with
| mk i b => cases b <=> simp at h
```

Part B: ホモトピーと正規形定理

グラフの道のホモトピーは、**行って戻る辺**の対 $e, \bar{e}$ (backtrack) の削除と、その逆の挿入で生成される同値関係 `Homotopic` として定義する。連続的な変形は出てこない。backtrack を含まない語を**既約**と呼ぶ。

この Part の目標は**正規形定理** (問題11) である。**2つの語がホモトピックであるのは、それぞれを簡約した既約語が一致するときに限る**。とくに、各ホモトピー類には既約な代表がちょうど1つある (問題12)。

証明の方針を先に述べておく。「簡約の順序によらず結果が一致する」という合流性を直接示すのではなく、簡約を関数 `reduce` として先に定義する。`reduce` は語を後ろから読み、既約な語の先頭に辺を1本ずつ足していく (`rcons`)。足す辺が先頭と打ち消し合えば消す。あとは、`reduce` の値が backtrack の削除で変わらないこと (問題9) と、どの語も `reduce` の値まで削除だけで縮められること (問題10) を示せば、正規形定理が出る。関数として定義しておけば、正規形は計算で求まる (Part B の末尾の例)。

`reduce` の定義には、辺が等しいかどうかの判定 (`DecidableEq G.E`) が要る。

```
namespace SGraph

variable {G : SGraph}
```

語が**既約**であること: backtrack (e の直後の $\bar{e}$) を含まない。

```
def IsReduced : List G.E → Prop
| [] => True
| [_] => True
| e :: f :: l => f ≠ G.bar e ∧ IsReduced (f :: l)

theorem IsReduced.tail : ∀ {l : List G.E} {e : G.E}, IsReduced (e :: l) → IsReduced l
| [], _, _ => trivial
| _ :: _, _, h => h.2
```

語のホモトピー: backtrack の削除 (と、その逆の挿入) で生成される同値関係。

```
inductive Homotopic (G : SGraph) : List G.E → List G.E → Prop
| cancel (l1 l2 : List G.E) (e : G.E) :
  Homotopic G (l1 ++ e :: G.bar e :: l2) (l1 ++ l2)
| refl (l : List G.E) : Homotopic G l l
| symm {l l' : List G.E} : Homotopic G l l' → Homotopic G l' l
```

```
| trans {l l' l'' : List G.E} :
  Homotopic G l l' → Homotopic G l' l'' → Homotopic G l l''
```

削除だけの一方方向の関係。

```
inductive Reduces (G : SGraph) : List G.E → List G.E → Prop
| cancel (l1 l2 : List G.E) (e : G.E) :
  Reduces G (l1 ++ e :: G.bar e :: l2) (l1 ++ l2)
| refl (l : List G.E) : Reduces G l l
| trans {l l' l'' : List G.E} :
  Reduces G l l' → Reduces G l' l'' → Reduces G l l''
```

補足: 削除だけの関係 Reduces を別に立てる理由

語のホモトピー Homotopic は語についての関係で、道であること（隣接条件）は見えていない。道の水準で考えると、backtrack の削除は道を道に写す（問題13）が、挿入はそうとは限らない。挿入する辺 e の始点が、挿入する位置の頂点と一致している必要があるからである。

そこで、道についての議論はすべて削除だけの関係 Reduces で行い、挿入は正規形定理の中に閉じ込める。どの語も削除だけで正規形に到達する（問題10）ので、2つのホモトピックな道は、どちらも削除だけで同じ正規形に縮められる。紙の上では「ホモトピーで変形する」と一言で済ませるところだが、形式化すると、この区別が避けられない形で表に出てくる。

```
theorem Reduces.toHomotopic {l l' : List G.E} (h : Reduces G l l') :
  Homotopic G l l' := by
induction h with
| cancel l1 l2 e => exact .cancel l1 l2 e
| refl l => exact .refl l
| trans _ _ ih1 ih2 => exact .trans ih1 ih2

theorem Reduces.cons {l l' : List G.E} (e : G.E) (h : Reduces G l l') :
  Reduces G (e :: l) (e :: l') := by
induction h with
| cancel l1 l2 f => exact .cancel (e :: l1) l2 f
| refl l => exact .refl _
| trans _ _ ih1 ih2 => exact .trans ih1 ih2

section Reduce

variable [DecidableEq G.E]
```

既約な語の先頭に辺を1本足す。先頭と打ち消し合うなら消す。

```

def rcons (e : G.E) : List G.E → List G.E
| [] => [e]
| f :: r => if f = G.bar e then r else e :: f :: r

theorem rcons_nil (e : G.E) : rcons e ([] : List G.E) = [e] := rfl

theorem rcons_cons (e f : G.E) (r : List G.E) :
  rcons e (f :: r) = if f = G.bar e then r else e :: f :: r := rfl

```

語の簡約（正規形の計算）。

```

def reduce : List G.E → List G.E
| [] => []
| e :: l => rcons e (reduce l)

theorem reduce_cons (e : G.E) (l : List G.E) : reduce (e :: l) = rcons e (reduce l) := rfl

```

問題6: 既約な語に `rcons` で辺を足しても既約であることを示せ。

ヒント: `m` で場合分けし、`f :: r` の場合は `rcons_cons` で展開して `by_cases h : f = G.bar e` で分ける。`if` は `if_pos h · if_neg h` で外せる。

```

theorem isReduced_rcons {e : G.E} : ∀ {m : List G.E}, IsReduced m → IsReduced (rcons e m)
:=
  sorry

```

問題7: 簡約の結果は既約であることを示せ。

```

theorem reduce_isReduced : ∀ l : List G.E, IsReduced (reduce l) :=
  sorry

```

問題8: 既約な語に `ē` と `e` を順に足すと、元に戻ることを示せ。

ヒント: `m` の場合分け。`f :: r` の場合は `f = G.bar (G.bar e)` かどうかで分ける。等しければ `bar_bar` により `f = e` で、`r` をさらに場合分けする (`r = g :: r'` なら、`m` が既約なので `g ≠ G.bar e`)。

```

theorem rcons_rcons_bar (e : G.E) : ∀ (m : List G.E), IsReduced m →
  rcons e (rcons (G.bar e) m) = m :=
  sorry

```

問題9: `reduce` の値は、`backtrack` の削除で変わらないことを示せ。

ヒント: `l1` についての再帰。 `[]` の場合は `reduce_cons` を 2 回展開して問題7・8。

```
theorem reduce_append_cancel (e : G.E) : ∀ (l1 l2 : List G.E),  
  reduce (l1 ++ e :: G.bar e :: l2) = reduce (l1 ++ l2) :=  
sorry
```

```
theorem reduces_cons_rcons (e : G.E) (m : List G.E) :  
  Reduces G (e :: m) (rcons e m) := by  
match m with  
| [] => exact .refl _  
| f :: r =>  
  by_cases h : f = G.bar e  
  · subst h  
    rw [rcons_cons, if_pos rfl]  
    exact Reduces.cancel [] r e  
  · rw [rcons_cons, if_neg h]  
    exact .refl _
```

問題10: どの語も、自分の正規形へ削除だけで到達することを示せ。

ヒント: 再帰。 `Reduces.cons` と、与えてある `reduces_cons_rcons` を `Reduces.trans` でつなぐ。

```
theorem reduces_reduce : ∀ l : List G.E, Reduces G l (reduce l) :=  
sorry
```

問題11 (正規形定理) : ホモトピックであることと、正規形が一致することは同値であることを示せ。

ヒント: $\rightarrow$ は `Homotopic` についての帰納法 (`induction h`) で、`cancel` の場合が問題9。 $\leftarrow$ は、問題10により `l` も `l'` も正規形まで削除で縮められることを使う (`Reduces.toHomotopic`)。

```
theorem homotopic_iff_reduce_eq {l l' : List G.E} :  
  Homotopic G l l'  $\leftrightarrow$  reduce l = reduce l' :=  
sorry  
  
theorem reduce_eq_self_of_isReduced : ∀ {l : List G.E}, IsReduced l  $\rightarrow$  reduce l = l  
| [], _ => rfl  
| [ _ ], _ => rfl  
| _ :: _ :: _, h => by  
  rw [reduce_cons, reduce_eq_self_of_isReduced h.2, rcons_cons, if_neg h.1]
```

問題12: ホモトピックな既約語は等しいことを示せ (既約な代表の一意性)。

ヒント: 与えてある `reduce_eq_self_of_isReduced` で両辺を正規形に書き換え、問題11。

```

theorem eq_of_homotopic_of_isReduced {l l' : List G.E}
  (hl : IsReduced l) (hl' : IsReduced l') (h : Homotopic G l l') : l = l' :=
  sorry

end Reduce

```

問題13: backtrack の削除は、道であることを保つことを示せ。

ヒント: `l1` についての再帰。 `[]` の場合は逆転補題を 2 回使い、問題1で `G.term (G.bar e) = G.init e` と書き換える。

```

theorem Wf.of_append_cancel : ∀ {v w : G.V} (l1 : List G.E) {l2 : List G.E} {e : G.E},
  Wf G v (l1 ++ e :: G.bar e :: l2) w → Wf G v (l1 ++ l2) w :=
  sorry

theorem Wf.of_reduces {v w : G.V} {l l' : List G.E} (h : Reduces G l l') :
  Wf G v l w → Wf G v l' w := by
  induction h with
  | cancel l1 l2 e => exact fun hw => Wf.of_append_cancel l1 hw
  | refl l => exact id
  | trans _ _ ih1 ih2 => exact fun hw => ih2 (ih1 hw)

```

問題14: 道の正規形は、同じ始点・終点の道であることを示せ。

ヒント: 与えてある `Wf.of_reduces` と問題10。

```

theorem Wf.reduce [DecidableEq G.E] {v w : G.V} {l : List G.E} (hw : Wf G v l w) :
  Wf G v (SGraph.reduce l) w :=
  sorry

end SGraph

```

花束では、正規形が計算で求まる。どちらも `rfl` (定義の展開による計算) で確かめられる。2 つ目は「正の向きに 2 周してから逆向きに 2 周するループは、ホモトピーで消える」ことに当たる。

花束では、正規形が計算で求まる。

```

example : SGraph.reduce (G := bouquet) [true, false, true] = [true] := rfl
example : SGraph.reduce (G := bouquet) [true, true, false, false] = [] := rfl

```

Part C: π_1 亜群

道のホモトピー類全体に、接続と逆道を入れる。類は商型 `Quot` で作り、接続 `comp` ・逆道 `inv` は `Quot.lift` で代表の取り方によらずに定義する。そのためにホモトピーが接続と反転と両立すること（`appendLeft` ・ `appendRight` ・ 問題15）を使う。

基点を固定しないので、得られるのは群ではなく**亜群**（すべての射が可逆な圏）である。基点 `v` から `v` へのループの類だけを見れば群になり、それが基本群である。Part F では花束の基点 `()` のループの類 `PathClass bouquet () ()` を扱う。

```
namespace SGraph

variable {G : SGraph}

theorem Homotopic.appendRight {l l' : List G.E} (r : List G.E)
  (h : Homotopic G l l') : Homotopic G (l ++ r) (l' ++ r) := by
  induction h with
  | cancel l1 l2 e =>
    rw [List.append_assoc, List.append_assoc]
    exact .cancel l1 (l2 ++ r) e
  | refl l => exact .refl _
  | symm _ ih => exact .symm ih
  | trans _ _ ih1 ih2 => exact .trans ih1 ih2

theorem Homotopic.appendLeft (r : List G.E) {l l' : List G.E}
  (h : Homotopic G l l') : Homotopic G (r ++ l) (r ++ l') := by
  induction h with
  | cancel l1 l2 e =>
    rw [← List.append_assoc, ← List.append_assoc]
    exact .cancel (r ++ l1) l2 e
  | refl l => exact .refl _
  | symm _ ih => exact .symm ih
  | trans _ _ ih1 ih2 => exact .trans ih1 ih2
```

問題15: ホモトピックな語を反転しても、ホモトピックであることを示せ。

ヒント: `induction h`。 `cancel` の場合、反転した語がまた `... ++ e :: G.bar e :: ...` の形になることを `simp [revWord_append, revWord_cons, G.bar_bar, List.append_assoc]` で示す。

```
theorem Homotopic.revWordCong {l l' : List G.E} (h : Homotopic G l l') :
  Homotopic G (revWord G l) (revWord G l') :=
  sorry
```

問題16: 逆向きにたどってから元の向きにたどると、ホモトピーで消えることを示せ。

ヒント: 再帰。 $e :: l$ の場合、 `revWord_cons` と `List.append_assoc` で書き換えると真ん中に `backtrack` $\bar{e} \ e$ が現れる (`Homotopic.cancel` を `G.bar e` に使い、 `bar_bar`)。

```
theorem homotopic_revWord_append : ∀ l : List G.E, Homotopic G (revWord G l ++ l) [] :=  
  sorry
```

問題17: 元の向きにたどってから逆向きにたどっても、ホモトピーで消えることを示せ。

ヒント: 再帰。 `appendRight` ・ `appendLeft` で内側の `l ++ revWord G l` を消し、最後に残る `[e,  $\bar{e}$ ]` を消す。

```
theorem homotopic_append_revWord : ∀ l : List G.E, Homotopic G (l ++ revWord G l) [] :=  
  sorry
```

v から w への道のホモトピー類。

```
def PathClass (G : SGraph) (v w : G.V) : Type :=  
  Quot (fun a b : {l : List G.E // Wf G v l w} => Homotopic G a.1 b.1)  
  
namespace PathClass  
  
def mk {v w : G.V} {l : List G.E} (hl : Wf G v l w) : PathClass G v w :=  
  Quot.mk _ (l, hl)
```

同じ語の類は等しい。

```
theorem mk_eq_of_eq {v w : G.V} {l l' : List G.E}  
  (h : l = l') (hl : Wf G v l w) (hl' : Wf G v l' w) : mk hl = mk hl' := by  
  subst h; rfl
```

ホモトピックな語の類は等しい。

```
theorem sound {v w : G.V} {l l' : List G.E}  
  (hl : Wf G v l w) (hl' : Wf G v l' w) (h : Homotopic G l l') : mk hl = mk hl' :=  
  Quot.sound h
```

定数道の類。

```
def refl (v : G.V) : PathClass G v v := mk (Wf.nil v)
```

接続。

```

def comp {v u w : G.V} (φ : PathClass G v u) (ψ : PathClass G u w) : PathClass G v w :=
  Quot.lift
    (fun a => Quot.lift (fun b => mk (Wf.append a.2 b.2))
      (fun _ _ hb => Quot.sound (Homotopic.appendLeft a.1 hb)) ψ)
    (fun _ _ ha => by
      induction ψ using Quot.ind with
      | mk b =>
        apply Quot.sound
        exact Homotopic.appendRight b.1 ha)
  φ

```

逆道。

```

def inv {v w : G.V} (φ : PathClass G v w) : PathClass G w v :=
  Quot.lift (fun a => mk a.2.revWord)
    (fun _ _ h => Quot.sound (Homotopic.revWordCong h)) φ

variable {v w : G.V}

```

問題18a～e は亜群の法則である。どれも `induction γ using Quot.ind` で代表に下ろしてから示す。単位律と結合律は、代表の語の等式 (`List.nil_append · List.append_nil · List.append_assoc`) なので `mk_eq_of_eq` で、逆元の法則は問題16・17と `sound` で示せる。道の証明を明示する必要があるときは `Wf.append` など組み立てる。

問題18a: 左単位律を示せ。

```

theorem refl_comp (γ : PathClass G v w) : (refl v).comp γ = γ :=
  sorry

```

問題18b: 右単位律を示せ。

```

theorem comp_refl (γ : PathClass G v w) : γ.comp (refl w) = γ :=
  sorry

```

問題18c: 結合律を示せ。

```

theorem comp_assoc {u z : G.V} (γ : PathClass G v u) (δ : PathClass G u w)
  (ε : PathClass G w z) : (γ.comp δ).comp ε = γ.comp (δ.comp ε) :=
  sorry

```

問題18d: 逆道を先にたどると定数道であることを示せ。

```

theorem inv_comp (γ : PathClass G v w) : γ.inv.comp γ = refl w :=
  sorry

```

問題18e: 逆道をあとでたどっても定数道であることを示せ。

```

theorem comp_inv (γ : PathClass G v w) : γ.comp γ.inv = refl v :=
  sorry

end PathClass

end SGraph

```

Part D: 被覆と持ち上げ

グラフの射は、頂点と辺を写し、始点と反転を保つ写像である。**被覆**は、グラフの射であって、各頂点 y から出る辺の集合 (y の star) を、 y の像から出る辺の集合へ **全単射**に写すものとする。位相空間の被覆の局所自明性が、グラフではこの条件になる。

被覆 `Covering` は、写像 (データ) と、それが star の上で全単射であるという性質 (命題) を 1 つに束ねた structure として定義する。06_Topology.lean の同相写像 `Homeomorph` (写像と連続性を束ねた structure) と同じ作りである。extends `GraphHom Y X` により、被覆 p から射としての成分 $p.toV \cdot p.toE \cdot p.bar_toE$ などがそのまま取り出せる。

この Part の中心は問題20 (**道の持ち上げ**) で、証明は語の長さについての帰納法である。位相空間版で区間のコンパクト性とルベーグ数が担う仕事を、ここではリストの再帰が担う。問題22・23は、ホモトピックな道の持ち上げが同じ点に着くこと (ホモトピーの持ち上げ) で、Part B の正規形定理に帰着させる。

```

namespace SGraph

```

グラフの射: 頂点と辺を写し、始点と反転を保つ。

```

structure GraphHom (Y X : SGraph) where
  toV : Y.V → X.V
  toE : Y.E → X.E
  init_toE : ∀ e, X.init (toE e) = toV (Y.init e)
  bar_toE : ∀ e, X.bar (toE e) = toE (Y.bar e)

variable {Y X : SGraph}

theorem GraphHom.term_toE (p : GraphHom Y X) (e : Y.E) :
  X.term (p.toE e) = p.toV (Y.term e) := by

```

```
rw [term, p.bar_toE, p.init_toE]
rfl
```

被覆: グラフの射であって、各頂点の `star`（そこから出る辺の集合）の上で全単射になるもの。写像（データ）と性質（命題）を束ねた `structure` で、`06_Topology` の `Homeomorph` と同じ作り。

```
structure Covering (Y X : SGraph) extends GraphHom Y X where
  star_surj : ∀ (y : Y.V) (f : X.E), X.init f = toV y →
    ∃ e : Y.E, Y.init e = y ∧ toE e = f
  star_inj : ∀ e e' : Y.E, Y.init e = Y.init e' → toE e = toE e' → e = e'
```

語を辺の写像で写す。

```
def mapWord {A B : Type} (f : A → B) (m : List A) : List B := m.map f

theorem mapWord_nil {A B : Type} (f : A → B) : mapWord f [] = [] := rfl

theorem mapWord_cons {A B : Type} (f : A → B) (a : A) (m : List A) :
  mapWord f (a :: m) = f a :: mapWord f m := rfl

theorem mapWord_append {A B : Type} (f : A → B) (m m' : List A) :
  mapWord f (m ++ m') = mapWord f m ++ mapWord f m' := by
  simp [mapWord]

theorem mapWord_split {A B : Type} (f : A → B) : ∀ (l₁ : List B) {l₂ : List B}
  {m : List A}, mapWord f m = l₁ ++ l₂ →
  ∃ m₁ m₂, m = m₁ ++ m₂ ∧ mapWord f m₁ = l₁ ∧ mapWord f m₂ = l₂
| [], _, m, h => ⟨[], m, rfl, rfl, h⟩
| _ :: _, _, [], h => by simp [mapWord] at h
| _ :: l₁, _, a :: _, h => by
  rw [mapWord_cons] at h
  injection h with h1 h2
  match mapWord_split f l₁ h2 with
  | ⟨m₁, m₂, hm, hmap₁, hmap₂⟩ =>
    refine ⟨a :: m₁, m₂, by rw [hm]; rfl, ?_, hmap₂⟩
    rw [mapWord_cons, h1, hmap₁]

end SGraph

open SGraph in
```

問題19: 直線グラフ $\rightarrow$ 花束（辺の向きだけを見る射）は被覆であることを示せ。写像の成分は与えてあるので、`star` の上の全射性と単射性を示す。

ヒント: 全射性は花束の辺 `f` で場合分けする。`false` の場合、`y` から出る逆向きの辺は `(y - 1, false)` である。ただし `y : zcover.V` のままでは引き算の記法が見つからないので、`∀ j : Int, ∃ i : Int, i + 1 = j` を先に `Int` の上で示して `y` に使うとよい。単射性は 2 本の辺を分解し、向きが等しいことを使って場合分けし、`omega` で。

```
def zcoverCovering : Covering zcover bouquet where
  toV _ := ()
  toE e := e.2
  init_toE _ := rfl
  bar_toE _ := rfl
  star_surj := sorry
  star_inj := sorry

namespace SGraph.Covering

variable {Y X : SGraph} (p : Covering Y X)
```

問題20: **道の持ち上げの存在**。始点の持ち上げを決めれば、道は持ち上がることを示せ。

ヒント: `l` についての再帰。`f :: l` の場合、`star` の全射性 `p.star_surj` で `f` の持ち上げ `e` を取り、`e` の終点から残りを再帰で持ち上げる。`e` の終点の像が `f` の終点であることは `GraphHom.term_toE` から出る。

```
theorem exists_lift_word :
  ∀ (l : List X.E) (y : Y.V) {w : X.V}, Wf X (p.toV y) l w →
  ∃ w' : Y.V, ∃ m : List Y.E, Wf Y y m w' ∧ mapWord p.toE m = l ∧ p.toV w' = w :=
  sorry
```

問題21: **持ち上げの一意性**。同じ点から始まり同じ語に写る持ち上げは、一致することを示せ。

ヒント: 両方の語で場合分けする。先頭の辺どうしは始点も像も同じなので、`star` の単射性 `p.star_inj` で等しい。

```
theorem lift_word_unique :
  ∀ {m m' : List Y.E} {y w1 w2 : Y.V},
  Wf Y y m w1 → Wf Y y m' w2 → mapWord p.toE m = mapWord p.toE m' → m = m' :=
  sorry
```

問題22 (発展) : `backtrack` の削除は、持ち上げの終点を変えずに持ち上げの語を縮めることを示せ。

ヒント: `Reduces` の構成子についての再帰。本体は `cancel l1 l2 f` の場合で、`mapWord_split` で `m` を `m1 ++ m2` に切り、`m2` から `ê :: g :: m3` の形を取り出す。`g` と `Y.bar ê` は始点も像も同じなので、`star` の単射性で等しい。つまり上の語にも `backtrack` があり、あとは問題13で消せる。

```

theorem endpoint_of_reduces :
  ∀ {l l' : List X.E}, Reduces X l l' →
  ∀ {y b : Y.V} {m : List Y.E}, Wf Y y m b → mapWord p.toE m = l →
  ∃ m' : List Y.E, Wf Y y m' b ∧ mapWord p.toE m' = l' :=
sorry

```

問題23: ホモトピックな道の持ち上げは同じ点に着くことを示せ。

ヒント: `reduce` を使うために `letI : DecidableEq X.E := fun _ _ => Classical.propDecidable _` と置く。両方の持ち上げを正規形まで縮める（問題10・22）。正規形は等しい（問題11）ので、縮めた持ち上げの語も等しく（問題21）、終点も等しい（問題3）。

```

theorem endpoint_eq {l l' : List X.E}
  (hh : Homotopic X l l') {y b b' : Y.V} {m m' : List Y.E}
  (hm : Wf Y y m b) (hmapm : mapWord p.toE m = l)
  (hm' : Wf Y y m' b') (hmapm' : mapWord p.toE m' = l') : b = b' :=
sorry

```

Part E: モノドロミー

頂点 `v` の上のファイバー `p.Fiber v` は、`v` に写る頂点の全体である。ファイバーの点 `a` と、`v` から `w` への道の類 `γ` が与えられたとき、`γ` の代表を `a` から持ち上げた終点は `w` の上のファイバーに入る。問題23により、これは代表の取り方によらない。こうして得られる写像 `p.transport γ : p.Fiber v → p.Fiber w` をモノドロミー（輸送）と呼ぶ。

持ち上げの存在（問題20）は「存在する」という命題なので、終点を取り出すには選択公理（`Classical.choose`）を使う（`liftEnd`）。取り出した終点は、持ち上げの語を1つ示せば決まる（`liftEnd_eq`）ので、以下の問題では `liftEnd` の中身を開かずに `liftEnd_eq` を使えばよい。

頂点 `v` の上のファイバー。

```

abbrev Fiber (v : X.V) : Type := {y : Y.V // p.toV y = v}

theorem fiberExt {v : X.V} {a b : p.Fiber v} (h : a.1 = b.1) : a = b := by
  cases a
  cases b
  cases h
  rfl

theorem fiber_wf {v w : X.V} (a : p.Fiber v) {l : List X.E}
  (hl : Wf X v l w) : Wf X (p.toV a.1) l w := by
  rw [a.2]; exact hl

```

道の持ち上げの終点。存在は問題20、取り出しは選択公理。

```
noncomputable def liftEnd {v w : X.V} (a : p.Fiber v) (l : List X.E)
  (hl : Wf X v l w) : p.Fiber w :=
  (Classical.choose (p.exists_lift_word l a.1 (p.fiber_wf a hl)),
   (Classical.choose_spec (Classical.choose_spec
     (p.exists_lift_word l a.1 (p.fiber_wf a hl))))).2.2)

theorem liftEnd_spec {v w : X.V} (a : p.Fiber v) {l : List X.E} (hl : Wf X v l w) :
  ∃ m, Wf Y a.1 m (p.liftEnd a l hl).1 ∧ mapWord p.toE m = l :=
  (Classical.choose (Classical.choose_spec (p.exists_lift_word l a.1 (p.fiber_wf a hl))),
   (Classical.choose_spec (Classical.choose_spec
     (p.exists_lift_word l a.1 (p.fiber_wf a hl))))).1,
  (Classical.choose_spec (Classical.choose_spec
    (p.exists_lift_word l a.1 (p.fiber_wf a hl))))).2.1)
```

`liftEnd` の特徴づけ: 持ち上げの語を1つ示せば、終点が決まる。

```
theorem liftEnd_eq {v w : X.V} (a : p.Fiber v) {l : List X.E} (hl : Wf X v l w)
  {b : Y.V} {m : List Y.E} (hm : Wf Y a.1 m b) (hmap : mapWord p.toE m = l) :
  (p.liftEnd a l hl).1 = b := by
  match p.liftEnd_spec a hl with
  | ⟨M, hM, hMmap⟩ =>
    have heq : M = m := p.lift_word_unique hM hm (by rw [hMmap, hmap])
    subst heq
    exact Wf.end_unique hM hm
```

ホモトピー類に沿った輸送 (モノドロミー)。問題23により代表の取り方によらない。

```
noncomputable def transport {v w : X.V} (γ : PathClass X v w) (a : p.Fiber v) :
  p.Fiber w :=
  Quot.lift (fun l => p.liftEnd a l.1 l.2)
  (fun l l' hll' => by
    apply p.fiberExt
    match p.liftEnd_spec a l.2, p.liftEnd_spec a l'.2 with
    | ⟨_, hM, hMmap⟩, ⟨_, hM', hM'map⟩ =>
      exact p.endpoint_eq hll' hM hMmap hM' hM'map)
  γ
```

問題24: 定数道に沿った輸送は恒等であることを示せ。

ヒント: `p.fiberExt` で点の等式に落とし、`liftEnd_eq` に長さ 0 の持ち上げ `.nil a.1` を渡す。

```

theorem transport_refl {v : X.V} (a : p.Fiber v) :
  p.transport (PathClass.refl v) a = a :=
  sorry

```

問題25: 接続に沿った輸送は、輸送の合成であることを示せ。

ヒント: $\gamma \cdot \delta$ を代表に下ろす。 `liftEnd_spec` で 2 つの持ち上げの語を取り出し、それらを接続した語 (問題2) を `liftEnd_eq` に渡す。

```

theorem transport_trans {v u w : X.V}
  (γ : PathClass X v u) (δ : PathClass X u w) (a : p.Fiber v) :
  p.transport (γ.comp δ) a = p.transport δ (p.transport γ a) :=
  sorry

```

問題26: モノドロミーは、ファイバーの間の全単射であることを示せ。

ヒント: 逆道 $\gamma.\text{inv}$ に沿った輸送が逆写像になる。問題24・25と問題18d・18e。

```

theorem transport_bijective {v w : X.V} (γ : PathClass X v w) :
  Function.Bijective (p.transport γ) :=
  sorry

end SGraph.Covering

```

補足: 問題24~26はグラフを使っていない

問題24~26の証明を振り返ると、グラフであることを直接には一度も使っていない。使ったのは、持ち上げが一意に定まること (`liftEnd_eq`) と、道の類の亜群の法則 (問題18) だけである。したがって、位相空間の被覆についても、道とホモトピーの持ち上げさえ示せば、モノドロミーについての同じ定理が同じ証明で得られる。被覆の理論のうち解析が要るのは持ち上げの部分だけで、そこから先は持ち上げの一意性から出る。

`open SGraph`

花束のループ (正の向きの辺 1 本からなる道) を、直線グラフの点 i から持ち上げると、 i から $i + 1$ への辺になる。つまりループの類はファイバー (整数全体) に $+1$ で作用する。Part F で示す $\pi_1 \cong \mathbb{Z}$ の対応の中身は、この作用である。

花束のループ (円周を1周する道の離散版)。

```

def loopWf : Wf bouquet () [true] () :=

```

```
Wf.cons (G := bouquet) true (Wf.nil (bouquet.term true))
```

ループの類は、ファイバー $\mathbb{Z}$ に $+1$ で作用する。

```
example (i : Int) :  
  (zcoverCovering.transport (PathClass.mk loopWf) (i, rfl)).1 = i + 1 :=  
  zcoverCovering.liftEnd_eq (i, rfl) loopWf  
  (Wf.cons (G := zcover) ((i, true) : Int × Bool) (Wf.nil (zcover.term (i, true)))) rfl
```

Part F: $\pi_1(\text{花束}) \simeq \mathbb{Z}$

花束の基本群から整数への写像 `deg` (**モノドロミー次数**) を、「ループを直線グラフの 0 から持ち上げ、終点の整数を読む」ことで定義する。主定理は、`deg` が接続を和に写す全単射であることである。証明は 3 つの部品からなり、それぞれ位相空間版の議論の部品に対応する。

- **準同型性** (問題31) : 直線グラフの平行移動 $i \mapsto i + k$ は、持ち上げと可換な変換 (**デッキ変換**) である (問題29・30)。 i から持ち上げた終点は `deg  $\gamma$  + i` になる
- **単射性** (問題33・34) : 直線グラフは**単連結**である。既約な道は一方向に単調に進むので、既約な閉じた道は空しかない (問題27・28)。持ち上げが 0 に戻るループは、上で削除だけで空にでき、それを底へ写せばループ自身が自明とわかる
- **全射性** (問題35) : ループの k 乗と、その逆道で、すべての整数が得られる

整数の等式・不等式は `omega` で閉じられる。ただし `omega` は `Int.ofNat k` をひとかたまりの未知数として扱うので、`Int.ofNat (k + 1) = Int.ofNat k + 1` のような等式は、定義の展開 (`rfl`・`exact`) で閉じる。

被覆でなくても、グラフの射は削除の列を底へ押し出す。

```
theorem mapWord_reduces {Y X : SGraph} (p : GraphHom Y X) {m m' : List Y.E}  
  (h : Reduces Y m m') : Reduces X (mapWord p.toE m) (mapWord p.toE m') := by  
  induction h with  
  | cancel m1 m2 e =>  
    rw [mapWord_append, mapWord_cons, mapWord_cons, ← p.bar_toE, mapWord_append]  
    exact .cancel _ _ _  
  | refl l => exact .refl _  
  | trans _ _ ih1 ih2 => exact .trans ih1 ih2
```

問題27: 直線グラフの既約な道は、最初の辺の向きに単調に進むことを示せ。

ヒント: 語についての再帰で、2 辺目以降の道に再帰した結果と、1 辺目の向きを比べる。向きが食い違っていると、隣接条件から 2 辺目が 1 辺目の反転になり、既約性に反する。辺は `cases e with | mk i b` で整数と向きに分解できる。

```

theorem zc_drift : ∀ {m : List zcover.E} {e : zcover.E} {v w : Int},
  Wf zcover v (e :: m) w → IsReduced (e :: m) →
  (e.2 = true ∧ v < w) v (e.2 = false ∧ w < v) :=
sorry

```

問題28: **直線グラフは単連結**: 既約な閉じた道は空であることを示せ。

ヒント: 空でなければ、問題27から $v < v$ が出る。

```

theorem zc_reduced_closed : ∀ {m : List zcover.E} {v : Int},
  Wf zcover v m v → IsReduced m → m = [] :=
sorry

```

語を k だけ平行移動する (デッキ変換の辺への作用)。

```

def shiftWord (k : Int) (m : List zcover.E) : List zcover.E :=
  m.map (fun e => (e.1 + k, e.2))

theorem mapWord_shift (k : Int) : ∀ (m : List zcover.E),
  mapWord zcoverCovering.toE (shiftWord k m) = mapWord zcoverCovering.toE m
| [] => rfl
| e :: m => congrArg (fun t => e.2 :: t) (mapWord_shift k m)

```

問題29: 平行移動した語も道であることを示せ (平行移動はデッキ変換)。

ヒント: m についての再帰。辺の向きで場合分けし、始点を `subst` で揃える。 $(i + 1) + k = (i + k) + 1$ のような書き換えは `show ... from by omega` で作る。

```

theorem shift_wf (k : Int) : ∀ (m : List zcover.E) {v w : Int},
  Wf zcover v m w → Wf zcover (v + k) (shiftWord k m) (w + k) :=
sorry

```

花束の定数ループの類 (基本群の単位元)。

```

abbrev triv : PathClass bouquet () () := PathClass.refl (G := bouquet) ()

```

ファイバーの基点 (0 の持ち上げ)。

```

def basePt : zcoverCovering.Fiber () := ⟨(0 : Int), rfl⟩

```

モノドロミー次数: ループを 0 から持ち上げ、終点の整数を読む。

```

noncomputable def deg (γ : PathClass bouquet () ()) : Int :=
  (zcoverCovering.transport γ basePt).1

```

問題30: i から持ち上げると、終点は $\deg \gamma + i$ であることを示せ。

ヒント: γ を代表に下ろし、0 からの持ち上げ (`liftEnd_spec`) を i だけ平行移動したもの (問題29・`mapWord_shift`) を `liftEnd_eq` に渡す。

```

theorem transport_apply (γ : PathClass bouquet () ()) (i : Int) :
  (zcoverCovering.transport γ (i, rfl)).1 = deg γ + i :=
  sorry

```

問題31: `deg` は接続を和に写すことを示せ。

ヒント: 問題25で $\gamma \cdot \delta$ に沿った輸送を分解し、 δ に沿った輸送の始点が $\deg \gamma$ であることから問題30を使う。

```

theorem deg_comp (γ δ : PathClass bouquet () ()) :
  deg (γ.comp δ) = deg γ + deg δ :=
  sorry

theorem deg_refl : deg triv = 0 :=
  congrArg Subtype.val (zcoverCovering.transport_refl basePt)

```

問題32: `deg` は逆道を符号反転に写すことを示せ。

ヒント: 問題31を $\gamma.\text{inv}$ と γ に使い、問題18d と `deg_refl`。

```

theorem deg_inv (γ : PathClass bouquet () ()) : deg γ.inv = - deg γ :=
  sorry

```

花束のループの類。

```

def loop : PathClass bouquet () () := PathClass.mk loopWf

theorem deg_loop : deg loop = 1 := by
  have h : deg loop = (0 : Int) + 1 :=
    zcoverCovering.liftEnd_eq basePt loopWf
    (Wf.cons (G := zcover) ((0, true) : Int × Bool) (Wf.nil (zcover.term (0, true))))
  rfl
  omega

```

ループの k 乗。

```
def loopPow : Nat → PathClass bouquet () ()
| 0 => triv
| k + 1 => (loopPow k).comp loop
```

問題33: 持ち上げが 0 に戻るループは自明であることを示せ。

ヒント: γ を代表に下ろし、0 から 0 への持ち上げの語 m を取る。 m の正規形は 0 から 0 への道 (問題 14) で既約 (問題7) なので、問題28により空である。つまり m は削除だけで空にできる (問題10)。それを底へ写し (mapWord_reduces)、PathClass.sound を使う。reduce を使うには DecidableEq zcover.E を Classical.propDecidable で置く。

```
theorem deg_eq_zero {γ : PathClass bouquet () ()} (h : deg γ = 0) :
  γ = triv :=
sorry
```

問題34: deg は単射であることを示せ。

ヒント: $\deg(\gamma \cdot \delta \cdot \gamma^{-1}) = \deg \gamma$ を示して問題33を使い、亜群の法則 (問題18) で $\gamma = \delta$ に直す。calc で等式をつなぐと見通しがよい。

```
theorem deg_injective : Function.Injective deg :=
sorry

theorem deg_loopPow : ∀ k : Nat, deg (loopPow k) = Int.ofNat k
| 0 => deg_refl
| k + 1 => by
  have h := deg_comp (loopPow k) loop
  rw [deg_loopPow k, deg_loop] at h
  exact h
```

問題35: deg は全射であることを示せ。

ヒント: n で場合分けする。Int.ofNat k には loopPow k、Int.negSucc k ($-(k+1)$) には (loopPow (k + 1)).inv を使う (deg_loopPow · 問題32)。

```
theorem deg_surjective : Function.Surjective deg :=
sorry
```

問題36 (主定理): $\pi_1(\text{花束}) \cong \mathbb{Z}$ 。モノドロミ一次数 deg は、連接を和に写す全単射であることを示せ。

```
theorem pi1_bouquet :
  (∀ γ δ : PathClass bouquet () (), deg (γ.comp δ) = deg γ + deg δ) ∧
```

```
Function.Bijective deg :=  
sorry
```

解答（11_CoveringSol.lean）では、主定理が依存する公理は `propext` ・ `Classical.choice` ・ `Quot.sound` の 3 つだけである。 `06_Topology.lean` の最終定理 ・ `09_Ascoli.lean` の主定理と同じである。問題を解き終えたら、次の出力に `sorryAx` が残っていないことを確かめよ。

```
#print axioms pi1_bouquet
```

```
import «07_ExercisesSol»
import «08_RealSol»
```

発展演習: 位相空間の被覆

`08_Real.lean` と `11_Covering.lean` のあとに読む演習問題集。 `11_Covering.lean` でグラフの上に組み立てた被覆の理論を、位相空間の上で組み立て直す。この章の目標は**ホモトピーの持ち上げ定理**（問題19）で、続く `13_FundamentalGroupoid.lean` でこれを使って円周の基本群が整数の群と同型であることを示す。

グラフ版との違い

グラフ版では、道の持ち上げがリストの長さの帰納法一発で済んだ。位相空間では、道は区間 $[0, 1]$ からの連続写像なので、ここに解析が要る。この章の仕事は、ほぼすべてこの一点に集まっている。

1. 区間を細かく切って、各小片の像が均等被覆近傍に収まるようにする（区間のコンパクト性とルベグ数の補題。 `08_Real.lean` Part G）
2. 小片ごとに持ち上げて、つなぐ（貼り合わせ補題）
3. つないだものが連続であることを示す

この章では、区間を 2 等分していく帰納法をとる。長さ 2^{-n} の区間ごとに像が小さければ、区間の前半と後半を別々に持ち上げ、 $t = 1/2$ で貼り合わせればよい（問題15）。

前提

- 実数は `08_Real.lean` の完備順序体 $\mathbb{R}$ 、単位区間 $[0, 1]$ はその部分空間である
- 部分空間・積の位相は `07_Exercises.lean` Part 6 のものを使う。そこではどれも「射影をすべて連続にする最も粗い位相」として上から定義されていた。この章は、その上で具体的な議論ができる形の補題（Part A）から始める
- 前の章は解答ファイル（`07_ExercisesSol.lean` ・ `08_RealSol.lean`）を import する。前の章の問題が解けていなくても、この章は使える

構成

- Part A: 部分空間と積
- Part B: 閉集合と貼り合わせ
- Part C: 単位区間

- Part D: 被覆と局所的な持ち上げ
- Part E: 持ち上げ定理

山場は Part D の局所的な持ち上げ（問題15）と、Part E のホモトピーの持ち上げ（問題19）である。

進め方

`sorry` を自分の証明で置き換える。解答は `12_CoveringSpaceSol.lean` にある。与えてある宣言（`sorry` のないもの）も、問題を解くときに使ってよい。

```
namespace CovSpace
```

Part A: 部分空間と積

`07_Exercises.lean` Part 6 の部分空間・積の位相は、始位相——「射影をすべて連続にする最も粗い位相」——として上から定義されていた。この定義は、連続写像の判定（普遍性）には便利だが、開集合が具体的にどんな形をしているかは定義から読めない。この Part では、以後の議論に要る具体的な事実を取り出す。

- 部分空間への写像・部分空間からの写像の連続性（07 の問題33・35 の特殊化として与える）
- **連続性の局所性**（問題2）：各点に、そこへの制限が連続になる開近傍があれば連続
- **積の開集合は、各点のまわりに開集合の直積 $U \times V$ を含む**（問題4）

問題4 は `07_Exercises.lean` の問題36 と同じ発想で示す。「各点のまわりに開集合の直積を含む集合」の全体が位相をなすこと（問題3）を示し、それを「始位相は最も粗い」（07 の問題33(2)）に使う。

```
section
```

```
variable {X Y Z : Type} [tX : TopologicalSpace X] [tY : TopologicalSpace Y] [tZ :  
TopologicalSpace Z]
```

部分空間

`W : Set X` を条件 `X → Prop` とみなした部分型 `Subtype W` にも、`07_Exercises.lean` の部分空間の位相を入れる。`Set X` は定義上 `X → Prop` だが、インスタンスの探索はこの定義を展開しないことがあるので、`Set` 用のインスタンスとして明示しておく。

```
instance instTopSubtypeSet (W : Set X) : TopologicalSpace (Subtype W) := instTopSubtype W
```

```

theorem continuous_subtype_val {p : X → Prop} : Continuous (Subtype.val : Subtype p → X)
:=
  continuous_fromInitial (Y := fun _ : Unit => X) (fun _ => tX)
    (fun _ => Subtype.val) ()

theorem continuous_subtype_mk {p : X → Prop} {f : Z → X} (hf : Continuous f)
  (h : ∀ z, p (f z)) : Continuous (fun z => ((f z, h z) : Subtype p)) :=
  (continuous_toInitial_iff (Y := fun _ : Unit => X) (fun _ => tX)
    (fun _ => Subtype.val) tZ _).mpr fun _ => hf

theorem continuous_restrict {p : X → Prop} {f : X → Y} (hf : Continuous f) :
  Continuous (fun a : Subtype p => f a.1) :=
  Continuous.comp hf continuous_subtype_val

```

問題1: 開集合への制限で開な集合は、全体でも開であることを示せ。

ヒント: `07_Exercises.lean` の問題36 (`isOpen_subtype_iff`) で `s = Subtype.val ⁻¹' u` と書き、問題の集合が `W ∩ u` に等しいことを示す。

```

theorem isOpen_of_subtype_open {W : Set X} (hW : IsOpen W) {s : Set (Subtype W)}
  (hs : IsOpen s) : IsOpen {x | ∃ h : x ∈ W, ((x, h) : Subtype W) ∈ s} :=
  sorry

```

問題2: **連続性は局所的**: 各点に、そこへの制限が連続になる開近傍があれば連続であることを示せ。

ヒント: `06_Topology.lean` の `isOpen_of_nhds` を使う。点 `x` の開近傍 `W` について、制限の逆像を問題1で全体の開集合に持ち上げる。

```

theorem continuous_of_locally {f : X → Y}
  (h : ∀ x, ∃ W : Set X, IsOpen W ∧ x ∈ W ∧ Continuous (fun a : Subtype W => f a.1)) :
  Continuous f :=
  sorry

```

積

```

section Prod

variable {A B C : Type} [tA : TopologicalSpace A] [tB : TopologicalSpace B] [tC :
  TopologicalSpace C]

theorem continuous_fst : Continuous (@Prod.fst A B) :=
  continuous_fromInitial (Y := fun b => cond b A B)
    (fun b => match b with | true => tA | false => tB)

```

```

    (fun b => match b with | true => Prod.fst | false => Prod.snd) true

theorem continuous_snd : Continuous (@Prod.snd A B) :=
  continuous_fromInitial (Y := fun b => cond b A B)
    (fun b => match b with | true => tA | false => tB)
    (fun b => match b with | true => Prod.fst | false => Prod.snd) false

theorem continuous_prod_mk {f : C → A} {g : C → B} (hf : Continuous f) (hg : Continuous g)
:
  Continuous (fun z => (f z, g z)) :=
    (continuous_toInitial_iff (Y := fun b => cond b A B)
      (fun b => match b with | true => tA | false => tB)
      (fun b => match b with | true => Prod.fst | false => Prod.snd) tC _).mpr fun b =>
      match b with
      | true => hf
      | false => hg

end Prod

```

問題3: 開集合の直積を含む集合の全体は位相をなすことを示せ。

ヒント: 共通部分では直積 $U_1 \cap U_2 \cdot V_1 \cap V_2$ を取る。合併では、点が属する 1 つの集合の直積をそのまま使う。

```

@[reducible] def rectTop : TopologicalSpace (X × Y) where
  IsOpen W := ∀ p ∈ W, ∃ U V, IsOpen U ∧ IsOpen V ∧ p.1 ∈ U ∧ p.2 ∈ V ∧
    ∀ q : X × Y, q.1 ∈ U → q.2 ∈ V → q ∈ W
  isOpen_univ := sorry
  isOpen_inter := sorry
  isOpen_sUnion := sorry

```

問題4: (発展) 積の開集合は、各点のまわりに開集合の直積を含むことを示せ。

ヒント: 2 つの射影が問題3の位相に関して連続であることを示し、`07_Exercises.lean` の `initial_coarsest` (問題33(2)) に渡す。型推論の都合で、位相は $(t' := @rectTop\ X\ Y\ tX\ tY)$ と明示する。

```

theorem exists_rect {W : Set (X × Y)} (hW : IsOpen W) {p : X × Y} (hp : p ∈ W) :
  ∃ U V, IsOpen U ∧ IsOpen V ∧ p.1 ∈ U ∧ p.2 ∈ V ∧ ∀ q : X × Y, q.1 ∈ U → q.2 ∈ V → q ∈ W :=
  sorry

end

```

Part B: 閉集合と貼り合わせ

区間を半分ずつ使ってつないだ写像の連続性は、**貼り合わせ補題**（問題8）で示す。空間が 2 つの閉集合 $A \cdot B$ で覆われていれば、写像の連続性は、 A と B への制限の連続性に帰着する。証明は閉集合の言葉で書く：閉集合の逆像を A 側と B 側に分け、それぞれが閉であることを示す。

section

open CompleteOrderedField

variable {X Y Z : Type} [tX : TopologicalSpace X] [tY : TopologicalSpace Y] [tZ : TopologicalSpace Z]

theorem isClosed_inter {s t : Set X} (hs : IsClosed s) (ht : IsClosed t) : IsClosed (s ∩ t) := **by**

have e : (s ∩ t)^c = s^c ∪ t^c := **by**

apply Set.ext; **intro** x

constructor

· **intro** h

by_cases hs' : x ∈ s

· **exact** .inr **fun** ht' => h (hs', ht')

· **exact** .inl hs'

· **intro** h (h1, h2)

rcases h **with** h | h

· **exact** h h1

· **exact** h h2

show IsOpen (s ∩ t)^c

rw [e]; **exact** isOpen_union hs ht

問題5: 2 つの閉集合の合併は閉であることを示せ。

ヒント: 合併の補集合は補集合の共通部分。

theorem isClosed_union {s t : Set X} (hs : IsClosed s) (ht : IsClosed t) : IsClosed (s ∪ t) :=

sorry

問題6: 閉集合の逆像が閉なら連続であることを示せ。

ヒント: 開集合 s について s^c は閉 (Set.compl_compl)。 $f^{-1} s = (f^{-1} s^c)^c$ 。

theorem continuous_of_closed {f : X → Y} (h : ∀ s, IsClosed s → IsClosed (f⁻¹ s)) : Continuous f :=

sorry

```
theorem isClosed_preimage {f : X → Y} (hf : Continuous f) {s : Set Y} (hs : IsClosed s) :
  IsClosed (f ⁻¹' s) := hf _ hs
```

問題7: 閉集合の部分空間で閉な集合は、全体でも閉であることを示せ。

ヒント: `isOpen_subtype_iff` を補集合に使い、問題の集合を $A \cap U^c$ と書き直す。

```
theorem isClosed_of_subtype_closed {A : Set X} (hA : IsClosed A) {s : Set (Subtype A)}
  (hs : IsClosed s) : IsClosed {x | ∃ h : x ∈ A, (⟨x, h⟩ : Subtype A) ∈ s} :=
  sorry
```

問題8: **貼り合わせ補題**: 2 つの閉集合で覆われた空間上の写像は、それぞれへの制限が連続なら連続であることを示せ。

ヒント: 問題6を使う。閉集合の逆像を A 側と B 側の合併に分け、問題7と問題5。

```
theorem continuous_of_closed_cover {A B : Set X} (hA : IsClosed A) (hB : IsClosed B)
  (hcov : ∀ x, x ∈ A ∨ x ∈ B) {f : X → Y}
  (hfA : Continuous (fun a : Subtype A => f a.1)) (hfB : Continuous (fun a : Subtype B
=> f a.1)) :
  Continuous f :=
  sorry
```

実数の閉集合

貼り合わせでは、区間を $t \leq 1/2$ と $1/2 \leq t$ の 2 つの閉集合に分ける。

```
variable {R : Type} [CompleteOrderedField R]
```

問題9: 半直線 $\{x \mid x \leq c\}$ は閉であることを示せ。

ヒント: 補集合の点 x ($c < x$) のまわりには、半径 $x - c$ の近傍が収まる。`abs_lt` を使う。

```
theorem isClosed_le_const (c : R) : IsClosed ({x | x ≤ c} : Set R) :=
  sorry

theorem isClosed_ge_const (c : R) : IsClosed ({x | c ≤ x} : Set R) := by
  intro x hx
  have hx' : x < c := not_le.mp hx
  refine ⟨c - x, sub_pos.mpr hx', fun y hy hyc => ?_⟩
  have h1 := add_lt_add_right (abs_lt.mp hy).2 x
  rw [sub_add_cancel, sub_add_cancel] at h1
  exact not_le.mpr h1 hyc
```

end

Part C: 単位区間

単位区間 $\text{UI } R$ は、 R の部分集合 $[0, 1]$ を部分型にしたもの（位相は部分空間位相）である。2 等分の帰納法のために、前半・後半への縮小 $\text{halfL } t = t / 2$ ・ $\text{halfR } t = (t + 1) / 2$ と、 2^{-n} を表す $\text{halfPow } n$ を用意する。問題12 は、`08_Real.lean` のアルキメデス性から出る。

section

open CompleteOrderedField

variable {R : Type} [CompleteOrderedField R]

単位区間 $[0, 1]$ （実数の部分空間）。

```
abbrev UI (R : Type) [CompleteOrderedField R] : Type := Subtype (fun t : R => t ∈ Icc (0 : R) 1)
```

```
theorem zero_le_one' : (0 : R) ≤ 1 := le_of_lt zero_lt_one
```

```
theorem le_of_eq' {a b : R} (h : a = b) : a ≤ b := h ▸ le_refl a
```

```
def ui0 : UI R := ⟨0, le_refl 0, zero_le_one'⟩
```

```
def ui1 : UI R := ⟨1, zero_le_one', le_refl 1⟩
```

```
theorem UI.ext {s t : UI R} (h : s.1 = t.1) : s = t := by  
  cases s; cases t; cases h; rfl
```

半分と 2 倍の計算

```
theorem div_two_le_div_two {a b : R} (h : a ≤ b) : a / 2 ≤ b / 2 :=  
  mul_le_mul_of_nonneg_right h (le_of_lt (inv_pos zero_lt_two))
```

```
theorem zero_div_two : (0 : R) / 2 = 0 := zero_mul _
```

```
theorem two_div_two : (2 : R) / 2 = 1 := mul_inv_cancel 2 two_ne_zero
```

```
theorem two_mul_div_two (t : R) : 2 * t / 2 = t := by  
  rw [mul_comm]; exact mul_div_cancel two_ne_zero
```

```

theorem two_mul_half : (2 : R) * (1 / 2) = 1 := mul_div_cancel_left 1 two_ne_zero

theorem half_nonneg' : (0 : R) ≤ 1 / 2 := le_of_lt (half_pos zero_lt_one)

theorem half_le_one : (1 : R) / 2 ≤ 1 := le_of_lt (half_lt_self zero_lt_one)

theorem add_div_two (a b : R) : (a + b) / 2 = a / 2 + b / 2 := add_mul a b _

theorem sub_div_two (a b : R) : (a - b) / 2 = a / 2 - b / 2 := sub_mul a b _

theorem abs_div_two (a : R) : abs (a / 2) = abs a / 2 := by
  rw [div_def, abs_mul, abs_of_nonneg (le_of_lt (inv_pos zero_lt_two))]; rfl

theorem two_mul_le_two_mul {a b : R} (h : a ≤ b) : 2 * a ≤ 2 * b :=
  mul_le_mul_of_nonneg_left h (le_of_lt zero_lt_two)

theorem two_mul_one : (2 : R) * 1 = 2 := mul_one 2

```

$t / 2 \in [0, 1]$ 。

```

def halfL (t : UI R) : UI R :=
  (t.1 / 2, le_trans _ _ _ (le_of_eq' zero_div_two.symm) (div_two_le_div_two t.2.1),
   le_trans _ _ _ (div_two_le_div_two t.2.2) half_le_one)

```

$(t + 1) / 2 \in [0, 1]$ 。

```

def halfR (t : UI R) : UI R :=
  ((t.1 + 1) / 2,
   le_trans _ _ _ half_nonneg' (by
     have := div_two_le_div_two (add_le_add_right t.2.1 1)
     rwa [zero_add] at this),
   by
     have := div_two_le_div_two (add_le_add_right t.2.2 1)
     rwa [← two_def, two_div_two] at this)

theorem halfL_one : halfL (ui1 : UI R) = halfR ui0 := UI.ext (by show (1 : R) / 2 = (0 + 1) / 2; rw [zero_add])

theorem halfL_zero : halfL (ui0 : UI R) = ui0 := UI.ext zero_div_two

```

問題10: $t \mapsto t / 2$ は連続であることを示せ。

ヒント: `continuous_subtype_mk` と、`08_Real.lean` の `continuous_mul`。

```

theorem continuous_halfL : Continuous (halfL : UI R → UI R) :=
  sorry

theorem continuous_halfR : Continuous (halfR : UI R → UI R) :=
  continuous_subtype_mk (continuous_mul (continuous_add continuous_subtype_val
    (continuous_const _))
    (continuous_const _)) _

theorem two_mul_mem {t : R} (h0 : 0 ≤ t) (h1 : t ≤ 1 / 2) : 2 * t ∈ Icc (0 : R) 1 :=
  (by have := two_mul_le_two_mul h0; rwa [mul_zero] at this,
    by have := two_mul_le_two_mul h1; rwa [two_mul_half] at this)

theorem two_mul_sub_one_mem {t : R} (h0 : 1 / 2 ≤ t) (h1 : t ≤ 1) : 2 * t - 1 ∈ Icc (0 :
  R) 1 := by
  refine (sub_nonneg.mpr ?_, ?_)
  · have := two_mul_le_two_mul h0; rwa [two_mul_half] at this
  · have := add_le_add_right (two_mul_le_two_mul h1) (-1)
    rw [two_mul_one, two_def, add_assoc, add_neg_cancel, add_zero] at this
    exact this

```

2 のべき

2^{-n} 。

```

noncomputable def halfPow : Nat → R
| 0 => 1
| n + 1 => halfPow n / 2

theorem halfPow_pos : ∀ n : Nat, (0 : R) < halfPow n
| 0 => zero_lt_one
| n + 1 => half_pos (halfPow_pos n)

theorem halfPow_le_one : ∀ n : Nat, (halfPow n : R) ≤ 1
| 0 => le_refl 1
| n + 1 => le_trans _ _ _ (le_of_lt (half_lt_self (halfPow_pos n))) (halfPow_le_one n)

```

問題11: $n \cdot 2^{-n} \leq 1$ を示せ。

ヒント: n の帰納法。 $(n + 1) \cdot 2^{-(n+1)} = (n \cdot 2^{-n} + 2^{-n}) / 2 \leq (1 + 1) / 2$
 (halfPow_le_one) 。

```

theorem natCast_mul_halfPow_le : ∀ n : Nat, natCast n * (halfPow n : R) ≤ 1 :=
  sorry

```

問題12: 十分大きい n で $2^{-n} < \delta$ となることを示せ。

ヒント: `08_Real.lean` のアルキメデス性で $\delta^{-1} < n$ となる n を取ると $1 < n \cdot \delta$ 。問題11と比べる。

```
theorem exists_halfPow_lt {δ : ℝ} (hδ : 0 < δ) : ∃ n : Nat, (halfPow n : ℝ) < δ :=  
  sorry  
  
end
```

Part D: 被覆と局所的な持ち上げ

開集合 U が $p : E \rightarrow X$ で均等に被覆される (EvenlyCovered) とは、 $p^{-1} U$ が互いに交わらない開集合 (シート) V_i に分かれ、各シートが p で U と同相に写ることである。同相であることは、逆写像 s_i を与え、 U の上で連続であることで表す。被覆 (CoveringMap) は、連続写像であって、底のどの点にも均等に被覆される開近傍があるもの。11_Covering.lean の Covering や 06_Topology.lean の Homeomorph と同じく、写像と性質を束ねた structure にする。

この Part の中心は局所的な持ち上げ (問題15) である。 $F : Y \times [0,1] \rightarrow X$ と、 $F(\cdot, 0)$ の持ち上げ F_0 が与えられ、 y_0 の近傍 N の上で「長さ 2^{-n} の区間ごとに像が均等被覆近傍に収まる」とする。このとき y_0 のより小さい近傍の上に、 F の連続な持ち上げがある。証明は n の帰納法で、 $n = 0$ (像が1つの均等被覆近傍に収まる場合) は問題14、 $n + 1$ は区間の前半と後半に分けて帰納法の仮定を2回使い、 $t = 1/2$ で貼り合わせる。

補足: 逆写像を X 全体の関数として持つ理由

シートの逆写像 s_i は、本来は U から V_i への写像である。ここでは $s_i : X \rightarrow E$ と X 全体の関数として持ち、 U の外での値は使わない (連続性も U への制限で述べる)。こうすると、 $s_i(F q)$ のような合成を書くたびに「 $F q \in U$ の証明」を添えずに済む。部分型を使うのは、連続性を述べる場所だけになる。

section

```
open CompleteOrderedField
```

問題13: 定数写像は連続であることを示せ。

ヒント: 逆像は、値が s に入るかどうかで `univ` か `∅`。

```
theorem continuous_const_map {A B : Type} [TopologicalSpace A] [TopologicalSpace B] (b :  
B) :  
  Continuous (fun _ : A => b) :=
```

sorry

```
variable {R : Type} [CompleteOrderedField R]
variable {E X : Type} [tE : TopologicalSpace E] [tX : TopologicalSpace X]
```

開集合 U が p で均等に被覆される: $p^{-1} U$ は互いに交わらない開集合 (シート) V_i に分かれ、各シートは p で U と同相に写る。 s_i はその逆写像 (U の上だけで意味を持つ)。

```
def EvenlyCovered (p : E → X) (U : Set X) : Prop :=
  ∃ (ι : Type) (V : ι → Set E) (s : ι → X → E),
    (∀ i, IsOpen (V i)) ∧
    (∀ e, p e ∈ U → ∃ i, e ∈ V i) ∧
    (∀ i j e, e ∈ V i → e ∈ V j → i = j) ∧
    (∀ i x, x ∈ U → s i x ∈ V i ∧ p (s i x) = x) ∧
    (∀ i e, e ∈ V i → s i (p e) = e) ∧
    (∀ i, Continuous (fun x : Subtype U => s i x.1))
```

被覆: 連続写像であって、底のどの点にも均等に被覆される開近傍があるもの。

```
structure CoveringMap (E X : Type) [TopologicalSpace E] [TopologicalSpace X] where
  toFun : E → X
  continuous_toFun : Continuous toFun
  evenly : ∀ x, ∃ U, IsOpen U ∧ x ∈ U ∧ EvenlyCovered toFun U

theorem sub_halfR (a b : R) : (a + 1) / 2 - (b + 1) / 2 = (a - b) / 2 := by
  rw [← sub_div_two, add_sub_add_right]
```

問題14: 局所的な持ち上げの基底: 像が 1 つの均等被覆近傍に収まるなら、シートの逆写像を合成すればよいことを示せ。

ヒント: `hsmall` を $\tau = 0$ に使うと、 $N \times [0, 1]$ 全体の像が 1 つの U に収まる。 $F_0 y_0$ を含むシート V_i を取り、 N' を「 N の点で F_0 の値が V_i に入るもの」(問題1で開)、 $G := s_i \circ F$ とする。

```
theorem lift_local_zero (p : CoveringMap E X) (Y : Type) [TopologicalSpace Y]
  (F : Y × UI R → X) (hF : Continuous F) (N : Set Y) (hN : IsOpen N)
  (F0 : Y → E) (hF0 : Continuous (fun y : Subtype N => F0 y.1))
  (h0 : ∀ y ∈ N, p.toFun (F0 y) = F (y, ui0))
  (hsmall : ∀ τ : UI R, ∃ U, EvenlyCovered p.toFun U ∧
    ∀ y ∈ N, ∀ t : UI R, abs (t.1 - τ.1) ≤ 1 → F (y, t) ∈ U)
  (y0 : Y) (hy0 : y0 ∈ N) : ∃ N' : Set Y, IsOpen N' ∧ y0 ∈ N' ∧ N' ⊆ N ∧ ∃ G : Y × UI R
→ E,
  (∀ y ∈ N', ∀ t, p.toFun (G (y, t)) = F (y, t)) ∧ (∀ y ∈ N', G (y, ui0) = F0 y) ∧
```

```
Continuous (fun q : Subtype (fun q : Y × UI R => q.1 ∈ N') => G q.1) :=
sorry
```

問題15: (発展) **局所的な持ち上げ** (2 等分の帰納法)。 y_0 の近傍 N の上で、長さ 2^{-n} の区間ごとに F の像が均等被覆近傍に収まるなら、 y_0 のより小さい近傍の上に連続な持ち上げがあることを示せ。

ヒント: n についての再帰。 0 は問題14。 $n + 1$ では $F_1(y, t) = F(y, t/2) \cdot F_2(y, t) = F(y, (t+1)/2)$ とし、 F_1 を F_0 から、 F_2 を F_1 の持ち上げの終点 $G_1(\cdot, 1)$ から持ち上げる (長さの条件は縮小で半分になる)。最後に $t \leq 1/2$ と $1/2 \leq t$ の2つの閉集合で貼り合わせる (問題8・問題9)。 $t = 1/2$ で両者が一致することに注意。 `if` による場合分けのために、証明の冒頭で `classical` とする。

```
theorem lift_local (p : CoveringMap E X) : ∀ (n : Nat) (Y : Type) [TopologicalSpace Y]
  (F : Y × UI R → X), Continuous F → ∀ (N : Set Y), IsOpen N →
  ∀ (F0 : Y → E), Continuous (fun y : Subtype N => F0 y.1) →
  (∀ y ∈ N, p.toFun (F0 y) = F (y, ui0)) →
  (∀ τ : UI R, ∃ U, EvenlyCovered p.toFun U ∧
    ∀ y ∈ N, ∀ t : UI R, abs (t.1 - τ.1) ≤ halfPow n → F (y, t) ∈ U) →
  ∀ y0 ∈ N, ∃ N' : Set Y, IsOpen N' ∧ y0 ∈ N' ∧ N' ⊆ N ∧ ∃ G : Y × UI R → E,
  (∀ y ∈ N', ∀ t, p.toFun (G (y, t)) = F (y, t)) ∧ (∀ y ∈ N', G (y, ui0) = F0 y) ∧
  Continuous (fun q : Subtype (fun q : Y × UI R => q.1 ∈ N') => G q.1) :=
sorry
```

```
end
```

Part E: 持ち上げ定理

Part D の局所的な持ち上げから、3つの定理を出す。

- **小ささの条件** (問題16) : 各点 y_0 のある近傍と n について、Part D の仮定が成り立つ。区間のコンパクト性とルベグ数から出る
- **道の持ち上げの一意性** (問題17) : 区間の連結性 (`08_Real.lean` の `Icc_connected`) から出る
- **ホモトピーの持ち上げ** (問題19) と、その特別な場合 (Y が1点) の**道の持ち上げ** (問題18)

問題19 では、持ち上げ G を点ごとに定義する: 各 y について、道 $t \mapsto F(y, t)$ を $F_0 y$ から持ち上げた道の値を $G(y, t)$ とする。連続性は局所的に示す (問題2)。各点のまわりでは、Part D の局所的な持ち上げが連続で、一意性 (問題17) によって G と一致するからである。

```
section
```

```
open CompleteOrderedField
```

```
variable {R : Type} [CompleteOrderedField R]
variable {E X : Type} [tE : TopologicalSpace E] [tX : TopologicalSpace X]
```

問題16: (発展) 小ささの条件: y_0 のある近傍と n について、長さ 2^{-n} の区間ごとに像が均等被覆近傍に収まることを示せ。

ヒント: 各 t で $F(y_0, t)$ の均等被覆近傍 U の逆像に問題4を使い、 $N_t \times J_t$ を取る (J_t は区間の開集合なので、`isOpen_subtype_iff` で R の開集合 0_t に直す)。区間のコンパクト性 (`isCompact_Icc`) で有限個の t に減らし、 N をそれらの N_t の共通部分 (`Set.interFin`) とする。 δ はルベーグ数 (`lebesgue`)、 n は問題12で取る。

```
theorem exists_small (p : CoveringMap E X) {Y : Type} [tY : TopologicalSpace Y]
  (F : Y × UI R → X) (hF : Continuous F) (y0 : Y) :
  ∃ N : Set Y, IsOpen N ∧ y0 ∈ N ∧ ∃ n : Nat, ∀ τ : UI R, ∃ U, EvenlyCovered p.toFun U ∧
    ∀ y ∈ N, ∀ t : UI R, abs (t.1 - τ.1) ≤ halfPow n → F (y, t) ∈ U :=
  sorry
```

問題17: 道の持ち上げの一意性: 同じ道の2つの持ち上げは、始点が一致すれば一致することを示せ。

ヒント: $\{t \mid f t = g t\}$ と $\{t \mid f t \neq g t\}$ がともに開であることを示し、`Icc_connected` を使う。前者では、 $f t$ を含むシート $V i$ を取ると、 f と g がともに $V i$ に入る近傍で $f = s i \circ p \circ f = s i \circ p \circ g = g$ 。

```
theorem lift_unique (p : CoveringMap E X) {f g : UI R → E} (hf : Continuous f) (hg :
  Continuous g)
  (hfg : ∀ t, p.toFun (f t) = p.toFun (g t)) (h0 : f ui0 = g ui0) : f = g :=
  sorry
```

1 点の空間の位相 (開集合はすべて)。

```
instance instTopUnit : TopologicalSpace Unit where
  IsOpen _ := True
  isOpen_univ := trivial
  isOpen_inter _ _ _ := trivial
  isOpen_sUnion _ _ := trivial
```

問題18: 道の持ち上げの存在: 始点の持ち上げを決めれば、道は持ち上がることを示せ。

ヒント: Y を1点の空間 `Unit` として、問題16・15。

```
theorem exists_lift_path (p : CoveringMap E X) (γ : UI R → X) (hγ : Continuous γ) (e0 : E)
  (he : p.toFun e0 = γ ui0) :
```

```

    ∃ γ' : UI R → E, Continuous γ' ∧ (∀ t, p.toFun (γ' t) = γ t) ∧ γ' ui0 = e0 :=
sorry

```

問題19: ホモトピーの持ち上げ: $F : Y \times [0,1] \rightarrow X$ と $F(\cdot, 0)$ の連続な持ち上げから、 F 全体の連続な持ち上げが得られることを示せ。

ヒント: 各 y で道 $t \mapsto F(y, t)$ の持ち上げ (問題18) を選び、 $G(y, t)$ をその値とする。連続性は問題2で局所的に示す: 点 (y_0, t_0) で問題16・15から $N' \times [0,1]$ 上の連続な持ち上げ G' を取ると、 $y \in N'$ では問題17により $G(y, \cdot) = G'(y, \cdot)$ 。

```

theorem exists_lift_homotopy (p : CoveringMap E X) {Y : Type} [tY : TopologicalSpace Y]
  (F : Y × UI R → X) (hF : Continuous F) (F0 : Y → E) (hF0 : Continuous F0)
  (h0 : ∀ y, p.toFun (F0 y) = F (y, ui0)) :
  ∃ G : Y × UI R → E, Continuous G ∧ (∀ q, p.toFun (G q) = F q) ∧ ∀ y, G (y, ui0) = F0 y
:=
sorry

end

end CovSpace

```

解答 (12_CoveringSpaceSol.lean) では、ホモトピーの持ち上げが依存する公理は `propext` ・ `Classical.choice` ・ `Quot.sound` の3つだけである。問題を解き終えたら、次の出力に `sorryAx` が残っていないことを確かめよ。

```

#print axioms CovSpace.exists_lift_homotopy

```

```
import «11_CoveringSol»
import «12_CoveringSpaceSol»
```

発展演習: 基本亜群と円周

`12_CoveringSpace.lean` の持ち上げ定理を使って、位相空間の基本亜群とモノドロミーを組み立て、**円周の基本群は整数の群と同型**（問題26）であることを示す。`11_Covering.lean`（グラフ版）の Part C〜F と同じ道筋を位相空間でたどり直し、途中（Part D）で、グラフ版と位相版に共通する部分を一度だけ書く枠組みを取り出す。

グラフ版との対応

11_Covering.lean（グラフ）	この章（位相空間）
道 = 隣接条件付きの辺のリスト	道 = $[0, 1]$ からの連続写像
接続 = リストの連結	接続 = 区間を半分ずつ使って貼り合わせる
亜群の法則は、リストの等式から	亜群の法則は、再パラメータ化の補題から
ホモトピー不変性は、正規形定理から	ホモトピー不変性は、ホモトピーの持ち上げから
直線グラフ $\rightarrow$ 花束	$\mathbb{R} \rightarrow S^1 = \mathbb{R}/\mathbb{Z}$
直線グラフは単連結（既約な閉道がない）	$\mathbb{R}$ は単連結（直線ホモトピー）

構成

- Part A: 道
- Part B: ホモトピーと亜群の法則
- Part C: モノドロミー
- Part D: 共通の枠組み
- Part E: 円周
- Part F: $\pi_1(S^1) \simeq \mathbb{Z}$

進め方

`sorry` を自分の証明で置き換える。解答は `13_FundamentalGroupoidSol.lean` にある。前の章は解答ファイル（`11_CoveringSol.lean` ・ `12_CoveringSpaceSol.lean`）を `import` するので、前の章の問題が解け

ていなくても、この章は使える。

`namespace` CovSpace

Part A: 道

道は、単位区間から連続な写像で、始点と終点を指定したもの（`Path x y`）である。2本の道の**接続**は、区間の前半で1本目を、後半で2本目を2倍の速さでたどる。この「半分ずつの貼り合わせ」を `glue` として定義しておく（`12_CoveringSpace.lean` の局所的な持ち上げで使ったものと同じ形）。

道の**ホモトピー**は、端点を止めた連続な変形 $H : [0,1] \times [0,1] \rightarrow X$ である。 $H(s, t)$ の s が道のパラメータ、 t が変形のパラメータ。

この Part の中心は**再パラメータ化の補題**（問題3）である。端点で一致する $\varphi, \psi : [0,1] \rightarrow [0,1]$ について、 $\gamma \circ \varphi$ と $\gamma \circ \psi$ はホモトピックになる。証明は直線ホモトピー $(1-t)\varphi(s) + t\psi(s)$ で、単位区間が凸であることしか使わない。Part B の亜群の法則は、すべてこの補題に帰着する。

`section`

`open` CompleteOrderedField

`variable` {R : Type} [CompleteOrderedField R]

`variable` {X : Type} [tX : TopologicalSpace X]

単位区間どうしの半分の貼り合わせ: $s \leq 1/2$ なら $f(2s)$ 、そうでなければ $g(2s - 1)$ 。

```
noncomputable def glue {A : Type} (f g : UI R → A) (s : UI R) : A :=
  open Classical in
  if h : s.1 ≤ 1 / 2 then f (2 * s.1, two_mul_mem s.2.1 h)
  else g (2 * s.1 - 1, two_mul_sub_one_mem (le_of_lt (not_le.mp h)) s.2.2)

theorem glue_of_le {A : Type} (f g : UI R → A) {s : UI R} (h : s.1 ≤ 1 / 2) :
  glue f g s = f (2 * s.1, two_mul_mem s.2.1 h) := by
  unfold glue; rw [dif_pos h]
```

問題1: $1/2 \leq s$ なら貼り合わせは後半の値であることを示せ。境目 $s = 1/2$ で両者が一致することを使う。

ヒント: `unfold glue` して $s.1 \leq 1/2$ かどうかで場合分けする。等号の場合は $s = 1/2$ なので、 $(2s) = 1 \cdot (2s - 1) = 0$ を `UI.ext` で示して書き換え、仮定 $f\ 1 = g\ 0$ を使う。

```

theorem glue_of_ge {A : Type} (f g : UI R → A) (hfg : f ui1 = g ui0) {s : UI R} (h : 1 / 2
≤ s.1) :
  glue f g s = g (2 * s.1 - 1, two_mul_sub_one_mem h s.2.2) :=
  sorry

```

問題2: 貼り合わせは連続であることを示せ（パラメータつき版）。

ヒント: `12_CoveringSpace.lean` の貼り合わせ補題 (`continuous_of_closed_cover`) を、第2成分が $1/2$ 以下・以上の2つの閉集合に使う。それぞれの側では `glue_of_le` ・問題1で関数を書き直し (`funext`)、連続性を示す。

```

theorem continuous_glue_param {Z A : Type} [TopologicalSpace Z] [TopologicalSpace A]
  (f g : Z × UI R → A) (hf : Continuous f) (hg : Continuous g)
  (hfg : ∀ z, f (z, ui1) = g (z, ui0)) :
  Continuous (fun q : Z × UI R => glue (fun s => f (q.1, s)) (fun s => g (q.1, s)) q.2)
:=
  sorry

theorem continuous_glue {A : Type} [TopologicalSpace A] {f g : UI R → A}
  (hf : Continuous f) (hg : Continuous g) (hfg : f ui1 = g ui0) : Continuous (glue f g)
:= by
  have := continuous_glue_param (Z := Unit) (fun q => f q.2) (fun q => g q.2)
  (hf.comp continuous_snd) (hg.comp continuous_snd) (fun _ => hfg)
  exact this.comp (continuous_prod_mk (continuous_const_map ()) continuous_id)

```

`x` から `y` への道。

```

structure Path (x y : X) where
  toFun : UI R → X
  continuous_toFun : Continuous toFun
  source : toFun ui0 = x
  target : toFun ui1 = y

variable {x y z w : X}

theorem Path.ext {γ δ : Path (R := R) x y} (h : ∀ s, γ.toFun s = δ.toFun s) : γ = δ := by
  cases γ; cases δ
  have : _ := funext h
  subst this
  rfl

```

定数道。

```
def Path.refl (x : X) : Path (R := R) x x :=
  {fun _ => x, continuous_const_map x, rfl, rfl}
```

連接。

```
noncomputable def Path.trans (γ : Path (R := R) x y) (δ : Path (R := R) y z) : Path (R :=
R) x z :=
  (glue γ.toFun δ.toFun, continuous_glue γ.continuous_toFun δ.continuous_toFun
    (by rw [γ.target, δ.source]),
    by rw [glue_of_le (s := ui0) _ _ half_nonneg']
      exact (congrArg γ.toFun (UI.ext (mul_zero 2))).trans γ.source,
    by rw [glue_of_ge (s := ui1) _ _ (by rw [γ.target, δ.source]) half_le_one]
      exact (congrArg δ.toFun (UI.ext (show 2 * 1 - 1 = (1 : R) by
        rw [two_mul_one, two_def, add_sub_cancel])))).trans δ.target)
```

区間の反転 $s \mapsto 1 - s$ 。

```
def uiRev (s : UI R) : UI R :=
  (1 - s.1, sub_nonneg.mpr s.2.2, by
    have := add_le_add_left _ _ (neg_le_neg s.2.1) 1
    rwa [neg_zero, add_zero] at this)

theorem continuous_uiRev : Continuous (uiRev : UI R → UI R) :=
  continuous_subtype_mk (continuous_sub (continuous_const _) continuous_subtype_val) _

theorem uiRev_zero : uiRev (ui0 : UI R) = ui1 := UI.ext (sub_zero 1)
theorem uiRev_one : uiRev (ui1 : UI R) = ui0 := UI.ext (sub_self 1)
```

逆道。

```
def Path.symm (γ : Path (R := R) x y) : Path (R := R) y x :=
  {fun s => γ.toFun (uiRev s), γ.continuous_toFun.comp continuous_uiRev,
    by show γ.toFun (uiRev ui0) = y; rw [uiRev_zero, γ.target],
    by show γ.toFun (uiRev ui1) = x; rw [uiRev_one, γ.source]}
```

道のホモトピー（端点を止める）。 $H(s, t)$ の s が道のパラメータ、 t が変形のパラメータ。

```
def Path.Homotopic (γ δ : Path (R := R) x y) : Prop :=
  ∃ H : UI R × UI R → X, Continuous H ∧ (∀ s, H(s, ui0) = γ.toFun s) ∧
    (∀ s, H(s, ui1) = δ.toFun s) ∧ (∀ t, H(ui0, t) = x) ∧ (∀ t, H(ui1, t) = y)
```

再パラメータ化の補題

凸結合 $(1 - t) a + t b$ は区間に入る。

```
def uiMix (a b t : UI R) : UI R :=
  ((1 - t.1) * a.1 + t.1 * b.1,
   add_nonneg (mul_nonneg _ _ (sub_nonneg.mpr t.2.2) a.2.1) (mul_nonneg _ _ t.2.1 b.2.1),
   by
     have h1 := mul_le_mul_of_nonneg_left a.2.2 (sub_nonneg.mpr t.2.2)
     have h2 := mul_le_mul_of_nonneg_left b.2.2 t.2.1
     have := add_le_add h1 h2
     rwa [mul_one, mul_one, sub_add_cancel] at this)

theorem uiMix_zero (a b : UI R) : uiMix a b ui0 = a :=
  UI.ext (show (1 - 0) * a.1 + 0 * b.1 = a.1 by rw [sub_zero, one_mul, zero_mul,
add_zero]))

theorem uiMix_one (a b : UI R) : uiMix a b ui1 = b :=
  UI.ext (show (1 - 1) * a.1 + 1 * b.1 = b.1 by rw [sub_self, zero_mul, one_mul,
zero_add]))

theorem uiMix_self (a t : UI R) : uiMix a a t = a :=
  UI.ext (show (1 - t.1) * a.1 + t.1 * a.1 = a.1 by rw [← add_mul, sub_add_cancel,
one_mul]))

theorem continuous_uiMix {Z : Type} [TopologicalSpace Z] {a b t : Z → UI R}
  (ha : Continuous a) (hb : Continuous b) (ht : Continuous t) :
  Continuous (fun z => uiMix (a z) (b z) (t z)) :=
  continuous_subtype_mk (continuous_add
    (continuous_mul (continuous_sub (continuous_const _) (continuous_subtype_val.comp ht))
      (continuous_subtype_val.comp ha))
    (continuous_mul (continuous_subtype_val.comp ht) (continuous_subtype_val.comp hb))) _
```

問題3: 再パラメータ化の補題: 端点で一致する $\varphi, \psi : [0, 1] \rightarrow [0, 1]$ について、 $\gamma \circ \varphi$ と $\gamma \circ \psi$ はホモトピックであることを示せ。

ヒント: $H(s, t) := \gamma (\text{uiMix } (\varphi s) (\psi s) t)$ (直線ホモトピー)。端点では φ と ψ の値が一致するので uiMix_self 。

```
theorem homotopic_reparam (γ : UI R → X) (hγ : Continuous γ) {φ ψ : UI R → UI R}
  (hφ : Continuous φ) (hψ : Continuous ψ) (h0 : φ ui0 = ψ ui0) (h1 : φ ui1 = ψ ui1)
  (P Q : Path (R := R) × γ) (hP : ∀ s, P.toFun s = γ (φ s)) (hQ : ∀ s, Q.toFun s = γ (ψ
s)) :
  P.Homotopic Q :=
  sorry
```

end

Part B: ホモトピーと亜群の法則

ホモトピックであることは同値関係で（問題6・7）、連接・逆道と両立する（問題8）。亜群の法則（単位律・結合律・逆元律）は、両辺を**同じ道の再パラメータ化**として書き直し、問題3を使って示す。たとえば定数道との連接 `refl · γ` は、`γ · glue (定数 0) id` に等しい。

書き直しの計算は、貼り合わせと写像の合成が可換であること（問題4）に帰着する。結合律（問題10）も、`1/4 · 3/4` のような数値計算をせず、貼り合わせどうしの等式だけで示せる。

section

open CompleteOrderedField

variable {R : Type} [CompleteOrderedField R]

variable {X : Type} [tX : TopologicalSpace X]

variable {x y z w : X}

貼り合わせの計算規則

問題4: 写像を合成してから貼り合わせても、貼り合わせてから合成しても同じであることを示せ。

ヒント: `unfold glue` して `s.1 ≤ 1/2` で場合分けし、両辺の `dif_pos · dif_neg` を外す。

```
theorem glue_comp {A B : Type} (h : A → B) (f g : UI R → A) (s : UI R) :  
  h (glue f g s) = glue (fun u => h (f u)) (fun u => h (g u)) s :=  
  sorry
```

```
theorem glue_congr {A : Type} {f f' g g' : UI R → A} (hf : ∀ u, f u = f' u) (hg : ∀ u, g u  
= g' u)  
  (s : UI R) : glue f g s = glue f' g' s := by  
  rw [funext hf, funext hg]
```

```
theorem glue_const {A : Type} {f g : UI R → A} {c : A} (hf : ∀ u, f u = c) (hg : ∀ u, g u  
= c)  
  (s : UI R) : glue f g s = c := by  
  unfold glue  
  by_cases hs : s.1 ≤ 1 / 2  
  · rw [dif_pos hs, hf]  
  · rw [dif_neg hs, hg]
```

```

theorem glue_zero {A : Type} (f g : UI R → A) : glue f g ui0 = f ui0 := by
  rw [glue_of_le (s := ui0) _ _ half_nonneg']
  exact congrArg f (UI.ext (mul_zero 2))

theorem glue_one {A : Type} (f g : UI R → A) (hfg : f ui1 = g ui0) : glue f g ui1 = g ui1
:= by
  rw [glue_of_ge (s := ui1) _ _ hfg half_le_one]
  exact congrArg g (UI.ext (show 2 * 1 - 1 = (1 : R) by rw [two_mul_one, two_def,
add_sub_cancel])))

```

問題5: 前半への縮小 `halfL` で貼り合わせを読むと、前半の写像になることを示せ。

ヒント: `glue_of_le` を `halfL u` に使い ($u / 2 \leq 1/2$)、 $2 * (u / 2) = u$ (`mul_div_cancel_left`)。

```

theorem glue_halfL {A : Type} (f g : UI R → A) (u : UI R) : glue f g (halfL u) = f u :=
  sorry

theorem glue_halfR {A : Type} (f g : UI R → A) (hfg : f ui1 = g ui0) (u : UI R) :
  glue f g (halfR u) = g u := by
  have h : (1 : R) / 2 ≤ (u.1 + 1) / 2 := by
    have := div_two_le_div_two (add_le_add_right u.2.1 1)
    rwa [zero_add] at this
  rw [glue_of_ge (s := halfR u) _ _ hfg h]
  exact congrArg g (UI.ext (show 2 * ((u.1 + 1) / 2) - 1 = u.1 by
    rw [mul_div_cancel_left _ two_ne_zero, add_sub_cancel])))

theorem Path.trans_halfL (γ : Path (R := R) x y) (δ : Path (R := R) y z) (u : UI R) :
  (γ.trans δ).toFun (halfL u) = γ.toFun u := glue_halfL _ _ u

theorem Path.trans_halfR (γ : Path (R := R) x y) (δ : Path (R := R) y z) (u : UI R) :
  (γ.trans δ).toFun (halfR u) = δ.toFun u :=
  glue_halfR _ _ (by rw [γ.target, δ.source]) u

```

同値関係

```

theorem Path.Homotopic.refl' (γ : Path (R := R) x y) : γ.Homotopic γ :=
  (fun q => γ.toFun q.1, γ.continuous_toFun.comp continuous_fst, fun _ => rfl, fun _ =>
  rfl,
  fun _ => γ.source, fun _ => γ.target)

```

問題6: ホモトピーは対称であることを示せ。

ヒント: $H(s, 1 - t)$ (`uiRev`)。

```

theorem Path.Homotopic.symm' {γ δ : Path (R := R) × y} (h : γ.Homotopic δ) : δ.Homotopic γ
:=
  sorry

```

問題7: ホモトピーは推移的であることを示せ。

ヒント: 変形のパラメータ `t` の側で、2 つのホモトピーを問題2で貼り合わせる。

```

theorem Path.Homotopic.trans' {γ δ ε : Path (R := R) × y} (h : γ.Homotopic δ) (k :
δ.Homotopic ε) :
  γ.Homotopic ε :=
  sorry

```

接続・逆道との両立

```

theorem continuous_swap {A B : Type} [TopologicalSpace A] [TopologicalSpace B] :
  Continuous (fun q : A × B => (q.2, q.1)) := continuous_prod_mk continuous_snd
continuous_fst

```

問題8: 接続はホモトピーと両立することを示せ。

ヒント: 今度は道のパラメータ `s` の側で貼り合わせる。問題2は第 2 成分での貼り合わせなので、成分を入れ替えて (`continuous_swap`) 使う。

```

theorem Path.Homotopic.trans_congr {γ γ' : Path (R := R) × y} {δ δ' : Path (R := R) y z}
  (h : γ.Homotopic γ') (k : δ.Homotopic δ') : (γ.trans δ).Homotopic (γ'.trans δ') :=
  sorry

theorem Path.Homotopic.symm_congr {γ γ' : Path (R := R) × y} (h : γ.Homotopic γ') :
  γ.symm.Homotopic γ'.symm := by
obtain ⟨H, hH, h0, h1, hx, hy⟩ := h
refine ⟨fun q => H (uiRev q.1, q.2), hH.comp (continuous_prod_mk
  (continuous_uiRev.comp continuous_fst) continuous_snd), fun s => h0 _, fun s => h1 _,
  fun t => ?_, fun t => ?_⟩
· show H (uiRev ui0, t) = y; rw [uiRev_zero, hy]
· show H (uiRev ui1, t) = x; rw [uiRev_one, hx]

```

亜群の法則（再パラメータ化の補題から）

```

theorem continuous_id' {A : Type} [TopologicalSpace A] : Continuous (fun a : A => a) :=
  continuous_id

```

問題9: 左単位律を示せ。

ヒント: 問題3を $\varphi = \text{glue } (\text{fun } _ \Rightarrow \text{ui0}) \text{ id}$ 、 $\psi = \text{id}$ に使う。 $\text{refl} \cdot \gamma$ の値が $\gamma (\varphi s)$ であることは問題4。

```
theorem Path.refl_trans (γ : Path (R := R) x y) : ((Path.refl x).trans γ).Homotopic γ :=
  sorry

theorem Path.trans_refl (γ : Path (R := R) x y) : (γ.trans (Path.refl y)).Homotopic γ :=
  by
    refine homotopic_reparam γ.toFun γ.continuous_toFun (φ := glue (fun u => u) (fun _ =>
      ui1))
      (ψ := fun u => u) (continuous_glue continuous_id' (continuous_const_map _) rfl)
      continuous_id'
      (glue_zero _ _) (glue_one _ _ rfl) _ _ (fun s => ?_) (fun s => rfl)
    show glue γ.toFun (fun _ => y) s = γ.toFun (glue (fun u => u) (fun _ => ui1) s)
    rw [glue_comp γ.toFun]
    exact glue_congr (fun _ => rfl) (fun _ => γ.target.symm) s
```

問題10: (発展) 結合律を示せ。

ヒント: $\Gamma := \gamma.\text{trans } (\delta.\text{trans } \varepsilon)$ とし、左辺を $\Gamma \circ \text{glue } \alpha \beta$ と書く。ここで $\alpha = \text{glue halfL } (\text{halfR} \circ \text{halfL})$ 、 $\beta = \text{halfR} \circ \text{halfR}$ 。 $\Gamma \circ \text{halfL} = \gamma$ 、 $\Gamma \circ \text{halfR} = \delta.\text{trans } \varepsilon (\text{trans_halfL} \cdot \text{trans_halfR})$ 。

```
theorem Path.trans_assoc (γ : Path (R := R) x y) (δ : Path (R := R) y z) (ε : Path (R :=
  R) z w) :
  ((γ.trans δ).trans ε).Homotopic (γ.trans (δ.trans ε)) :=
  sorry
```

問題11: 逆道を先にたどると定数道にホモトピックであることを示せ。

ヒント: 問題3を $\varphi = \text{glue uiRev id}$ 、 $\psi = \text{定数 } 1$ に使う。

```
theorem Path.symm_trans (γ : Path (R := R) x y) : (γ.symm.trans γ).Homotopic (Path.refl y)
:=
  sorry

theorem Path.trans_symm (γ : Path (R := R) x y) : (γ.trans γ.symm).Homotopic (Path.refl x)
:= by
  refine homotopic_reparam γ.toFun γ.continuous_toFun (φ := glue (fun u => u) uiRev)
    (ψ := fun _ => ui0) (continuous_glue continuous_id' continuous_uiRev uiRev_zero.symm)
    (continuous_const_map _) (glue_zero _ _) (by rw [glue_one _ _ uiRev_zero.symm,
      uiRev_one]) _ _
    (fun s => ?_) (fun s => γ.source.symm)
```

```

show glue γ.toFun (fun u => γ.toFun (uiRev u)) s = γ.toFun (glue (fun u => u) uiRev s)
rw [glue_comp γ.toFun]

end

```

Part C: モノドロミー

道のホモトピー類を商型 `Quot` で作り (`PathClass`)、接続と逆道を入れる。亜群の法則は Part B から出る。

被覆 p について、点 x の上のファイバーの点 a と、 x から y への道 γ が与えられたとき、 γ を a から持ち上げた道の終点を `liftEnd  $\gamma$  a` とする。これがホモトピー類だけで決まること (問題13) は、`12_CoveringSpace.lean` のホモトピーの持ち上げから出る。グラフ版では正規形定理から出したところである。こうして得られる写像 `transport` がモノドロミーである。

モノドロミーがファイバーの間の全単射であることは、この Part では示さず、次の Part でグラフ版と一緒に一度だけ示す。

section

open CompleteOrderedField

variable {R : Type} [CompleteOrderedField R]

variable {X : Type} [tX : TopologicalSpace X]

道のホモトピー類。

```

def PathClass (R : Type) [CompleteOrderedField R] (x y : X) : Type :=
  Quot (fun γ δ : Path (R := R) x y => γ.Homotopic δ)

```

namespace PathClass

variable {x y z w : X}

def mk (γ : Path (R := R) x y) : PathClass R x y := Quot.mk _ γ

theorem sound {γ δ : Path (R := R) x y} (h : γ.Homotopic δ) : mk γ = mk δ := Quot.sound h

def refl (x : X) : PathClass R x x := mk (Path.refl x)

```

noncomputable def comp (φ : PathClass R x y) (ψ : PathClass R y z) : PathClass R x z :=
  Quot.lift (fun γ => Quot.lift (fun δ => mk (γ.trans δ))
    (fun _ _ h => sound (Path.Homotopic.trans_congr (Path.Homotopic.refl' γ) h)) ψ)

```

```

    (fun _ _ h => by
      induction  $\psi$  using Quot.ind with
      | mk  $\delta$  => exact sound (Path.Homotopic.trans_congr h (Path.Homotopic.refl'  $\delta$ ))
     $\phi$ 

def inv ( $\phi$  : PathClass R x y) : PathClass R y x :=
  Quot.lift (fun  $\gamma$  => mk  $\gamma$ .symm) (fun _ _ h => sound (Path.Homotopic.symm_congr h))  $\phi$ 

theorem refl_comp ( $\phi$  : PathClass R x y) : (refl x).comp  $\phi$  =  $\phi$  := by
  induction  $\phi$  using Quot.ind with
  | mk  $\gamma$  => exact sound (Path.refl_trans  $\gamma$ )

theorem comp_refl ( $\phi$  : PathClass R x y) :  $\phi$ .comp (refl y) =  $\phi$  := by
  induction  $\phi$  using Quot.ind with
  | mk  $\gamma$  => exact sound (Path.trans_refl  $\gamma$ )

theorem comp_assoc ( $\phi$  : PathClass R x y) ( $\psi$  : PathClass R y z) ( $\chi$  : PathClass R z w) :
  ( $\phi$ .comp  $\psi$ ).comp  $\chi$  =  $\phi$ .comp ( $\psi$ .comp  $\chi$ ) := by
  induction  $\phi$  using Quot.ind with
  | mk  $\gamma$  =>
    induction  $\psi$  using Quot.ind with
    | mk  $\delta$  =>
      induction  $\chi$  using Quot.ind with
      | mk  $\epsilon$  => exact sound (Path.trans_assoc  $\gamma$   $\delta$   $\epsilon$ )

theorem inv_comp ( $\phi$  : PathClass R x y) :  $\phi$ .inv.comp  $\phi$  = refl y := by
  induction  $\phi$  using Quot.ind with
  | mk  $\gamma$  => exact sound (Path.symm_trans  $\gamma$ )

theorem comp_inv ( $\phi$  : PathClass R x y) :  $\phi$ .comp  $\phi$ .inv = refl x := by
  induction  $\phi$  using Quot.ind with
  | mk  $\gamma$  => exact sound (Path.trans_symm  $\gamma$ )

end PathClass

```

モノドロミー

```

variable {E : Type} [tE : TopologicalSpace E]

namespace CoveringMap

variable (p : CoveringMap E X)

```

点 x の上のファイバー。

```
abbrev Fiber (x : X) : Type := {e : E // p.toFun e = x}
```

```
theorem fiberExt {x : X} {a b : p.Fiber x} (h : a.1 = b.1) : a = b := by
  cases a; cases b; cases h; rfl
```

```
variable {x y z : X}
```

```
theorem fiber_start (γ : Path (R := R) x y) (a : p.Fiber x) : p.toFun a.1 = γ.toFun ui0 :=
  by
    rw [a.2, γ.source]
```

道の持ち上げの終点。

```
noncomputable def liftEnd (γ : Path (R := R) x y) (a : p.Fiber x) : p.Fiber y :=
  (Classical.choose (exists_lift_path p γ.toFun γ.continuous_toFun a.1 (p.fiber_start γ
a)) ui1,
  by rw [(Classical.choose_spec (exists_lift_path p γ.toFun γ.continuous_toFun a.1
(p.fiber_start γ a))).2.1, γ.target])
```

問題12: 持ち上げの終点は、持ち上げを 1 つ示せば決まることを示せ。

ヒント: 選んだ持ち上げと与えられた持ち上げに、12_CoveringSpace.lean の lift_unique を使う。

```
theorem liftEnd_eq (γ : Path (R := R) x y) (a : p.Fiber x) {f : UI R → E} (hf : Continuous
f)
  (hfp : ∀ t, p.toFun (f t) = γ.toFun t) (hf0 : f ui0 = a.1) : (p.liftEnd γ a).1 = f ui1
:=
  sorry
```

問題13: ホモトピックな道の持ち上げは同じ点に着くことを示せ。

ヒント: exists_lift_homotopy を H に使い、γ の持ち上げから始まる持ち上げ G を得る。G(0, ·) と G(1, ·) は定数道の持ち上げなので定数 (lift_unique)。すると G(·, 1) は δ の持ち上げで、始点は a、終点は γ の持ち上げの終点。

```
theorem liftEnd_homotopic {γ δ : Path (R := R) x y} (h : γ.Homotopic δ) (a : p.Fiber x) :
  p.liftEnd γ a = p.liftEnd δ a :=
  sorry
```

ホモトピー類に沿った輸送 (モノドロミー)。

```
noncomputable def transport (φ : PathClass R x y) (a : p.Fiber x) : p.Fiber y :=
  Quot.lift (fun γ => p.liftEnd γ a) (fun _ _ h => p.liftEnd_homotopic h a) φ
```

```

theorem transport_refl (a : p.Fiber x) : p.transport (PathClass.refl (R := R) x) a = a :=
  p.fiberExt (p.liftEnd_eq (Path.refl x) a (continuous_const_map a.1) (fun _ => a.2) rfl)

```

問題14: 接続に沿った輸送は、輸送の合成であることを示せ。

ヒント: γ の持ち上げ γ' と、その終点からの δ の持ち上げ δ' を貼り合わせる (`continuous_glue`) と、 $\gamma.trans \delta$ の持ち上げになる (問題4で p と貼り合わせを入れ替える)。

```

theorem transport_trans (φ : PathClass R x y) (ψ : PathClass R y z) (a : p.Fiber x) :
  p.transport (φ.comp ψ) a = p.transport ψ (p.transport φ a) :=
  sorry

end CoveringMap

end

```

Part D: 共通の枠組み

`11_Covering.lean` の Part E の補足で、「モノドロミーの 3 つの問題 (24~26) はグラフであることを一度も使っていない」と述べた。使ったのは、持ち上げが一意に定まることと、道のホモトピー類の亜群の法則だけである。この Part では、それを定理の形にする。

- `PathSystem`: 道のホモトピー類の亜群 (接続・定数道・逆道と、その法則)
- `LiftSystem`: ファイバーの族と、道のホモトピー類に沿った輸送 (定数道・接続との関係)

「モノドロミーはファイバーの間の全単射」(問題15) を、この抽象的な形で一度だけ証明する。グラフの被覆 (`11_Covering.lean`) と位相空間の被覆 (Part C) は、どちらもこの枠組みの**実例**になる。2 つの被覆理論の類似は、ここで「同じ定理の 2 つの実例」という形式化された事実になる。

section

道のホモトピー類の接続構造 (亜群) の抽象化。

```

structure PathSystem (V : Type) where
  Hom : V → V → Type
  refl : (x : V) → Hom x x
  trans : {x y z : V} → Hom x y → Hom y z → Hom x z
  symm : {x y : V} → Hom x y → Hom y x
  refl_trans : ∀ {x y : V} (γ : Hom x y), trans (refl x) γ = γ
  trans_refl : ∀ {x y : V} (γ : Hom x y), trans γ (refl y) = γ
  trans_assoc : ∀ {x y z w : V} (γ : Hom x y) (δ : Hom y z) (ε : Hom z w),

```

```

trans (trans γ δ) ε = trans γ (trans δ ε)
symm_trans : ∀ {x y : V} (γ : Hom x y), trans (symm γ) γ = refl y
trans_symm : ∀ {x y : V} (γ : Hom x y), trans γ (symm γ) = refl x

```

被覆の抽象化: ファイバーの族と、道のホモトピー類に沿った輸送。

```

structure LiftSystem {V : Type} (P : PathSystem V) (F : V → Type) where
  transport : {x y : V} → P.Hom x y → F x → F y
  transport_refl : ∀ (x : V) (a : F x), transport (P.refl x) a = a
  transport_trans : ∀ {x y z : V} (γ : P.Hom x y) (δ : P.Hom y z) (a : F x),
    transport (P.trans γ δ) a = transport δ (transport γ a)

namespace LiftSystem

variable {V : Type} {P : PathSystem V} {F : V → Type} (L : LiftSystem P F)

```

問題15: **共通の定理**: モノドロミーはファイバーの間の全単射であることを示せ。

ヒント: 逆道 `P.symm γ` に沿った輸送が逆写像になる。 `transport_trans · transport_refl` と、`PathSystem` の法則 `trans_symm · symm_trans`。 `11_Covering.lean` の問題26と同じ証明である。

```

theorem transport_bijective {x y : V} (γ : P.Hom x y) : Function.Bijective (L.transport γ)
where
  injective := sorry
  surjective := sorry

end LiftSystem

```

実例 1: グラフの被覆 (`11_Covering.lean`)

```

def graphPathSystem (G : SGraph) : PathSystem G.V where
  Hom := SGraph.PathClass G
  refl := SGraph.PathClass.refl
  trans φ ψ := φ.comp ψ
  symm φ := φ.inv
  refl_trans := SGraph.PathClass.refl_comp
  trans_refl := SGraph.PathClass.comp_refl
  trans_assoc := SGraph.PathClass.comp_assoc
  symm_trans := SGraph.PathClass.inv_comp
  trans_symm := SGraph.PathClass.comp_inv

noncomputable def graphLiftSystem {Y X : SGraph} (p : SGraph.Covering Y X) :
  LiftSystem (graphPathSystem X) p.Fiber where

```

```

transport := p.transport
transport_refl _ a := p.transport_refl a
transport_trans := p.transport_trans

```

実例 2: 位相空間の被覆

```

open CompleteOrderedField

variable {R : Type} [CompleteOrderedField R]

noncomputable def topPathSystem (R : Type) [CompleteOrderedField R] (X : Type)
  [TopologicalSpace X] : PathSystem X where
  Hom := PathClass R
  refl := PathClass.refl
  trans  $\phi$   $\psi$  :=  $\phi$ .comp  $\psi$ 
  symm  $\phi$  :=  $\phi$ .inv
  refl_trans := PathClass.refl_comp
  trans_refl := PathClass.comp_refl
  trans_assoc := PathClass.comp_assoc
  symm_trans := PathClass.inv_comp
  trans_symm := PathClass.comp_inv

noncomputable def topLiftSystem {E X : Type} [TopologicalSpace E] [TopologicalSpace X]
  (p : CoveringMap E X) : LiftSystem (topPathSystem R X) p.Fiber where
  transport := p.transport
  transport_refl _ a := p.transport_refl a
  transport_trans := p.transport_trans

```

同じ定理の 2 つの実例。

```

example {Y X : SGraph} (p : SGraph.Covering Y X) {v w : X.V} ( $\gamma$  : SGraph.PathClass X v w)
:
  Function.Bijective (p.transport  $\gamma$ ) :=
    (graphLiftSystem p).transport_bijective  $\gamma$ 

example {E X : Type} [TopologicalSpace E] [TopologicalSpace X] (p : CoveringMap E X) {x y
: X}
  ( $\gamma$  : PathClass R x y) : Function.Bijective (p.transport  $\gamma$ ) :=
    (topLiftSystem p).transport_bijective  $\gamma$ 

end

```

Part E: 円周

円周を $S^1 = \mathbb{R}/\mathbb{Z}$ (整数だけずれた 2 数を同一視した商) として作り、 `07_Exercises.lean` Part 6 の商位相を入れる。射影 `proj : R → R/ℤ` が被覆であることを示す。

点 `[a]` の均等被覆近傍は、`a` のまわりの半径 `1/2` の開区間の像で、シートは整数 `n` だけずらした開区間 $V\ n = (a + n - 1/2, a + n + 1/2)$ 、シートへの逆写像は `circleSec a n` である。逆写像の連続性は、`proj` が開写像であること (問題18) から出る。

section

open CompleteOrderedField

variable {R : Type} [CompleteOrderedField R]

整数の計算

```
theorem natCast_succ_ge_one (k : Nat) : (1 : R) ≤ natCast (k + 1) := by
  rw [natCast_succ]
  have := add_le_add_right (natCast_nonneg (R := R) k) 1
  rwa [zero_add] at this
```

問題16: 絶対値が 1 未満の整数は 0 であることを示せ。

ヒント: `Int.ofNat` と `Int.negSucc` で場合分けする。 `natCast (k + 1) ≥ 1` (`natCast_succ_ge_one`)。

```
theorem int_eq_zero_of_abs_lt_one : ∀ {k : Int}, abs (intCast k : R) < 1 → k = 0 :=
  sorry

theorem intCast_sub (m n : Int) : (intCast (m - n) : R) = intCast m - intCast n := by
  rw [Int.sub_eq_add_neg, intCast_add, intCast_neg]; rfl

theorem intCast_injective {m n : Int} (h : (intCast m : R) = intCast n) : m = n := by
  have : m - n = 0 := int_eq_zero_of_abs_lt_one (R := R)
    (by rw [intCast_sub, h, sub_self, abs_zero]; exact zero_lt_one)
  omega
```

問題17: 差が整数で、どちらも `a` から `1/2` 未満の距離にある 2 数は等しいことを示せ。

ヒント: `intCast m = (w - a) + (a - r)` の絶対値は 1 未満なので、問題16で `m = 0`。

```

theorem eq_of_close {a r w : R} {m : Int} (hr : abs (r - a) < 1 / 2) (hw : abs (w - a) < 1 / 2)
  (hm : w = r + intCast m) : w = r :=
  sorry

```

円周 $\mathbb{R} / \mathbb{Z}$

整数だけずれた 2 数を同一視する。

```

def circleSetoid (R : Type) [CompleteOrderedField R] : Setoid R where
  r a b := ∃ n : Int, b = a + intCast n
  iseqv := {
    refl := fun a => ⟨0, (add_zero a).symm⟩
    symm := fun {a b} ⟨n, h⟩ => ⟨-n, by rw [h, intCast_neg, add_assoc, add_neg_cancel,
add_zero]⟩
    trans := fun {a b c} ⟨m, h₁⟩ ⟨n, h₂⟩ => ⟨m + n, by rw [h₂, h₁, intCast_add,
add_assoc]⟩
  }

```

円周。

```

def Circle (R : Type) [CompleteOrderedField R] : Type := Quotient (circleSetoid R)

instance : TopologicalSpace (Circle R) :=
  inferInstanceAs (TopologicalSpace (Quot (circleSetoid R).r))

```

射影 $\mathbb{R} \rightarrow \mathbb{R}/\mathbb{Z}$ 。

```

def proj (r : R) : Circle R := Quotient.mk _ r

theorem proj_eq_iff {a b : R} : proj a = proj b ↔ ∃ n : Int, b = a + intCast n :=
  (fun h => Quotient.exact h, fun h => Quotient.sound h)

theorem proj_add_int (a : R) (n : Int) : proj (a + intCast n) = proj a :=
  (proj_eq_iff.mpr ⟨n, rfl⟩).symm

theorem continuous_proj : Continuous (proj : R → Circle R) :=
  continuous_quotMk (circleSetoid R).r

theorem isOpen_circle_iff {s : Set (Circle R)} : IsOpen s ↔ IsOpen (proj ⁻¹' s) := Iff.rfl

```

問題18: 射影 $\text{proj} : \mathbb{R} \rightarrow \mathbb{R}/\mathbb{Z}$ は開写像であることを示せ。

ヒント: 商位相での開集合は、 R への逆像が開いたもの (`isOpen_circle_iff`)。 $w \in W$ と $r = w + n$ なら、 r のまわりの近傍を n だけずらすと W の中に入る。

```
theorem isOpen_image_proj {W : Set R} (hW : IsOpen W) :
  IsOpen ({z | ∃ w, w ∈ W ∧ proj w = z} : Set (Circle R)) :=
  sorry
```

$R \rightarrow R/\mathbb{Z}$ は被覆

点 a のまわりの均等被覆近傍の、シート n の逆写像。

```
noncomputable def circleSec (a : R) (n : Int) (z : Circle R) : R :=
  open Classical in
  if h : ∃ r, abs (r - a) < 1 / 2 ∧ proj r = z then Classical.choose h + intCast n else 0

theorem circleSec_spec {a : R} {z : Circle R} (h : ∃ r, abs (r - a) < 1 / 2 ∧ proj r = z)
(n : Int) :
  ∃ r, abs (r - a) < 1 / 2 ∧ proj r = z ∧ circleSec a n z = r + intCast n := by
  refine (Classical.choose h, (Classical.choose_spec h).1, (Classical.choose_spec h).2, ?
_)
  unfold circleSec; rw [dif_pos h]
```

a から $1/2$ 未満の点 r の類については、逆写像の値は $r + n$ 。

```
theorem circleSec_proj {a r : R} (hr : abs (r - a) < 1 / 2) (n : Int) :
  circleSec a n (proj r) = r + intCast n := by
  obtain ⟨r', hr', hpr', hs⟩ := circleSec_spec ⟨r, hr, rfl⟩ n
  obtain ⟨m, hm⟩ := proj_eq_iff.mp hpr'
  rw [hs, ← eq_of_close hr' hr hm]

theorem abs_shift (r a : R) (n : Int) : abs (r + intCast n - (a + intCast n)) = abs (r -
a) := by
  rw [add_sub_add_right]
```

問題19: (発展) a のまわりの $1/2$ -近傍の像は均等に被覆されることを示せ。

ヒント: シートは `ball (a + n) (1/2)`、逆写像は `circleSec a n`。合併は `proj_eq_iff`、交わらないことは問題17、シートの点での逆写像は `circleSec_proj`。逆写像の連続性は、 0 の逆像が `proj` による開集合 $\{r \mid |r - a| < 1/2 \wedge r + n \in 0\}$ の像になることと問題18。

```
theorem evenlyCovered_circle (a : R) :
  EvenlyCovered (proj : R → Circle R) {z | ∃ r, r ∈ ball a (1 / 2) ∧ proj r = z} :=
```

sorry

$\mathbb{R} \rightarrow \mathbb{R}/\mathbb{Z}$ は被覆。

```
def circleCovering : CoveringMap R (Circle R) where
  toFun := proj
  continuous_toFun := continuous_proj
  evenly := by
    intro x
    induction x using Quotient.ind with
    | _ a =>
      exact (_, isOpen_image_proj (isOpen_ball a (1 / 2)), {a, mem_ball_self (half_pos
zero_lt_one),
      rfl}, evenlyCovered_circle a)

end
```

Part F: $\pi_1(S^1) \simeq \mathbb{Z}$

基点 $[0]$ のループの類 φ に、 0 から持ち上げた終点の整数を対応させる。これが**モノドロミー次数** $\deg$ である。主定理（問題26）は、 $\deg$ が連接を和に写す全単射であること。証明は `11_Covering.lean` の Part F と同じ 3 つの部品からなる。

- **準同型性**（問題22）：整数 m だけの平行移動（**デッキ変換**）は持ち上げと可換（問題21）
- **単射性**（問題23・24）： $\mathbb{R}$ は単連結。持ち上げた閉じた道は、直線ホモトピーで定数に縮み、それを `proj` で押し出すと、もとのループが定数ループにホモトピックとわかる
- **全射性**（問題25）： n 周するループ $t \mapsto [n t]$ の持ち上げは $t \mapsto n t$ で、終点は n

section

open CompleteOrderedField

variable {R : Type} [CompleteOrderedField R]

円周の基点 $[0]$ 。

```
def base : Circle R := proj 0

theorem proj_intCast (m : Int) : proj (intCast m : R) = base := by
  have := proj_add_int (0 : R) m
```

```
rw [zero_add] at this
exact this
```

基点の上のファイバーの点 m (整数)。

```
def fibInt (m : Int) : circleCovering.Fiber (base : Circle R) := {intCast m, proj_intCast m}
```

問題20: 基点の上のファイバーの点は整数であることを示せ。

ヒント: $\text{proj } a = \text{proj } 0$ から、ある n で $0 = a + n$ (proj_eq_iff)。

```
theorem fiber_int (a : circleCovering.Fiber (base : Circle R)) :  $\exists n : \text{Int}, a.1 = \text{intCast } n$  :=
  sorry
```

問題21: **デッキ変換**: 整数 m から持ち上げると、 0 から持ち上げた終点を m だけずらした点に着くことを示せ。

ヒント: 0 からの持ち上げ γ' を m だけずらした $\gamma' + m$ は、 m からの持ち上げ (proj_add_int)。問題12。

```
theorem transport_fibInt ( $\phi$  : PathClass R (base : Circle R) base) (m : Int) :
  (circleCovering.transport  $\phi$  (fibInt m)).1 = (circleCovering.transport  $\phi$  (fibInt 0)).1
+ intCast m :=
  sorry
```

モノドロミー次数: ループを 0 から持ち上げた終点の整数。

```
noncomputable def deg ( $\phi$  : PathClass R (base : Circle R) base) : Int :=
  Classical.choose (fiber_int (circleCovering.transport  $\phi$  (fibInt 0)))

theorem deg_spec ( $\phi$  : PathClass R (base : Circle R) base) :
  circleCovering.transport  $\phi$  (fibInt 0) = fibInt (deg  $\phi$ ) :=
  circleCovering.fiberExt (Classical.choose_spec (fiber_int (circleCovering.transport  $\phi$ 
    (fibInt 0))))
```

問題22: モノドロミー次数は連接を和に写すことを示せ。

ヒント: transport_trans で分解し、 deg_spec と問題21。整数の等式に戻すのは intCast_injective 。

```

theorem deg_comp (φ ψ : PathClass R (base : Circle R) base) :
  deg (φ.comp ψ) = deg φ + deg ψ :=
  sorry

theorem deg_refl : deg (PathClass.refl (R := R) (base : Circle R)) = 0 := by
  apply intCast_injective (R := R)
  have h := congrArg Subtype.val (deg_spec (PathClass.refl (R := R) (base : Circle R)))
  rw [circleCovering.transport_refl] at h
  exact h.symm

theorem deg_inv (φ : PathClass R (base : Circle R) base) : deg φ.inv = -deg φ := by
  have h := deg_comp φ.inv φ
  rw [PathClass.inv_comp, deg_refl] at h
  omega

```

問題23: 次数 0 のループは定数ループにホモトピックであることを示せ。

ヒント: γ の持ち上げ γ' は 0 から 0 への閉じた道。直線ホモトピー $(1 - t) \gamma'(s)$ を `proj` で押し出すと、 γ から定数ループへのホモトピーになる。

```

theorem deg_eq_zero {φ : PathClass R (base : Circle R) base} (h : deg φ = 0) :
  φ = PathClass.refl base :=
  sorry

```

問題24: モノドロミー次数は単射であることを示せ。

ヒント: $\deg(\phi \cdot \psi \cdot \phi^{-1}) = 0$ を示して問題23を使い、亜群の法則で $\phi = \psi$ に直す。
11_Covering.lean の問題34と同じ。

```

theorem deg_injective : Function.Injective (deg (R := R)) :=
  sorry

```

n 周するループ $t \mapsto [n \ t]$ 。

```

def loopN (n : Int) : Path (R := R) (base : Circle R) base where
  toFun t := proj (intCast n * t.1)
  continuous_toFun := continuous_proj.comp (continuous_mul (continuous_const _))
  continuous_subtype_val
  source := by show proj (intCast n * 0) = base; rw [mul_zero]; rfl
  target := by show proj (intCast n * 1) = base; rw [mul_one]; exact proj_intCast n

```

問題25: モノドロミー次数は全射であることを示せ。

ヒント: n 周するループ `loopN n` の持ち上げは $t \mapsto n \ t$ (問題12)。

```

theorem deg_surjective : Function.Surjective (deg (R := R)) :=
  sorry

```

問題26: 主定理 $\pi_1(S^1) \simeq \mathbb{Z}$: モノドロミー次数は、接続を和に写す全単射であることを示せ。

ヒント: 問題22・24・25。

```

theorem pi1_circle :
  (∀ φ ψ : PathClass R (base : Circle R) base, deg (φ.comp ψ) = deg φ + deg ψ) ∧
  Function.Bijective (deg (R := R)) :=
  sorry

end

end CovSpace

```

解答（13_FundamentalGroupoidSol.lean）では、主定理が依存する公理は `propext` ・ `Classical.choice` ・ `Quot.sound` の3つだけである（11_Covering.lean の主定理と同じ）。問題を解き終えたら、次の出力に `sorryAx` が残っていないことを確かめよ。

```

#print axioms CovSpace.pi1_circle

```